

The Art of Debugging with GDB, DDD, and Eclipse. Русский перевод

Русский перевод книги Norman Matloff и Peter Jay Salzman об отладке программ с помощью GDB, DDD и Eclipse.

Больше крутых материалов в моём телеграм-канале [@grogrigs](#)

Предисловие

"Слушайте, эта штука и правда работает!" Так сказал один из наших студентов, Эндрю, после того как впервые всерьез воспользовался отладчиком. О средствах отладки он узнал тремя годами раньше, на курсах программирования для первокурсников, но тогда отмахнулся от них как от очередной темы, которую надо выучить к выпускному экзамену. Теперь, уже на четвертом курсе, преподаватель убедил Эндрю перестать отлаживать программы с помощью операторов печати и начать пользоваться настоящими отладочными инструментами. К своему удовольствию и удивлению, он обнаружил, что с правильными инструментами время отладки можно резко сократить.

Таких "Эндрю" вокруг много - и среди студентов, и среди работающих программистов, - и мы надеемся, что эта книга поможет им пережить похожее озарение. Но еще больше мы хотим обратиться к тем, кто уже пользуется отладчиками, но не вполне представляет, что можно делать в специальных ситуациях, и хотел бы лучше разобраться как в самих средствах отладки, так и в стоящей за ними философии.

Как заметил редактор этой книги, во многих сообществах существует обширное знание об отладке, но оно живет в виде своеобразного фольклора и не попадает в книги. Что ж, эта книга должна это изменить. Мы будем отвечать, например, на такие вопросы:

- Как отлаживать код с потоками?
- Почему точки останова иногда оказываются немного не там, где вы их поставили?
- Почему команда GDB `until` иногда переносит выполнение в "удивительное" место?
- Какие интересные приемы можно применять в DDD и Eclipse?
- Есть ли в эпоху графических интерфейсов какая-нибудь ценность у текстовых приложений вроде GDB?
- Почему ошибка сегментации не возникла, когда ваш ошибочный код вышел за границы массива?
- Почему одна из наших примерных структур данных называется `nsp`? (Извините, это внутренняя шутка с нашим издателем.)

Эта книга не является ни прославленным руководством пользователя, ни отвлеченным трактатом по когнитивной теории процесса отладки. Она находится где-то между этими двумя жанрами. С одной стороны, мы действительно даем сведения в духе "как это сделать" для конкретных команд GDB, DDD и Eclipse; с другой - формулируем и часто используем общие принципы процесса отладки.

В качестве основных инструментов для примеров мы выбрали GDB, DDD и Eclipse из-за их популярности в сообществах Linux и open source. Наши примеры немного больше тяготеют к GDB не только потому, что текстовая природа этого инструмента позволяет компактнее показывать материал на странице, но и потому, что, как уже было сказано выше, мы считаем: текстовые команды по-прежнему играют ценную роль в процессе отладки.

Eclipse стал широко использоваться далеко не только для той отладочной роли, которую мы рассматриваем здесь, и действительно предоставляет привлекательный, гибкий инструмент для отладки. С другой стороны, DDD гораздо компактнее и включает несколько мощных возможностей, которых нет в Eclipse.

Глава 1, "Некоторые предварительные сведения для новичков и профессионалов", представляет собой обзор. У многих опытных программистов может возникнуть соблазн пропустить ее, но мы настоятельно советуем прочитать ее полностью, поскольку в ней сформулирован ряд простых, но сильных общих рекомендаций, которые мы советуем применять при отладке.

Затем глава 2, "Остановка, чтобы осмотреться", рассматривает рабочую лошадку отладки - точку останова - и обсуждает все подробности: установку, удаление и отключение точек останова; переход от одной точки останова к следующей; просмотр подробной информации о точках останова и так далее.

Когда вы добрались до точки останова, что дальше? Глава 3, "Просмотр и установка переменных", отвечает на этот вопрос. Наш сквозной пример здесь связан с кодом, который обходит дерево. Ключевая задача - удобно показать содержимое узла дерева, когда мы достигаем точки останова. Здесь GDB раскрывается особенно хорошо, предоставляя очень гибкие возможности, позволяющие эффективно выводить интересующую информацию каждый раз, когда программа приостанавливается. Мы также показываем особенно удачную возможность DDD для графического отображения деревьев и других связанных структур данных.

Глава 4, "Когда программа аварийно завершается", посвящена пугающим ошибкам времени выполнения, возникающим из-за нарушений сегментации. Сначала мы рассказываем, что происходит на нижних уровнях, включая распределение памяти для программы и совместную роль аппаратуры и операционной системы. Читатели с хорошим системным кругозором могут просмотреть этот материал бегло, но мы считаем, что многим другим будет полезно получить такую основу. Затем мы переходим к core-файлам: как они создаются, как использовать их для "посмертного" анализа и так далее. Завершается глава расширенным примером сеанса отладки, в котором несколько ошибок приводят к нарушениям сегментации.

Название главы 5 - "Отладка в контексте нескольких видов активности" - выбрано, чтобы подчеркнуть: мы рассматриваем не только параллельное программирование, но и сетевой код. Клиент-серверное сетевое программирование тоже является разновидностью параллельной обработки, причем даже наши инструменты используются параллельно - например, два окна GDB, одно для клиента и одно для сервера. Поскольку сетевой код связан с системными вызовами, мы дополняем отладочные инструменты переменной C/C++ `errno` и командой Linux `strace`. Следующая часть главы 5 посвящена программированию с потоками. Здесь мы снова начинаем с обзора инфраструктуры: разделение времени, процессы и потоки, состояния гонки и так далее. Мы приводим технические подробности работы с потоками в GDB, DDD и Eclipse и снова обсуждаем общие принципы, которые полезно держать в голове, например случайность моментов, в которые происходят переключения контекста потоков. Последняя часть главы 5 касается параллельного программирования с популярными пакетами MPI и OpenMP. Завершаем мы расширенным примером на OpenMP.

Глава 6, "Специальные темы", охватывает несколько важных разрозненных вопросов. Отладчик не поможет, если ваш код даже не компилируется, поэтому мы обсуждаем подходы к такой ситуации. Затем рассматриваем проблему неудачной компоновки из-за отсутствующих библиотек; и здесь мы снова сочли полезным дать немного "теории" - например, о типах библиотек и о том, как они связываются с основным кодом. А как быть с отладкой программ с графическим интерфейсом? Для простоты мы ограничиваемся "полу-GUI" случаем - программированием с `curses` - и показываем, как заставить GDB, DDD и Eclipse взаимодействовать с событиями в окне `curses`.

Как уже отмечалось, процесс отладки можно значительно усилить с помощью дополнительных инструментов; некоторые из них представлены в главе 7, "Другие инструменты". Там мы продолжаем разговор об `errno` и `strace`, даем материал о `lint` и советы по эффективному использованию текстового редактора.

Хотя книга сосредоточена на C/C++, в главе 8, "Использование GDB/DDD/Eclipse для других языков", мы рассматриваем и другие языки: Java, Python, Perl и язык ассемблера.

Приносим извинения, если каким-то образом обошли вниманием любимую тему читателя, связанную с отладкой, но мы постарались охватить тот материал, который сами находили полезным в собственной практике программирования.

Мы глубоко признательны многим сотрудникам No Starch Press, которые помогали нам в этом проекте на протяжении его долгой истории. Особенно мы благодарим основателя и редактора издательства Билла Поллока. Он с самого начала поверил в этот необычный проект и проявлял удивительное терпение к нашим многочисленным задержкам.

Дэниел Джейкобовиц проделал по-настоящему блестящую работу, рецензируя рукопись и дав множество бесценных советов. Нил Чинг, формально приглашенный для литературного редактирования, на деле оказался "подсадным" специалистом с дипломом по информатике! Он поднял ряд важных вопросов, связанных с ясностью наших технических объяснений. Благодаря отзывам Дэниела и Нила качество книги существенно выросло. Разумеется, необходимо сделать обычную оговорку: любые ошибки остаются нашими собственными.

Норм говорит: я хочу сказать xie xie и todah rabah моей жене Гамис и дочери Лоре, двум удивительным людям, с которыми мне посчастливилось быть родным. Их подход к решению задач, искрящийся юмор и joie de vivre пронизывают эту книгу, хотя они не прочитали в ней ни слова. Я также благодарю многих студентов, которых учил за эти годы: они учат меня не меньше, чем я их, и дают мне ощущение, что я все-таки правильно выбрал профессию. Я всегда стремился "что-то изменить" и надеюсь, что эта книга хотя бы немного в этом поможет.

Пит добавляет: я благодарю Николь Карлсон, Марка Кима и Ронду Зальцман за то, что они провели много часов за чтением глав, исправлениями и предложениями - без всякой иной причины, кроме того, что вы читаете прямо сейчас. Я также хочу поблагодарить участников Linux Users Group of Davis, которые годами отвечали на мои вопросы. Знакомство с вами сделало меня умнее. Todah адресовано Эвелин, которая во всем улучшила мою жизнь. Особое упоминание - Джорди ("J-Train" из Сан-Франциско), который бескорыстно использовал собственный кошачий вес, чтобы страницы не разлетались, всегда согревал мое место и следил, чтобы комната никогда не пустовала. Нам тебя очень не хватает каждый день. Мурчи дальше, малыш. Привет, мама! Смотри, что я сделал!

Норм Мэтлофф и Пит Зальцман

9 июня 2008 г.

Глава 1. Некоторые предварительные сведения для новичков и профессионалов

Некоторые читатели, особенно профессионалы, могут захотеть пропустить эту главу. Тем не менее мы предлагаем каждому хотя бы просмотреть ее. Многие профессионалы найдут здесь что-то новое для себя, и в любом случае важно, чтобы все читатели были знакомы с представленным здесь материалом: он будет использоваться на протяжении всей остальной книги. Новичкам, конечно, следует прочитать эту главу внимательно.

В первых нескольких разделах этой главы мы дадим обзор процесса отладки и роли отладочных инструментов, а затем подробно разберем расширенный пример в разделе 1.7.

1.1 Отладочные инструменты, используемые в этой книге

В этой книге мы излагаем основные принципы отладки и иллюстрируем их в контексте следующих отладочных инструментов:

GDB

Самый часто используемый отладочный инструмент среди Unix-программистов - это GDB, GNU Project Debugger, разработанный Ричардом Столлманом, заметным лидером движения open source, сыгравшего ключевую роль в развитии Linux.

В большинстве систем Linux GDB должен быть уже установлен. Если его нет, необходимо скачать пакет компилятора GCC.

DDD

В связи с более поздней популярностью графических пользовательских интерфейсов (GUI) было разработано несколько GUI-отладчиков, работающих под Unix. Большинство из них являются графическими оболочками над GDB: пользователь вводит команды через GUI, а тот, в свою очередь, передает их GDB. Один из таких инструментов - DDD, Data Display Debugger.

Если в вашей системе DDD еще не установлен, его можно скачать. Например, в системах Fedora Linux команда

```
yum install ddd
```

полностью выполнит весь процесс установки. В Ubuntu Linux для этого можно использовать похожую команду apt-get.

Eclipse

Некоторые читатели могут пользоваться интегрированными средами разработки (IDE). IDE - это больше, чем просто отладочный инструмент: она объединяет редактор, средство сборки, отладчик и другие вспомогательные средства разработки в одном пакете. В этой книге в качестве примерной IDE выступает чрезвычайно популярная система Eclipse. Как и DDD, Eclipse работает поверх GDB или какого-либо другого отладчика.

Eclipse можно установить через `yum` или `apt-get`, как описано выше, либо просто скачать файл `.zip` и распаковать его в подходящий каталог, например `/usr/local`. В этой книге мы используем Eclipse версии 3.3.

1.2 Фокус на языках программирования

Основное внимание в этой книге направлено на программирование на C/C++, и большинство наших примеров будут относиться именно к этому контексту. Однако в главе 8 мы обсудим и другие языки.

1.3 Принципы отладки

Хотя отладка является скорее искусством, чем наукой, существуют вполне определенные принципы, которые направляют ее практику. Некоторые из них мы обсудим в этом разделе. По крайней мере одно из наших правил, фундаментальный принцип подтверждения, носит довольно формальный характер.

1.3.1 Суть отладки: принцип подтверждения

Следующее правило составляет саму суть отладки:

Фундаментальный принцип подтверждения

Исправление программы с ошибками - это процесс последовательного подтверждения того, что множество вещей, которые вы считаете истинными относительно кода, действительно истинны. Когда вы обнаруживаете, что одно из ваших предположений неверно, вы находите подсказку о местонахождении ошибки, если не о ее точной природе.

Иначе это можно сформулировать так:

Сюрпризы - это хорошо!

Когда одна из вещей, которые вы считаете истинными относительно программы, не подтверждается, вы удивляетесь. Но это хороший сюрприз, потому что такое открытие может привести вас к месту, где находится ошибка.

1.3.2 Какую ценность отладочный инструмент дает принципу подтверждения?

Классическая техника отладки состоит в том, чтобы просто добавить в программу трассировочный код, который по мере выполнения программы печатает значения переменных, например с помощью операторов `printf()` или `cout`. Вы можете спросить: "Разве этого недостаточно? Зачем использовать отладочный инструмент вроде GDB, DDD или Eclipse?"

Прежде всего такой подход требует постоянного цикла: стратегически добавлять трассировочный код, заново компилировать программу, запускать программу и анализировать вывод трассировочного кода, удалять трассировочный код после исправления ошибки, а затем повторять эти шаги для каждой новой обнаруженной ошибки. Это отнимает очень много времени и изматывает. Что важнее всего, эти действия отвлекают вас от настоящей задачи и снижают способность сосредоточиться на рассуждении, необходимом для поиска ошибки.

Напротив, в графических отладочных инструментах вроде DDD и Eclipse, чтобы посмотреть значение переменной, достаточно навести указатель мыши на экземпляр этой переменной в отображаемом коде, и вы увидите ее текущее значение. Зачем во время ночного сеанса отладки утомлять себя еще больше и дольше, чем необходимо, делая это с помощью операторов `printf()`? Сделайте себе одолжение: сократите время, которое приходится тратить, и монотонность, которую приходится терпеть, используя отладочный инструмент.

Отладочный инструмент дает вам гораздо больше, чем просто возможность смотреть на переменные. Во многих ситуациях отладчик может сообщить приблизительное место ошибки. Предположим, например, что ваша программа аварийно завершается с `segmentation fault`, то есть с ошибкой доступа к памяти. Как вы увидите в нашем примерном сеансе отладки далее в этой главе, GDB/DDD/Eclipse может сразу указать место нарушения сегментации, которое обычно находится в самой точке ошибки или рядом с ней.

Точно так же отладчик позволяет устанавливать `watchpoints`, которые могут сообщить, в какой момент выполнения программы значение некоторой переменной достигло подозрительного значения или диапазона. Такую информацию бывает трудно вывести, просто глядя на результат вызовов `printf()`.

1.3.3 Другие принципы отладки

Начинайте с малого

В начале процесса отладки следует запускать программу на легких, простых случаях. Это может не выявить все ваши ошибки, но, скорее всего, обнаружит хотя бы несколько из них. Если, например, ваш код содержит большой цикл, проще всего найти ошибки, возникающие на первой или второй итерации.

Используйте нисходящий подход

Вероятно, вы знаете о нисходящем, или модульном, подходе к написанию кода: основная программа не должна быть слишком длинной и должна в основном состоять из вызовов функций, выполняющих существенную работу. Если одна из таких функций велика, стоит, в свою очередь, разбить ее на меньшие модули.

Код следует писать в нисходящем стиле, но и отлаживать его тоже следует сверху вниз.

Например, предположим, что ваша программа использует функцию `f()`. Когда вы проходите код с помощью отладочного инструмента и встречаете вызов `f()`, отладчик даст вам выбор, где произойдет следующая остановка выполнения: либо на первой строке внутри вызываемой

функции, либо на операторе, следующем за вызовом функции. Во многих случаях второй вариант лучше подходит в качестве начального выбора: вы выполняете вызов, а затем проверяете значения переменных, зависящих от результатов вызова, чтобы понять, правильно ли отработала функция. Если да, то вы избежали трудоемкого и ненужного прохода по коду внутри функции, которая в данном случае не вела себя неправильно.

Используйте отладочный инструмент для определения места нарушения сегментации

Самый первый шаг при возникновении seg fault должен состоять в том, чтобы запустить программу внутри отладчика и воспроизвести нарушение сегментации. Отладчик сообщит строку кода, на которой произошла ошибка. Затем можно получить дополнительную полезную информацию, вызвав средство backtrace отладчика, которое показывает последовательность вызовов функций, приведшую к вызову той функции, где произошла ошибка.

В некоторых случаях нарушение сегментации бывает трудно воспроизвести, но если у вас есть core-файл, вы все равно можете выполнить backtrace, чтобы определить ситуацию, вызвавшую seg fault. Это будет обсуждаться в главе 4.

Определяйте место бесконечного цикла с помощью прерывания

Если вы подозреваете, что в программе есть бесконечный цикл, войдите в отладчик и снова запустите программу, позволив ей выполняться достаточно долго, чтобы попасть в этот цикл. Затем используйте команду прерывания отладчика, чтобы приостановить программу, и выполните backtrace, чтобы увидеть, какая точка тела цикла достигнута и как программа туда попала. Программа при этом не уничтожена; при желании выполнение можно продолжить.

Используйте двоичный поиск

Вы, вероятно, встречали двоичный поиск в контексте отсортированных списков. Например, предположим, что у вас есть массив `x[ ]` из 500 чисел, расположенных по возрастанию, и вы хотите определить, куда вставить новое число `y`. Начните со сравнения `y` с `x[250]`. Если `y` окажется меньше этого элемента, следующим сравнением будет `x[125]`; но если `y` больше `x[250]`, то следующая проверка будет с `x[375]`. Во втором случае, если `y` меньше `x[375]`, вы затем сравните его с `x[312]`, то есть с элементом посередине между `x[250]` и `x[375]`, и так далее. На каждой итерации вы будете делить область поиска пополам и быстро найдете точку вставки.

Этот принцип можно применять и при отладке. Предположим, вы знаете, что значение, хранящееся в некоторой переменной, становится неправильным где-то в течение первых 1000 итераций цикла. Один способ, который может помочь найти итерацию, где значение впервые портится, - использовать watchpoint, продвинутый прием, который мы обсудим в разделе 1.5.3. Другой подход - двоичный поиск, в данном случае по времени, а не по пространству. Сначала вы проверите значение переменной на 500-й итерации; если в этой точке оно все еще правильно, затем проверите значение на 750-й итерации и так далее.

Еще один пример: предположим, что один из исходных файлов вашей программы вообще не компилируется. Строка кода, указанная в сообщении компилятора, порожденном синтаксической ошибкой, иногда находится далеко от настоящего места ошибки, и поэтому определить это место может быть трудно. Здесь может помочь двоичный поиск: вы удаляете или комментируете половину кода в единице компиляции, перекомпилируете оставшийся код и смотрите, сохраняется ли сообщение об ошибке. Если сохраняется, значит ошибка находится во второй половине; если сообщение не появляется, значит ошибка находится в той половине, которую вы удалили. Определив, какая половина кода содержит ошибку, вы далее ограничиваете область поиска половиной этой части и продолжаете, пока не найдете проблему. Конечно, перед началом такого процесса следует сделать копию исходного кода, а еще лучше - использовать возможность undo в текстовом редакторе. Советы по эффективному использованию редактора при программировании приведены в главе 7.

1.4 Текстовые и GUI-ориентированные отладочные инструменты, а также компромисс между ними

GUI, обсуждаемые в этой книге, DDD и Eclipse, служат оболочками для GDB при работе с C и C++, а также для других отладчиков. Хотя GUI выглядят привлекательно и могут быть удобнее текстового GDB, наша точка зрения в этой книге состоит в том, что текстовые и GUI-ориентированные отладчики, включая IDE, полезны все, но в разных контекстах.

1.4.1 Краткое сравнение интерфейсов

Чтобы быстро понять различия между текстовыми и GUI-ориентированными отладочными инструментами, рассмотрим ситуацию, которую будем использовать как сквозной пример в этой главе. Программа в примере называется `insert_sort`. Она скомпилирована из исходного файла `ins.c` и выполняет сортировку вставками.

1.4.1.1 GDB: обычный текст

Чтобы начать сеанс отладки этой программы в GDB, в командной строке Unix нужно набрать:

```
$ gdb insert_sort
```

После этого GDB предложит вам вводить команды, показав свое приглашение:

```
(gdb)
```

1.4.1.2 DDD: GUI-отладчик

При использовании DDD вы начнете сеанс отладки командой:

```
$ ddd insert_sort
```

в командной строке Unix. После этого появится окно DDD, и команды можно будет вводить через GUI.

Типичный вид окна DDD показан на рисунке 1-1. Как видно, окно DDD располагает информацией в нескольких подокнах:

- Окно Source Text отображает исходный код. DDD начинает показ с вашей функции `main()`, но, конечно, можно перейти к другим частям исходного файла, используя полосу прокрутки у правого края окна.
- Menu Bar представляет различные категории меню, включая File, Edit и View.
- Command Tool перечисляет наиболее распространенные команды DDD, такие как Run, Interrupt, Step и Next, чтобы к ним можно было быстро обращаться.
- Console: напомним, что DDD - это просто GUI-оболочка для GDB и других отладчиков. DDD переводит выбор, сделанный мышью, в соответствующие команды GDB. Эти команды и их вывод отображаются в Console. Кроме того, команды можно отправлять в GDB напрямую через Console; это удобная возможность, поскольку не у всех команд GDB есть аналоги в DDD.
- Окно Data показывает значения переменных, которые вы попросили отображать постоянно. Это подокно не появится, пока вы не сделаете такой запрос, поэтому на данном рисунке его нет.

Вот быстрый пример того, как типичная отладочная команда передается отладчику при каждом типе пользовательского интерфейса. При отладке `insert_sort` вы можете захотеть приостановить выполнение программы - поставить breakpoint - на строке 16, скажем, функции `get_args()`. Полный исходный код `insert_sort` вы увидите в разделе 1.7. Чтобы сделать это в GDB, нужно набрать:

```
(gdb) break 16
```

в приглашении GDB.

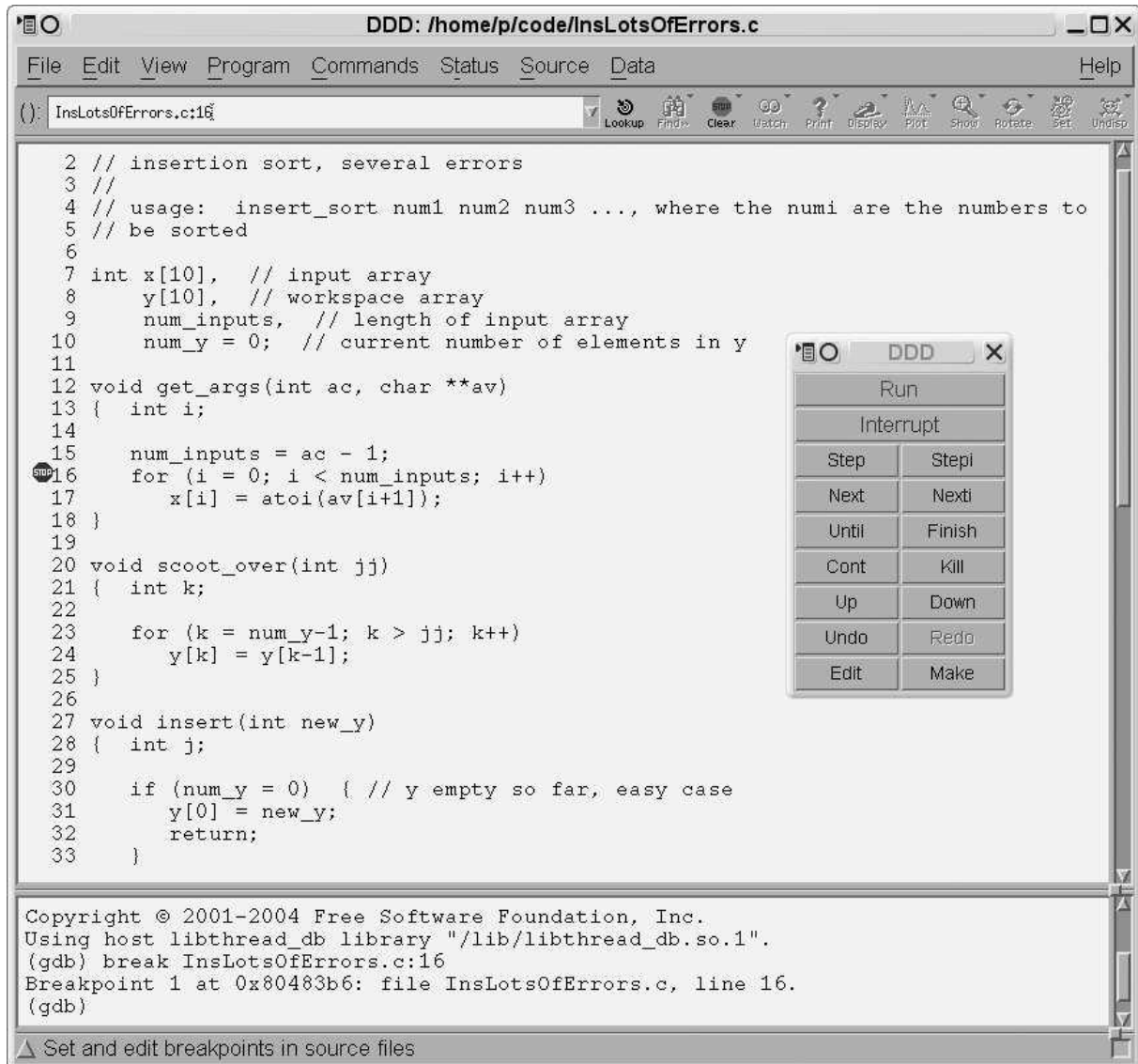


Рисунок 1-1. Схема окна DDD

Полное имя команды - `break`, но GDB разрешает сокращения, если они не создают неоднозначности, и большинство пользователей GDB набрали бы здесь `b 16`. Чтобы облегчить понимание тем, кто только знакомится с GDB, сначала мы будем использовать полные имена команд, а позже в книге, когда команды станут привычнее, перейдем к сокращениям.

При использовании DDD вы посмотрели бы на окно Source Text, щелкнули бы в начале строки 16, а затем нажали бы значок Break в верхней части экрана DDD. Можно также щелкнуть правой кнопкой мыши в начале строки и затем выбрать Set Breakpoint. Еще один вариант - просто дважды щелкнуть строку кода в любом месте слева от начала строки. В любом случае DDD подтвердит выбор, показав на этой строке маленький знак остановки, как на рисунке 1-2. Так вы сможете видеть свои точки останова с первого взгляда.

1.4.1.3 Eclipse: GUI-отладчик и многое другое

Теперь рисунок 1-3 знакомит с общей средой Eclipse. В терминологии Eclipse мы сейчас находимся в перспективе Debug. Eclipse - это общая платформа для разработки множества различных видов программного обеспечения. У каждого языка программирования есть собственный подключаемый GUI - перспектива - внутри Eclipse. Более того, для одного и того же языка может существовать несколько конкурирующих перспектив. В нашей работе с Eclipse в этой книге мы будем использовать перспективу C/C++ для разработки на C/C++, перспективу Pydev для написания программ на Python и так далее. Существует также перспектива Debug для собственно отладки, с некоторыми языково-специфичными возможностями; именно ее вы видите на рисунке.

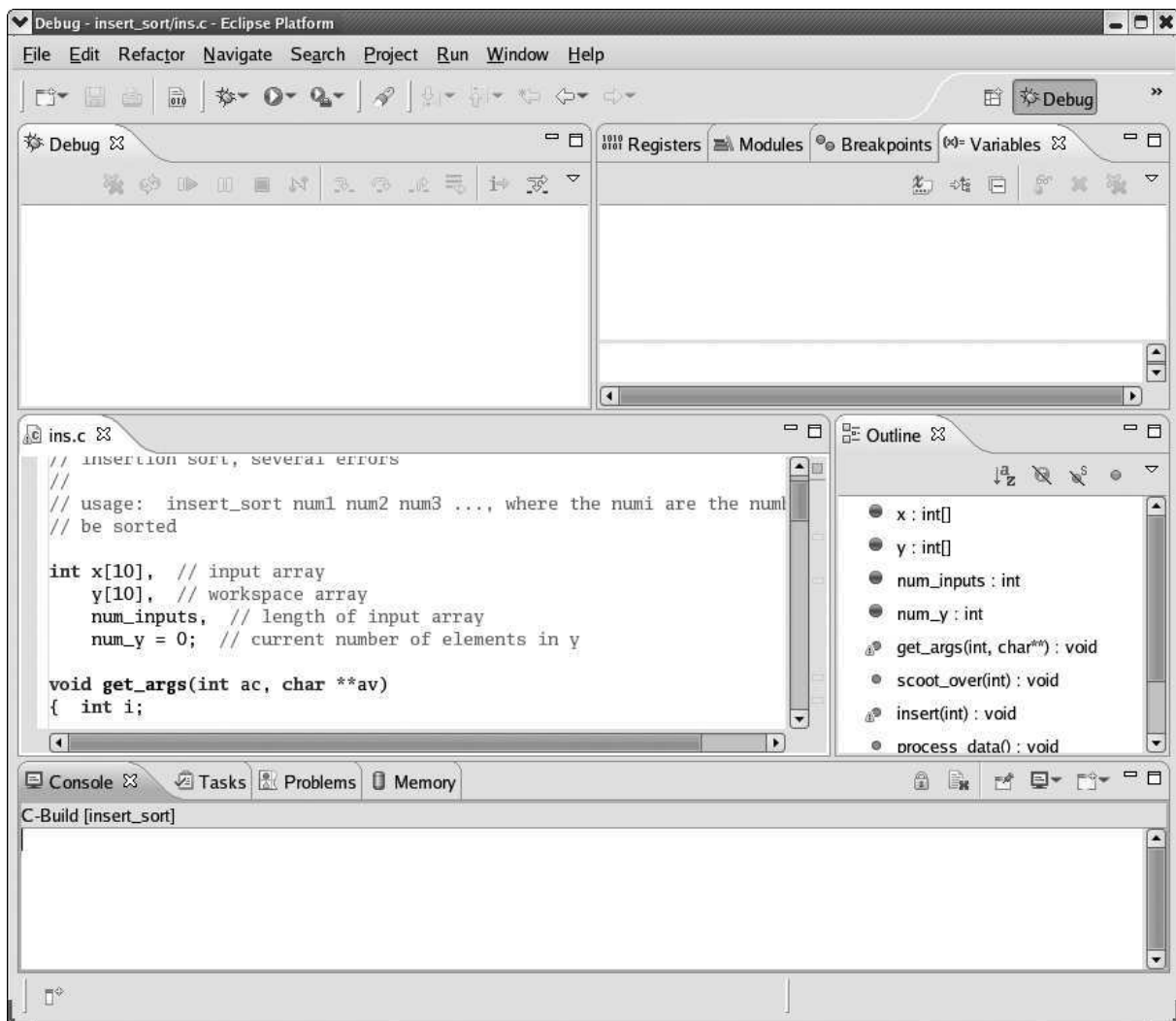


Рисунок 1-2. Установленная точка останова

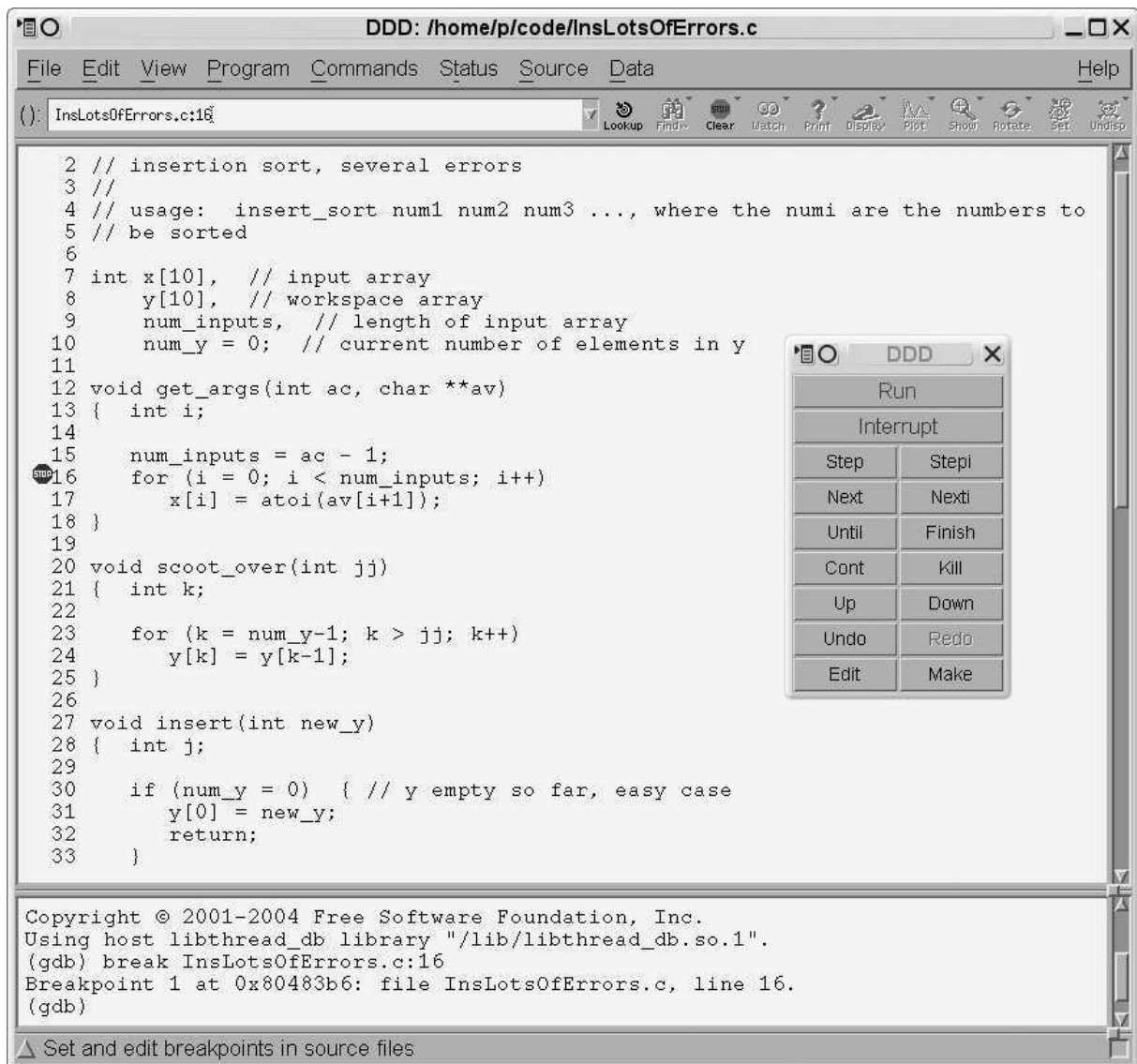


Рисунок 1-3. Среда Eclipse

Перспектива C/C++ является частью подключаемого модуля CDT. За кулисами CDT вызывает GDB, подобно тому как это происходит в случае DDD.

Детали этого рисунка в целом похожи на то, что мы выше описали для DDD. Перспектива разбита на окна с вкладками, называемые *views*. Слева видно *view* для исходного файла `ins.c`; есть *view Variables* для просмотра значений переменных (на рисунке пока нет ни одной); есть *view Console*, функция которого очень похожа на одноименное подокно в DDD; и так далее.

Точки останова и подобные элементы можно устанавливать визуально, как в DDD. Например, на рисунке 1-4 строка

```
for (i = 0; i < num_inputs; i++)
```

в окне исходного файла имеет синий символ на левом поле, обозначающий, что там установлена точка останова.

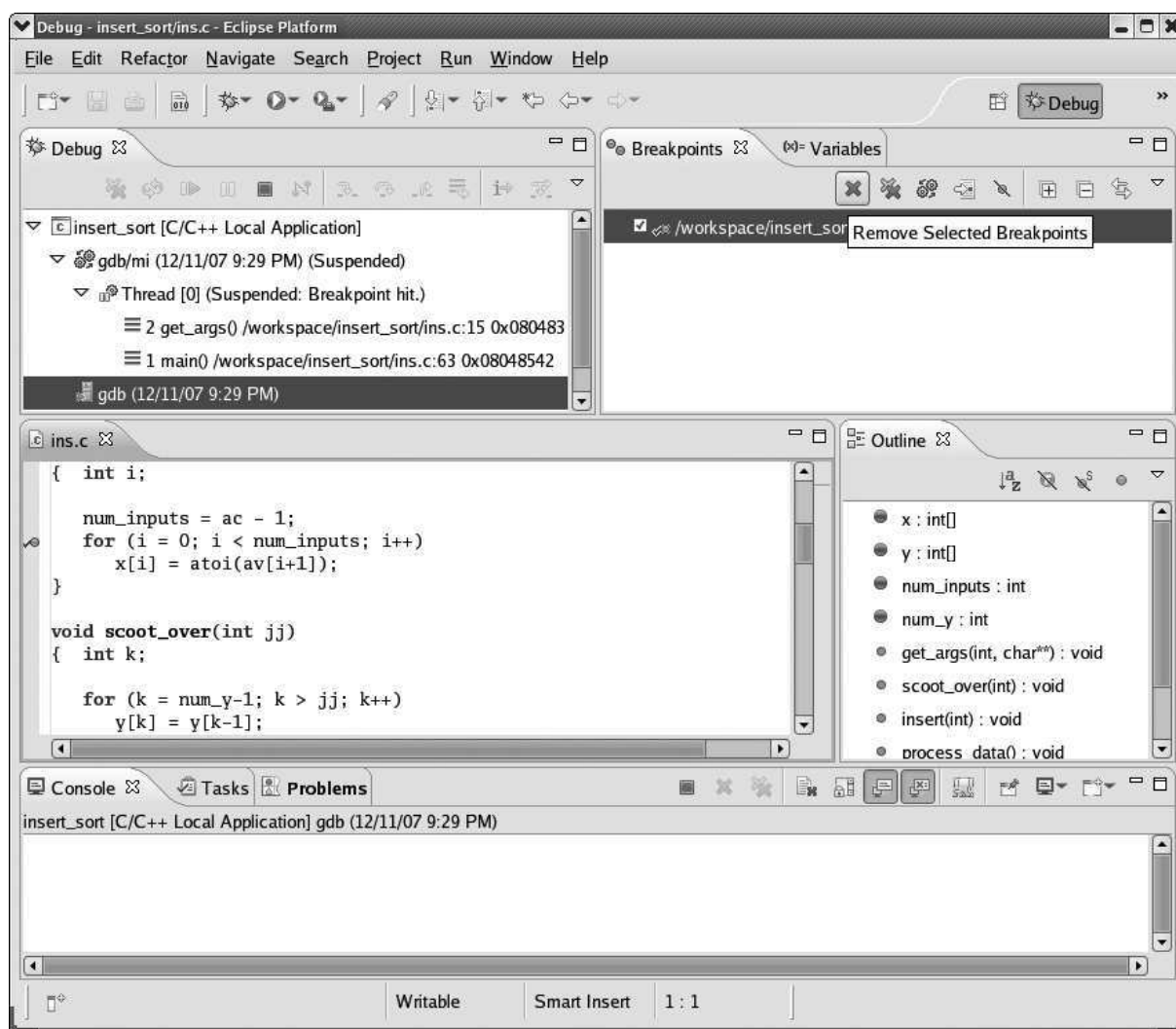


Рисунок 1-4. Удаление точки останова в Eclipse

1.4.1.4 Eclipse против DDD

В Eclipse есть и некоторые вспомогательные средства, отсутствующие в DDD. Например, ближе к правой стороне обратите внимание на view Outline, где перечислены переменные, функции и тому подобное. Если щелкнуть, скажем, запись вашей функции `scoot_over()`, курсор в view исходного файла перейдет к этой функции. Более того, если вы временно перейдете из перспективы Debug обратно в перспективу C/C++, где выполняете редактирование и компиляцию для этого проекта (не показано), view Outline будет доступен и там. В больших проектах это может быть весьма полезно.

Eclipse также лучше объединяет процессы редактирования и компиляции. Если у вас есть ошибки компиляции, они ясно отмечаются прямо в редакторе. Это можно сделать и в редакторе Vim, который оба автора этой книги обычно предпочитают IDE, но IDE справляется с этим гораздо лучше.

С другой стороны, видно, что Eclipse, как и большинство IDE, занимает значительную площадь на экране (и, разумеется, на страницах этой книги!). View Outline занимает драгоценное экранное пространство независимо от того, часто вы им пользуетесь или нет. Правда, Outline можно скрыть, щелкнув X в его правом углу (а если захотите вернуть его, выберите Window | Show Views | Outline), что освободит немного места; кроме того, вкладки можно перетаскивать в разные места внутри

окна Eclipse. Но в целом эффективно использовать экранное пространство в Eclipse может быть трудно.

Помните, что в Console DDD всегда можно выполнять команды GDB напрямую. Поэтому у вас есть гибкость: выполнять отладочные команды самым удобным из доступных способов, иногда через интерфейс DDD, а иногда через командную строку GDB. В разных местах этой книги вы увидите, что GDB позволяет выполнять целый ряд действий, способных заметно упростить отладочную жизнь.

Напротив, для пользователей Eclipse GDB в основном прозрачен, и хотя старая поговорка "меньше знаешь - крепче спишь" часто бывает уместна, такая прозрачность означает потерю удобного доступа к действиям, экономящим труд и возможным при прямом использовании GDB. На момент написания книги настойчивый пользователь все еще может получить прямой доступ к GDB, щелкнув поток GDB в Debug и затем используя Console, хотя уже без приглашений GDB. Однако эта "недокументированная возможность" может не пережить будущие версии.

1.4.1.5 Преимущества GUI

GUI-интерфейсы, предоставляемые DDD и Eclipse, визуально привлекательнее интерфейса GDB. Кроме того, они обычно удобнее. Например, предположим, что вы больше не хотите, чтобы выполнение останавливалось на строке 16 функции `get_args()`, то есть хотите убрать точку останова. В GDB вы очистили бы точку останова, набрав:

```
(gdb) clear 16
```

Однако для этого нужно помнить номер строки точки останова, а это непросто, если одновременно активно много точек останова. Можно освежить память с помощью команды GDB `info break`, получив список всех точек останова, но это все равно потребует некоторой работы и отвлечет от поиска ошибки.

В DDD задача была бы намного проще: чтобы убрать точку останова, достаточно щелкнуть знак остановки на нужной строке, затем нажать Clear, и знак остановки исчезнет, показывая, что точка останова очищена.

В Eclipse вы перешли бы к `view Breakpoints`, выделили бы точку или точки останова, которые хотите удалить, а затем переместили бы указатель мыши к серому X, обозначающему операцию `Remove Selected Breakpoints` (см. рисунок 1-4). Другой вариант - щелкнуть правой кнопкой мыши синий символ точки останова в окне исходного кода и выбрать `Toggle Breakpoint`.

Одна задача, в которой GUI безусловно выигрывают, - пошаговое выполнение кода. Делать это в DDD или Eclipse гораздо проще и приятнее, чем в GDB, потому что вы можете наблюдать свое движение по коду в окне исходного кода GUI. Следующая строка исходного кода, которая будет выполнена, обозначается стрелкой, как показано для DDD на рисунке 1-5. В Eclipse следующая строка подсвечивается зеленым. Поэтому вы сразу видите, где находитесь относительно других интересующих операторов программы.

1.4.1.6 Преимущества GDB

Итак, GUI имеют много преимуществ перед текстовым GDB. И все же широкий вывод на основе этого примера, будто GUI лучше GDB, был бы необоснован.

Молодые программисты, выросшие на GUI почти для всех своих действий в сети, естественно предпочитают GUI вместо GDB, как и многие их старшие коллеги. С другой стороны, у GDB тоже есть вполне определенные преимущества:

- GDB запускается быстрее DDD, что является большим преимуществом, когда нужно просто быстро что-то проверить в коде. В случае Eclipse разница во времени запуска еще больше.
- В некоторых случаях отладка выполняется удаленно через SSH- или telnet-соединение, например с публичного терминала. Если у вас нет настройки X11, GUI вообще нельзя использовать; но даже при наличии X11 операции обновления экрана в GUI могут быть медленными.
- При отладке нескольких программ, работающих во взаимодействии друг с другом, например клиент-серверной пары в сетевой среде, для каждой программы нужно отдельное окно отладки. В Eclipse это немного лучше, чем в DDD, поскольку Eclipse позволит отлаживать две программы одновременно в одном окне, но это усугубляет проблемы с пространством, упомянутые ранее. Поэтому небольшая визуальная площадь, занимаемая GDB на экране по сравнению с большей площадью GUI, является крупным преимуществом.
- Если отлаживаемая программа имеет GUI и вы используете GUI-отладчик вроде DDD, они могут конфликтовать. GUI-события - нажатия клавиш, щелчки мыши и перемещения мыши - одного могут мешать событиям другого, и программа может вести себя иначе при запуске под отладчиком, чем при самостоятельном запуске. Это может серьезно осложнить поиск ошибок.

Тем, кто не привык к объему набора текста, требуемому GDB по сравнению с удобными операциями мышью в GUI, следует отметить, что GDB включает несколько средств экономии ввода, делающих его текстовую природу более приемлемой. Мы уже упоминали, что у большинства команд GDB есть короткие сокращения, и большинство людей использует именно их вместо полных форм. Кроме того, сочетания клавиш CTRL-P и CTRL-N позволяют прокручивать предыдущие команды и при желании редактировать их. Простое нажатие клавиши ENTER повторяет последнюю выданную команду, что очень полезно при многократном выполнении команды `next` для пошагового прохода по коду по одной строке; также есть команда `define`, позволяющая пользователю определять сокращения и макросы. Подробности этих возможностей будут представлены в главах 2 и 3.

1.4.1.7 Итог: у каждого инструмента есть своя ценность

Мы считаем важными инструментами и GDB, и GUI, и эта книга будет показывать примеры GDB, DDD и Eclipse. Рассмотрение любой конкретной темы мы всегда будем начинать с GDB, поскольку он является общей основой для этих инструментов, а затем покажем, как этот материал переносится на GUI.

1.4.2 Компромиссы

Начиная с версии 6.1, GDB предлагает компромисс между текстовым и графическим взаимодействием с пользователем в виде режима TUI (Terminal User Interface). В этом режиме GDB делит экран терминала на аналоги окна Source Text и Console из DDD: в первом можно следить за ходом выполнения программы, а во втором - вводить команды GDB. В качестве альтернативы можно использовать другую программу, CGDB, которая предоставляет похожую функциональность.

1.4.2.1 GDB в режиме TUI

Чтобы запустить GDB в режиме TUI, можно либо указать параметр `-tui` в командной строке при вызове GDB, либо набрать `CTRL-X-A` внутри GDB, находясь не в режиме TUI. Последняя команда также выводит вас из режима TUI, если вы сейчас в нем находитесь.

В режиме TUI окно GDB делится на два подокна: одно для команд GDB и одно для просмотра исходного кода. Предположим, вы запускаете GDB в режиме TUI для `insert_sort`, а затем выполняете пару отладочных команд. После этого ваш экран GDB может выглядеть так:

```
11
12 void get_args(int ac, char **av)
13 { int i;
14
15   num_inputs = ac - 1;
16   for (i = 0; i < num_inputs; i++)
17     x[i] = atoi(av[i+1]);
18 }
19
20 void scoot_over(int jj)
21 { int k;
22
23   for (k = num_y-1; k > jj; k++) .

File: ins.c Procedure: get_args Line: 17 pc: 0x80484b8
-----
(gdb) break 16
Breakpoint 1 at 0x804849f: file ins.c, line 16.
(gdb) run 12 5 6
Starting program: /debug/insert_sort 12 5 6
Breakpoint 1, get_args (ac=4, av=0xbffff094) at ins.c:16
(gdb) next
(gdb)
```

Нижнее подокно показывает ровно то, что вы увидели бы при использовании GDB без TUI. Здесь это подокно показывает следующее:

- Мы выдали команду `break`, чтобы установить точку останова на строке 16 в текущем исходном файле.
- Мы выполнили `run`, чтобы запустить программу, передав ей аргументы командной строки 12, 5 и 6; после этого отладчик остановил выполнение на указанной точке останова. (`run` и другие команды GDB будут объяснены позже.) GDB напоминает нам, что точка останова находится на строке 16 файла `ins.c`, и сообщает, что машинный код для этой исходной строки расположен по адресу памяти `0x804849f`.
- Мы выдали команду `next`, чтобы перейти к следующей строке кода, строке 17.

Верхнее подокно дает дополнительную, визуально полезную информацию. Здесь TUI показывает исходный код вокруг строки, выполняемой в данный момент, так же как это сделали бы DDD и Eclipse. Благодаря этому гораздо легче видеть, где мы находимся в коде. Точка останова и текущая выполняемая строка обозначаются соответственно звездочкой и знаком `>`, аналогично значку `stop sign` и зеленой стрелке в DDD.

Можно переходить к другим частям кода, прокручивая его клавишами со стрелками вверх и вниз. Когда вы не в режиме TUI, клавиши со стрелками можно использовать для прокрутки предыдущих команд GDB, чтобы изменить или повторить их. В режиме TUI клавиши со стрелками прокручивают подокно исходного кода, а предыдущие команды GDB прокручиваются с помощью `CTRL-P` и `CTRL-N`. Кроме того, в режиме TUI область кода, отображаемую в подокне исходного кода, можно менять с помощью команды GDB `list`. Это особенно полезно при работе с несколькими исходными файлами.

Используя режим TUI в GDB и его сокращения ввода, мы можем получить значительную часть дополнительной функциональности GUI, не принимая на себя недостатки GUI. Однако обратите внимание: в некоторых обстоятельствах TUI может вести себя не совсем так, как вам хотелось бы, и тогда придется найти обходной прием.

1.4.2.2 CGDB

Еще один интерфейс к GDB, который стоит рассмотреть, - CGDB, доступный по адресу <http://cgdb.sourceforge.net/>. CGDB также предлагает компромисс между текстовым подходом и GUI-подходом. Как и GUI, он служит оболочкой для GDB. Он похож на терминальную идею TUI, но с дополнительными привлекательными свойствами: он цветной, позволяет просматривать подокно исходного кода и устанавливать точки останова прямо там. Кроме того, похоже, что он лучше справляется с обновлением экрана, чем GDB/TUI.

Вот несколько базовых команд и соглашений CGDB:

- Нажмите ESC, чтобы перейти из командного окна в окно исходного кода; нажмите ее снова, чтобы вернуться.
- Находясь в окне исходного кода, перемещайтесь с помощью клавиш со стрелками или клавиш в стиле vi (j вниз, k вверх, / для поиска).
- Следующая строка, которая будет выполнена, отмечена стрелкой.
- Чтобы установить точку останова на строке, которая сейчас выделена курсором, просто нажмите пробел.
- У строк с точками останова номера строк выделены красным.

1.5 Основные операции отладчика

Здесь мы дадим обзор основных типов операций, которые предлагает отладчик.

1.5.1 Пошаговое прохождение исходного кода

Вы уже видели, что для запуска программы в GDB используется команда run, а в DDD нужно щелкнуть Run. В подробностях, которые будут представлены позже, вы увидите, что Eclipse обрабатывает это похожим образом.

Можно также организовать остановку выполнения программы в определенных точках, чтобы проверять значения переменных и получать подсказки о том, где находится ошибка. Вот некоторые методы, которыми можно для этого пользоваться:

Точки останова

Как уже говорилось, отладочный инструмент приостанавливает выполнение программы в указанных точках останова. В GDB это делается командой break вместе с номером строки; в DDD вы щелкаете правой кнопкой мыши в любом месте пустого пространства на соответствующей строке и выбираете Set Breakpoint; в Eclipse вы дважды щелкаете на поле слева от строки.

Пошаговое выполнение

Команда GDB `next`, которая также уже упоминалась, приказывает GDB выполнить следующую строку и затем остановиться. Команда `step` похожа на нее, но при вызове функции она войдет внутрь функции, тогда как `next` приведет к тому, что следующая остановка выполнения произойдет на строке после вызова функции. В DDD есть соответствующие пункты меню `Next` и `Step`, а в Eclipse для тех же действий есть значки `Step Over` и `Step Into`.

Возобновление выполнения

В GDB команда `continue` приказывает отладчику возобновить выполнение и продолжать его до достижения точки останова. В DDD есть соответствующий пункт меню, а в Eclipse для этого имеется значок `Resume`.

Временные точки останова

В GDB команда `tbreak` похожа на `break`, но устанавливает точку останова, которая действует только до первого достижения указанной строки. В DDD это выполняется щелчком правой кнопкой мыши в любом месте пустого пространства на нужной строке в окне `Source Text` и последующим выбором `Set Temporary Breakpoint`. В Eclipse нужно выделить нужную строку в окне исходного кода, затем щелкнуть правой кнопкой мыши и выбрать `Run to Line`.

В GDB также есть команды `until` и `finish`, которые создают специальные виды одноразовых точек останова. В окне `Command DDD` им соответствуют пункты меню `Until` и `Finish`, а в Eclipse - `Step Return`. Они обсуждаются в главе 2.

Типичный шаблон выполнения программы при отладке выглядит так (на примере GDB): после достижения точки останова вы некоторое время двигаетесь по коду по одной строке, или выполняете `single-step`, с помощью команд GDB `next` и `step`. Это позволяет внимательно изучить состояние и поведение программы рядом с точкой останова. Когда вы закончите, можно приказать отладчику продолжать выполнение программы без остановок до следующей точки останова с помощью команды `continue`.

1.5.2 Просмотр переменных

После того как отладчик приостанавливает выполнение нашей программы, можно выдавать команды для отображения значений переменных программы. Это могут быть локальные переменные, глобальные переменные, элементы массивов и структур C, переменные-члены классов C++ и так далее. Если у переменной обнаруживается неожиданное значение, обычно это серьезная подсказка о месте и природе ошибки. DDD даже может строить графики массивов, что позволяет сразу увидеть подозрительные значения или тенденции внутри массива.

Самый базовый вид отображения переменной - простая печать текущего значения. Например, предположим, что вы поставили точку останова на строке 37 функции `insert()` в `ins.c`. Полный исходный код снова будет приведен в разделе 1.7, но сейчас детали не важны. Когда вы достигнете этой строки, можно проверить значение локальной переменной `j` в этой функции. В GDB вы использовали бы команду `print`:

```
(gdb) print j
```

В DDD это еще проще: достаточно навести указатель мыши на любой экземпляр `j` в окне `Source Text`, и значение `j` на секунду или две появится в маленьком желтом поле рядом с указателем мыши. Такое поле называется `value tip`. См. рисунок 1-5, где проверяется значение переменной `new_y`. В Eclipse все работает так же, как видно на рисунке 1-6, где мы запрашиваем значение `num_y`.

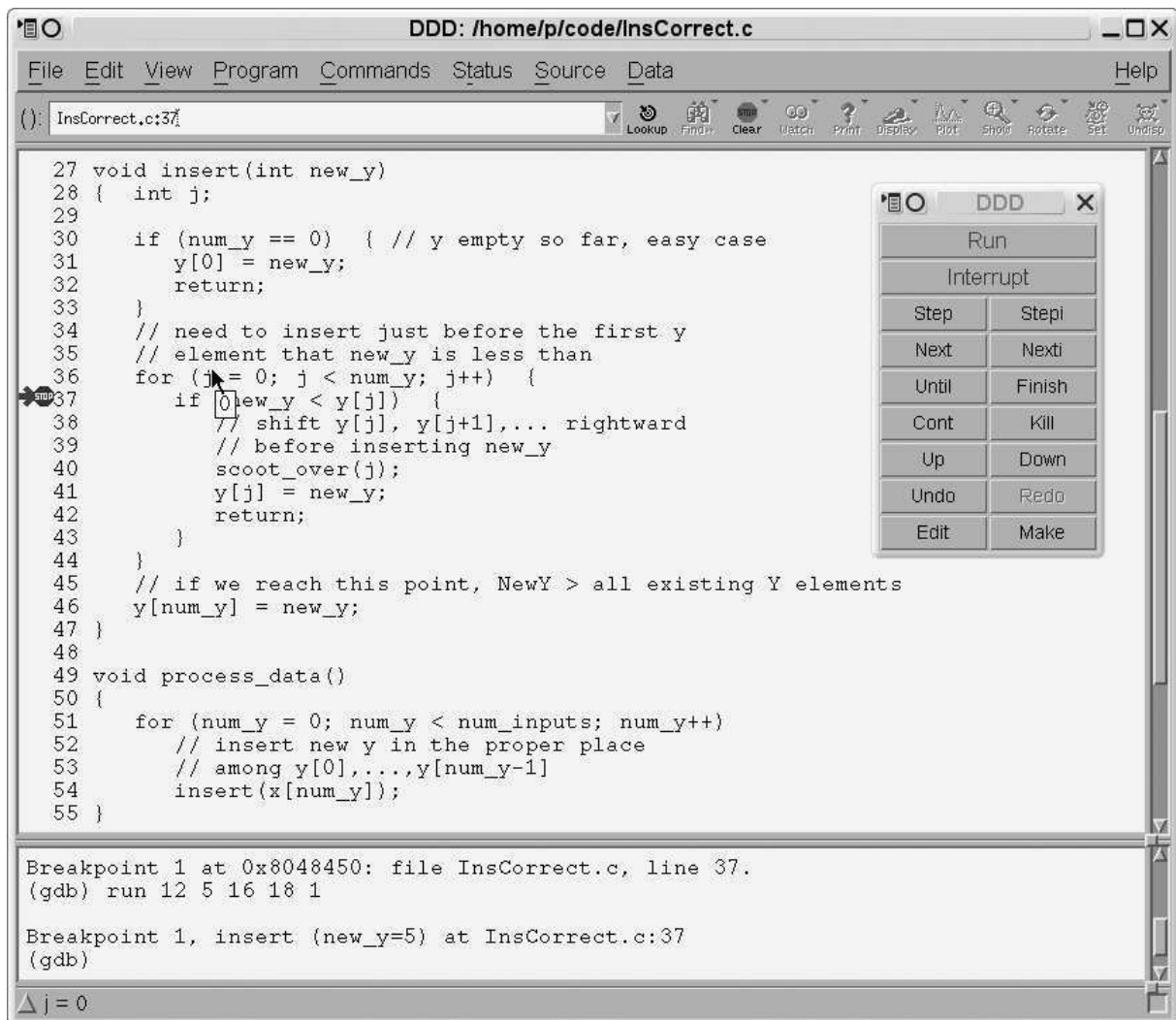


Рисунок 1-5. Просмотр переменной в DDD

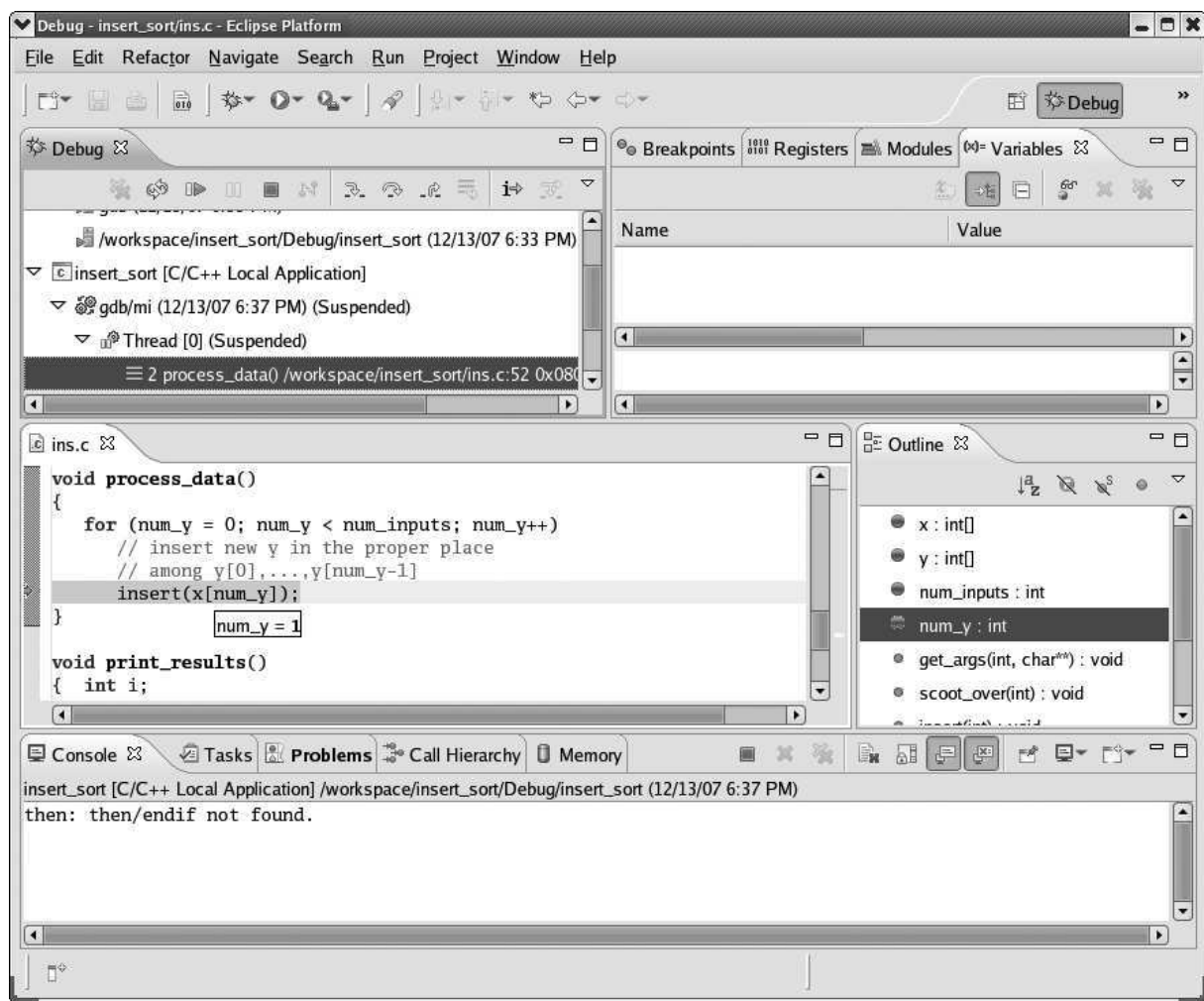


Рисунок 1-6. Просмотр переменной в Eclipse

Как вы увидите в главе 2, в GDB или DDD можно также организовать постоянное отображение переменной, чтобы не запрашивать ее значение повторно. У DDD есть особенно удобная возможность для отображения связанных списков, деревьев и других структур данных, содержащих указатели: можно щелкнуть исходящую ссылку любого узла такой структуры, чтобы найти следующий узел.

1.5.3 Рассылка "ориентировки" при изменении переменной

Watchpoint объединяет идеи точки останова и просмотра переменной. В самой базовой форме он предписывает отладчику приостанавливать выполнение программы всякий раз, когда значение указанной переменной меняется.

Например, предположим, что вы хотите исследовать состояние программы в те моменты ее выполнения, когда переменная `z` меняет значение. В GDB можно выдать команду:

```
(gdb) watch z
```

Когда вы запустите программу, GDB будет приостанавливать выполнение при каждом изменении значения `z`. В DDD вы установили бы watchpoint, щелкнув любой экземпляр `z` в окне Source Text и затем нажав значок Watch в верхней части окна DDD.

Еще лучше то, что watchpoints можно задавать на основе условных выражений. Допустим, например, что вы хотите найти первую точку выполнения программы, в которой значение `z` превышает 28. Это можно сделать, установив watchpoint на основе выражения `(z > 28)`. В GDB вы набрали бы:

```
(gdb) watch (z > 28)
```

В DDD эту команду можно выдать в Console DDD. Напомним, что в C выражение `(z > 28)` имеет булев тип и вычисляется либо как `true`, либо как `false`, где `false` представляется 0, а `true` - любым ненулевым целым числом, обычно 1. Когда `z` впервые примет значение больше 28, значение выражения `(z > 28)` изменится с 0 на 1, и GDB приостановит выполнение программы.

В Eclipse watchpoint можно установить, щелкнув правой кнопкой мыши в окне исходного кода, выбрав Add a Watch Expression и затем введя нужное выражение в диалоговом окне.

Watchpoints обычно не так полезны для локальных переменных, как для переменных с более широкой областью видимости, потому что watchpoint, установленный на локальную переменную, отменяется сразу после выхода переменной из области видимости, то есть когда завершается функция, в которой эта переменная определена. Однако локальные переменные в `main()` являются очевидным исключением, поскольку такие переменные не освобождаются до завершения выполнения программы.

1.5.4 Перемещение вверх и вниз по стеку вызовов

Во время вызова функции информация времени выполнения, связанная с этим вызовом, хранится в области памяти, известной как `stack frame`. Фрейм содержит значения локальных переменных функции и ее параметров, а также запись о месте, из которого функция была вызвана. Каждый раз, когда происходит вызов функции, создается новый фрейм и помещается в стек, поддерживаемый системой; фрейм на вершине стека представляет текущую выполняемую функцию, а при выходе из функции снимается со стека и освобождается.

Например, предположим, что вы приостанавливаете выполнение примерной программы `insert_sort`, находясь в функции `insert()`. Данные в текущем `stack frame` укажут, что вы попали туда через вызов функции в конкретном месте, которое оказывается внутри функции `process_data()` (она вызывает `insert()`). Фрейм также сохранит текущее значение единственной локальной переменной `insert()`, которая, как вы увидите позже, называется `j`.

Stack frames для других активных вызовов функций будут содержать похожую информацию, и при желании их тоже можно просмотреть. Например, хотя выполнение сейчас находится в `insert()`, вы можете захотеть взглянуть на предыдущий фрейм в стеке вызовов, то есть на фрейм `process_data()`. В GDB это можно сделать командой:

```
(gdb) frame 1
```

При выдаче команды GDB `frame` фрейм текущей выполняемой функции имеет номер 0, его родительский фрейм, то есть `stack frame` вызывающей функции, имеет номер 1, родитель родителя - номер 2 и так далее. Команда GDB `up` переносит вас к следующему родителю в стеке вызовов, например от фрейма 0 к фрейму 1, а `down` переносит в противоположном направлении. Такие операции очень полезны, поскольку значения локальных переменных в некоторых более ранних `stack frames` могут подсказать, что вызвало ошибку.

Проход по стеку вызовов не меняет путь выполнения: в этом примере следующей строкой `insert_sort`, которая будет выполнена, все равно останется текущая строка в `insert()`. Но он позволяет посмотреть на фреймы-предки и тем самым изучить значения локальных переменных для вызовов функций, приведших к текущему месту. Опять же, это может дать подсказки о том, где искать ошибку.

Команда GDB `backtrace` покажет весь стек, то есть весь набор фреймов, существующих в данный момент.

Аналогичная операция в DDD вызывается щелчком `Status | Backtrace`; появится окно со всеми фреймами, после чего можно щелкнуть тот, который вы хотите изучить. В интерфейсе DDD также есть кнопки `Up` и `Down`, которыми можно вызывать команды GDB `up` и `down`.

В Eclipse стек постоянно виден прямо в перспективе `Debug`. На рисунке 1-7 посмотрите на вкладку `Debug` в левом верхнем углу. Вы увидите, что сейчас мы находимся во фрейме 2, в функции `get_args()`, которая была вызвана из фрейма 1 в `main()`. Тот фрейм, который выделен, отображается в окне исходного кода, поэтому любой фрейм можно показать, щелкнув его в стеке вызовов.

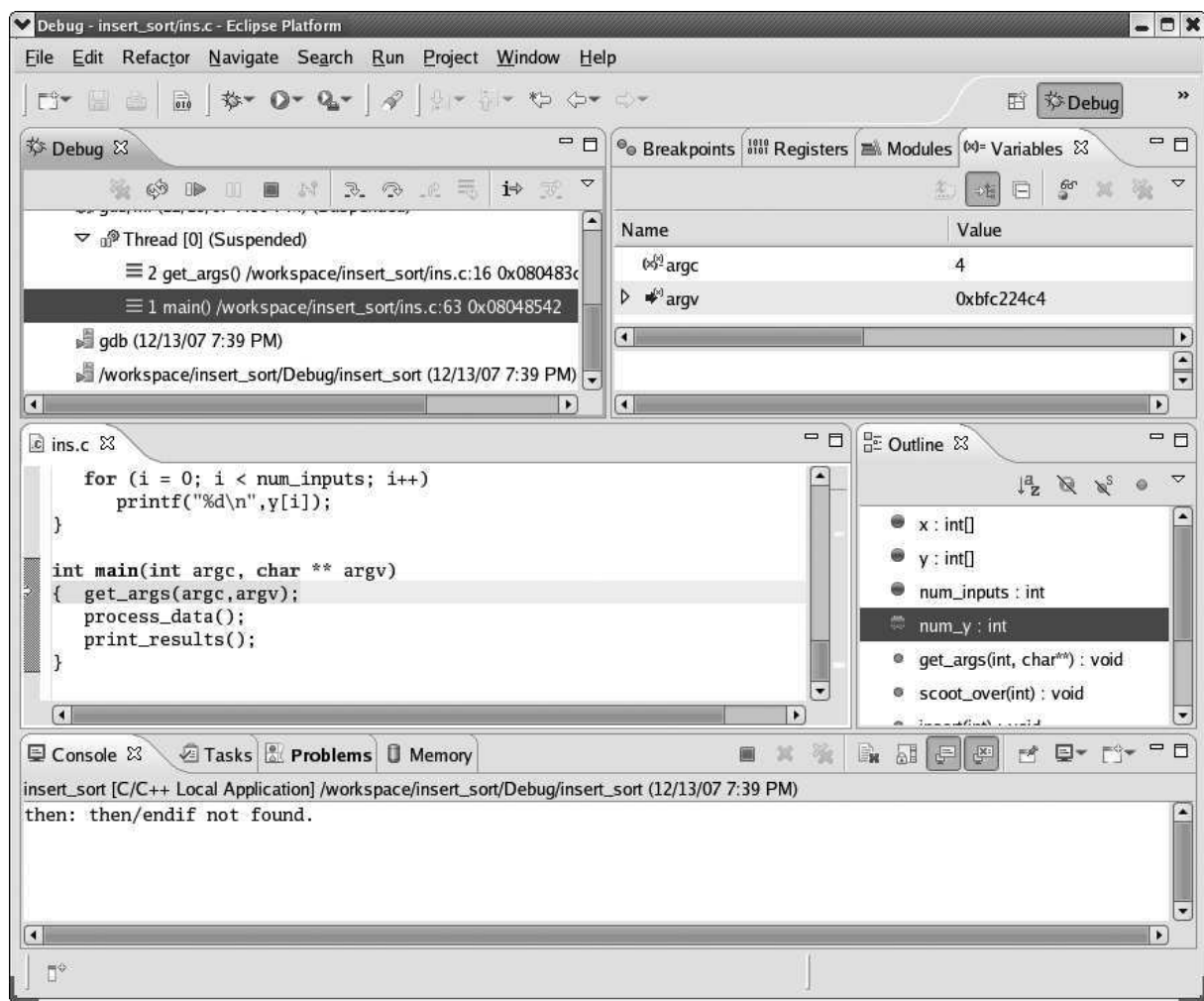


Рисунок 1-7. Перемещение внутри стека в Eclipse

1.6 Online Help

В GDB документацию можно получить с помощью команды `help`. Например,

```
(gdb) help breakpoints
```

выдаст документацию по точкам останова. Команда GDB `help` без аргументов выводит меню категорий команд, которые можно использовать как аргументы для `help`.

В DDD и Eclipse множество материалов доступно щелчком Help.

1.7 Вводный сеанс отладки

Теперь мы представим полный сеанс отладки. Как уже говорилось, примерная программа находится в исходном файле `ins.c` и выполняет сортировку вставками. Конечно, это неэффективный метод сортировки, но простота кода делает его удобным для иллюстрации отладочных операций. Вот код:

```

//
// insertion sort, several errors
//
// usage: insert_sort num1 num2 num3 ..., where the numi are the numbers to
// be sorted
int x[10], // input array
y[10], // workspace array
num_inputs, // length of input array
num_y = 0; // current number of elements in y

void get_args(int ac, char **av)
{ int i;
  num_inputs = ac - 1;
  for (i = 0; i < num_inputs; i++)
    x[i] = atoi(av[i+1]);
}

void scoot_over(int jj)
{ int k;
  for (k = num_y-1; k > jj; k++)
    y[k] = y[k-1];
}

void insert(int new_y)
{ int j;
  if (num_y == 0) { // y empty so far, easy case
    y[0] = new_y;
    return;
  }
  // need to insert just before the first y
  // element that new_y is less than
  for (j = 0; j < num_y; j++) {
    if (new_y < y[j]) {
      // shift y[j], y[j+1],... rightward
      // before inserting new_y
      scoot_over(j);
      y[j] = new_y;
      return;
    }
  }
}

void process_data()
{
  for (num_y = 0; num_y < num_inputs; num_y++)
    // insert new y in the proper place
    // among y[0],...,y[num_y-1]
    insert(x[num_y]);
}

void print_results()
{ int i;
  for (i = 0; i < num_inputs; i++)
    printf("%d\n",y[i]);
}

int main(int argc, char ** argv)
{ get_args(argc,argv);
  process_data();
  print_results();
}

```


Ниже приведено псевдокодированное описание программы. Вызовы функций обозначены операторами `call`, а псевдокод каждой функции показан с отступом под соответствующими вызовами:

```
call main():
  set y array to empty
  call get_args():
    get num_inputs numbers x[i] from command line
  call process_data():
    for i = 1 to num_inputs
      call insert(x[i]):
        new_y = x[i]
        find first y[j] for which new_y < y[j]
        call scoot_over(j):
          shift y[j], y[j+1], ... to right,
          to make room for new_y
        set y[j] = new_y
```

Скомпилируем и запустим код:

```
$ gcc -g -Wall -o insert_sort ins.c
```

Важно: параметр `-g` для GCC позволяет сообщить компилятору, что нужно сохранить таблицу символов, то есть список адресов памяти, соответствующих переменным и строкам кода вашей программы, внутри сгенерированного исполняемого файла, которым здесь является `insert_sort`. Это совершенно необходимый шаг, позволяющий во время сеанса отладки ссылаться на имена переменных и номера строк в исходном коде. Без этого шага (и аналогичный шаг потребовался бы при использовании компилятора, отличного от GCC) вы не смогли бы попросить отладчик, например, "остановиться на строке 30" или "напечатать значение x".

Теперь запустим программу. Следуя принципу "начинайте с малого" из раздела 1.3.3, сначала попробуйте отсортировать список всего из двух чисел:

```
$ insert_sort 12 5
(execution halted by user hitting ctrl-C)
```

Программа не завершилась и не напечатала никакого вывода. По-видимому, она попала в бесконечный цикл, и нам пришлось остановить ее нажатием CTRL-C. Сомнений нет: что-то не так.

В следующих разделах мы сначала представим сеанс отладки этой ошибочной программы с помощью GDB, а затем обсудим, как те же операции выполняются в DDD и Eclipse.

1.7.1 Подход GDB

Чтобы найти первую ошибку, выполните программу в GDB и дайте ей некоторое время поработать, прежде чем приостановить ее с помощью CTRL-C. Затем посмотрите, где вы оказались. Таким образом можно определить место бесконечного цикла.

Сначала запустите отладчик GDB для `insert_sort`:

```
$ gdb insert_sort -tui
```

Теперь экран будет выглядеть так:

```
63 { get_args(argc,argv);
64 process_data();
65 print_results();
66 }
67
68
69
File: ins.c Procedure: ?? Line: ?? pc: ??
-----
(gdb)
```

Верхнее подокно отображает часть исходного кода, а в нижнем подокне вы видите приглашение GDB, готовое принять команды. Есть также приветственное сообщение GDB, которое мы опустили ради краткости.

Если при вызове GDB не запрашивать режим TUI, вы получили бы только приветственное сообщение и приглашение GDB, без верхнего подокна с исходным кодом программы. Затем можно было бы войти в режим TUI с помощью команды GDB CTRL-X-A. Эта команда переключает вас в режим TUI и обратно; она полезна, например, если вы хотите временно выйти из TUI, чтобы удобнее читать встроенную справку GDB или чтобы видеть больше истории команд GDB на одном экране.

Теперь запустите программу изнутри GDB, выдав команду `run` вместе с аргументами командной строки вашей программы, а затем нажмите CTRL-C, чтобы приостановить ее. Теперь экран выглядит так:

```
46
47 void process_data()
48 {
49 for (num_y = 0; num_y < num_inputs; num_y++)
50 // insert new y in the proper place
51 // among y[0],...,y[num_y-1]
> 52 insert(x[num_y]);
53 }
54
55 void print_results()
56 { int i;
57
58 for (i = 0; i < num_inputs; i++)
59 printf("%d\n",y[i]);
60}.
File: ins.c Procedure: process_data Line: 52 pc: 0x8048483
-----
(gdb) run 12 5
Starting program: /debug/insert_sort 12 5
Program received signal SIGINT, Interrupt.
0x08048483 in process_data () at ins.c:52
(gdb)
```

Это сообщает, что когда вы остановили программу, `insert_sort` находилась в функции `process_data()`, и должна была выполняться строка 52 исходного файла `ins.c`.

Мы нажали CTRL-C в случайный момент и остановились в случайном месте кода. Иногда полезно два или три раза приостановить и возобновить программу, переставшую отвечать, выдавая `continue` между нажатиями CTRL-C, чтобы увидеть, где вы останавливаетесь каждый раз.

Теперь строка 52 является частью цикла, который начинается на строке 49. Это и есть бесконечный цикл? Цикл не выглядит так, будто должен выполняться бесконечно, но принцип подтверждения говорит: это нужно проверить, а не просто предположить. Если цикл не завершается потому, что вы каким-то образом неправильно установили верхнюю границу для переменной `num_y`, то после некоторого времени работы программы значение `num_y` будет огромным. Так ли это? Снова,

выглядит так, будто не должно быть, но нужно подтвердить. Проверим текущее значение `num_y`, попросив GDB напечатать его.

```
(gdb) print num_y
$ 1=1
```

Вывод этого запроса к GDB показывает, что значение `num_y` равно 1. Метка \$1 означает, что это первое значение, которое вы попросили GDB напечатать. Значения, обозначенные \$1, \$2, \$3 и так далее, вместе называются value history сеанса отладки. Они могут быть очень полезны, как вы увидите в последующих главах. Итак, похоже, мы находимся только на второй итерации цикла на строке 49. Если бы этот цикл был бесконечным, к этому моменту он ушел бы далеко за вторую итерацию.

Поэтому посмотрим внимательнее, что происходит, когда `num_y` равно 1. Скажите GDB остановиться в `insert()` во время второй итерации цикла на строке 49, чтобы можно было осмотреться и попытаться выяснить, что идет не так в этом месте и в этот момент выполнения программы:

```
(gdb) break 30
Breakpoint 1 at 0x80483fc: file ins.c, line 30.
(gdb) condition 1 num_y==1
```

Первая команда ставит точку останова на строке 30, то есть в начале `insert()`. В качестве альтернативы можно было указать эту точку останова командой `break insert`, то есть остановиться на первой строке `insert()` (здесь это строка 30). У последней формы есть преимущество: если вы измените код программы так, что функция `insert()` больше не будет начинаться на строке 30 файла `ins.c`, точка останова останется действительной, если она задана через имя функции, но не останется, если задана через номер строки.

Обычно команда `break` заставляет выполнение останавливаться всякий раз, когда программа достигает указанной строки. Однако вторая команда здесь, `condition 1 num_y==1`, делает эту точку останова условной: GDB будет приостанавливать выполнение программы в точке останова 1 только тогда, когда выполняется условие `num_y==1`.

Обратите внимание, что в отличие от команды `break`, которая принимает номера строк или имена функций, `condition` принимает номер точки останова. Всегда можно использовать команду `info break`, чтобы узнать номер нужной точки останова. Эта команда дает и другую полезную информацию, например сколько раз каждая точка останова уже была достигнута.

Мы могли бы объединить команды `break` и `condition` в один шаг, используя `break if` следующим образом:

```
(gdb) break 30 if num_y==1
```

Затем снова запустите программу командой `run`. Если вы просто хотите повторно использовать старые аргументы командной строки, их не нужно указывать заново. Здесь именно такой случай, поэтому можно просто набрать `run`. Поскольку программа уже выполняется, GDB спросит, хотите ли вы перезапустить ее с начала, и вы ответите "yes".

Теперь экран будет выглядеть так:

```
24 y[k] = y[k-1];
25 }
26
27 void insert(int new_y)
28 { int j;
29
*> 30 if (num_y = 0) { // y empty so far, easy case
31 y[0] = new_y;

32 return;
33 }
34 // need to insert just before the first y
35 // element that new_y is less than
36 for (j = 0; j < num_y; j++) {
37 if (new_y < y[j]) {
38 // shift y[j], y[j+1],... rightward .
File: ins.c Procedure: insert Line: 30 pc: 0x80483fc
-----
(gdb) condition 1 num_y==1
(gdb) run
The program being debugged has been started already.
Start it from the beginning? (y or n)
Starting program: /debug/insert_sort 12 5
Breakpoint 1, insert (new_y=5) at ins.c:30
(gdb)
```

Снова применим принцип подтверждения: поскольку num_y равно 1, строка 31 должна быть пропущена, а выполнение должно перейти к строке 36. Но нам нужно подтвердить это, поэтому мы выдаем команду next, чтобы перейти к следующей строке:

```
24 y[k] = y[k-1];
25 }
26
27 void insert(int new_y)
28 { int j;
29
* 30 if (num_y = 0) { // y empty so far, easy case
31 y[0] = new_y;
32 return;
33 }
34 // need to insert just before the first y
35 // element that new_y is less than
> 36 for (j = 0; j < num_y; j++) {
37 if (new_y < y[j]) {
38 // shift y[j], y[j+1],... rightward .
File: ins.c Procedure: insert Line: 36 pc: 0x8048406
-----
(gdb) run
The program being debugged has been started already.
Start it from the beginning? (y or n)
Starting program: /debug/insert_sort 12 5
Breakpoint 1, insert (new_y=5) at ins.c:30
(gdb) next
(gdb)
```

Стрелка в верхнем подокне теперь находится на строке 36, поэтому наше ожидание подтверждено: мы действительно пропустили строку 31. Теперь продолжим пошагово проходить программу, попутно подтверждая предположения о коде. Сейчас вы находитесь в начале цикла, поэтому выдайте команду `next` еще несколько раз и посмотрите, как цикл продвигается строка за строкой:

```
39 // before inserting new_y
40 scoot_over(j);
41 y[j] = new_y;
42 return;
43 }
44 }
>4 5 }
46
47 void process_data()
48 {
49 for (num_y = 0; num_y < num_inputs; num_y++)
50 // insert new y in the proper place
51 // among y[0],...,y[num_y-1]
52 insert(x[num_y]);
53 } .
File: ins.c Procedure: insert Line: 45 pc: 0x804844d
-----
The program being debugged has been started already.
Start it from the beginning? (y or n)
Starting program: /debug/insert_sort 12 5
Breakpoint 1, insert (new_y=5) at ins.c:30
(gdb) next
(gdb) next
(gdb) next
(gdb) next
```

Посмотрите, где теперь находится стрелка в верхнем подокне: мы перешли прямо со строки 37 на строку 45! Это настоящий сюрприз. Мы не выполнили ни одной итерации цикла. Но помните: сюрпризы - это хорошо, потому что они дают подсказки о местонахождении ошибок.

Единственный способ, при котором цикл на строке 36 мог не выполнить ни одной итерации, состоит в том, что условие `j < num_y` в строке 36 не выполнялось даже тогда, когда `j` было равно 0. Но вы знаете, что `num_y` равно 1, потому что сейчас вы находитесь в этой функции после того, как наложили условие `num_y==1` на точку останова. Или, по крайней мере, вы думаете, что знаете это. Снова: вы этого не подтвердили. Проверьте это сейчас:

```
(gdb) print num_y
$ 2=0
```

Действительно, условие `num_y==1` выполнялось, когда вы вошли в `insert()`, но, по-видимому, с тех пор `num_y` изменилось. Каким-то образом `num_y` стало равно 0 после входа в эту функцию. Но как?

Как уже говорилось, принцип подтверждения не говорит вам, в чем именно ошибка, но дает подсказки о том, где она, вероятно, находится. В данном случае вы теперь обнаружили, что место ошибки где-то между строками 30 и 36. И этот диапазон можно сузить еще сильнее, потому что вы видели, что строки 31-33 были пропущены, а строки 34-35 являются комментариями. Другими словами, загадочное изменение значения `num_y` произошло либо на строке 30, либо на строке 36.

После короткого перерыва - часто это лучшая стратегия отладки! - мы внезапно понимаем, что ошибка классическая, часто допускаемая начинающими (и, что неловко, опытными) C-программистами: в строке 30 мы использовали `=` вместо `==`, превратив проверку равенства в присваивание.

Видите, как из-за этого возникает бесконечный цикл? Ошибка на строке 30 создает вечные качели: часть `num_y++` строки 49 снова и снова увеличивает `num_y` с 0 до 1, а ошибка в строке 30 снова и снова устанавливает значение этой переменной обратно в 0.

Итак, мы исправляем эту унижительную ошибку (а какие ошибки не унижительны?), перекомпилируем программу и пробуем запустить ее снова:

```
$ insert_sort 12 5
5
0
```

Бесконечного цикла больше нет, но правильного вывода тоже нет.

Вспомните по псевдокоду, что должна делать программа в этом месте: изначально массив `y` пуст. Первая итерация цикла на строке 49 должна поместить 12 в `y[0]`. Затем на второй итерации 12 должно быть сдвинуто на одну позицию массива, чтобы освободить место для вставки 5. Вместо этого 5, похоже, заменило 12.

Проблема возникает со вторым числом (5), поэтому снова следует сосредоточиться на второй итерации. Поскольку мы разумно остались в сеансе GDB, а не вышли из GDB после обнаружения и исправления первой ошибки, точка останова и ее условие, которые мы установили ранее, сейчас все еще действуют. Поэтому мы просто снова запускаем программу и останавливаемся, когда программа начинает обрабатывать второй входной элемент:

```
24 y[k] = y[k-1];
25 }
26
27 void insert(int new_y)
28 { int j;
29
*> 30 if (num_y == 0) { // y empty so far, easy case
31 y[0] = new_y;
32 return;
33 }
34 // need to insert just before the first y
35 // element that new_y is less than

36 for (j = 0; j < num_y; j++) {
37 if (new_y < y[j]) {
38 // shift y[j], y[j+1],... rightward .
File: ins.c Procedure: insert Line: 30 pc: 0x80483fc
-----
The program being debugged has been started already.
Start it from the beginning? (y or n)
`/debug/insert_sort' has changed; re-reading symbols.
Starting program: /debug/insert_sort 12 5
Breakpoint 1, insert (new_y=5) at ins.c:30
(gdb)
```

Обратите внимание на строку, которая сообщает:

```
`/debug/insert_sort' has changed; re-reading symbols.
```

Она показывает, что GDB увидел: мы перекомпилировали программу, и автоматически перечитал новый бинарный файл и новую таблицу символов перед запуском программы.

Еще раз: то, что перед перекомпиляцией программы не пришлось выходить из GDB, является большим удобством по нескольким причинам. Во-первых, не нужно заново указывать аргументы командной строки; достаточно набрать `run`, чтобы снова запустить программу. Во-вторых, GDB сохраняет установленную ранее точку останова, так что ее не нужно вводить заново. Здесь у вас только одна точка останова, но обычно их несколько, и тогда это становится реальной проблемой. Эти удобства экономят набор текста и, что важнее, избавляют от практических отвлечений, позволяя лучше сосредоточиться на самой отладке.

Точно так же во время сеанса отладки не следует постоянно выходить из текстового редактора и запускать его заново: это тоже отвлекает и тратит время. Просто держите текстовый редактор открытым в одном окне, GDB (или DDD) - в другом, а третье окно используйте для пробных запусков программы.

Теперь снова попробуем пройти код пошагово. Как и раньше, программа должна пропустить строку 31, но, будем надеяться, на этот раз она достигнет строки 37, в отличие от прежней ситуации.

Проверим это, дважды выдав команду `next`:

```
31 y[0] = new_y;
32 return;
33 }
34 // need to insert just before the first y
35 // element that new_y is less than
36 for (j = 0; j < num_y; j++) {
> 37 if (new_y < y[j]) {
38 // shift y[j], y[j+1],... rightward
39 // before inserting new_y
40 scoot_over(j);
41 y[j] = new_y;

42 return;
43 }
44 }
45 } .
File: ins.c Procedure: insert Line: 37 pc: 0x8048423
-----
`/debug/insert_sort' has changed; re-reading symbols.
Starting program: /debug/insert_sort 12 5
Breakpoint 1, insert (new_y=5) at ins.c:30
(gdb) next
(gdb) next
(gdb)
```

Мы действительно достигли строки 37.

В этой точке мы считаем, что условие в `if` на строке 37 должно выполняться, потому что `new_y` должно быть равно 5, а `y[0]` должно быть равно 12 после первой итерации. Вывод GDB подтверждает первое предположение. Проверим второе:

```
(gdb) print y[0]
$ 3=12
```

Теперь, когда это предположение тоже подтверждено, выдайте команду `next`, которая приведет вас к строке 40. Функция `scoot_over()` должна сдвинуть 12 в следующую позицию массива, чтобы освободить место для 5. Нужно проверить, делает она это или нет. Здесь перед вами важный выбор. Можно снова выдать команду `next`, из-за чего GDB остановится на строке 41; функция `scoot_over()` будет выполнена, но GDB не остановится внутри этой функции. Однако если вместо этого вы выдадите команду `step`, GDB остановится на строке 23, и это позволит выполнять пошаговый проход внутри `scoot_over()`.

Следуя нисходящему подходу к отладке, описанному в разделе 1.3.3, на строке 40 мы выбираем команду `next`, а не `step`. Когда GDB остановится на строке 41, можно посмотреть на `y` и понять, правильно ли функция выполнила свою работу. Если эта гипотеза подтвердится, вы избежите трудоемкого изучения подробной работы функции `scoot_over()`, которое ничего не дало бы для исправления текущей ошибки. Если же подтвердить правильную работу функции не удастся, можно снова запустить программу в отладчике и войти в функцию с помощью `step`, чтобы изучить ее подробную работу и, будем надеяться, определить, где она сбивается.

Итак, когда вы достигнете строки 40, наберите next, получив:

```
31 y[0] = new_y;
32 return;
33 }
34 // need to insert just before the first y
35 // element that new_y is less than
36 for (j = 0; j < num_y; j++) {
37 if (new_y < y[j]) {

38 // shift y[j], y[j+1],... rightward
39 // before inserting new_y
40 scoot_over(j);
> 41 y[j] = new_y;
42 return;
43 }
44 }
45 } .
File: ins.c Procedure: insert Line: 41 pc: 0x8048440
```

```
-----
(gdb) next
(gdb) next
(gdb)
```

Сдвинула ли scoot_over() значение 12 правильно? Проверим:

```
(gdb) print y
$4 = {12, 0, 0, 0, 0, 0, 0, 0, 0, 0}
```

Очевидно, нет. Проблема действительно находится в scoot_over(). Удалим точку останова в начале insert() и поставим одну в scoot_over(), снова с условием, что мы остановимся там во время второй итерации строки 49:

```
(gdb) clear 30
Deleted breakpoint 1
(gdb) break 23
Breakpoint 2 at 0x80483c3: file ins.c, line 23.
(gdb) condition 2 num_y==1
```

Теперь снова запустим программу:

```
15 num_inputs = ac - 1;
16 for (i = 0; i < num_inputs; i++)
17 x[i] = atoi(av[i+1]);
18 }
19
20 void scoot_over(int jj)
21 { int k;
22
*> 23 for (k = num_y-1; k > jj; k++)
24 y[k] = y[k-1];
25 }
26
27 void insert(int new_y)
28 { int j;
29 .
File: ins.c Procedure: scoot_over Line: 23 pc: 0x80483c3
```

```
-----
(gdb) condition 2 num_y==1
(gdb) run
The program being debugged has been started already.
Start it from the beginning? (y or n)
Starting program: /debug/insert_sort 12 5
Breakpoint 2, scoot_over (jj=0) at ins.c:23
(gdb)
```


Снова следуйте принципу подтверждения: подумайте, что, как вы ожидаете, должно произойти, а затем попытайтесь подтвердить, что это действительно происходит. В данном случае функция должна сдвинуть 12 на следующую позицию в массиве y, а значит цикл на строке 23 должен пройти ровно одну итерацию. Пройдем программу пошагово, несколько раз выдавая команду next, чтобы проверить это ожидание:

```
15 num_inputs = ac - 1;
16 for (i = 0; i < num_inputs; i++)
17 x[i] = atoi(av[i+1]);
18 }
19
20 void scoot_over(int jj)
21 { int k;
22
* 23 for (k = num_y-1; k > jj; k++)
24 y[k] = y[k-1];
>2 5 }
26
27 void insert(int new_y)
28 { int j;
29 .
File: ins.c Procedure: scoot_over Line: 25 pc: 0x80483f1
-----
The program being debugged has been started already.
Start it from the beginning? (y or n)
Starting program: /debug/insert_sort 12 5
Breakpoint 2, scoot_over (jj=0) at ins.c:23
(gdb) next
(gdb) next
(gdb) next
```

Здесь мы снова получаем сюрприз: теперь мы на строке 25, ни разу не коснувшись строки 24; цикл выполнил ноль итераций, а не одну, как мы ожидали. По-видимому, в строке 23 есть ошибка.

Как и с предыдущим циклом, который неожиданно не выполнил ни одной итерации своего тела, должно быть так, что условие цикла не было истинным с самого начала. Так ли это здесь? Условие цикла на строке 23 - $k > jj$. Из этой же строки мы знаем, что начальное значение k равно $\text{num_y}-1$, а из условия нашей точки останова знаем, что последняя величина равна 0. Наконец, экран GDB сообщает, что jj равно 0. Значит, условие $k > jj$ не выполнялось, когда цикл начался.

Следовательно, мы неверно задали либо условие цикла $k > jj$, либо инициализацию $k = \text{num_y}-1$. Учитывая, что 12 должно было переместиться из $y[0]$ в $y[1]$ на первой и единственной итерации цикла, то есть строка 24 должна была выполняться при $k=1$, мы понимаем, что неверна инициализация цикла. Нужно было написать $k = \text{num_y}$.

Исправьте ошибку, перекомпилируйте программу и снова запустите ее вне GDB:

```
$ insert_sort 12 5
Segmentation fault
```

Segmentation faults, подробно обсуждаемые в главе 4, возникают, когда выполняющаяся программа пытается обратиться к памяти, к которой у нее нет разрешения на доступ. Обычно причина состоит в выходе индекса массива за границы или в ошибочном значении указателя. Seg faults могут возникать и из-за обращений к памяти, где явно не используются переменные-указатели или массивы. Один пример можно увидеть в другой классической ошибке C-программиста: забыть амперсанд в параметре функции, который передается через call-by-reference, например написать

```
scanf("%d",x);
```

ВМЕСТО

```
scanf("%d",&x);
```

В общем случае главная ценность отладочного инструмента вроде GDB или DDD состоит в облегчении процесса проверки ваших предположений о коде, но в случае seg faults отладочный инструмент дает дополнительную, ощутимую и немедленную помощь: он сообщает, где в программе произошла ошибка.

Чтобы этим воспользоваться, нужно запустить `insert_sort` в GDB и воспроизвести seg fault. Сначала удалите точку останова. Как было видно ранее, для этого нужно указать номер строки точки останова. Возможно, вы уже помните его, но найти его легко: либо прокрутите окно TUI клавишами со стрелками вверх и вниз, отыскивая строки, помеченные звездочками, либо используйте команду GDB `info break`. Затем удалите точку останова командой `clear`:

```
(gdb) clear 30
```

Теперь снова запустите программу в GDB:

```
19
20 void scoot_over(int jj)
21 { int k;
22
23 for (k = num_y; k > jj; k++)
> 24 y[k] = y[k-1];
25 }
26
27 void insert(int new_y)
28 { int j;
29
30 if (num_y == 0) { // y empty so far, easy case
31 y[0] = new_y; .
File: ins.c Procedure: scoot_over Line: 24 pc: 0x8048538
-----
Start it from the beginning? (y or n)
`/debug/insert_sort' has changed; re-reading symbols.
Starting program: /debug/insert_sort 12 5
Program received signal SIGSEGV, Segmentation fault.
0x08048538 in scoot_over (jj=0) at ins.c:24
(gdb)
```

Как и обещано, GDB сообщает нам точно, где произошел seg fault: на строке 24. И действительно, здесь явно участвует индекс массива, а именно `k`. Либо `k` было достаточно велико, чтобы превысить число элементов в `y`, либо `k-1` было отрицательным. Очевидно, первое дело - определить значение `k`:

```
(gdb) print k
$4 = 584
```

Ого! Код объявил массив `y` только из 10 элементов, так что это значение `k` действительно далеко за пределами диапазона. Теперь нужно отследить причину.

Прежде всего определите итерацию большого цикла на строке 49, во время которой произошел этот seg fault.

```
(gdb) print num_y
$ 5=1
```

Итак, это случилось во время второй итерации, то есть в первый раз, когда выполняется функция `scoot_over()`. Другими словами, не было такого, что строка 23 прекрасно работала в первых нескольких вызовах `scoot_over()`, а позже дала сбой. С этой строкой кода по-прежнему что-то фундаментально не так. И поскольку единственный оставшийся кандидат - оператор

```
k++
```

(напомним, что две другие части этой строки вы уже проверили ранее), виновником должен быть он. После еще одного перерыва, чтобы прочистить голову, мы с некоторым смущением понимаем, что здесь должно было быть `k--`.

Исправьте эту строку, снова перекомпилируйте и запустите программу:

```
$ insert_sort 12 5
5
12
```

Вот это уже прогресс! Но работает ли программа на большем наборе данных? Попробуем:

```
$ insert_sort 12 5 19 22 6 1
1
5
6
12
0
0
```

Теперь уже виден свет в конце туннеля. Большая часть массива сортируется правильно. Первое число в списке, которое не сортируется правильно, - 19, поэтому поставьте точку останова на строке 36, на этот раз с условием `new_y == 19`:^[1]

```
(gdb) b 36
Breakpoint 3 at 0x804840d: file ins.c, line 36.
(gdb) cond 3 new_y==19
```

Затем запустите программу в GDB, обязательно используя те же аргументы: 12 5 19 22 6 1. Когда вы достигнете точки останова, подтвердите, что массив `y` к этому моменту отсортирован правильно:

```
31 y[0] = new_y;
32 return;
33 }
34 // need to insert just before the first y
35 // element that new_y is less than
*> 36 for (j = 0; j < num_y; j++) {
37 if (new_y < y[j]) {
38 // shift y[j], y[j+1],... rightward
39 // before inserting new_y
40 scoot_over(j);
41 y[j] = new_y;

42 return;
43 } .
File: ins.c Procedure: insert Line: 36 pc: 0x8048564
-----
Start it from the beginning? (y or n)
Starting program: /debug/insert_sort 12 5 19 22 6 1
Breakpoint 2, insert (new_y=19) at ins.c:36
(gdb) p y
$1 = {5, 12, 0, 0, 0, 0, 0, 0, 0}
(gdb)
```

Пока все хорошо. Теперь попробуем определить, как программа "проглатывает" 19. Мы будем проходить код по одной строке. Обратите внимание: поскольку 19 не меньше 5 или 12, мы не ожидаем, что условие в операторе `if` на строке 37 выполнится. Несколько раз нажав `n`, мы оказываемся на строке 45:

```
35 // element that new_y is less than
* 36 for (j = 0; j < num_y; j++) {
37 if (new_y < y[j]) {
38 // shift y[j], y[j+1],... rightward
39 // before inserting new_y
40 scoot_over(j);
41 y[j] = new_y;
42 return;
43 }
44 }
>4 5 }
46
47 void process_data() .
```

File: ins.c Procedure: insert Line: 45 pc: 0x80485c4

```
-----  
(gdb) n  
(gdb) n  
(gdb) n  
(gdb) n  
(gdb) n  
(gdb)
```

Мы на строке 45, вот-вот выйдем из цикла, так ничего и не сделав с 19! Небольшая проверка показывает, что наш код не был написан для важного случая: когда `new_y` больше любого элемента, обработанного к этому моменту. На этот недосмотр указывают и комментарии в строках 34 и 35:

```
// need to insert just before the first y  
// element that new_y is less than
```

Чтобы обработать этот случай, добавьте следующий код сразу после строки 44:

```
// one more case: new_y > all existing y elements  
y[num_y] = new_y;
```

Затем перекомпилируйте программу и попробуйте снова:

```
$ insert_sort 12 5 19 22 6 1  
1  
5  
6  
12  
19  
22
```

Это правильный вывод, и последующее тестирование тоже дает правильные результаты.

1.7.2 Тот же сеанс в DDD

Посмотрим, как описанный выше сеанс GDB выполнялся бы в DDD. Конечно, нет необходимости повторять все шаги; сосредоточимся только на отличиях от GDB.

Запуск DDD похож на запуск GDB. Скомпилируйте исходный код с помощью GCC с параметром `-g`, а затем наберите:

```
$ ddd insert_sort
```

чтобы вызвать DDD. В GDB вы начинали выполнение программы командой `run`, включая аргументы, если они были нужны. В DDD вы щелкаете Program | Run, после чего увидите экран, показанный на рисунке 1-8.

Появилось окно Run, в котором предлагается история предыдущих наборов аргументов командной строки, которые вы использовали. Пока предыдущих наборов нет, но если бы они были, вы могли бы выбрать один щелчком мыши или ввести новый набор аргументов, как показано здесь. Затем нажмите Run.

В сеансе отладки GDB мы некоторое время выполняли программу в отладчике, а затем приостанавливали ее с помощью CTRL-C, чтобы исследовать очевидный бесконечный цикл. В DDD мы приостанавливаем программу, нажимая Interrupt в Command Tool. Теперь экран DDD выглядит как на рисунке 1-9. Поскольку DDD действует как оболочка для GDB, этот щелчок мышью переводится в операцию CTRL-C в GDB, что можно увидеть в Console.

Следующим шагом в приведенном выше сеансе GDB был просмотр переменной `num_y`. Как было показано ранее в разделе 1.5, в DDD это делается наведением указателя мыши на любой экземпляр `num_y` в Source Window.

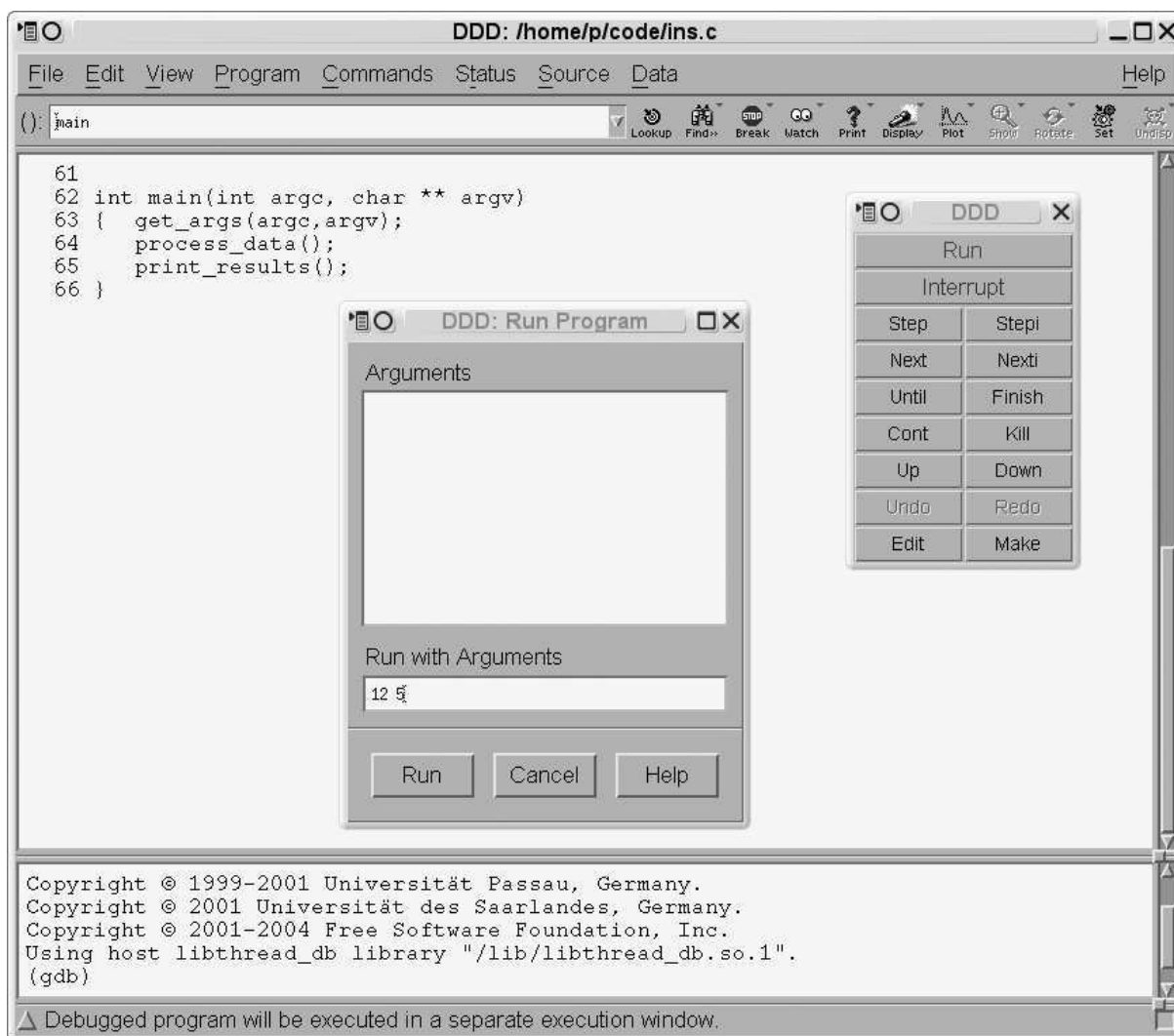


Рисунок 1-8. Команда Run в DDD

Таким же образом можно просматривать и целые массивы. Например, в один момент сеанса GDB вы напечатали весь массив `y`. В DDD вы просто навели бы указатель мыши на любой экземпляр `y` в окне Source. Если переместить курсор на `y` в выражении `y[j]` на строке 30, экран будет выглядеть так, как показано на рисунке 1-10. Рядом с этой строкой появилось поле value tip, показывающее содержимое `y`.

Следующим действием в сеансе GDB была установка точки останова на строке 30. Мы уже объяснили, как устанавливать точки останова в DDD, но как насчет условия для точки останова, которое было необходимо в данном случае? Условие можно задать, щелкнув правой кнопкой мыши значок stop sign на строке точки останова и затем выбрав Properties. Появится всплывающее окно, показанное на рисунке 1-11.

Затем введите условие `num_y==1`.

Чтобы затем снова запустить программу, нужно нажать Run в Command Tool. Как и команда GDB `run` без аргументов, эта кнопка запускает программу с последним предоставленным набором аргументов.

Аналоги команд GDB `n` и `s` в DDD - это кнопки Next и Step в Command Tool. Аналог команды GDB `c` - кнопка Cont.

Этого обзора достаточно, чтобы начать работать с DDD. В следующих главах мы рассмотрим некоторые расширенные возможности DDD, например его чрезвычайно полезную способность визуально отображать сложные структуры данных, такие как связанные списки и двоичные деревья.

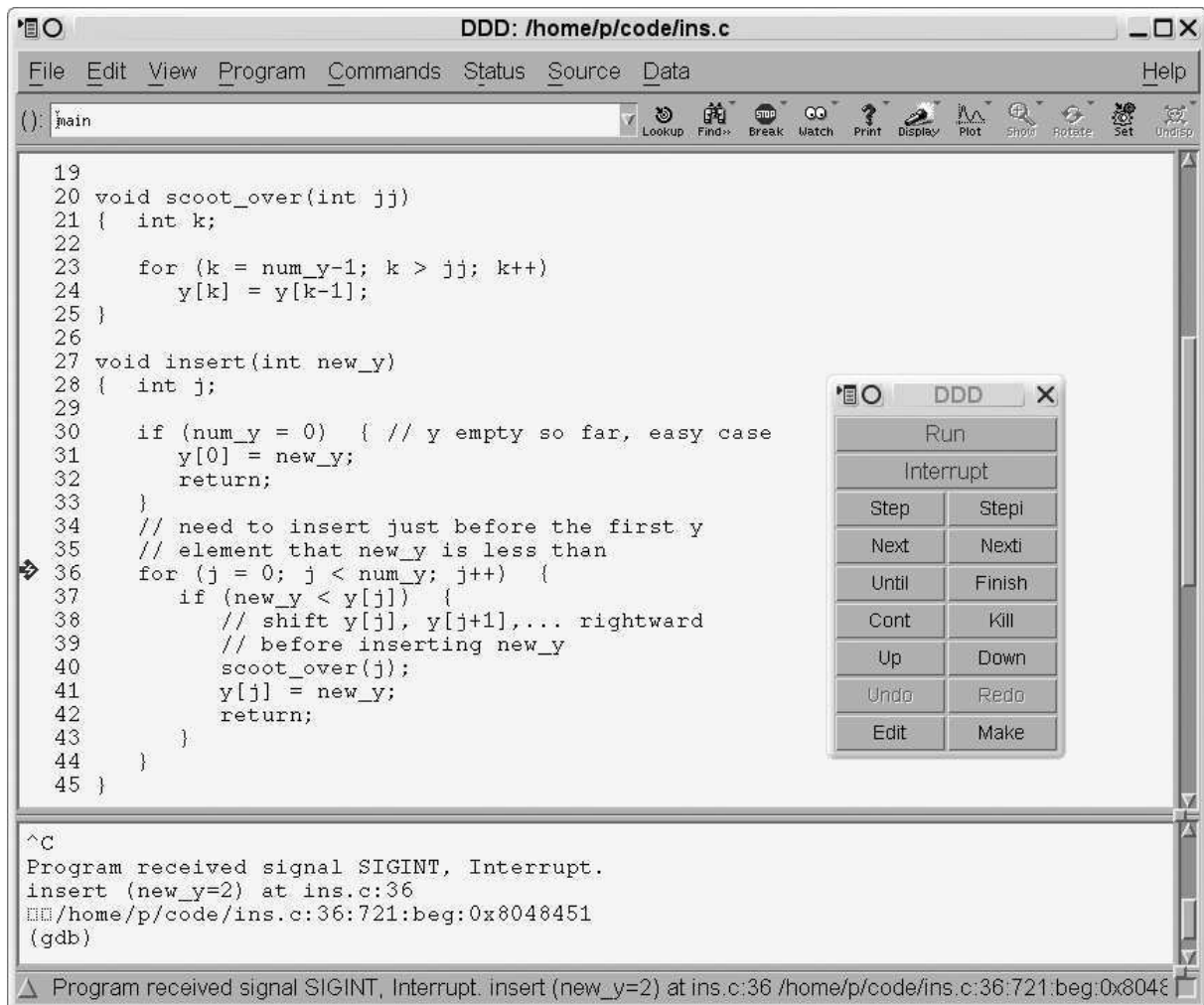


Рисунок 1-9. После прерывания

1.7.3 Сеанс в Eclipse

Теперь посмотрим, как описанный выше сеанс GDB выполнялся бы в Eclipse. Как и в изложении про DDD, нет необходимости повторять все шаги; мы просто сосредоточимся на отличиях от GDB.

Учтите, что Eclipse может быть довольно привередливым. Хотя он предлагает множество способов выполнить определенную задачу, если вы не следуете строго необходимой последовательности шагов, можно оказаться в затруднительном положении без интуитивного решения, кроме как перезапустить часть процесса отладки.

Здесь мы предполагаем, что вы уже создали свой проект C/C++. ^[2]

При первом запуске/отладке программы вам понадобятся конфигурации `run` и `debug`. Они задают имя исполняемого файла и проект, к которому он относится, его аргументы командной строки (если есть), специальное окружение переменных `shell` (если есть), выбранный отладчик и так далее. Конфигурация `run` используется для запуска программы вне отладчика, а конфигурация `debug` - внутри отладчика. Обязательно создайте обе конфигурации, именно в этом порядке, следующим образом:

1. Выберите `Run | Open Run Dialog`.
2. Щелкните правой кнопкой `C/C++ Local Applications` и выберите `New`.
3. Выберите вкладку `Main` и заполните конфигурацию запуска, проект и имена исполняемых файлов (Eclipse, вероятно, предложит их сам), а также отметьте `Connect process input and output to a terminal`, если у вас есть терминальный ввод-вывод.
4. Если у вас есть аргументы командной строки или специальные переменные окружения, щелкните вкладку `Arguments` или `Environment` и заполните нужные настройки.
5. Выберите вкладку `Debugger`, чтобы увидеть, какой отладчик используется. Скорее всего, вам не придется ничего здесь менять, но полезно понимать, что в основе есть отладчик, вероятно `GDB`.
6. Нажмите `Apply` (если будет предложено) и `Close`, чтобы завершить создание конфигурации `run`.
7. Начните создание конфигурации `debug`, выбрав `Run | Open Debug Dialog`. Eclipse, вероятно, повторно использует информацию, которую вы ввели в конфигурации `run`, как показано на рисунке 1-12, или вы можете изменить ее при желании. Снова нажмите `Apply` (если будет предложено) и `Close`, чтобы завершить создание конфигурации `debug`.

Можно создать несколько конфигураций `run/debug`, обычно с разными наборами аргументов командной строки.

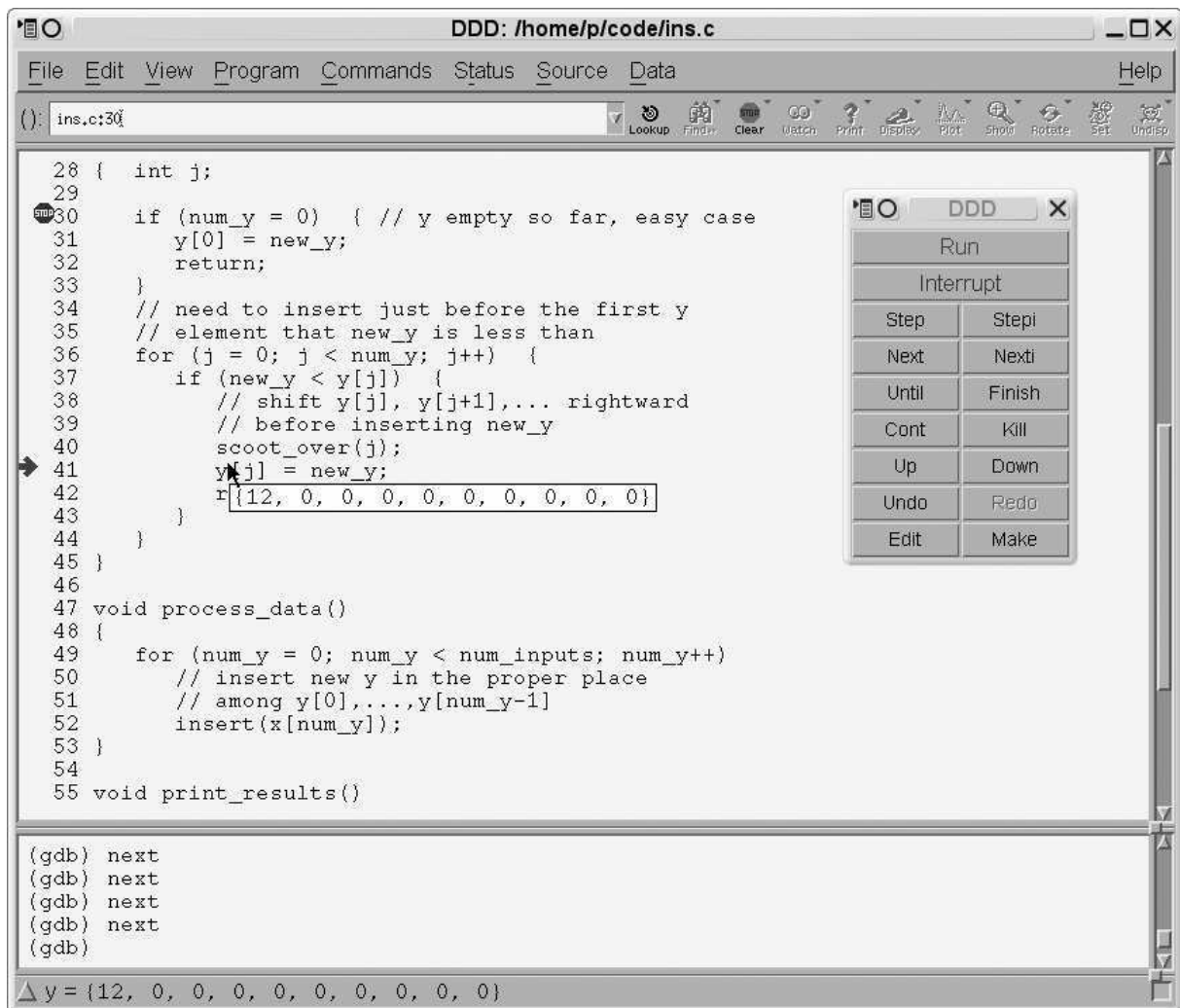


Рисунок 1-10. Просмотр массива

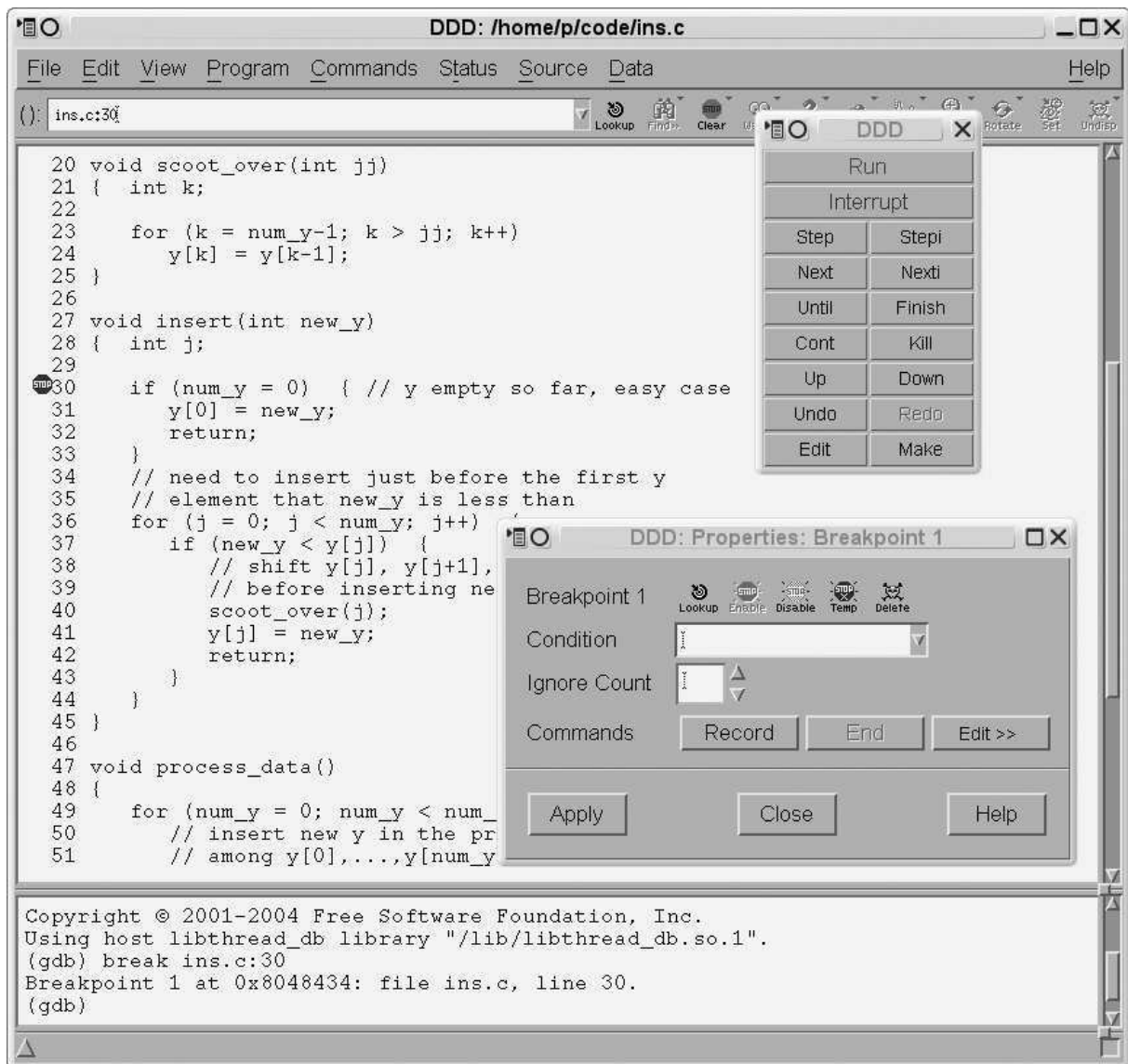


Рисунок 1-11. Наложение условия на точку останова

Чтобы начать сеанс отладки, нужно перейти в перспективу Debug, выбрав Window | Open Perspective | Debug. Существуют разные сокращения, которые мы оставим вам для самостоятельного обнаружения.

Первый раз, когда вы фактически выполняете действие run или debug, вы снова делаете это через Run | Open Run Dialog или Run | Open Debug Dialog, в зависимости от случая, чтобы указать, какую конфигурацию использовать. После этого, однако, достаточно выбрать Run | Run или Run | Debug; любой из этих вариантов повторно запустит последнюю конфигурацию debug.

На самом деле для отладки есть более быстрый способ запустить debug run: щелкнуть значок Debug прямо под Navigate (см. рисунок 1-13). Однако внимательно учтите: всякий раз, когда вы запускаете новый debug run, нужно завершать существующие, щелкая красный квадрат Terminate; один находится на панели инструментов view Debug, другой - в view Console. В view Debug также есть значок с двойным X, Remove All Terminated Launches.

Рисунок 1-13 показывает экран после запуска отладки. Начальную строку можно задать в диалогах отладки Eclipse, но обычно по умолчанию ставится автоматическая точка останова на первой выполняемой строке кода. На рисунке это видно по символу точки останова на левом поле строки:

```
{ get_args(argc, argv);
```

Эта строка также подсвечена, поскольку именно ее вы сейчас собираетесь выполнить. Выполните ее, щелкнув значок Resume на панели инструментов view Debug (над полем, которое всплыло в окне, потому что вы навели указатель мыши на этот значок).

Вспомните, что в примерном сеансе GDB первая версия программы имела бесконечный цикл, и программа зависала. Здесь, конечно, вы увидите тот же симптом: никакого вывода в view Console. Программу нужно остановить. Однако не следует делать это, нажимая один из красных квадратов Terminate, потому что это также завершит базовый сеанс GDB. Вы хотите остаться в GDB, чтобы посмотреть, где находились в коде, то есть найти место бесконечного цикла, изучить значения переменных и так далее. Поэтому вместо операции Terminate выберите Suspend, щелкнув значок справа от Resume на панели инструментов view Debug. В литературе Eclipse эта кнопка иногда называется Pause, поскольку ее символ похож на символ паузы в медиапроигрывателях.

После щелчка Suspend экран выглядит как на рисунке 1-14. Вы увидите, что непосредственно перед этой операцией Eclipse собирался выполнить строку:

```
for (j = 0; j < num_y; j++) {
```

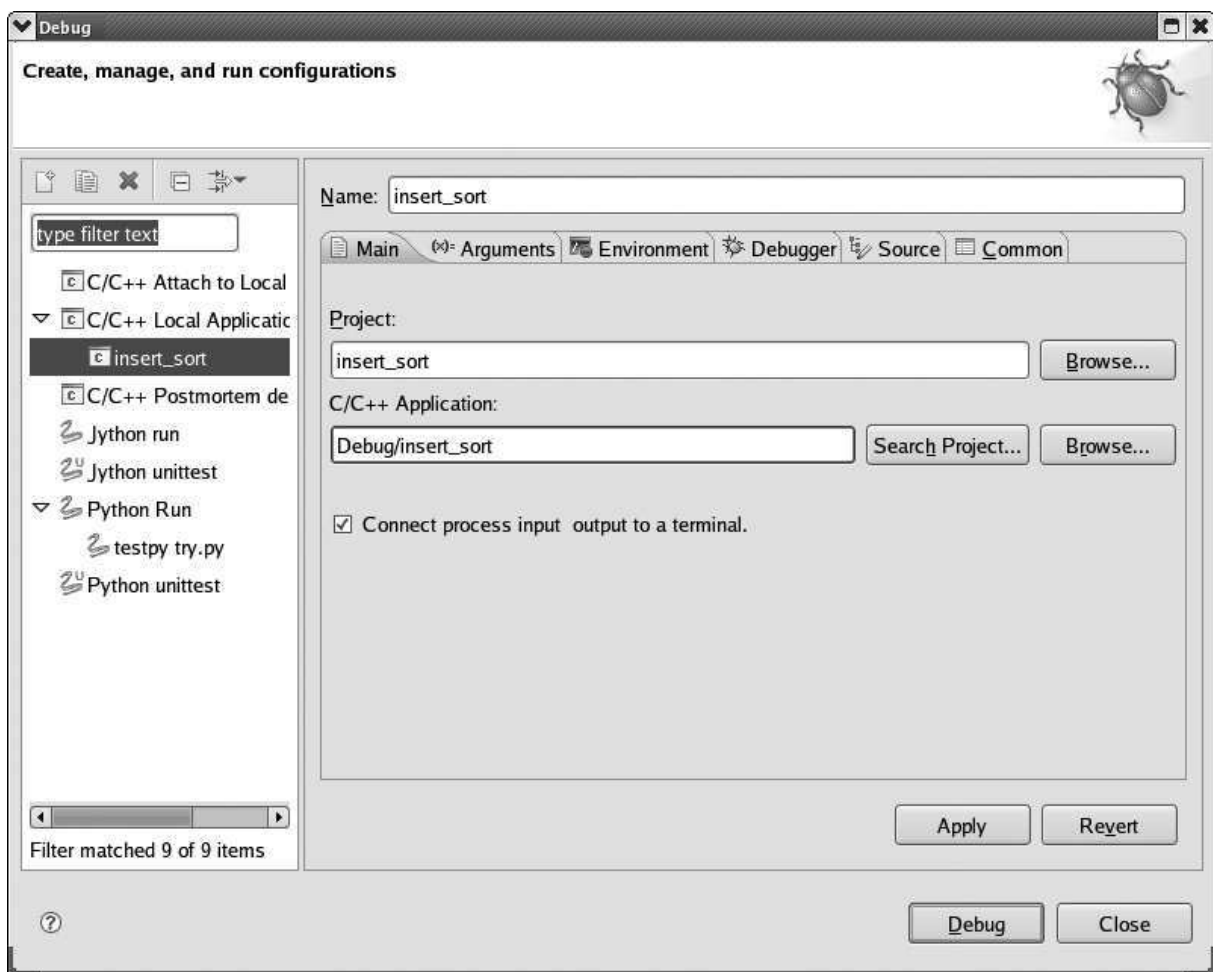


Рисунок 1-12. Диалог конфигурации отладки

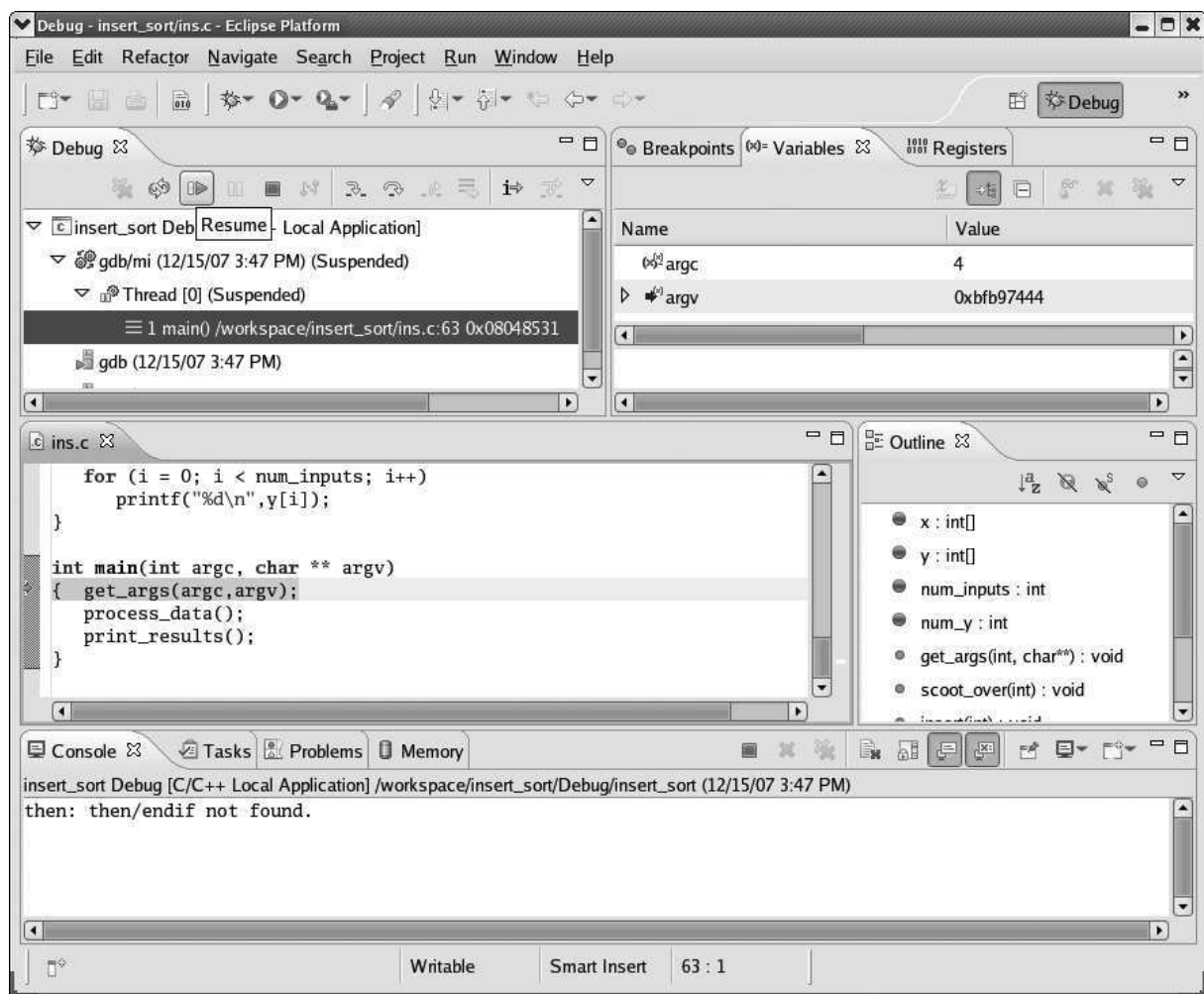


Рисунок 1-13. Начало debug run

Теперь можно изучить значение `num_y`, наведя указатель мыши на любой экземпляр этой переменной в окне исходного кода (вы обнаружите, что значение равно 0), и так далее.

Снова вспомним наш сеанс GDB выше. После исправления пары ошибок программа получила `segmentation fault`. Рисунок 1-15 показывает ваш экран Eclipse в этой точке.

Произошло следующее: мы щелкнули `Resume`, поэтому программа выполнялась, но внезапно остановилась на строке

```
y[k] = y[k-1];
```

из-за `seg fault`. Как ни странно, Eclipse не сообщает об этом на вкладке `Problems`, но сообщает на вкладке `Debug`, с сообщением об ошибке:

```
(Suspended 'SIGSEGV' received. Description: Segmentation fault.)
```

которое снова видно на рисунке 1-15.

На этой вкладке видно, что ошибка произошла в функции `scoot_over()`, которая была вызвана из `insert()`. Снова можно запрашивать значения переменных и, например, обнаружить, что `k = 544`, то есть далеко за пределами диапазона, как в примере GDB.

В примере GDB вы также устанавливали условные точки останова. Напомним, что в Eclipse точка останова устанавливается двойным щелчком на левом поле строки исходного кода.

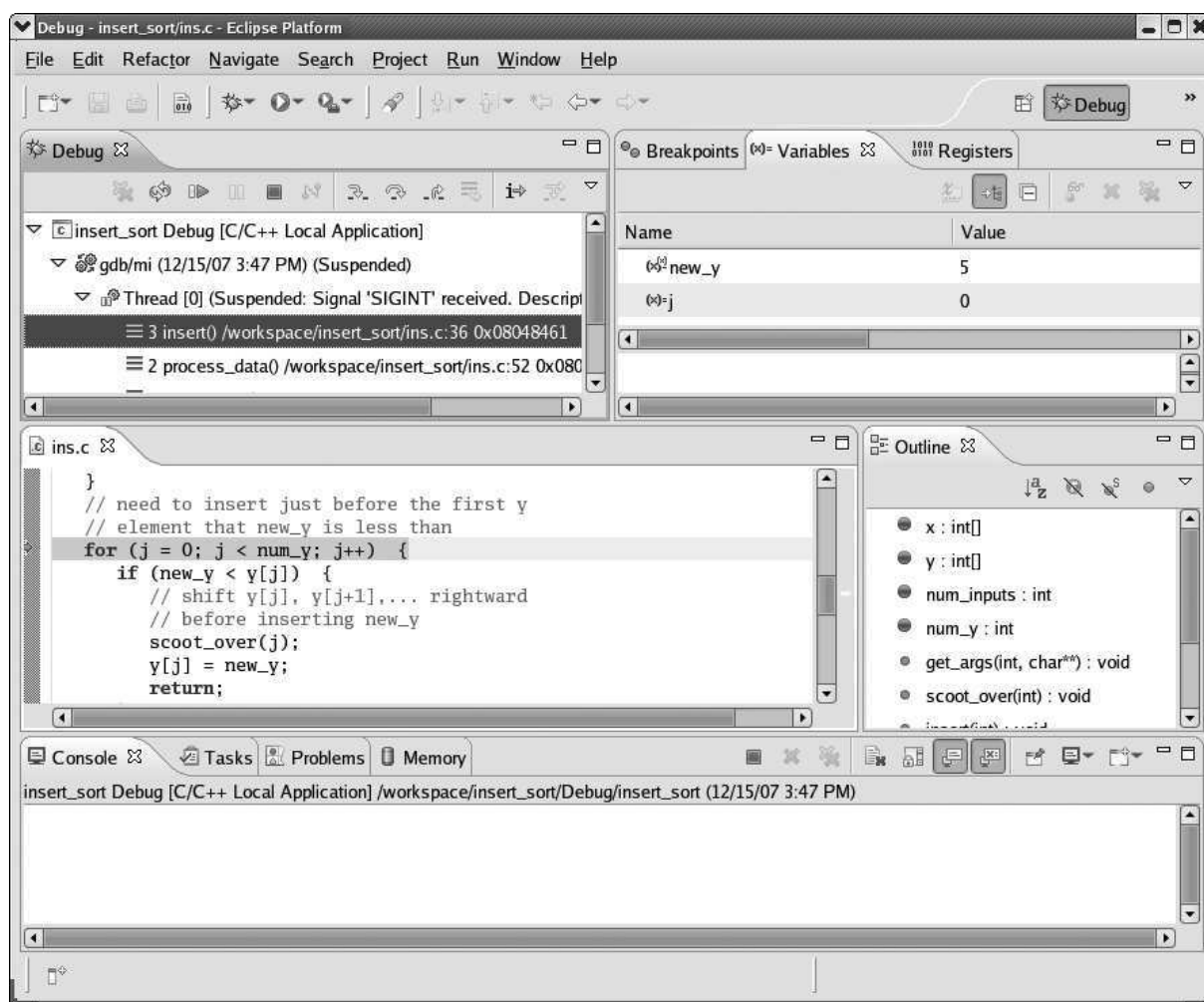


Рисунок 1-14. Программа приостановлена

Чтобы сделать такую точку останова условной, щелкните правой кнопкой мыши символ точки останова на нужной строке и выберите Breakpoint Properties... | New | Common, затем заполните условие в диалоговом окне. Диалог показан на рисунке 1-16.

Также вспомните, что в сеансе GDB вы иногда выполняли программу вне GDB, в отдельном окне терминала. Это легко сделать и в Eclipse, выбрав Run | Run. Результаты, как обычно, будут находиться в view Console.

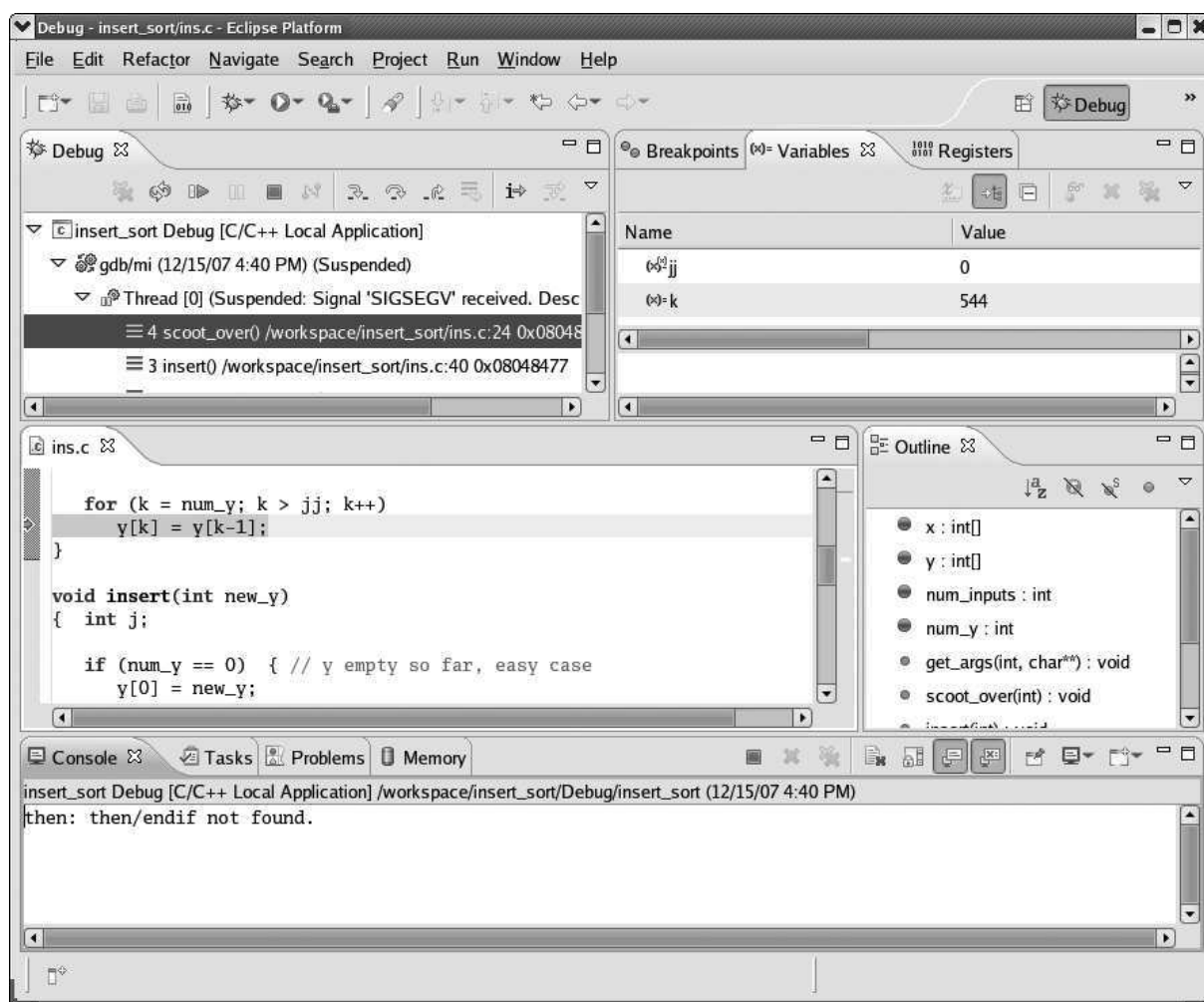


Рисунок 1-15. Нарушение сегментации

1.8 Использование startup-файлов

Как уже говорилось, обычно полезно не выходить из GDB, пока вы перекомпилируете код. Так сохраняются ваши точки останова и различные другие настроенные действия, например команды `display`, которые будут обсуждаться в главе 3. Если выйти из GDB, все это придется вводить заново.

Однако может понадобиться выйти из GDB до того, как отладка закончена. Если вы прекращаете работу на перерыв или до следующего дня и не можете оставаться вошедшим в компьютер, придется выйти из GDB. Чтобы не потерять команды для точек останова и других настроек, можно поместить их в startup-файл GDB, и тогда они будут автоматически загружаться при каждом запуске GDB.

Startup-файлы GDB по умолчанию называются `.gdbinit`. Один такой файл может находиться в домашнем каталоге для общих целей, а другой - в каталоге конкретного проекта для задач, специфичных для этого проекта. Например, команды установки точек останова следует помещать в startup-файл в последнем каталоге. В файле `.gdbinit` домашнего каталога можно хранить макросы общего назначения, которые вы разработали, как обсуждается в главе 2.

GDB читает startup-файл из домашнего каталога до загрузки исполняемого файла. Поэтому если в `.gdbinit` вашего домашнего каталога есть команда вроде

```
break g
```

означающая остановку на функции `g()`, GDB всегда будет жаловаться при запуске, что не знает такой функции. Однако та же строка будет вполне уместна в startup-файле локального каталога проекта, потому что локальный startup-файл читается после загрузки исполняемого файла и его таблицы символов. Обратите внимание: эта особенность GDB означает, что программные проекты лучше не размещать прямо в домашнем каталоге, поскольку тогда вы не сможете поместить проектно-специфичную информацию в `.gdbinit`.

Startup-файл можно указать в момент вызова GDB. Например,

```
$ gdb -command=z x
```

означало бы: запустить GDB для исполняемого файла `x`, сначала прочитав команды из файла `z`. Кроме того, поскольку DDD является лишь оболочкой для GDB, запуск DDD также приведет к использованию startup-файлов GDB.

Наконец, DDD можно настраивать различными способами, выбрав `Edit | Preferences`. Для Eclipse соответствующая последовательность - `Window | Preferences`.

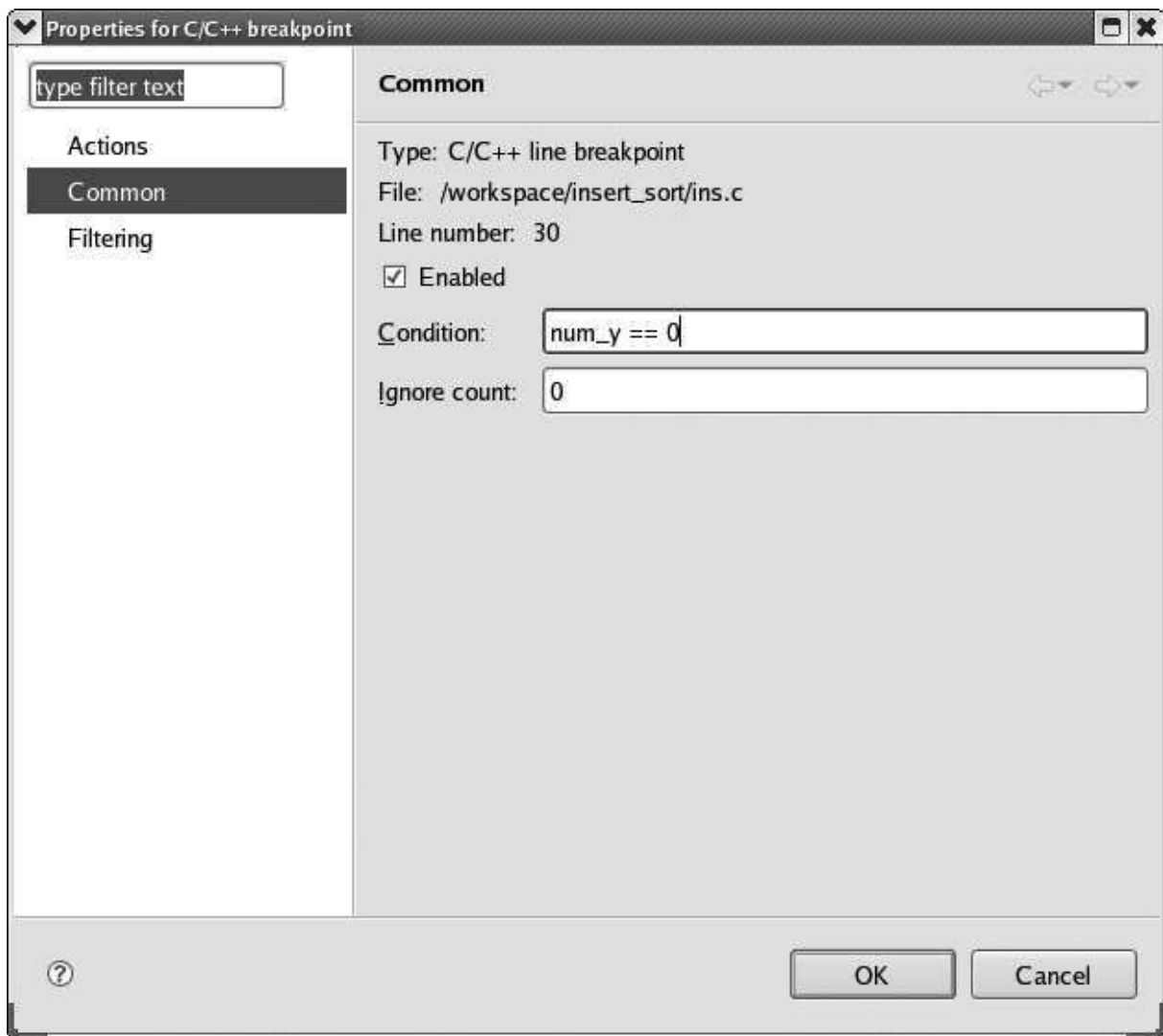


Рисунок 1-16. Создание условной точки останова

Сноски

[1] [\[назад\]](#) Уже пора начать использовать распространенные сокращения команд. В их число входят `b` для `break`, `ib` для `info break`, `cond` для `condition`, `r` для `run`, `n` для `next`, `s` для `step`, `c` для `continue`, `p` для `print` и `bt` для `backtrace`.

[2] [\[назад\]](#) Поскольку это книга об отладке, а не об управлении проектами, мы не будем много говорить здесь о создании и сборке проектов в Eclipse. Краткое резюме, однако, таково: проект создается следующим образом: выберите `File | New | Project`; выберите `C (или C++) Project`; заполните имя проекта; выберите `Executable | Finish`. Makefile создается автоматически. Проект собирается, то есть компилируется и компоуется, выбором `Project | Build Project`.

Глава 2. Остановка, чтобы осмотреться

Символьный отладчик вроде GDB может запускать вашу программу так же, как это можете делать вы. Однако благодаря волшебству включения отладочных символов в исполняемый файл отладчик создает иллюзию выполнения программы строка за строкой исходного кода, а не инструкция за инструкцией скомпилированного машинного кода. Именно этот, на вид скромный, факт и делает символьный отладчик настолько полезным при отладке программ.

Если бы отладчик умел только запускать программу, пользы от него было бы немного. Мы, конечно, могли бы сделать то же самое, причем эффективнее. Полезность отладчика в том, что мы можем приказывать ему приостановить выполнение программы. После остановки отладчик дает возможность изучить переменные, проследить путь выполнения и многое другое.

2.1 Механизмы остановки

Есть три способа приказывать GDB приостановить выполнение вашей программы:

- Breakpoint сообщает GDB, что нужно приостановить выполнение в определенном месте программы.
- Watchpoint сообщает GDB, что нужно приостановить выполнение, когда конкретная ячейка памяти или выражение, включающее одну или несколько ячеек, меняет значение.
- Catchpoint сообщает GDB, что нужно приостановить выполнение, когда происходит определенное событие.

Сначала это сбивает с толку, но в документации GDB все три механизма вместе называются breakpoints. Возможно, причина в том, что у них есть много общих атрибутов и команд. Например, вы узнаете о команде GDB delete, которая, как говорит справочная подсказка, удаляет breakpoint:

```
(gdb) help delete
Delete some breakpoints or auto-display expressions.
Arguments are breakpoint numbers with spaces in between.
To delete all breakpoints, give no argument.
```

Однако опытный пользователь GDB знает, что справочная подсказка на самом деле означает: команда delete удаляет breakpoints, watchpoints и catchpoints!

2.2 Обзор точек останова

Breakpoint похож на сигнальную растяжку внутри программы: вы устанавливаете breakpoint в определенном "месте" программы, и когда выполнение достигает этой точки, отладчик приостанавливает выполнение программы (а в случае текстового отладчика вроде GDB дает вам командное приглашение).

GDB очень гибко понимает слово "место": оно может означать самые разные вещи, например строку исходного кода, адрес кода, номер строки внутри исходного файла или вход в функцию.

Ниже показан фрагмент сеанса отладки, иллюстрирующий, что происходит, когда GDB останавливается на строке кода. Во фрагменте мы выводим часть исходного кода, ставим breakpoint на строке 35 программы, а затем запускаем программу. GDB достигает breakpoint и приостанавливается.


```

(gdb) list
30
31 /* Get the size of file in bytes */
32 if ((fd = open(c.filename, O_RDONLY)) == -1)
33 (void) die(1, "Can't open file.");
34 (void) stat(c.filename, &fstat);
35 c.filesize = fstat.st_size;
36
(gdb) break 35
Breakpoint 1 at 0x8048ff3: file bed.c, line 35.
(gdb) run
Starting program: binary_editor/bed

Breakpoint 1, main (argc=1, argv=0xbfa3e1f4) at bed.c:35
35 c.filesize = fstat.st_size;
(gdb)

```

Будем совершенно точны в том, что здесь произошло: GDB выполнил строки 30-34, но строка 35 еще не выполнена. Это может сбивать с толку, поскольку многие думают, что GDB показывает последнюю выполненную строку кода, тогда как на самом деле он показывает, какая строка кода будет выполнена следующей. В данном случае GDB сообщает, что строка 35 - следующая строка исходного кода для выполнения. Когда выполнение в GDB достигает breakpoint на строке 35, можно представить, что GDB сидит и ждет между строками 34 и 35 исходного кода.

Однако, как вы, возможно, знаете, GDB работает с инструкциями машинного языка, а не со строками исходного кода, и одной строке кода может соответствовать несколько строк машинного языка. GDB может работать со строками исходного кода благодаря дополнительной информации, включенной в исполняемый файл. Сейчас этот факт может казаться не слишком важным, но он будет иметь последствия, когда на протяжении этой главы мы будем обсуждать пошаговое выполнение программы.

2.3 Отслеживание точек останова

Каждому созданному breakpoint, включая breakpoints, watchpoints и catchpoints, назначается уникальный целочисленный идентификатор, начиная с 1. Этот идентификатор используется для выполнения различных операций с breakpoint. Отладчик также включает средство для вывода списка всех ваших breakpoints и их свойств.

2.3.1 Списки breakpoints в GDB

Когда вы создаете breakpoint, GDB сообщает назначенный ему номер. Например, breakpoint, установленный в этом примере:

```

(gdb) break main
Breakpoint 2 at 0x8048824: file efh.c, line 16.

```

получил номер 2. Если вы забудете, какой номер назначен какому breakpoint, можно напомнить себе это командой `info breakpoints`:

```
(gdb) info breakpoints
Num Type Disp Enb Address What
1 breakpoint keep y 0x08048846 in Initialize_Game at efh.c:26
2 breakpoint keep y 0x08048824 in main at efh.c:16
breakpoint already hit 1 time
3 hw watchpoint keep y efh.level
4 catch fork keep y
```

Мы увидим, что эти идентификаторы используются для выполнения разных операций с breakpoints. Чтобы сделать это более конкретным, уместен очень короткий пример.

В предыдущем разделе вы видели команду `delete`. Можно удалить breakpoint 1, watchpoint 3 и catchpoint 4 с помощью команды `delete` и идентификаторов этих breakpoints:

```
(gdb) delete 1 3 4
```

В следующих разделах вы увидите много других способов использования идентификаторов breakpoints.

2.3.2 Списки breakpoints в DDD

Пользователи DDD в основном выполняют операции управления breakpoints через интерфейс point-and-click, поэтому идентификаторы breakpoints для них менее важны, чем для пользователей GDB. При выборе Source | Breakpoints появляется окно Breakpoints and Watchpoints, в котором перечислены все ваши breakpoints, как показано на рисунке 2-1.

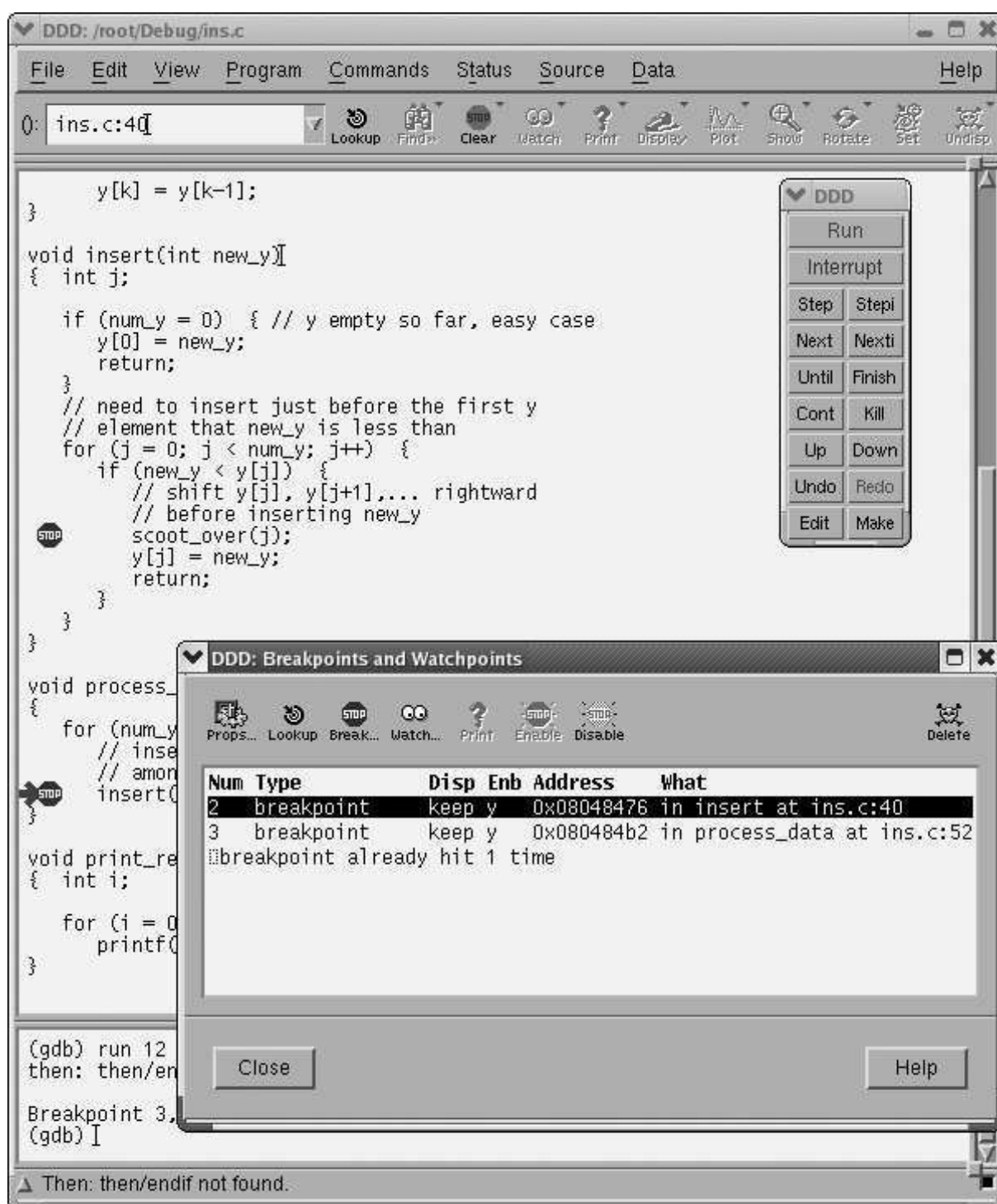


Рисунок 2-1. Просмотр breakpoints в DDD

Тем не менее помните, что DDD позволяет пользоваться как командным интерфейсом GDB, так и предоставленным GUI. В некоторых случаях GDB предоставляет операции с breakpoints, недоступные через GUI DDD, но пользователь DDD может получить доступ к этим специальным операциям GDB через Console DDD. В таких случаях идентификаторы breakpoints по-прежнему могут быть полезны пользователям DDD.

Обратите внимание, что при желании можно держать окно Breakpoints and Watchpoints постоянно открытым, перетаскив его в удобную часть экрана.

2.3.3 Списки breakpoints в Eclipse

Перспектива Debug включает view Breakpoints. Например, на рисунке 2-2 видно, что сейчас у вас есть две точки останова, на строках 30 и 52 файла `ins.c`.

Можно щелкнуть правой кнопкой мыши запись любого breakpoint, чтобы изучить или изменить его свойства. Кроме того, двойной щелчок по записи приведет к тому, что фокус окна исходного файла переместится к этому breakpoint.

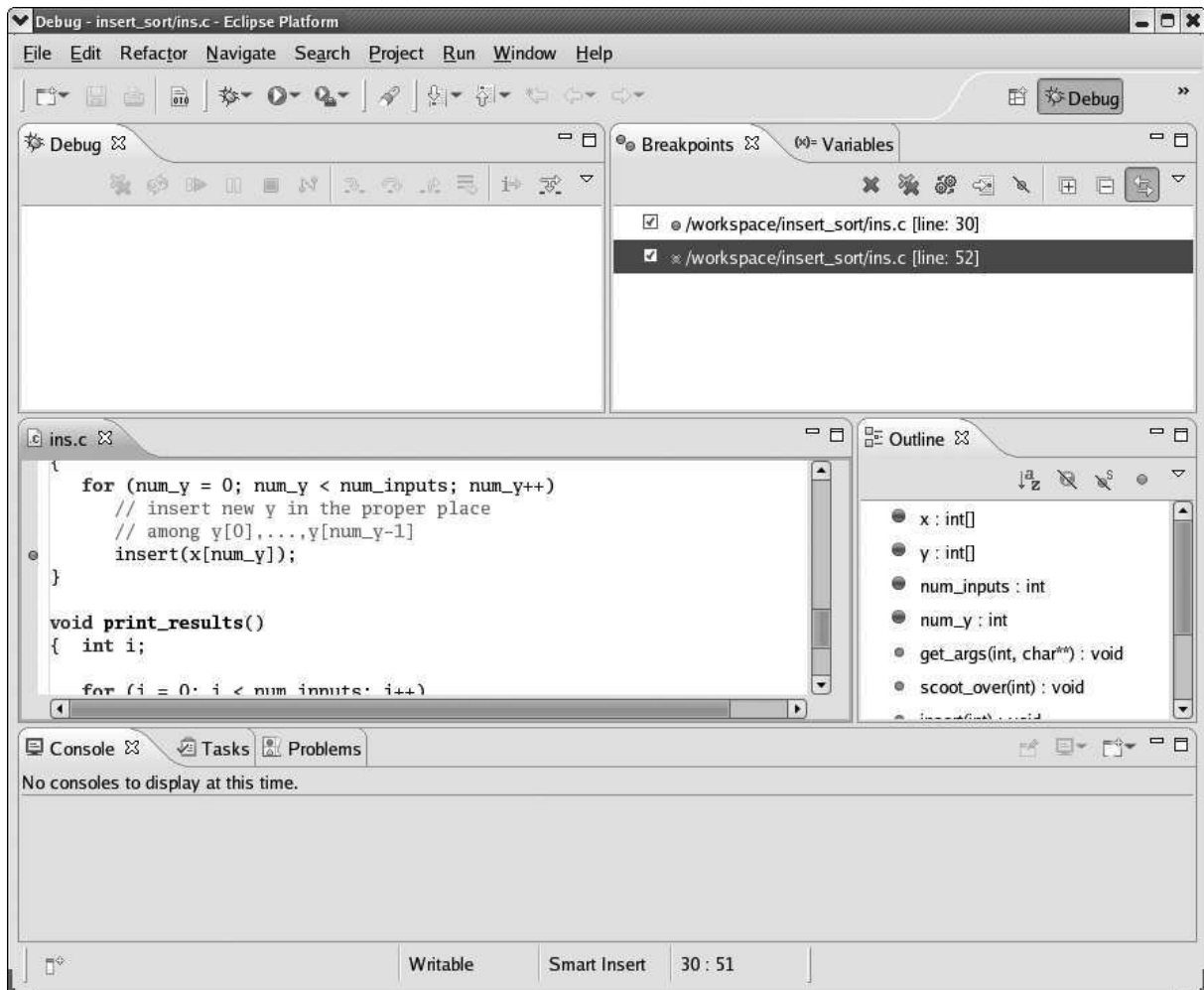


Рисунок 2-2. Просмотр breakpoints в Eclipse

2.4 Установка точек останова

Отладочные инструменты обычно предлагают множество механизмов для установки breakpoints.

2.4.1 Установка breakpoints в GDB

Вы узнали, что GDB создает иллюзию выполнения программы строка за строкой исходного кода. Вы также узнали, что для сколько-нибудь полезной работы с GDB нужно указать ему, где приостанавливать выполнение, чтобы можно было выполнять отладочные действия через командное приглашение GDB. В этом разделе вы узнаете, как устанавливать breakpoints, которые сообщают GDB, где останавливаться в вашем исходном коде.

В GDB есть много разных способов указать breakpoint; вот несколько наиболее распространенных методов:

`break function`

Установить breakpoint на входе, то есть на первой выполняемой строке, функции `function()`. Вы видели пример этого в разделе 2.3.1; команда

```
(gdb) break main
```

устанавливает breakpoint на входе в `main()`.

`break line_number`

Установить breakpoint на строке `line_number` текущего активного файла исходного кода. Для программ из нескольких файлов это либо файл, содержимое которого вы в последний раз просматривали командой `list`, либо файл, содержащий `main()`. Пример этого был в разделе 2.2:

```
(gdb) break 35
```

эта команда установила breakpoint на строке 35 файла `bed.c`.

`break filename:line_number`

Установить breakpoint на строке `line_number` исходного файла `filename`. Если `filename` не находится в текущем рабочем каталоге, можно указать относительный или полный путь, чтобы помочь GDB найти файл, например:

```
(gdb) break source/bed.c:35
```

`break filename:function`

Установить breakpoint на входе в функцию `function()` внутри файла `filename`. Использование этой формы может потребоваться для перегруженных функций или программ, где есть одноименные static-функции, например:

```
(gdb) break bed.c:parseArguments
```

Как мы увидим, когда breakpoint установлен, он остается действующим, пока вы не удалите его, не отключите его или не выйдете из GDB. Однако temporary breakpoint - это breakpoint, который автоматически удаляется после первого достижения. Temporary breakpoint устанавливается командой `tbreak`, принимающей аргументы того же типа, что и `break`. Например, `tbreak foo.c:10` устанавливает temporary breakpoint на строке 10 файла `foo.c`.

Нужно сделать замечание о функциях с одинаковыми именами. C++ позволяет перегружать функции, то есть определять функции с одним и тем же именем. Даже C позволяет делать это, если использовать квалификатор `static`, чтобы объявить функции с файловой областью видимости. Использование `break function` установит breakpoint на всех функциях с таким именем. Если вы хотите установить breakpoint на конкретном экземпляре функции, нужно устранить неоднозначность, например указав номер строки внутри исходного файла в команде `break`.

Место, в котором GDB фактически устанавливает breakpoint, может отличаться от того, куда вы попросили его поставить. Это может немного сбивать с толку.

Другие аргументы break

Есть и другие, возможно менее часто используемые, аргументы, которые можно применять с командой break.

- С помощью `break +offset` или `break -offset` можно установить breakpoint на offset строк впереди или позади выполняемой строки исходного кода в текущем выбранном stack frame.
- Форма `break *address` может использоваться для установки breakpoint на виртуальном адресе памяти. Это необходимо для участков программы, у которых нет отладочной информации, например когда исходный код недоступен или речь идет о shared libraries.

Поэтому посмотрим на короткий пример, демонстрирующий эту особенность. Рассмотрим следующую короткую программу test1.c:

```
1 int main(void)
2 {
3     int i;
4     i=3 ;
5
6     return 0;
7 }
```

Скомпилируйте эту программу без оптимизации и попробуйте установить breakpoint на входе в `main()`. Можно подумать, что breakpoint будет установлен в верхней части функции - на строке 1, строке 2 или строке 3. Это были бы хорошие предположения о местонахождении breakpoint, но они неверны. На самом деле breakpoint устанавливается на строке 4.

```
$ gcc -g3 -Wall -Wextra -o test1 test1.c
$ gdb test1
(gdb) break main
Breakpoint 1 at 0x6: file test1.c, line 4.
```

Строка 4 едва ли является первой строкой `main()`, так что же произошло? Как вы, возможно, догадались, одна из причин в том, что эта строка является исполняемой. Напомним: GDB на самом деле работает с инструкциями машинного языка, но благодаря расширенной таблице символов создает иллюзию работы со строками исходного кода. Обычно этот факт не слишком важен, но здесь он становится значимым. Вообще говоря, объявление `i` действительно генерирует машинный код, но это не тот код, который GDB считает полезным для наших целей отладки.^[1] Поэтому, когда вы сказали GDB остановиться в начале `main()`, он установил breakpoint на строке 4.

Другие типы breakpoints

Кроме того, есть еще несколько команд в стиле `break`, которые эта книга не будет рассматривать подробно, поскольку они довольно специализированные.

Команда `hbreak` устанавливает hardware assisted breakpoint. Это breakpoint, который можно установить в ячейке памяти без изменения содержимого этой ячейки. Для этого требуется аппаратная поддержка, и главным образом это полезно при отладке EEPROM/ROM. Кроме того, есть команда `thbreak`, которая устанавливает temporary, hardware-assisted breakpoint. Команды `hbreak` и `thbreak` принимают аргументы того же типа, что `break` и `tbreak`.

Команда `rbreak` принимает регулярное выражение в стиле `grep` и устанавливает breakpoint на входе в любую функцию, соответствующую этому регулярному выражению. При работе с `rbreak` нужно помнить две вещи. Во-первых, важно помнить, что `rbreak` использует регулярные выражения в стиле `grep`, а не регулярные выражения в стиле Perl и не shell-style globbing. Это означает, например, что `rbreak func*` поставит breakpoints на функции с именами `func` и `func()`, но не поставит breakpoint на `function()`. Во-вторых, перед вашим аргументом к `rbreak` и после него неявно стоит `*`, поэтому если вы не хотите, чтобы `rbreak func` устанавливал breakpoint на `afunc()`, следует использовать `rbreak ^func`.

Проблема может стать хуже, если вы скомпилируете программу с оптимизациями. Давайте посмотрим. Перекомпилируйте программу с включенной оптимизацией:

```
$ gcc -O9 -g3 -Wall -Wextra -o test1 test1.c
$ gdb test1
(gdb) break main
Breakpoint 1 at 0x3: file test1.c, line 6.
```

Мы попросили поставить breakpoint в начале `main()`, но GDB поставил его на последней строке `main()`. Что произошло теперь? Ответ тот же, что и раньше, но GCC сыграл более активную роль. При включенной оптимизации GCC заметил, что хотя `i` присваивается значение, оно никогда не используется. Поэтому, стремясь сгенерировать более эффективный код, GCC просто оптимизировал строки 3 и 4 до исчезновения. GCC не создал машинных инструкций для этих строк. Следовательно, первой строкой исходного кода, породившей машинные инструкции, оказалась последняя строка `main()`. Это одна из причин, по которым не следует оптимизировать код, пока вы не закончили его отлаживать. ^[2]

Catchpoints

Программистам C++ стоит изучить команду `catch`, которая устанавливает catchpoints. Catchpoints похожи на breakpoints, но могут срабатывать по разным причинам: выброшенные исключения, пойманные исключения, сигналы, вызовы `fork()`, загрузка и выгрузка библиотек и многие другие события.

В этой книге мы рассмотрим breakpoints и watchpoints, а за дополнительной информацией о catchpoints отсылаем читателя к документации GDB.

Итог всего этого таков: если вы обнаружите, что установка breakpoint не создает breakpoint точно там, где вы его ожидали, теперь вы знаете почему. Не удивляйтесь.

В связи с этим мы хотели бы упомянуть, что происходит, когда более одного breakpoint находится на одной и той же строке исходного кода. Когда GDB останавливается на строке исходного кода с несколькими breakpoints, он остановится там только один раз. Иными словами, после достижения этой строки кода, если вы возобновите выполнение, другие breakpoints, которые случайно находятся на той же строке, будут проигнорированы. На самом деле GDB отслеживает, какой breakpoint "сработал" и остановил выполнение программы. На строке кода с несколькими breakpoints breakpoint, вызвавший остановку, будет breakpoint с наименьшим идентификатором.

2.4.2 Установка breakpoints в DDD

Чтобы установить breakpoints в DDD, найдите строку кода, на которой хотите поставить breakpoint, в Source Window. Поместите курсор над любым пустым пространством на этой строке и щелкните правой кнопкой мыши, чтобы открыть всплывающее меню. Перетащите мышью вниз, пока не будет выделен пункт `Set Breakpoint`, затем отпустите кнопку мыши. Рядом со строкой кода, где вы установили breakpoint, должен появиться красный знак остановки. Если вы не делаете с breakpoint ничего специального, например не делаете его условным (это будет обсуждаться в разделе 2.10), можно использовать сокращение: просто дважды щелкнуть нужную строку.

Если вы экспериментировали с DDD, то могли заметить, что при нажатии правой кнопки рядом со строкой кода всплывающее меню содержит пункт `Set Temporary Breakpoint`. Так в DDD устанавливается temporary breakpoint, то есть breakpoint, исчезающий после первого достижения; при этом вызывается команда GDB `tbreak`.

Еще раз: не забывайте, что DDD на самом деле является оболочкой GDB. В DDD можно выдавать любые команды `break` в стиле GDB, используя Console Window DDD. Иногда это желательно; если у вас очень большая программа или программа из нескольких файлов, может быть удобнее выдать `breakpoint`, используя семантику GDB. Более того, иногда это необходимо, потому что не все команды GDB для `breakpoints` доступны из интерфейса DDD.

2.4.3 Установка `breakpoints` в Eclipse

Чтобы установить `breakpoint` на заданной строке в Eclipse, дважды щелкните эту строку. Появится символ `breakpoint`, как, например, на строке

```
insert(x[num_y]);
```

на рисунке 2-2.

Чтобы установить `temporary breakpoint`, щелкните строку, затем щелкните правой кнопкой мыши в окне исходного кода и выберите `Run to Line`. Однако обратите внимание: операция `Run to Line` работает только если целевая строка находится в той же функции, что и ваша текущая позиция, и если вы не выходите из функции до повторного входа и достижения этой строки.

2.5 Расширенный пример GDB

Информации было много, поэтому уместен короткий пример установки `breakpoints`, который можно выполнить самостоятельно. Рассмотрим следующий многофайловый код C:

`main.c`:

```
#include <stdio.h>
void swap(int *a, int *b);
int main(void)
{
    int i = 3;
    int j = 5;
    printf("i: %d, j: %d\n", i, j);
    swap(&i, &j);
    printf("i: %d, j: %d\n", i, j);
    return 0;
}
```

`swapper.c`:

```
void swap(int *a, int *b)
{
    int c = *a;
    *a = *b;
    *b = c;
}
```


Скомпилируйте этот код и запустите GDB для исполняемого файла:

```
$ gcc -g3 -Wall -Wextra -c main.c swapper.c
$ gcc -o swap main.o swapper.o
$ gdb swap
```

NB. Это первый раз в книге, когда мы компилируем многофайловую C-программу, поэтому нужно сказать несколько слов. Первая строка процесса компиляции выше создает два объектных файла, содержащих неразрешенный объектный код с отладочной информацией. Вторая строка связывает объектные файлы в исполняемый файл, содержащий всю отладочную информацию. Использовать параметр GCC -g во время процесса компоновки не нужно.

Установка breakpoint на `main()` очень распространена при начале сеанса отладки. Это устанавливает breakpoint на первой строке этой функции. ^[3]

```
(gdb) break main
Breakpoint 1 at 0x80483f6: file main.c, line 6.
```

Все приведенные ниже примеры устанавливают breakpoint на первой строке функции `swap()`. Хотя они выглядят по-разному, все они делают одно и то же: останавливаются в начале `swap()`.

```
(gdb) break swapper.c:1
Breakpoint 2 at 0x8048454: file swapper.c, line 1.
(gdb) break swapper.c:swap
Breakpoint 3 at 0x804845a: file swapper.c, line 3.
(gdb) break swap
Note: breakpoint 3 also set at pc 0x804845a.
Breakpoint 4 at 0x804845a: file swapper.c, line 3.
```

В любой момент у GDB есть, за неимением лучшего слова, фокус, который можно понимать как текущий "активный" файл. Это означает, что если вы не уточняете команды, они выполняются по отношению к файлу, находящемуся в фокусе GDB. По умолчанию начальный фокус GDB находится на файле, содержащем функцию `main()`, но фокус меняется на другой файл, когда происходит любое из следующих действий:

- Вы применяете команду `list` к другому исходному файлу.
- Вы входите пошаговым выполнением в код, находящийся в другом исходном файле.
- GDB достигает breakpoint при выполнении кода в другом исходном файле.

Рассмотрим пример. Хотя breakpoints были установлены в `swapper.c`, мы еще не выводили код из этого файла. Следовательно, фокус все еще направлен на `main.c`. Это можно проверить, установив breakpoint на строке 6. Когда вы устанавливаете этот breakpoint без имени файла, GDB поставит breakpoint на строке 6 текущего активного файла:

```
(gdb) break 6
Breakpoint 5 at 0x8048404: file main.c, line 6.
```

Действительно, `main.c` находится в фокусе: когда вы запускаете GDB, breakpoint, заданный только номером строки, устанавливается внутри файла, содержащего `main()`. Фокус можно изменить, выведя код из `swapper.c`:

```
(gdb) list swap
1 void swap(int *a, int *b)
2 {
3     int tc = *a;
4     *a = *b;
5     *b = tc;
6 }
```

Проверим, что теперь в фокусе `swapper.c`, попробовав установить еще один breakpoint на строке 6:

```
(gdb) break 6
Breakpoint 6 at 0x8048474: file swapper.c, line 6.
```

Да, breakpoint был установлен на строке 6 файла `swapper.c`. Затем вы установите temporary breakpoint на строке 4 файла `swapper.c`:

```
(gdb) tbreak swapper.c:4
Breakpoint 7 at 0x8048462: file swapper.c, line 4.
```

Наконец, используйте команду `info breakpoints`, представленную в разделе 2.3.1, чтобы полюбоваться всеми установленными breakpoints:

```
(gdb) info breakpoints
Num Type Disp Enb Address What
1 breakpoint keep y 0x080483f6 in main at main.c:6
2 breakpoint keep y 0x08048454 in swap at swapper.c:1
3 breakpoint keep y 0x0804845a in swap at swapper.c:3
4 breakpoint keep y 0x0804845a in swap at swapper.c:3
5 breakpoint keep y 0x08048404 in main at main.c:9
6 breakpoint keep y 0x08048474 in swap at swapper.c:6
7 breakpoint del y 0x08048462 in swap at swapper.c:4
```

Гораздо позже, когда вы закончите сеанс GDB, используйте команду `quit`, чтобы выйти из GDB:

```
(gdb) quit
$
```

2.6 Сохранение breakpoints

Выше мы сказали "гораздо позже", чтобы подчеркнуть: во время сеанса отладки не следует выходить из GDB. Например, когда вы нашли и исправили одну ошибку, но другие ошибки остались, не нужно выходить из GDB и снова входить в него, чтобы использовать новую версию программы. Это лишние хлопоты, а главное, вам пришлось бы заново вводить breakpoints.

Если не выходить из GDB после изменения и перекомпиляции кода, при следующей выдаче команды GDB `run` GDB обнаружит, что код изменился, и автоматически перезагрузит новую версию.

Однако обратите внимание, что ваши breakpoints могут "сдвинуться". Например, рассмотрим следующую простую программу:

```
1 main()
2 { int x,y;
3   x=1 ;
4   y=2 ;
5 }
```

Мы компилируем ее, входим в GDB и устанавливаем breakpoint на строке 4:

```
(gdb) l
1 main()
2 { int x,y;
3   x = 1;
4   y = 2;
5 }
(gdb) b 4
Breakpoint 1 at 0x804830b: file a.c, line 4.
(gdb) r
Starting program: /usr/home/matloff/Tmp/tmp1/a.out
Breakpoint 1, main () at a.c:4
4y = 2;
```

Пока все хорошо. Но предположим, что теперь вы добавляете строку исходного кода:

```
1 main()
2 { int x,y;
3 x=1 ;
4 x++;
5 y=2 ;
6 }
```

Затем перекомпилируете - снова помните, что вы не выходили из GDB, - и снова выдаете команду GDB run:

```
(gdb) r
The program being debugged has been started already.
Start it from the beginning? (y or n) y
`/usr/home/matloff/Tmp/tmp1/a.out' has changed; re-reading symbols.
Starting program: /usr/home/matloff/Tmp/tmp1/a.out
Breakpoint 1, main () at a.c:4
4 x++;
```

GDB действительно перезагрузил новый код, но breakpoint вроде бы сдвинулся с оператора:

```
y=2 ;
```

на

```
x++;
```

Если присмотреться внимательнее, видно, что breakpoint на самом деле вообще не перемещался: он был на строке 4 и по-прежнему находится на строке 4. Но эта строка больше не содержит оператор, на котором вы изначально установили breakpoint. Поэтому breakpoint нужно переместить, удалив этот и поставив новый. В DDD это можно сделать намного проще; см. раздел 2.7.5.

В конце концов текущий сеанс отладки закончится, например потому что пора есть, спать или отдыхать. Если обычно вы не держите компьютер постоянно включенным, вам придется выйти из отладчика. Есть ли способ сохранить breakpoints?

Для GDB и DDD ответ в некоторой степени положительный. Breakpoints можно поместить в startup-файл .gdbinit в каталоге, где находится исходный код, или в каталоге, из которого вы вызываете GDB.

Если вы работаете в Eclipse, вам повезло, потому что все ваши breakpoints будут автоматически сохранены и восстановлены в следующем сеансе Eclipse.

2.7 Удаление и отключение breakpoints

В ходе сеанса отладки может оказаться, что breakpoint уже исчерпал свою полезность. Если вы уверены, что breakpoint больше не понадобится, его можно удалить.

Может быть и так, что, по вашему мнению, breakpoint пригодится позже в этом же сеансе отладки. Возможно, вы предпочитаете не удалять breakpoint, а настроить все так, чтобы пока отладчик пропускал остановки в этой точке кода. Это называется отключением breakpoint. Позже, если и когда он снова станет полезен, его можно включить.

Этот раздел посвящен удалению и отключению breakpoints. Все сказанное относится и к watchpoints.

2.7.1 Удаление breakpoints в GDB

Если рассматриваемый breakpoint действительно больше не нужен (возможно, конкретная ошибка была исправлена!), его можно удалить. В GDB для удаления breakpoints используются две команды. Команда `delete` удаляет breakpoints по их идентификаторам, а команда `clear` удаляет breakpoints с использованием того же синтаксиса, который применяется для создания breakpoints, как обсуждалось в разделе 2.4.1.

```
delete breakpoint_list
```

Удаляет breakpoints по их числовым идентификаторам, которые были объяснены в разделе 2.3. Это может быть одно число, например `delete 2`, удаляющее второй breakpoint, или список чисел, например `delete 2 4`, удаляющий второй и четвертый breakpoints.

```
delete
```

Удаляет все breakpoints. GDB попросит подтвердить эту операцию, если только вы не выдали команду `set confirm off`, которую также можно поместить в startup-файл `.gdbinit`.

```
clear
```

Очищает breakpoint на следующей инструкции, которую выполнит GDB. Этот метод полезен, когда вы хотите удалить breakpoint, которого GDB только что достиг.

- `clear function`
- `clear filename:function`
- `clear line_number`
- `clear filename:line_number`

Эти формы очищают breakpoint по его местоположению и работают аналогично соответствующим формам `break`.

Например, предположим, что вы установили breakpoint на входе в `foo()` с помощью:

```
(gdb) break foo
Breakpoint 2 at 0x804843a: file test.c, line 22.
```

Этот breakpoint можно удалить либо так:

```
(gdb) clear foo
Deleted breakpoint 2
```

либо так:

```
(gdb) delete 2
Deleted breakpoint 2
```

2.7.2 Отключение breakpoints в GDB

Каждый breakpoint может быть включен или отключен. GDB приостановит выполнение программы только при достижении включенного breakpoint; отключенные breakpoints он игнорирует. По умолчанию breakpoints создаются включенными.

Зачем может понадобиться отключать breakpoints? В ходе сеанса отладки может накопиться большое число breakpoints. Для циклических структур или часто повторяющихся функций частые остановки GDB могут быть крайне неудобны. Если вы хотите сохранить breakpoints для дальнейшего использования, но пока не хотите, чтобы GDB останавливал выполнение, их можно отключить и позже включить снова.

Breakpoint отключается командой `disable breakpoint-list` и включается командой `enable breakpoint-list`, где `breakpoint-list` - это разделенный пробелами список одного или нескольких идентификаторов breakpoints. Например:

```
(gdb) disable 3
```

отключит третий breakpoint. Аналогично:

```
(gdb) enable 1 5
```

включит первый и пятый breakpoints.

Выдача команды `disable` без аргументов отключит все существующие breakpoints. Аналогично команда `enable` без аргументов включит все существующие breakpoints.

Есть также команда `enable once`, которая заставляет breakpoint стать отключенным после следующего раза, когда он остановит выполнение GDB. Синтаксис таков:

```
enable once breakpoint-list
```

Например, `enable once 3` приведет к тому, что breakpoint 3 станет отключенным в следующий раз, когда заставит GDB остановить выполнение программы. Это очень похоже на команду `tbreak`, но при достижении breakpoint отключается, а не удаляется.

2.7.3 Удаление и отключение breakpoints в DDD

Удалять breakpoints в DDD так же просто, как устанавливать их. Поместите курсор на красный знак остановки и щелкните правой кнопкой мыши, как при установке breakpoint. Одним из пунктов всплывающего меню будет `Delete Breakpoint`. Удерживая правую кнопку мыши, перетащите мышь вниз, пока этот пункт не будет выделен. Затем отпустите кнопку, и вы увидите, что красный знак остановки исчез, показывая, что breakpoint удален.

Отключение breakpoints в DDD очень похоже на их удаление. Щелкните правой кнопкой мыши красный знак остановки и удерживайте кнопку, затем выберите `Disable Breakpoint`. Красный знак остановки станет серым, показывая, что breakpoint все еще существует, но пока отключен.

Другой вариант - использовать окно `Breakpoints and Watchpoints DDD`, показанное на рисунке 2-1. В нем можно щелкнуть запись breakpoint, чтобы выделить ее, а затем выбрать `Delete`, `Disable` или `Enable`.

На самом деле в этом окне можно выделить несколько записей, протащив по ним мышь, как видно на рисунке 2-3. Так можно удалить, отключить или включить сразу несколько breakpoints.

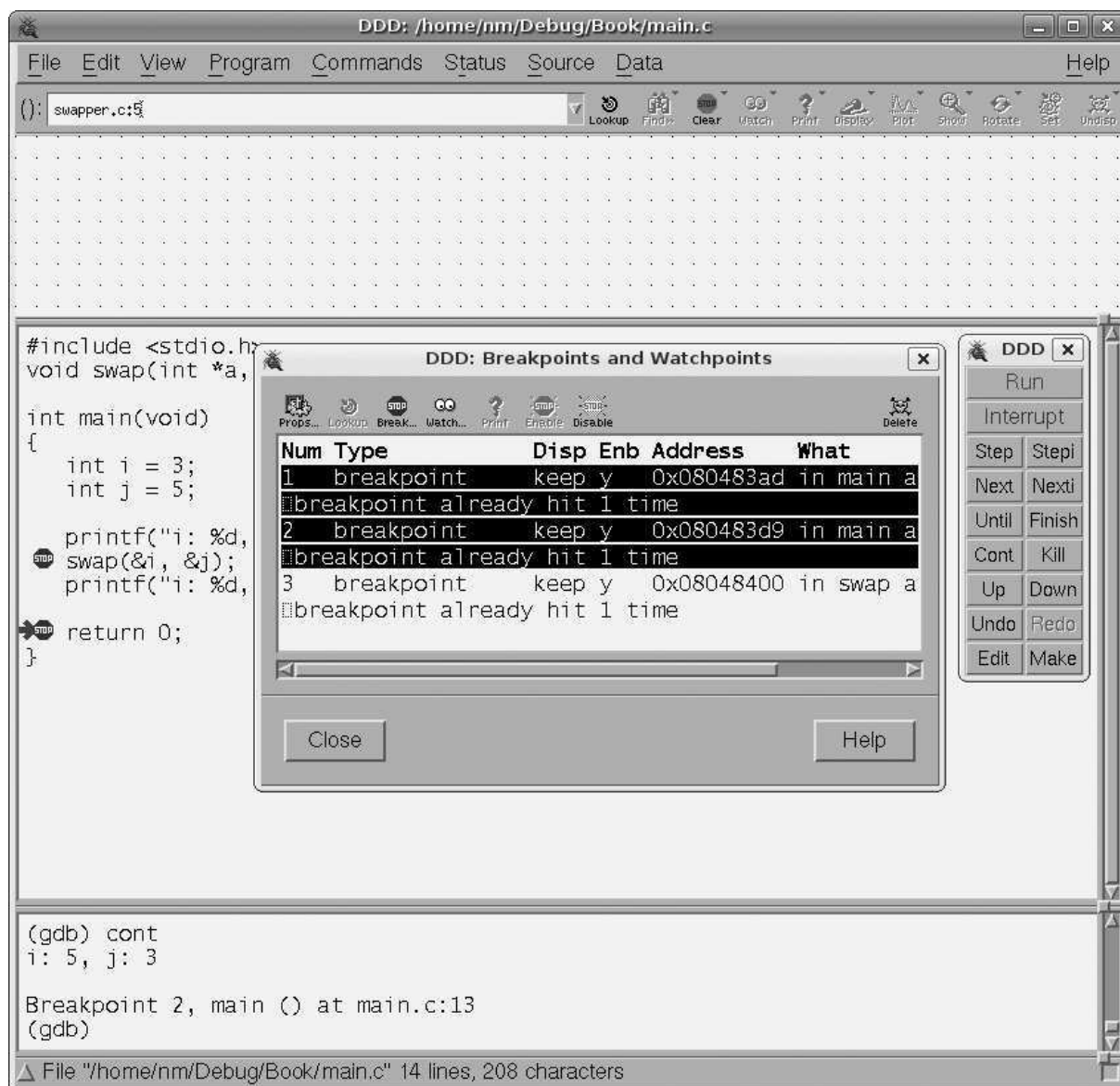


Рисунок 2-3. Удаление, отключение и включение нескольких breakpoints в DDD

После щелчка Delete в окне breakpoints экран выглядит как на рисунке 2-4. Действительно, две старые точки останова теперь исчезли.

2.7.4 Удаление и отключение breakpoints в Eclipse

Как и в DDD, в Eclipse можно удалить или отключить breakpoint, щелкнув правой кнопкой мыши символ breakpoint на соответствующей строке. Появится меню, показанное на рисунке 2-5. Обратите внимание: пункт Toggle означает удаление breakpoint, тогда как Disable/Enable означает очевидное.

Подобно окну breakpoints в DDD, Eclipse имеет view Breakpoints, который можно увидеть в правой верхней части рисунка 2-5. В отличие от случая DDD, где окно Breakpoints and Watchpoints нужно было вызвать, view Breakpoints в Eclipse автоматически отображается в перспективе Debug. Его, впрочем, можно скрыть, если вам не хватает места на экране, щелкнув X в его правом верхнем углу. Если позже захотите вернуть его, выберите Window | Show Views | Breakpoints.

Одна приятная сторона view Breakpoints в Eclipse состоит в том, что можно щелкнуть значок с двойным X, Remove All Breakpoints. Такая необходимость возникает чаще, чем можно подумать. В какой-то момент долгого сеанса отладки может оказаться, что ни один из ранее установленных breakpoints больше не полезен, и тогда вы захотите удалить их все.

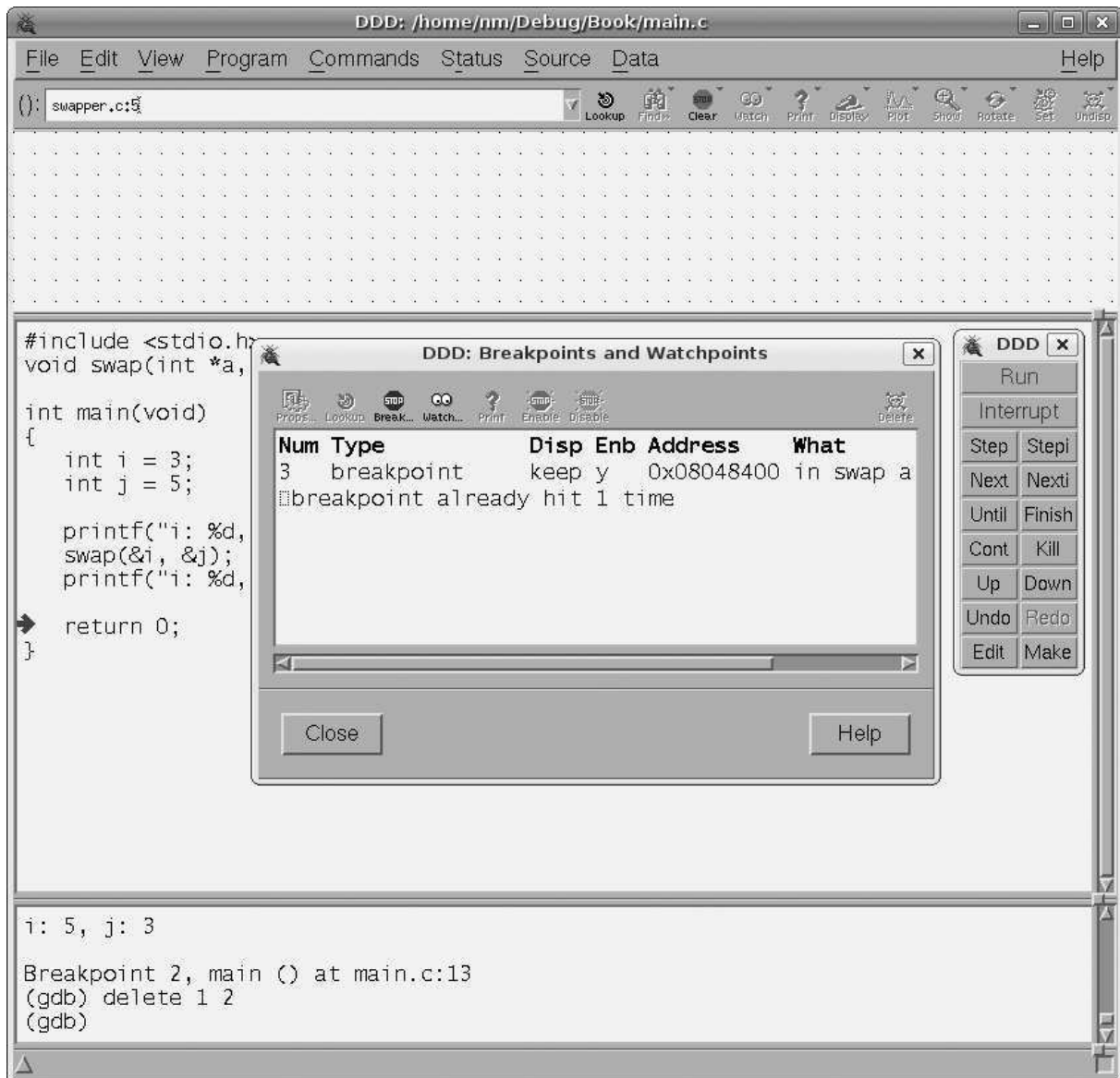


Рисунок 2-4. Две точки останова удалены

2.7.5 "Перемещение" breakpoints в DDD

У DDD есть еще одна действительно приятная возможность: breakpoints можно перетаскивать. Щелкните левой кнопкой мыши знак остановки breakpoint в Source Window и удерживайте кнопку. Пока левая кнопка нажата, breakpoint можно перетащить в другое место исходного кода. "За кулисами" происходит следующее: DDD удаляет исходный breakpoint и устанавливает новый breakpoint с теми же атрибутами. В результате новый breakpoint полностью эквивалентен старому, за исключением числового идентификатора. Конечно, это можно сделать и в GDB, но DDD ускоряет процесс.

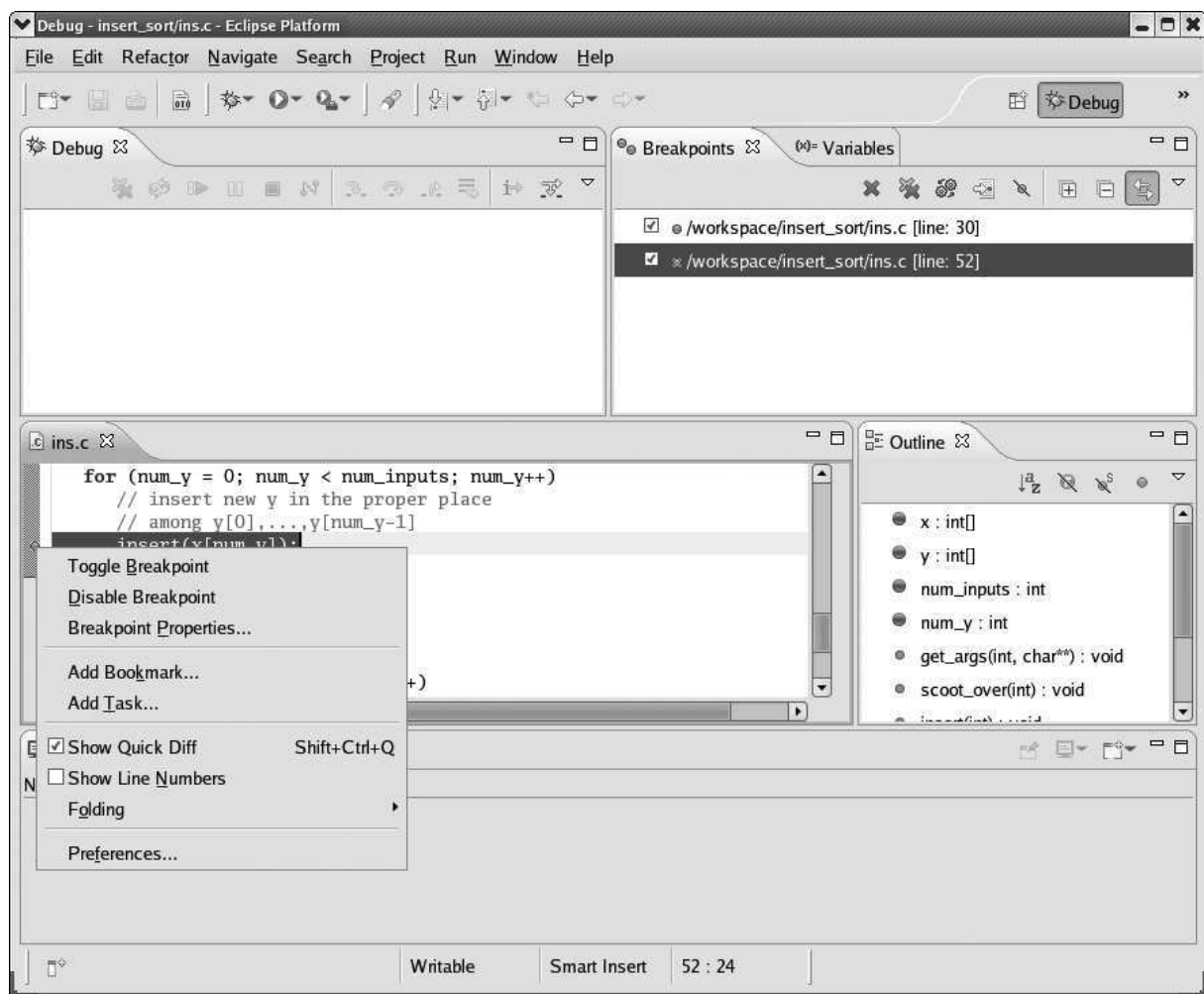


Рисунок 2-5. Удаление и отключение breakpoints в Eclipse

Рисунок 2-6 иллюстрирует это: это снимок, сделанный в середине операции перемещения breakpoint. Breakpoint находился на строке

```
if (new_y < y[i]) {
```

и мы хотели переместить его на строку

```
scoot_over(j);
```

Мы щелкнули знак остановки на старой строке и перетаскивали его на новую строку. На рисунке мы еще не отпустили кнопку мыши, поэтому исходный знак остановки все еще был на месте, а на новой строке появился "пустой" знак остановки. Когда мы отпустили кнопку мыши, знак остановки на старой строке исчез, а знак на новой строке заполнился.

Почему это так полезно? Во-первых, если вы находитесь в ситуации, когда хотите удалить один breakpoint и добавить другой, это выполняет обе операции одним махом. Но что важнее, как было отмечено ранее в разделе 2.6, когда вы добавляете или удаляете строки исходного кода, номера некоторых оставшихся строк меняются, тем самым "сдвигая" существующие breakpoints на строки, где вы не собирались их иметь. В GDB это вынуждает выполнить действие удаления breakpoint и создания нового breakpoint для каждого затронутого breakpoint. Это утомительно, особенно если к некоторым из них были привязаны условия.

Но в DDD можно просто перетаскивать значок stop sign на новые места - очень приятно и удобно.

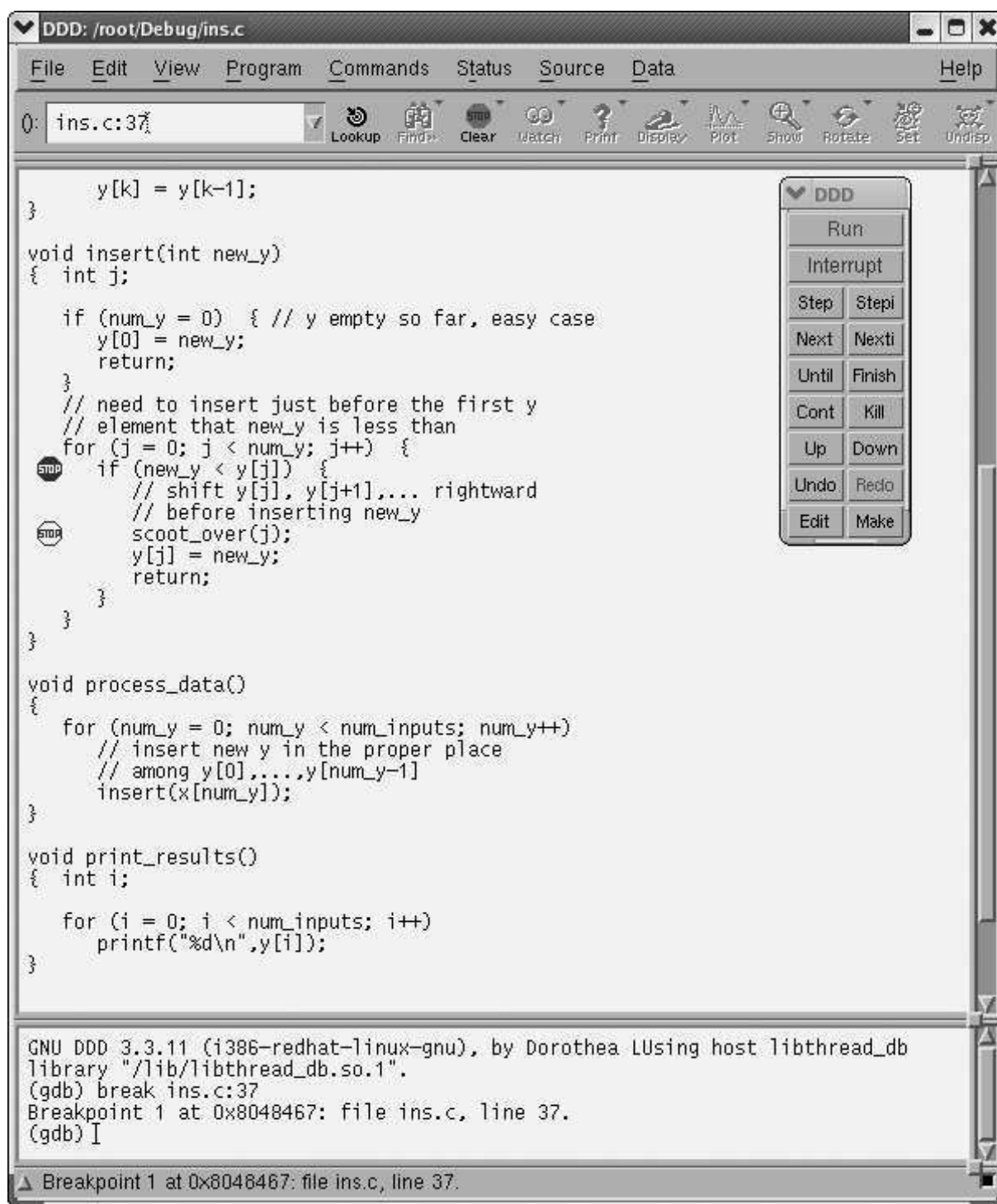


Рисунок 2-6. "Перемещение" breakpoint в DDD

2.7.6 Undo/Redo действий с breakpoints в DDD

Одна из действительно приятных возможностей DDD - Undo/Redo, доступная через щелчок Edit. В качестве примера рассмотрим ситуацию на рисунках 2-3 и 2-4. Предположим, вы внезапно понимаете, что не хотели удалять эти два breakpoints, а, возможно, хотели лишь отключить их. Можно выбрать Edit | Undo Delete, как показано на рисунке 2-7. Можно также щелкнуть Undo в Command Tool, но путь через Edit имеет преимущество: DDD напомним нам, что именно будет отменено.

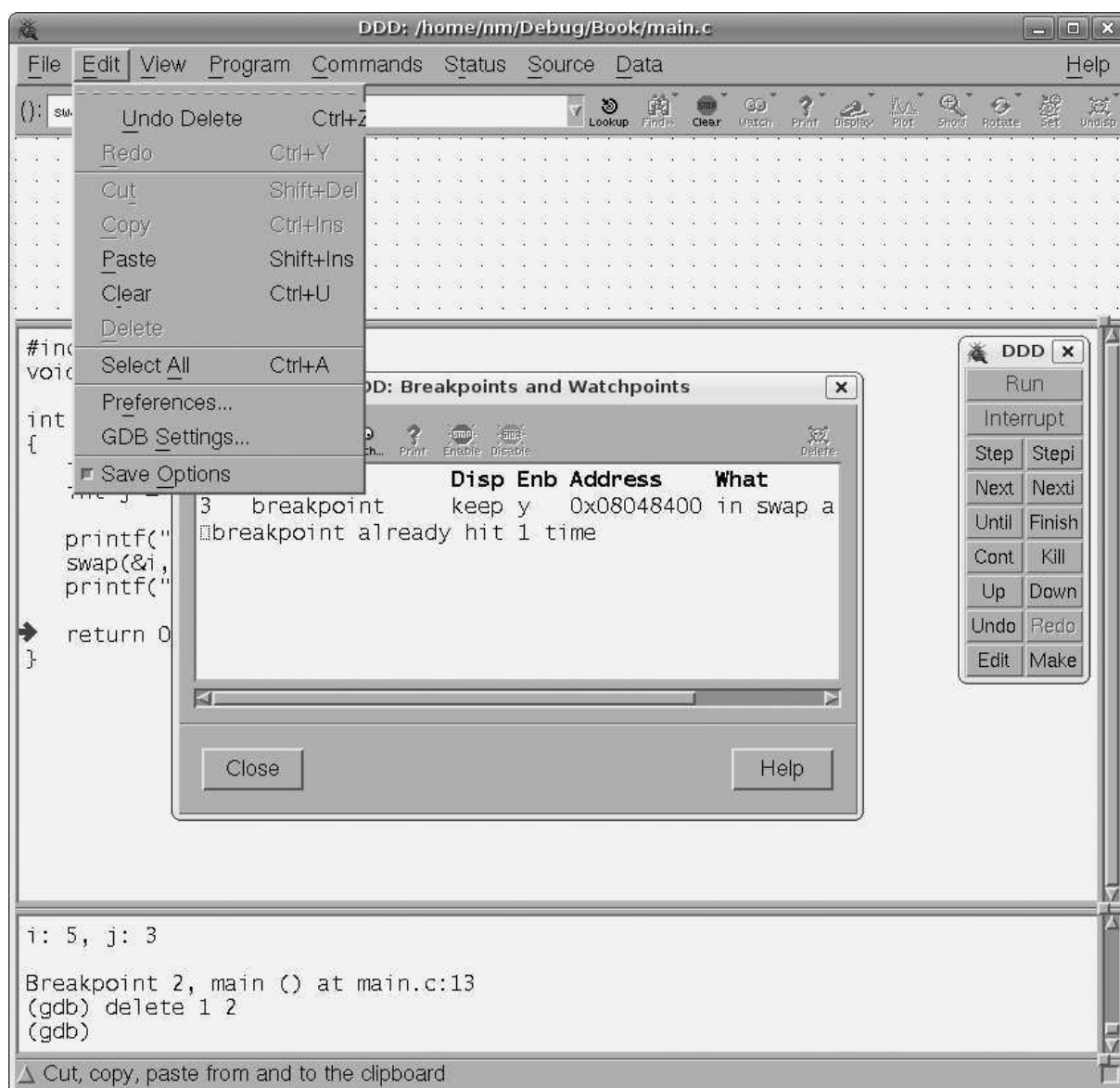


Рисунок 2-7. Возможность восстановить два breakpoints в DDD

2.8 Подробнее о просмотре атрибутов breakpoints

У каждого breakpoint есть различные атрибуты: номер строки, наложенное на него условие (если оно есть), текущий статус enabled/disabled и так далее. В разделе 2.3 мы немного показали, как отслеживать эти атрибуты, а теперь рассмотрим это подробнее.

2.8.1 GDB

Как вы видели в разделе 2.3.1, каждому созданному breakpoint назначается уникальный целочисленный идентификатор. Первому установленному breakpoint назначается 1, а каждому последующему новому breakpoint назначается целое число на единицу больше предыдущего назначенного идентификатора. У каждого breakpoint также есть несколько атрибутов, которые управляют его поведением и уточняют его. Используя уникальные идентификаторы, можно настраивать атрибуты каждого breakpoint отдельно.

Команду `info breakpoints`, сокращенно `ib`, можно использовать для получения списка всех установленных breakpoints вместе с их атрибутами. Вывод `info breakpoints` будет выглядеть примерно так:

```
(gdb) info breakpoints
Num Type Disp Enb Address What

1 breakpoint keep y 0x08048404 in main at int_swap.c:9
  breakpoint already hit 1 time
2 breakpoint keep n 0x08048447 in main at int_swap.c:14
3 breakpoint keep y 0x08048460 in swap at int_swap.c:20
  breakpoint already hit 1 time
4 hw watchpoint keep y counter
```

Рассмотрим этот вывод `info breakpoints` подробнее:

Identifier (Num)

Уникальный идентификатор breakpoint.

Type (Type)

Это поле сообщает, является ли breakpoint обычным breakpoint, watchpoint или catchpoint.

Disposition (Disp)

У каждого breakpoint есть disposition, указывающий, что произойдет с breakpoint после следующего раза, когда он приостановит выполнение программы в GDB. Возможны три disposition:

`keep` - breakpoint останется без изменений после следующего достижения. Это disposition по умолчанию для вновь созданных breakpoints.

`del` - breakpoint будет удален после следующего достижения. Это disposition назначается любому breakpoint, созданному командой `tbreak` (см. раздел 2.4.1).

`dis` - breakpoint будет отключен при следующем достижении. Это задается командой `enable once` (см. раздел 2.7.2).

Enable Status (Enb)

Это поле сообщает, включен breakpoint в данный момент или отключен.

Address (Address)

Это место в памяти, где установлен breakpoint. Оно в основном полезно программистам на языке ассемблера или тем, кто пытается отлаживать исполняемый файл, скомпилированный без расширенной таблицы символов.

Location (What)

Как обсуждалось, каждый breakpoint находится на определенной строке исходного кода. Поле `What` показывает номер строки и имя файла местонахождения breakpoint.

Для watchpoints это поле показывает, за какой переменной ведется наблюдение. Это логично, потому что переменная фактически является адресом памяти с именем, а адрес памяти - это местоположение.

Как видите, помимо перечисления всех breakpoints и их атрибутов, команда `ib` также сообщает, сколько раз конкретный breakpoint уже заставлял GDB останавливать выполнение программы. Если, например, у вас есть breakpoint внутри цикла, это сразу покажет, сколько итераций цикла уже выполнено, что может быть очень полезно.

2.8.2 DDD

Как вы видели на рисунке 2-1, окно Breakpoints and Watchpoints в DDD предоставляет ту же информацию, что и команда GDB `info breakpoints`. Однако оно удобнее, чем GDB, поскольку это окно можно отображать постоянно, например сбоку экрана, тем самым избегая выдачи команды каждый раз, когда вы хотите просмотреть breakpoints.

2.8.3 Eclipse

И снова, как было видно ранее (рисунок 2-2), view Breakpoints в Eclipse постоянно показывает ваши breakpoints и их свойства. Здесь Eclipse немного менее информативен, чем DDD, поскольку не сообщает, сколько раз breakpoint уже был достигнут; этой информации нет даже в окне Properties.

2.9 Возобновление выполнения

Знать, как приказать отладчику, где или когда приостанавливать выполнение программы, важно, но не менее важно знать, как приказать ему возобновить выполнение. В конце концов, просмотра переменных может быть недостаточно. Иногда нужно знать, как значения переменных взаимодействуют с остальным кодом.

Вспомните принцип подтверждения из главы 1: вы продолжаете подтверждать, что определенные переменные имеют значения, которые, как вы думаете, они должны иметь, пока не встретите переменную, не соответствующую ожиданию. Такое несоответствие затем станет подсказкой о вероятном месте ошибки. Но обычно несоответствие не возникнет, пока вы не приостановите и не возобновите выполнение на ряде breakpoints или несколько раз на одном и том же breakpoint. Поэтому возобновление выполнения в breakpoint так же важно, как и сама установка breakpoint, и потому отладочный инструмент обычно имеет достаточно богатый набор методов возобновления выполнения.

Существует три класса методов возобновления выполнения. Первый включает "single stepping" по программе с помощью `step` и `next`, выполняя только следующую строку кода и затем снова останавливаясь. Второй состоит в использовании `continue`, что заставляет GDB безусловно возобновить выполнение программы до достижения другого breakpoint или завершения программы. Последний класс методов связан с условиями: возобновление с помощью команд `finish` или `until`. В этом случае GDB возобновит выполнение, и программа будет работать, пока не выполнится некоторое заранее заданное условие, например не будет достигнут конец функции, пока не будет достигнут другой breakpoint или пока программа не завершится.

Мы рассмотрим каждый метод возобновления выполнения по очереди для GDB, а затем покажем, как выполнять такие операции в DDD и Eclipse.

2.9.1 В GDB

Начнем этот раздел с обсуждения разных способов возобновить выполнение после того, как GDB остановился на breakpoint.

2.9.1.1 Пошаговое выполнение с `step` и `next`

Когда GDB останавливается на breakpoint, команды `next` (сокращенно `n`) и `step` (сокращенно `s`) используются для пошагового прохода по коду. После срабатывания breakpoint и остановки GDB можно использовать `next` и `step`, чтобы выполнить только самую следующую строку кода. После выполнения строки GDB снова остановится и выдаст командное приглашение. Посмотрим, как это действует. Рассмотрим программу `swapflaw.c`:

```
1 /* swapflaw.c: A flawed function that swaps two integers. */
2 #include <stdio.h>
3 void swap(int a, int b);
4
5 int main(void)
6 {
7     int i = 4;
8     int j = 6;
9
10    printf("i: %d, j: %d\n", i, j);
11    swap(i, j);
12    printf("i: %d, j: %d\n", i, j);
13
14    return 0;
15 }
16
17 void swap(int a, int b)
18 {
19     int c = a;
20     a=b ;
21     b=c ;
22 }
```

Листинг 2-1. swapflaw.c

Мы установим breakpoint на входе в `main()` и запустим программу в GDB.

```
$ gcc -g3 -Wall -Wextra -o swapflaw swapflaw.c
$ gdb swapflaw
(gdb) break main
Breakpoint 1 at 0x80483f6: file swapflaw.c, line 7.
(gdb) run
Starting program: swapflaw

Breakpoint 1, main () at swapflaw.c:7
7 int i = 4;
```

GDB теперь находится на строке 7 программы, что означает: строка 7 еще не выполнена. Можно использовать команду `next`, чтобы выполнить только эту строку кода, оставив нас прямо перед строкой 8:

```
(gdb) next
8 int j = 6;
```

Мы используем `step`, чтобы выполнить следующую строку кода, строку 8, что перенесет нас к строке 10.

```
(gdb) step
10 printf("i: %d, j: %d\n", i, j);
```

Мы видим, что и `next`, и `step` выполняют следующую строку кода. Поэтому главный вопрос таков: "Чем эти команды отличаются?" Обе, похоже, выполняют следующую строку кода. Разница между ними в том, как они обрабатывают вызовы функций: `next` выполнит функцию, не останавливаясь внутри нее, а затем остановится на первом операторе после вызова. `step`, напротив, остановится на первом операторе внутри функции.

На строке 11 приближается вызов `swap()`. Посмотрим на действие `next` и `step` рядом.

Использование step:

```
(gdb) step
i: 4, j: 6
11 swap(i, j);
(gdb) step
swap (a=4, b=6) at swapflaw.c:19
19 i n tc=a ;
(gdb) step
20 a = b;
(gdb) step
21 b = c;
(gdb) step
22 }
(gdb) step
main () at swapflaw.c:12
12 printf("i: %d, j: %d\n", i, j);
(gdb) step
i: 4, j: 6
14 return 0;
```

Использование next:

```
(gdb) next
i: 4, j: 6
11 swap(i, j);
(gdb) next
12 printf("i: %d, j: %d\n", i, j);
(gdb) next
i: 4, j: 6
14 return 0;
```

Команда `step` работает так, как можно было бы ожидать. Она выполнила `printf()` на строке 10, затем вызов `swap()` на строке 11, ^[4] а затем начала выполнять строки кода внутри `swap()`. Это называется `stepping into` функции. После того как мы пошагово проходим все строки `swap()`, `step` возвращает нас в `main()`.

Напротив, кажется, что `next` никогда не покидала `main()`. Это главное различие между двумя командами. `next` рассматривает вызов функции как одну строку кода и выполняет всю функцию за одну операцию; это называется `stepping over` функции.

Однако не обманывайтесь: может выглядеть так, будто `next` перепрыгнула через тело `swap()`, но на самом деле она ни через что не "перешагнула". GDB молча выполнил каждую строку `swap()`, не показывая нам подробностей (хотя он показывает любой экранный вывод, который может напечатать `swap()`), и не предлагая выполнять отдельные строки этой функции.

Разница между `stepping into` функции (что делает `step`) и `stepping over` функции (что делает `next`) настолько важна, что, рискуя повториться, мы продемонстрируем различие между `next` и `step` с помощью схемы, показывающей выполнение программы стрелками.

Рисунок 2-8 иллюстрирует поведение команды `step`. Представьте, что программа приостановлена на первом операторе `printf()`. Рисунок показывает, куда приведет каждый оператор `step`:

```
step
step
int main(void)
{
    int i = 4;
    int j = 6;
    printf("i: %d, j: %d\n", i, j);
    swap(i, j);
    printf("i: %d, j: %d\n", i, j);
    return 0;
}
void swap(int a, int b)
{
    int c = a;
```

```

    a = b;
    b = c;
}
step
step
step

```

Рисунок 2-8. step входит внутрь функции.

Рисунок 2-9 показывает то же самое, но с использованием next.

```

next
int main(void)
{
    int i = 4;
    int j = 6;
    printf("i: %d, j: %d\n", i, j);
    swap(i, j);
    printf("i: %d, j: %d\n", i, j);
    return 0;
}
void swap(int a, int b)
{
    int c = a;
    a = b;
    b = c;
}
next

```

Рисунок 2-9. next перешагивает через функцию.

Использовать next или step - на самом деле вопрос того, что именно вы пытаетесь сделать. Если вы находитесь в части кода без вызовов функций, не имеет значения, какую из них использовать. В этом случае две команды полностью эквивалентны.

Однако если вы отлаживаете программу и оказываетесь перед входом в функцию, о которой знаете, что она не содержит ошибок (или не относится к ошибке, которую вы пытаетесь отследить), очевидно, стоит использовать next, чтобы не проходить пошагово каждую строку функции, которая вас не интересует.

Один из общих принципов отладки, изложенных в главе 1, состоял в нисходящем подходе к отладке. Если вы пошагово проходите исходный код и встречаете вызов функции, обычно лучше использовать next, а не step. Сразу после использования next в такой ситуации вы проверили бы, правильным ли был результат вызова. Если да, ошибка, вероятно, не в функции, а значит использование next вместо step сэкономило время и усилия, которые ушли бы на проход по каждой строке функции. С другой стороны, если результат функции неверен, можно повторно запустить программу с temporary breakpoint на вызове функции и затем использовать step, чтобы войти в функцию.

Команды next и step принимают необязательный числовой аргумент, указывающий, сколько дополнительных строк нужно пройти с помощью next или step. Иными словами, next 3 эквивалентно вводу next три раза подряд (или однократному вводу next с последующим двукратным нажатием ENTER).^[5] Рисунок 2-10 иллюстрирует, что делает next 3:

```

void swapper(int *a, int *b)
{
    int c = *a;
    *a = *b;
    *b = c;
    printf("swapped!\n");
}
next 3

```

Рисунок 2-10. next со счетчиком

2.9.1.2 Возобновление выполнения программы с помощью continue

Второй метод возобновления выполнения - команда `continue`, сокращенно `c`. В отличие от `step` и `next`, которые выполняют только одну строку кода, эта команда заставляет GDB возобновить выполнение программы до срабатывания breakpoint или завершения программы.

Команда `continue` может принимать необязательный целочисленный аргумент `n`. Это число сообщает GDB, что нужно проигнорировать следующие `n` breakpoints. Например, `continue 3` сообщает GDB возобновить выполнение программы и проигнорировать следующие 3 breakpoints.

2.9.1.3 Возобновление выполнения программы с помощью finish

После срабатывания breakpoint команды `next` и `step` используются для выполнения программы строка за строкой. Иногда это может быть мучительным занятием. Например, предположим, что GDB достиг breakpoint внутри функции. Вы изучили несколько переменных и собрали всю информацию, которую собирались получить. В этот момент вас не интересует пошаговый проход по остальной части функции. Вы хотели бы вернуться к вызывающей функции, где GDB находился до входа в вызванную функцию. Однако установка лишнего breakpoint и использование `continue` выглядят расточительно, если все, что вы хотите сделать, - пропустить остаток функции. Здесь и нужна `finish`.

Команда `finish`, сокращенно `fin`, приказывает GDB возобновить выполнение до момента сразу после завершения текущего stack frame. На обычном языке это означает, что если вы находитесь в функции, отличной от `main()`, команда `finish` заставит GDB возобновить выполнение до момента сразу после возврата из функции. Рисунок 2-11 иллюстрирует использование `finish`.

Хотя вместо `finish` можно было бы набрать `next 3`, легче набрать первое; второе требует подсчета строк, а все, что больше примерно полудюжины, было бы ненужной морокой.

Может выглядеть так, будто `finish` не выполняет каждую строку кода, пока переносит вас к нижней части функции, но это не так. GDB выполняет каждую строку без остановок, ^[6] кроме показа вывода программы.

Еще одно распространенное использование `finish` - когда вы случайно вошли в функцию, через которую собирались перешагнуть, иными словами использовали `step`, когда хотели использовать `next`. В таком случае `finish` помещает вас ровно туда, где вы оказались бы, если бы использовали `next`.

Если вы находитесь внутри рекурсивной функции, `finish` поднимет вас только на один уровень рекурсии. Это потому, что каждый вызов считается самостоятельным вызовом функции, поскольку у каждого есть собственный stack frame. Если вы хотите полностью выйти из рекурсивной функции при высоком уровне рекурсии, больше подходит temporary breakpoint вместе с `continue` или использование команды `until`. Сейчас мы обсудим `until`.

```
void swapper(int *a, int *b)
{
    int c = *a;
    *a = *b;
    *b = c;
    printf("swapped!\n");
}
finish
```

Рисунок 2-11. finish возобновляет выполнение до возврата текущей функции.

2.9.1.4 Возобновление выполнения программы с помощью `until`

Напомним, что команда `finish` завершает выполнение текущей функции без дальнейших остановок внутри функции, за исключением любых промежуточных breakpoints. Аналогично команда `until`, сокращенно просто `u`, обычно используется для завершения выполняющегося цикла без дальнейших остановок внутри цикла, за исключением любых промежуточных breakpoints внутри цикла. Рассмотрим следующий фрагмент кода.

```
...previous code...
int i = 9999;
while (i--) {
    printf("i is %d\n", i);
    ... lots of code ...
}
...future code...
```

Предположим, GDB остановлен на breakpoint у оператора `while`, вы изучили несколько переменных, а теперь хотите покинуть цикл, чтобы отлаживать "future code".

Проблема в том, что `i` настолько велико, что использовать `next` для завершения цикла пришлось бы бесконечно долго. Нельзя использовать `finish`, потому что эта команда пройдет прямо через "future code" и выведет нас из функции. Можно было бы установить temporary breakpoint на future code и использовать `continue`; однако это как раз та ситуация, для которой предназначена `until`.

Использование `until` выполнит остаток цикла, оставив GDB остановленным на первой строке кода после цикла. Рисунок 2-12 показывает, что сделает использование `until`:

```
...previous code...
int i = 9999;
while (i--) {
    printf("i is %d\n", i);
    ...lots of code...
}
...future code...
until
```

Рисунок 2-12. `until` переносит нас к следующей более высокой строке исходного кода.

Конечно, если GDB встретит breakpoint до выхода из цикла, он все равно остановится там. Если бы в операторе `printf()` на рисунке 2-12 был breakpoint, вы, конечно, захотели бы отключить его.

GDB User's Guide дает официальное определение `until`:

Execute until the program reaches a source line greater than the current [one].

Однако документация также предупреждает, что это может быть немного контринтуитивно. Чтобы показать почему, рассмотрим следующую программу:

```
#include <stdio.h>
int main(void)
{
    int i;
    for (i=0; i<10; ++i)
        printf("hello world!");
    return 0;
}
```

Листинг 2-2. `until-anomaly.c`

Мы установим breakpoint на входе в `main()`, запустим программу и используем `until`, чтобы добраться до оператора `return`.

```
$ gdb until-anomaly
Using host libthread_db library "/lib/tls/libthread_db.so.1".
(gdb) break main
```

```

Breakpoint 1 at 0x80483b4: file until-anomaly.c, line 7.
(gdb) run
Starting program: until-anomaly
Breakpoint 1, main () at until-anomaly.c:7
7 for (i=0; i<10; ++i)
(gdb) until
8 printf("hello world!");
(gdb) until
7 for (i=0; i<10; ++i)
(gdb) until
10 return 0;
(gdb)

```

Ого! Используя `until`, GDB перешел со строки 7 на строку 8, а затем обратно на строку 7. Неужели это означает, что исходная строка 7 больше исходной строки 8? На самом деле да. Возможно, к этому моменту вы уже можете догадаться об ответе, поскольку это, похоже, общая тема. В конечном счете GDB работает с машинными инструкциями. Хотя конструкция `for` написана с проверкой цикла в верхней части тела, GCC скомпилировал программу так, что условие оказалось внизу тела цикла. Поскольку условие связано со строкой 7 исходного кода, выглядит так, будто GDB пошел назад по исходному коду. На самом деле `until` выполняет программу до достижения машинной инструкции с более высоким адресом памяти, чем текущая, а не до достижения большего номера строки в исходном коде.

На практике такое может возникать не слишком часто, но приятно понимать, что происходит, когда это все же случается. Кроме того, глядя на некоторые странности поведения GDB, можно почерпнуть сведения о том, как компиляторы превращают исходный код в машинные инструкции, а это неплохое знание.

Если вам любопытно и вы знаете язык ассемблера своей машины, можно быстро взглянуть на машинный код с помощью команды GDB `disassemble`, а затем `p/x $pc`, чтобы напечатать текущее местоположение.^[7] Это покажет, что сделает `until`. Но это всего лишь особенность поведения, и в практическом смысле она не является проблемой. Если вы находитесь в конце цикла и команда `until` вызывает переход обратно к началу цикла, просто выдайте `until` второй раз, и вы выйдете из цикла, как хотели.

Команда `until` также может принимать аргумент, задающий местоположение в исходном коде. Фактически она принимает те же аргументы, что и команда `break`, обсуждавшаяся в разделе 2.4.1. Возвращаясь к листингу 2-1: если GDB сработал на breakpoint при входе в `main()`, все следующие варианты являются эквивалентными способами удобно выполнить программу до входа в `swap()`:

- `until 17`
- `until swap`
- `until swapflaw.c:17`
- `until swapflaw.c:swap`

2.9.2 В DDD

Начнем этот раздел с обсуждения разных способов возобновить выполнение после того, как DDD остановился на breakpoint.

2.9.2.1 Стандартные операции

В DDD есть кнопки для next и step в Command Tool. Кроме того, step и next можно выполнить функциональными клавишами F5 и F6 соответственно.

Если вы хотите использовать next или step с аргументом в DDD, то есть выполнить то, что в GDB делалось бы так:

```
(gdb) next 3
```

придется использовать сам GDB в Console Window DDD.

В DDD есть кнопка для continue в Command Tool, но снова: если нужно выполнить continue с аргументом, используйте GDB Console. Можно щелкнуть левой кнопкой мыши Source Window, чтобы вызвать вариант "continue until here", который на самом деле устанавливает temporary breakpoint (см. раздел 2.4.1) на этой строке исходного кода. Точнее, "continue until here" означает "продолжать до этой точки, но также остановиться на любых промежуточных breakpoints".

В GDB можно добиться того же эффекта, что и от finish, с помощью next с числовым аргументом, так что finish был бы лишь немного удобнее. Но в DDD использование finish явно выигрывает, требуя одного щелчка мышью в Command Tool.

Если вы используете DDD, можно выполнить until с помощью кнопки Until в Command Tool, либо щелкнуть Program | Until в строке меню, либо использовать клавиатурное сокращение F7. Среди всех этих вариантов обязательно найдется удобный! Как и со многими другими командами GDB, если вы хотите использовать until с аргументом, придется передать его напрямую GDB в Console Window DDD.

2.9.2.2 Undo/Redo

Как отмечалось в разделе 2.7.6, в DDD есть бесценная возможность Undo/Redo. В том разделе мы показали, как отменить случайное удаление breakpoint. Ее также можно использовать для таких действий, как Run, Next, Step и так далее.

Например, рассмотрим ситуацию, изображенную на рисунке 2-13. Мы достигли breakpoint у вызова `swap()` и собирались выполнить операцию Step, но случайно нажали Next. Однако щелчком Undo можно откатить время назад, как показано на рисунке 2-14. DDD напоминает, что вы что-то отменили, отображая курсор текущей строки контуром, а не сплошным зеленым цветом, который он обычно использует.

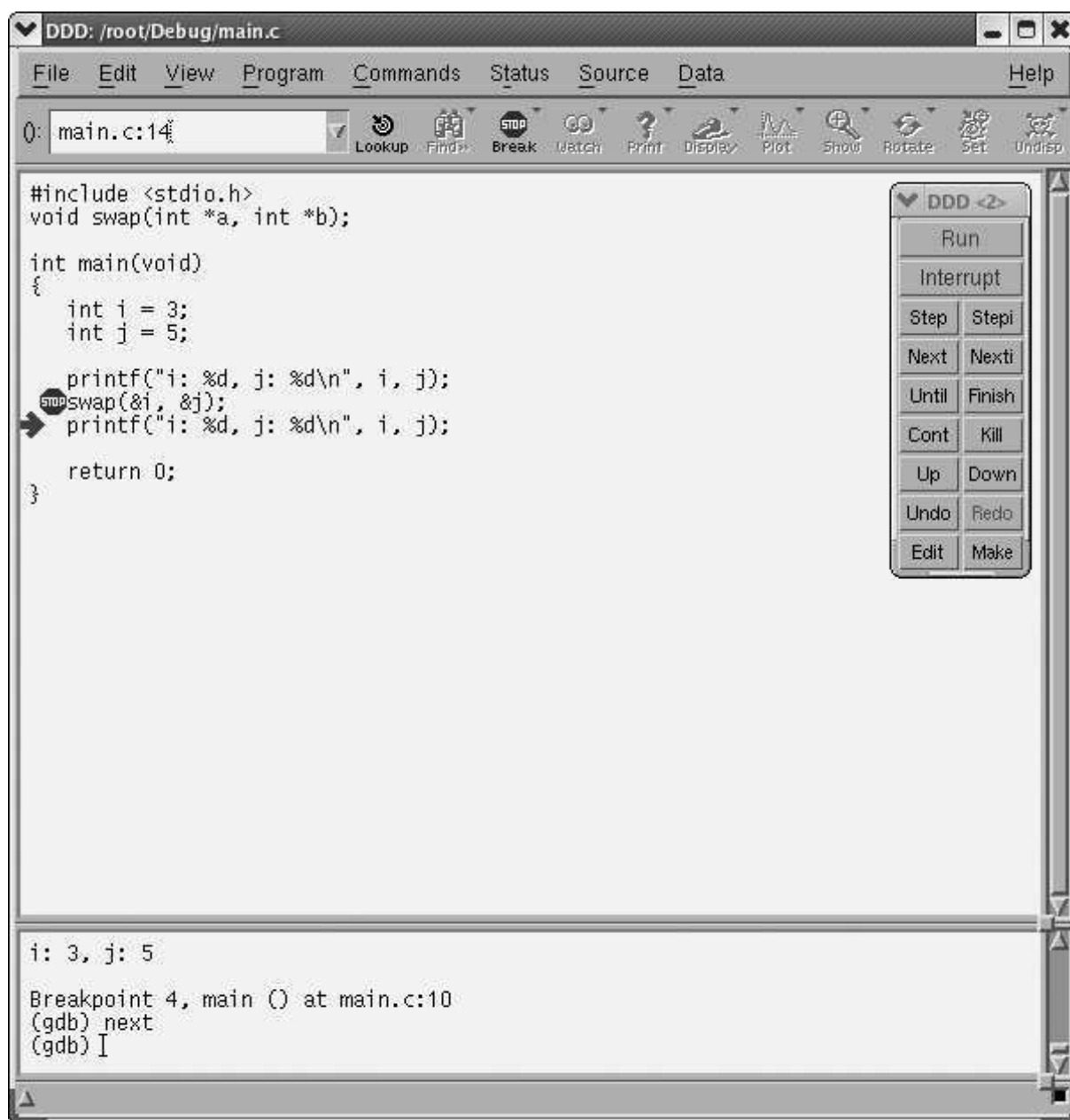


Рисунок 2-13. Упс!

2.9.3 В Eclipse

Аналоги step и next в Eclipse - это значки Step Into и Step Over. Значок Step Into виден во view Debug (слева сверху) на рисунке 2-15; указатель мыши временно вызвал появление подписи значка. Значок Step Over находится сразу справа от него. Обратите внимание: следующий оператор, который вы выполните, будет вызовом `get_args()`, поэтому щелчок Step Into приведет к тому, что следующая остановка выполнения произойдет на первом операторе внутри этой функции, а выбор Step Over будет означать, что следующая остановка будет на вызове `process_data()`.

В Eclipse есть значок Step Return (рядом со Step Over), выполняющий finish. Ничего точно соответствующего until там нет. Операция Run to Line, вызываемая щелчком целевой строки и затем правым щелчком в окне исходного кода, обычно выполняет то, что нужно от until.

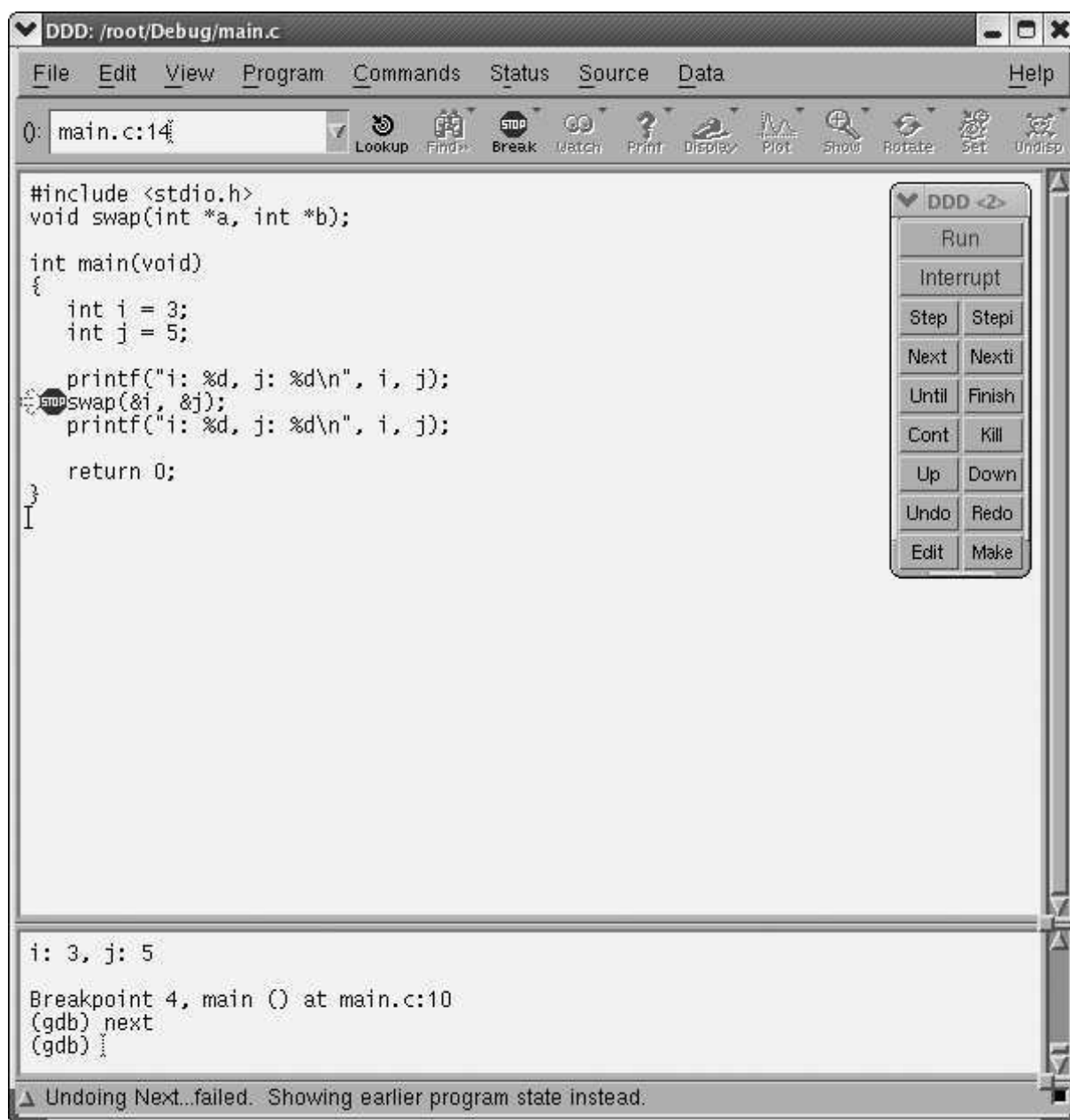


Рисунок 2-14. Возврат зубной пасты обратно в тюбик

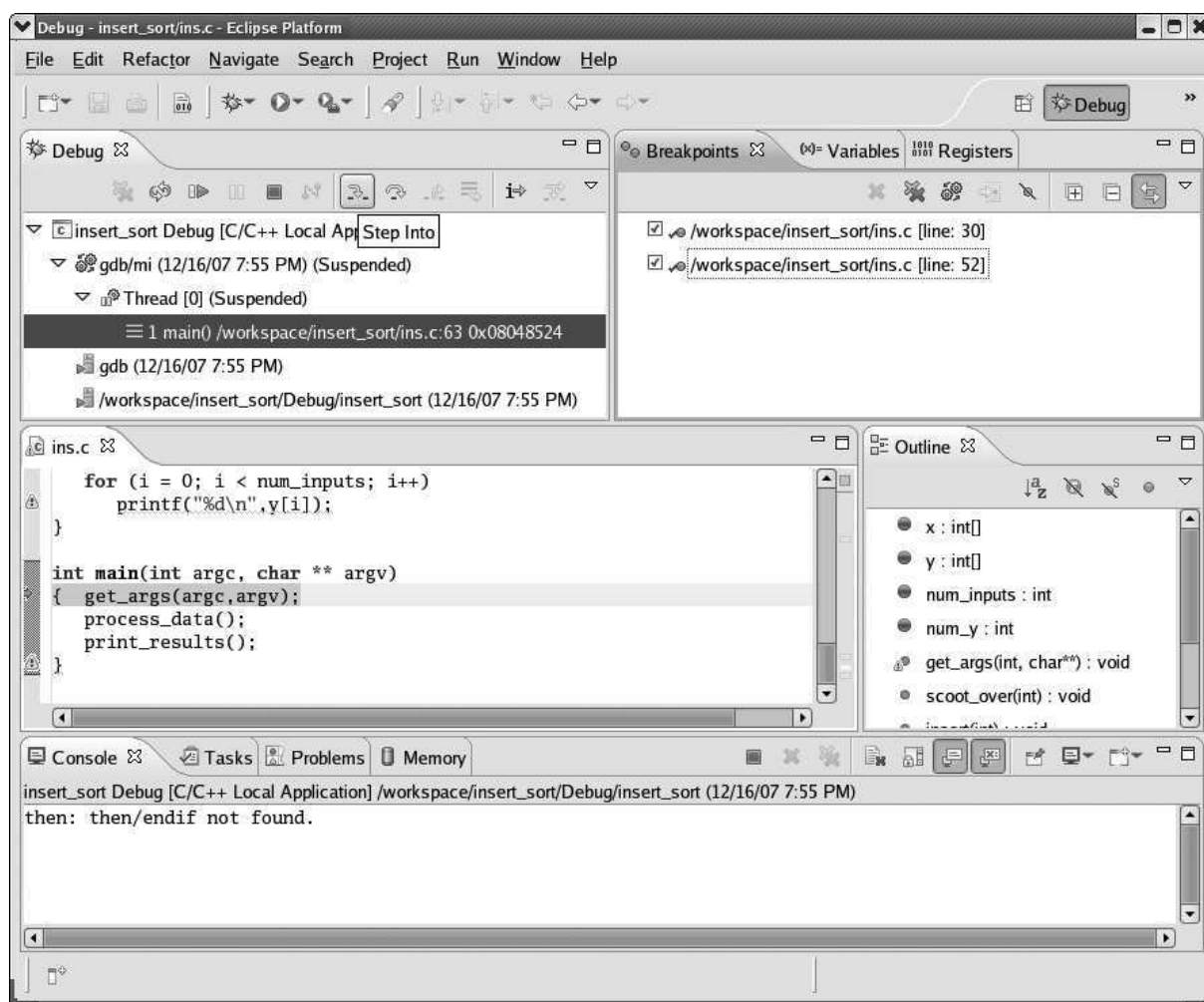


Рисунок 2-15. Step Into в Eclipse

2.10 Условные breakpoints

Пока breakpoint включен, отладчик всегда останавливается на этом breakpoint. Однако иногда полезно сказать отладчику останавливаться на breakpoint только при выполнении некоторого условия, например когда переменная имеет особенно интересное значение.

Это похоже на работу watchpoints, но есть важное различие. Если у вас есть подозрение, где переменная получает ошибочное значение, conditional breakpoint предпочтительнее watchpoint. Watchpoint остановится всякий раз, когда эта переменная изменит значение. Conditional breakpoint остановится только в подозреваемом проблемном коде и только тогда, когда переменная примет ошибочное значение. В этом смысле watchpoints хороши, когда у вас нет никакой догадки, где переменная получает ошибочное значение. Это особенно полезно для глобальных переменных или локальных переменных, которые постоянно передаются между функциями. Но в большинстве других случаев удачно поставленный conditional breakpoint полезнее и удобнее.

2.10.1 GDB

Синтаксис установки conditional breakpoint таков:

```
break break-args if (condition)
```

где `break-args` - любые аргументы, которые можно передать `break`, чтобы указать местоположение breakpoint, как обсуждалось в разделе 2.4.1, а `condition` - булево выражение, определенное в разделе 2.12.2. Скобки вокруг `condition` необязательны. Возможно, с ними некоторым С-программистам будет привычнее, но, с другой стороны, вам может понравиться более свободный вид без лишних знаков.

Например, вот как можно остановиться в `main()`, если пользователь ввел для программы аргументы командной строки: ^[8]

```
break main if argc > 1
```

Conditional breaking чрезвычайно полезен, особенно в циклических конструкциях, где что-то плохое происходит при конкретном значении индексной переменной. Рассмотрим следующий фрагмент кода:

```
for (i=0; i<=75000; ++i) {  
    retval = process(i);  
    do_something(retval);  
}
```

Предположим, вы знаете, что программа начинает вести себя безумно, когда `i` равно 70000. Вы хотите остановиться в верхней части цикла, но не хотите выполнять `next` через 69999 итераций. Именно здесь conditional breaking проявляет себя особенно хорошо. Можно установить breakpoint в верхней части цикла, но только когда `i` равно 70000:

```
break if (i == 70000)
```

Конечно, того же эффекта можно было бы добиться, набрав, скажем, `continue 69999`, но это было бы менее удобно.

Conditional breaking также чрезвычайно гибок. Можно делать гораздо больше, чем просто проверять переменную на равенство или неравенство. Что можно использовать в условии? Почти любое выражение, которое можно использовать в корректном условном операторе С. Все, что вы используете, должно иметь булево значение, то есть `true` (ненулевое) или `false` (ноль). Это включает:

- Операторы равенства, логические операторы и операторы неравенства (`<`, `<=`, `==`, `!=`, `>`, `>=`, `&&`, `||` и т. д.); например:

```
break 180 if string==NULL && i < 0
```

- Побитовые операторы и операторы сдвига (`&`, `|`, `^`, `>>`, `<<` и т. д.); например:

```
break test.c:34 if (x & y) == 1
```

- Арифметические операторы (+, -, x, /, %); например:

```
break myfunc if i % (j + 3) != 0
```

- Ваши собственные функции, если они скомпонованы в программу; например:

```
break test.c:myfunc if ! check_variable_sanity(i)
```

- Библиотечные функции, если библиотека скомпонована с вашим кодом; например:

```
break 44 if strlen(mystring) == 0
```

Правила приоритета операций действуют, поэтому вокруг конструкций вроде $(x \& y) == 0$ может потребоваться использовать скобки.

Кроме того, если вы используете библиотечную функцию в выражении GDB, а библиотека не была скомпилирована с отладочными символами (что почти наверняка так), единственные возвращаемые значения, которые можно использовать в условиях breakpoint, - это значения типа `int`. Иными словами, без отладочной информации GDB предполагает, что возвращаемое значение функции имеет тип `int`. Когда это предположение неверно, возвращаемое значение функции будет истолковано неправильно.

```
(gdb) print cos(0.0)
$1 = 14368
```

К сожалению, приведение типа тоже не помогает:

```
(gdb) print (double) cos(0.0)
$2 = 14336
```

Если ваша тригонометрия подзаржавела, косинус 0 равен 1.

Использование функций, возвращающих не `int`

На самом деле есть способ использовать в выражениях GDB функции, которые не возвращают `int`, но он довольно замысловат. Хитрость состоит в том, чтобы определить `convenience variable` GDB правильного типа данных, указывающую на функцию.

```
(gdb) set $p = (double (*)(double)) cos
(gdb) ptype $p
type = double (*)()
(gdb) p cos(3.14159265)
$2 = 14368
(gdb) p $p(3.14159265)
$ 4=- 1
```

Если ваша тригонометрия все еще подзаржавела, 3.14159265 приблизительно равно π , а косинус π равен -1.

Можно задавать условия для обычных breakpoints, превращая их в `conditional breakpoints`.

Например, если вы установили breakpoint 3 как безусловный, но теперь хотите добавить условие `i == 3`, просто введите:

```
(gdb) cond 3 i == 3
```


Если позже вы захотите удалить условие, но сохранить breakpoint, просто введите:

```
(gdb) cond 3
```

2.10.2 DDD

В DDD можно задавать conditional breakpoints с помощью семантики GDB в Console Window. Либо используйте DDD следующим образом. Установите обычный (то есть безусловный) breakpoint в том месте кода, где вам нужен conditional breakpoint. Щелкните правой кнопкой мыши и удерживайте красный знак stop sign, чтобы открыть меню, и выберите Properties. Появится всплывающее окно с текстовым полем, помеченным Condition. Введите условие в это поле, щелкните Apply, а затем Close. Теперь breakpoint стал conditional breakpoint.

Это показано на рисунке 2-16. Мы задаем условие `j == 0` для breakpoint 4. Кстати, stop sign на этой строке теперь будет содержать вопросительный знак, чтобы напоминать нам, что это conditional break.

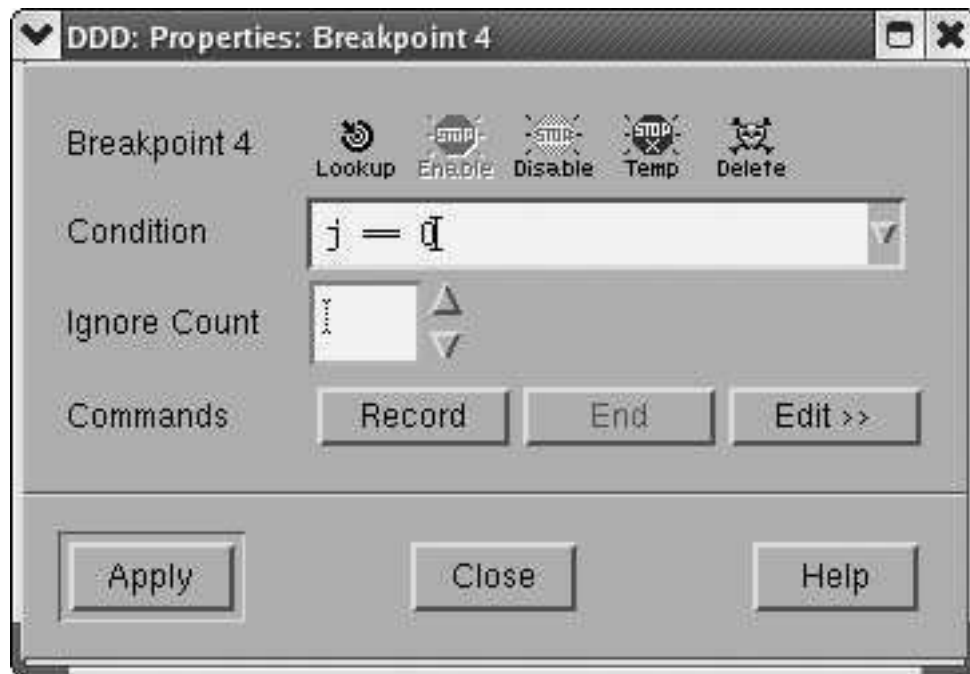


Рисунок 2-16. Наложение условия на breakpoint в DDD

2.10.3 Eclipse

Чтобы сделать breakpoint условным, щелкните правой кнопкой мыши символ breakpoint для этой строки, выберите Breakpoint Properties... | Common и заполните условие в диалоговом окне. Диалоговое окно изображено на рисунке 2-17.

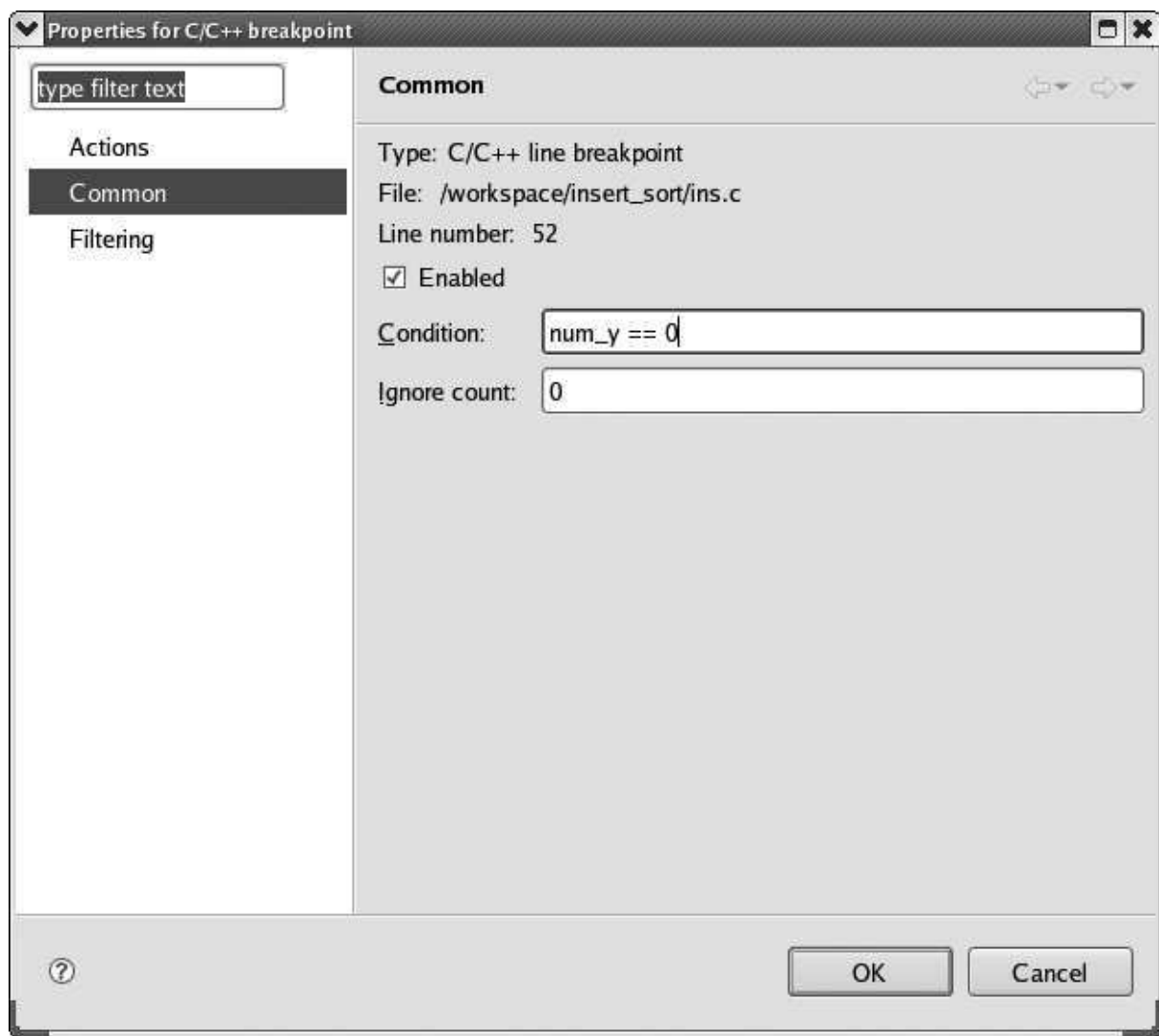


Рисунок 2-17. Наложение условия на breakpoint в Eclipse

2.11 Списки команд breakpoint

После того как GDB достигает breakpoint, вы почти всегда будете проверять переменную. Если тот же breakpoint срабатывает многократно (как breakpoint внутри цикла), вы будете многократно проверять одну и ту же переменную. Разве не было бы удобно автоматизировать эту процедуру, сказав GDB автоматически выполнять набор команд каждый раз, когда он достигает breakpoint?

На самом деле именно это можно сделать с помощью "списков команд breakpoint". Для иллюстрации command lists мы используем команду GDB `printf`. Формально вы с ней еще не знакомились, но `printf` в GDB в основном работает так же, как и в C, только круглые скобки необязательны.

Вы задаете command lists с помощью команды commands:

```
commands breakpoint-number
...
commands
...
end
```

где breakpoint-number - идентификатор breakpoint, к которому вы хотите добавить команды, а commands - список любых допустимых команд GDB, разделенных переводами строк. Вы вводите команды одну за другой, а затем вводите end, чтобы обозначить, что закончили ввод команд. После этого всякий раз, когда GDB будет останавливаться на этом breakpoint, он выполнит все команды, которые вы ему дали. Посмотрим пример. Рассмотрим следующую программу:

```
#include <stdio.h>
int fibonacci(int n);
int main(void)
{
    printf("Fibonacci(3) is %d.\n", fibonacci(3));
    return 0;
}
int fibonacci(int n)
{
    if (n <= 0 || n == 1)
        return 1;
    else
        return fibonacci(n-1) + fibonacci(n-2);
}
```

Листинг 2-3. fibonacci.c

Мы хотели бы увидеть, какие значения передаются в fibonacci() и в каком порядке. Однако вы не хотите вставлять операторы printf() и перекомпилировать код. Во-первых, в книге об отладке это было бы дурным тоном, не так ли? Но что важнее, потребовалось бы время, чтобы вставить код и перекомпилировать/перекомпоновать программу, а затем удалить этот код и снова перекомпилировать/перекомпоновать после исправления именно этой ошибки, особенно если программа велика. Более того, это захлестило бы код операторами, не относящимися к самому коду, тем самым усложнив его чтение во время отладки.

Можно было бы пошагово проходить код и печатать n при каждом вызове fibonacci(), но command lists лучше, потому что избавляют от необходимости снова и снова набирать команду print. Посмотрим.

Сначала установите breakpoint в начале fibonacci(). Этому breakpoint будет назначен идентификатор 1, поскольку это первый установленный вами breakpoint. Затем задайте для breakpoint 1 команду, печатающую переменную n.

```
$ gdb fibonacci
(gdb) break fibonacci
Breakpoint 1 at 0x80483e0: file fibonacci.c, line 13.
(gdb) commands 1
Type commands for when breakpoint 1 is hit, one per line.
End with a line saying just "end".
>printf "fibonacci was passed %d.\n", n
>end
(gdb)
```

Теперь запустите программу и посмотрите, что произойдет.

```
(gdb) run
Starting program: fibonacci
Breakpoint 1, fibonacci (n=3) at fibonacci.c:13
13 if ( n <= 0 || n == 1 )
    fibonacci was passed 3.
(gdb) continue
Continuing.
```

```

Breakpoint 1, fibonacci (n=2) at fibonacci.c:13
13 if ( n <= 0 || n == 1 )
fibonacci was passed 2.
(gdb) continue
Continuing.
Breakpoint 1, fibonacci (n=1) at fibonacci.c:13
13 if ( n <= 0 || n == 1 )
fibonacci was passed 1.
(gdb) continue
Continuing.
Breakpoint 1, fibonacci (n=0) at fibonacci.c:13
13 if ( n <= 0 || n == 1 )
fibonacci was passed 0.
(gdb) continue
Continuing.
Breakpoint 1, fibonacci (n=1) at fibonacci.c:13
13 if ( n <= 0 || n == 1 )
fibonacci was passed 1.
(gdb) continue
Continuing.
Fibonacci(3) is 3.
Program exited normally.
(gdb)

```

Что ж, это примерно то, чего мы ожидали, но вывод слишком многословен. В конце концов, мы уже знаем, где находится breakpoint. К счастью, можно заставить GDB сообщать о срабатывании breakpoints тише, используя команду `silent`, которая должна быть первым элементом в `command list`. Посмотрим на `silent` в действии. Обратите внимание, как мы переопределяем `command list`, помещая новый `command list` "поверх" ранее заданного:

```

(gdb) commands 1
Type commands for when breakpoint 1 is hit, one per line.
End with a line saying just "end".
>silent
>printf "fibonacci was passed %d.\n", n
>end
(gdb)

```

А ВОТ ВЫВОД:

```

(gdb) run
Starting program: fibonacci
fibonacci was passed 3.
(gdb) continue
Continuing.
fibonacci was passed 2.
(gdb) continue
Continuing.
fibonacci was passed 1.
(gdb) continue
Continuing.
fibonacci was passed 0.
(gdb) continue
Continuing.
fibonacci was passed 1.
(gdb) continue
Continuing.
Fibonacci(3) is 3.
Program exited normally.
(gdb)

```

Хорошо. Осталась последняя возможность для демонстрации: если последняя команда в `commands list - continue`, GDB автоматически продолжит выполнение программы после завершения команд из `commands list`:

```
(gdb) command 1
Type commands for when breakpoint 1 is hit, one per line.
End with a line saying just "end".
>silent
>printf "fibonacci was passed %d.\n", n
>continue
>end
(gdb) run
Starting program: fibonacci
fibonacci was passed 3.
fibonacci was passed 2.
fibonacci was passed 1.
fibonacci was passed 0.
fibonacci was passed 1.
Fibonacci(3) is 3.
Program exited normally.
(gdb)
```

Возможно, вы захотите делать такого рода вещи в других программах или в других строках этой программы, так что сделаем из этого макрос с помощью команды GDB `define`.

Сначала определим макрос, который назовем `print_and_go`:

```
(gdb) define print_and_go
Redefine command "print_and_go"? (y or n) y
Type commands for definition of "print_and_go".
End with a line saying just "end".
>printf $arg0, $arg1
>continue
>end
```

Чтобы использовать его, как выше, вы ввели бы:

```
(gdb) commands 1
Type commands for when breakpoint 1 is hit, one per line.
End with a line saying just "end".
>silent
>print_and_go "fibonacci() was passed %d\n" n
>end
```

Обратите внимание, что между аргументами `print_and_go` нет запятой. Тогда при запуске программы вы получили бы тот же вывод, что и раньше, но смысл в том, что теперь это можно использовать в общем виде, где угодно в коде. Более того, это можно поместить в файл `.gdbinit` для использования в других программах. Кстати, допускается до десяти аргументов, хотя в этом примере их всего два.

Список всех макросов можно получить, набрав `show user`.

Command lists очень полезны, но их также можно сочетать с `conditional breaking`, и это мощно. При таком виде условного ввода/вывода у вас даже может возникнуть искушение выбросить C и просто использовать GDB как любимый язык программирования. Конечно, мы шутим.

Команды и выражения

На самом деле любое допустимое выражение GDB, как описано в разделе 2.12.2, может входить в `commands list`. Вы можете использовать библиотечные функции или даже свои собственные функции, если они скомпонованы в исполняемый файл, и можете использовать их возвращаемое значение, если они возвращают `int`. Например:

```
(gdb) command 2
Type commands for when breakpoint 2 is hit, one per line.
End with a line saying just "end".
>silent
>printf "string has a length of %d\n", strlen(string)
>end
```

Command lists в DDD похожи на conditional breakpoints в DDD. Сначала установите breakpoint. Щелкните правой кнопкой мыши красный stop sign и выберите Properties. Появится всплывающее окно. Справа будет большое подокно (если вы не видите большого подокна, щелкните левой кнопкой кнопку Edit, которая переключает видимость окна команд). Команды можно вводить прямо в это окно. Там также есть кнопка Record. Если щелкнуть эту кнопку правой кнопкой мыши, можно вводить команды в GDB Console.

Похоже, что в Eclipse функции command-list нет.

2.12 Watchpoints

Watchpoint - это особый вид breakpoint, который, как и обычный breakpoint, является инструкцией, говорящей GDB приостановить выполнение вашей программы. Разница в том, что watchpoints не "живут" на строке исходного кода. Вместо этого watchpoint - это инструкция, говорящая GDB приостанавливать выполнение всякий раз, когда выражение изменяет значение.^[9] Это выражение может быть совсем простым, например именем переменной:

```
(gdb) watch i
```

что заставит GDB приостанавливаться всякий раз, когда `i` изменяет значение. Выражение может быть и весьма сложным:

```
(gdb) watch (i | j > 12) && i > 24 && strlen(name) > 6
```

Можно думать о watchpoint как о чем-то "прикрепленном" к выражению; когда значение этого выражения изменяется, GDB приостанавливает выполнение программы.

Хотя watchpoints и breakpoints управляются одинаково, между ними есть важное различие. Breakpoint связан с местом в исходном коде. Поскольку ваш код не изменяется, нет риска, что строка кода "выйдет из области видимости". Так как в C действуют жесткие правила областей видимости, вы можете наблюдать только за переменной, которая существует и находится в области видимости. Как только переменная больше не существует ни в одном frame call stack (когда функция, содержащая локальную переменную, возвращает управление), GDB автоматически удаляет watchpoint.

Например, в разделе 2.5 мы рассматривали программу с именем `swar`. В этой программе локальная переменная `s` использовалась для временного хранения. Вы не смогли бы установить watch на `s`, пока GDB не достигнет строки 3 файла `swapper.c`, где `s` определена. Кроме того, если бы вы установили watch на `s`, этот watch был бы автоматически удален после возврата GDB из `swapper()`, поскольку `s` больше не существовала бы.

Остановка GDB каждый раз, когда переменная изменяется, может стать некоторой помехой в тесно свернутых циклах или повторяющемся коде. Хотя watchpoints звучат замечательно, хорошо

размещенные breakpoints могут оказаться гораздо полезнее. Однако watchpoints бесценны, если одна из ваших переменных изменяется, особенно глобальная переменная, а вы понятия не имеете, где или как это произошло. Если вы имеете дело с многопоточным кодом, полезность watchpoints ограничена; GDB способен наблюдать за переменной только в пределах одного потока. Подробнее об этом можно прочитать в главе 5.

2.12.1 Установка watchpoints

Когда переменная `var` существует и находится в области видимости, можно установить на нее watchpoint с помощью команды:

```
watch var
```

что заставит GDB останавливаться всякий раз, когда значение переменной `var` изменяется. Именно так многие представляют себе watchpoints, потому что это просто и удобно; однако история на этом не заканчивается. На самом деле GDB останавливается, если место в памяти для `var` изменяет значение. Обычно не важно, думаете ли вы о watchpoint как о наблюдении за переменной или за адресом переменной, но это может быть важно в особых случаях, например при работе с указателями на указатели.

Hardware watchpoints?

При установке watchpoint вы можете увидеть сообщение о "hardware" watchpoint:

```
(gdb) watch i
Hardware watchpoint 2: i
```

или при выводе списка breakpoints:

```
(gdb) info breakpoints
Num Type Disp Enb Address What
2 hw watchpoint keep y i
```

Очевидно, 2 относится к идентификатору watchpoint, а `i` - это наблюдаемая переменная, но что означает это "hardware"?

На многих платформах есть специализированное аппаратное обеспечение, которое можно использовать для реализации watchpoint. GDB попытается использовать аппаратное обеспечение, если оно доступно, потому что реализация чего-либо аппаратно быстрее.

Возможно, GDB не сможет установить hardware watchpoints (у платформы может не быть необходимого аппаратного обеспечения, или аппаратное обеспечение может быть занято чем-то другим). В таком случае GDB попытается реализовать watchpoint с помощью методов VM, что тоже быстро.

Если ни один из этих методов недоступен, GDB реализует watchpoint сам, программно, а это может быть очень, очень медленно.

Ваш CPU может реализовать только ограниченное число hardware-assisted breaks, включая watchpoints и hardware-assisted breakpoints. Это число зависит от архитектуры, но если вы превысите этот предел, GDB выведет сообщение:

```
Stopped; cannot insert breakpoints.  
You may have requested too many hardware breakpoints and watchpoints.
```

Если вы увидите это сообщение, следует удалить или отключить часть ваших watchpoints или hardware-assisted breakpoints.

Рассмотрим один пример сценария, в котором watchpoints были бы очень полезны. Предположим, у вас есть две переменные `int`, `x` и `y`, и где-то в коде вы выполняете `p=&y`, хотя хотели выполнить `p=&x`. Это может привести к тому, что `y` где-то в коде таинственно изменит значение. Фактическое место возникшей ошибки может быть хорошо скрыто, так что breakpoint может оказаться не очень полезным. Однако, установив watchpoint, вы могли бы мгновенно узнать, когда и где `y` изменяет значение.

И это еще не все. Вы не ограничены наблюдением за переменной. На самом деле можно наблюдать за выражением, включающим переменные. Всякий раз, когда выражение изменяет значение, GDB остановится. В качестве примера рассмотрим следующий код:

```
1 #include <stdio.h>  
2 int i = 0;  
3  
4 int main(void)  
5 {  
6 i=3 ;  
7 printf("i is %d.\n", i);  
8  
9 i=5 ;  
10 printf("i is %d.\n", i);  
11  
12 return 0;  
13 }
```

Мы хотели бы знать всякий раз, когда `i` становится больше 4. Поэтому установим breakpoint на входе в `main()`, чтобы ввести `i` в область видимости, и установим watchpoint, который сообщит, когда `i` становится больше 4. Нельзя установить watchpoint на `i`, потому что до запуска программы `i` не существует. Поэтому сначала нужно установить breakpoint на `main()`, а затем установить watchpoint на `i`:

```
(gdb) break main  
Breakpoint 1 at 0x80483b4: file test2.c, line 6.  
(gdb) run  
Starting program: test2  
Breakpoint 1, main () at test2.c:6
```

Теперь, когда `i` находится в области видимости, установите watchpoint и скажите GDB продолжить выполнение программы. Мы полностью ожидаем, что `i > 4` станет true на строке 9.

```
1 (gdb) watch i > 4  
2 Hardware watchpoint 2: i > 4  
3 (gdb) continue  
4 Continuing.  
5 Hardware watchpoint 2: i > 4  
6  
7 Old value = 0  
8 New value = 1  
9 main () at test2.c:10
```

И действительно, GDB приостановлен между строками 9 и 10, где выражение `i > 4` изменило значение с 0 (не true) на 1 (true).

Этот метод можно использовать для установки watchpoints в DDD через Console Window; однако вам может показаться удобнее использовать GUI-интерфейс. В Source Window найдите

переменную, на которую хотите установить watchpoint. Это не обязательно должно быть первое появление переменной; можно использовать любое упоминание переменной в исходном коде. Щелкните переменную левой кнопкой мыши, чтобы выделить ее, затем щелкните левой кнопкой символ watchpoint в icon Menu Bar. Индикатора того, что watchpoint установлен, вроде красного stop sign для breakpoints, не будет. Чтобы увидеть, какие watchpoints вы установили, придется фактически вывести список всех breakpoints, что мы рассматривали в разделе 2.3.2.

В Eclipse можно установить watchpoint, щелкнув правой кнопкой мыши в окне исходного кода, выбрав Add a Watch Expression, а затем заполнив нужное выражение в диалоговом окне.

2.12.2 Выражения

Мы видели пример использования выражения с командой GDB watch. Оказывается, существует довольно много команд GDB, например print, которые также принимают выражения в качестве аргументов. Поэтому, вероятно, стоит сказать о них немного больше.

Выражение в GDB может содержать многое:

- GDB convenience variables
- любую переменную вашей программы, находящуюся в области видимости, например `i` из предыдущего примера
- любую строковую, числовую или символьную константу
- макросы препроцессора, если программа была скомпилирована так, чтобы включать отладочную информацию препроцессора^[10]
- условные выражения, вызовы функций, приведения типов и операторы, определенные используемым вами языком

Итак, если бы вы отлаживали программу на Fortran 77 и хотели узнать, когда переменная `i` становится больше 4, вместо `watch i > 4`, как в предыдущем разделе, вы использовали бы `watch i .GT. 4`.

Вы часто увидите руководства и документацию, которые используют синтаксис C для выражений GDB, но это связано с повсеместным распространением C и C++; если вы используете язык, отличный от C, выражения GDB строятся из элементов этого языка.

Сноски

[1] [\[назад\]](#) Машинный код, сгенерированный GCC, можно посмотреть с помощью параметра `-S`.

[2] [\[назад\]](#) На самом деле некоторые отладчики могут очень серьезно спотыкаться об исполняемые файлы, скомпилированные с включенной оптимизацией. GDB отличается тем, что является одним из немногих отладчиков, способных отлаживать оптимизированный код, но, как вы видели, результаты могут быть сомнительными.

[3] [\[назад\]](#) Напомним, что breakpoint может оказаться не точно на первой строке `main()`, но он будет рядом. В отличие от предыдущего примера на эту тему, однако, строка здесь является исполняемой, поскольку присваивает значение `i`.

[4] [\[назад\]](#) Вы можете удивиться, почему `step` не перенес вас на первую строку функции `printf()`. Причина в том, что GDB не останавливается внутри кода, для которого у него нет отладочной информации, то есть таблицы символов. Функция `printf()`, подключаемая из библиотеки C, является примером такого кода.

[5] [\[назад\]](#) Пользователи Vim должны чувствовать себя вполне привычно с идеей указания счетчика для заданной команды.

[6] [\[назад\]](#) Если есть какие-либо промежуточные breakpoints, `finish` остановится на них.

[7] [\[назад\]](#) Машинный код также можно посмотреть с помощью параметра GCC `-S`. Он создаст файл на языке ассемблера с суффиксом `.s`, показывающий код, произведенный компилятором. Обратите внимание: при этом создается только файл на языке ассемблера, а не исполняемый файл.

[8] [\[назад\]](#) При условии, что вы объявили `argc` и `argv` как аргументы `main()`. Конечно, если вы объявили их с другими именами, скажем `as` и `av`, используйте эти имена. Кстати, обратите внимание: программа всегда получает по крайней мере один аргумент - имя самой программы, на которое указывает `argv[0]`; здесь мы не считаем его "пользовательским аргументом".

[9] [\[назад\]](#) Выражения будут обсуждаться подробнее в разделе 2.12.2.

[10] [\[назад\]](#) На момент написания этой книги в официальном GNU GDB User's Manual говорится, что макросы препроцессора нельзя использовать в выражениях; однако это неверно. Если скомпилировать программу с параметром GCC -g3, макросы препроцессора можно использовать в выражениях.

Глава 3. Проверка и установка переменных

В главе 1 вы узнали, что в GDB можно печатать значение переменной с помощью команды `print` и что в `DDD` и `Eclipse` это можно делать, перемещая указатель мыши к экземпляру переменной в любом месте исходного кода. Но и GDB, и GUI также предлагают гораздо более мощные способы проверять переменные и структуры данных, как мы увидим в этой главе.

3.1 Наш основной пример кода

Ниже приведена прямолинейная (хотя не обязательно эффективная, модульная и т. д.) реализация бинарного дерева:

```
// bintree.c: routines to do insert and sorted print of a binary tree
#include <stdio.h>
#include <stdlib.h>

struct node {
    int val; // stored value
    struct node *left; // ptr to smaller child
    struct node *right; // ptr to larger child
};
typedef struct node *nsp;
nsp root;
nsp makenode(int x)
{
    nsp tmp;
    tmp = (nsp) malloc(sizeof(struct node));
    tmp->val = x;
    tmp->left = tmp->right = 0;
    return tmp;
}
void insert(nsp *btp, int x)
{
    nsp tmp = *btp;
    if (*btp == 0) {
        *btp = makenode(x);
        return;
    }
    while (1)
    {
        if (x < tmp->val) {
            if (tmp->left != 0) {
                tmp = tmp->left;
            } else {
                tmp->left = makenode(x);
                break;
            }
        } else {
            if (tmp->right != 0) {
                tmp = tmp->right;
            } else {
                tmp->right = makenode(x);
                break;
            }
        }
    }
}
void printtree(nsp bt)
{
    if (bt == 0) return;
    printtree(bt->left);
    printf("%d\n", bt->val);
    printtree(bt->right);
}
```

```

}
int main(int argc, char *argv[])
{ int i;
  root = 0;
  for (i = 1; i < argc; i++)
    insert(&root, atoi(argv[i]));
  printtree(root);
}

```

В каждом узле все элементы левого поддерева меньше значения в данном узле, а все элементы правого поддерева больше или равны значению в данном узле. Функция `insert()` создает новый узел и помещает его в правильную позицию дерева. Функция `printtree()` отображает элементы любого поддерева в возрастающем числовом порядке, а `main()` запускает тест, печатая весь отсортированный массив. ^[1]

Для здешних примеров отладки предположим, что вы случайно закодировали второй вызов `makenode()` в `insert()` как:

```
tmp->left = makenode(x);
```

ВМЕСТО:

```
tmp->right = makenode(x);
```

Если запустить этот ошибочный код, что-то сразу пойдет не так:

```

$ bintree 12 8 5 19 16
16
12

```

Исследуем, как различные команды проверки в отладочных инструментах могут помочь ускорить поиск ошибки.

3.2 Расширенная проверка и установка переменных

Наш пример с деревом здесь сложнее, и поэтому нужны более изощренные методы. Рассмотрим некоторые из них.

3.2.1 Проверка в GDB

В предыдущих главах вы использовали базовую команду GDB `print`. Как можно было бы использовать ее здесь? Что ж, основная работа, очевидно, выполняется в `insert()`, так что это было бы хорошее место для начала. Запустив GDB внутри цикла `while` в этой функции, можно было бы выдавать набор из трех команд GDB каждый раз, когда вы достигаете `breakpoint`:

```

(gdb) p tmp->val
$ 1=1 2
(gdb) p tmp->left
$2 = (struct node *) 0x8049698
(gdb) p tmp->right
$3 = (struct node *) 0x0

```

(Вспомните из главы 1, что вывод GDB помечается \$1, \$2 и так далее, причем эти величины вместе называются `value history`. Мы обсудим их подробнее в разделе 3.4.1.)

Здесь вы обнаружили бы, что узел, на который сейчас указывает tmp, содержит 12, с ненулевым левым указателем, но нулевым правым указателем. Конечно, фактическое значение левого указателя, то есть фактический адрес памяти, здесь, вероятно, не представляет прямого интереса, но важен сам факт, что указатель ненулевой или нулевой. Суть в том, что вы видите: ниже 12 сейчас есть левое поддерево, но нет правого поддерева.

Первое улучшение: печатать struct целиком

Было бы довольно утомительно продолжать вводить эти три команды print каждый раз, когда мы достигали breakpoint. Вот как можно сделать то же самое всего одной командой print:

```
(gdb) p *tmp
$4 = {val = 12, left = 0x8049698, right = 0x0}
```

Поскольку tmp указывает на struct, *tmp является самой структурой, и поэтому GDB показывает нам все ее содержимое.

Второе улучшение: использовать команду GDB display

Ввод p *tmp выше экономит время и силы. Каждый раз, когда вы достигаете breakpoint, вам нужно было бы вводить только одну команду GDB, а не три. Но если вы знаете, что будете вводить ее каждый раз при достижении breakpoint, можно сэкономить еще больше времени и усилий с помощью команды GDB display, сокращенно disp. Эта команда говорит GDB автоматически печатать указанный элемент каждый раз, когда происходит пауза в выполнении (из-за breakpoint, команд next или step и так далее):

```
(gdb) disp *tmp
1: *tmp = {val = 12, left = 0x8049698, right = 0x0}
(gdb) c
Continuing.
Breakpoint 1, insert (btp=0x804967c, x=5) at bintree.c:37
37 if (x < tmp->val) {
1: *tmp = {val = 8, left = 0x0, right = 0x0}
```

Как видно здесь, GDB автоматически напечатал *tmp после достижения breakpoint, поскольку вы ранее выдали команду display.

Конечно, переменная в display list будет отображаться только в те моменты, когда она находится в области видимости.

Третье улучшение: использовать команду GDB commands

Предположим, вы хотите посмотреть значения в дочерних узлах, когда находитесь в заданном узле. Вспоминая команду GDB commands из главы 1, можно было бы сделать что-то вроде этого:

```
(gdb) b 37
Breakpoint 1 at 0x8048403: file bintree.c, line 37.
(gdb) commands 1
Type commands for when breakpoint 1 is hit, one per line.
End with a line saying just "end".
>p tmp->val
>if (tmp->left != 0)
>p tmp->left->val
>else
>printf "%s\n", "none"
>end
```

```

>if (tmp->right != 0)
>p tmp->right->val
>else
>printf "%s\n", "none"
>end
>end

```

Обратите внимание в этом примере, что у команды GDB print есть более мощный родственник, printf(), с форматированием, похожим на форматирование ее тезки из языка C.

Вот фрагмент получившейся сессии GDB:

```

Breakpoint 1, insert (btp=0x804967c, x=8) at bintree.c:37
37 if (x < tmp->val)
$ 7=1 2
none
none
(gdb) c
Continuing.
Breakpoint 1, insert (btp=0x804967c, x=5) at bintree.c:37
37 if (x < tmp->val)
$ 6=1 2
$ 7=8
none
(gdb) c
Continuing.
Breakpoint 1, insert (btp=0x804967c, x=5) at bintree.c:37
37 if (x < tmp->val)
$ 8=8
none
none

```

Конечно, можно было бы сделать вывод красивее, добавив метки и так далее.

Четвертое улучшение: использовать команду GDB call

Обычный подход при отладке - изолировать первый элемент данных, на котором проявляется проблема. В здешнем контексте этого можно было бы добиться, печатая все дерево каждый раз после завершения вызова insert(). Поскольку в вашем исходном файле уже есть функция для этого - printtree() - можно было бы просто добавить вызов этой функции сразу после вызова insert() в исходном коде:

```

for (i = 1; i < argc; i++) {
insert(&root,atoi(argv[i]));
printtree(root);
}

```

Однако это было бы нежелательно с разных точек зрения. Это означало бы, например, что вам пришлось бы потратить время на редактирование исходного файла и его перекомпиляцию. Первое отвлекало бы, а второе могло бы занять некоторое время, если программа велика. В конце концов, именно от этого вы и пытаетесь уйти, используя отладчик.

Вместо этого было бы удобно сделать то же самое изнутри GDB. Это можно сделать с помощью команды GDB `call`. Например, можно установить breakpoint на строке 57, в конце `insert()`, а затем выполнить следующее:

```
(gdb) commands 2
Type commands for when breakpoint 1 is hit, one per line.
End with a line saying just "end".
>printf "***** current tree *****"
>call printtree(root)
>end
```

Фрагмент получившейся сессии GDB:

```
Breakpoint 2, insert (btp=0x8049688, x=12) at bintree.c:57
57 }
***** current tree *****
12
(gdb) c
Continuing.
Breakpoint 2, insert (btp=0x8049688, x=8) at bintree.c:57
57 }
***** current tree *****
8
12
(gdb) c
Continuing.
Breakpoint 2, insert (btp=0x8049688, x=5) at bintree.c:57
57 }
***** current tree *****
5
8
12
(gdb) c
Continuing.
Breakpoint 2, insert (btp=0x8049688, x=19) at bintree.c:57
57 }
***** current tree *****
19
12
```

Обратите внимание, что это показывает: первым элементом данных, вызвавшим проблему, было число 19. Эта информация позволит очень быстро сузить поиск ошибки. Вы заново запустили бы программу с теми же данными, установив breakpoint в начале `insert()`, но с условием `x == 19`, а затем исследовали бы, что там происходит.

Набор команд для заданного breakpoint можно изменять динамически или просто отменить, переопределив пустой набор:

```
(gdb) commands 1
Type commands for when breakpoint 1 is hit, one per line.
End with a line saying just "end".
>end
```

3.2.2 Проверка в DDD

Как вы уже знаете, любую команду GDB можно вызвать из DDD в DDD Console Window. Но одно из настоящих преимуществ DDD состоит в том, что многие команды GDB можно выполнять в DDD удобнее, а в некоторых случаях DDD способен выполнять мощные операции, которых нет в GDB, например отображение связанных структур данных, описанное ниже.

Как упоминалось в предыдущих главах, проверять значение переменной в DDD очень удобно: просто переместите указатель мыши к любому экземпляру переменной в Source Code Window. Но есть и другие полезные возможности, вполне заслуживающие использования, особенно для программ, применяющих связанные структуры данных. Мы проиллюстрируем это на том же примере бинарного дерева, что и ранее в этой главе.

Вспомните иллюстрацию команды GDB `display`:

```
(gdb) disp *tmp
```

Здесь содержимое структуры, на которую указывает `tmp`, печатается автоматически каждый раз, когда вы достигаете breakpoint или иным образом вызываете паузу в выполнении программы. Поскольку `tmp` был указателем на текущий узел дерева, такая автоматическая печать помогала следить за продвижением при обходе дерева.

Несколько аналогична этому команда `Display` в DDD. Если щелкнуть правой кнопкой мыши любой экземпляр переменной, скажем `root` в данном примере, в Source Code Window, всплывает меню, как показано на рисунке 3-1. Как видно, здесь есть несколько вариантов просмотра `root`. Варианты `Print root` и `Print *root` работают точно так же, как их аналоги в GDB, и фактически их вывод появляется в Console DDD (где команды GDB отображаются/вводятся). Но в нашем случае самый интересный вариант - `Display *root`. Результат выбора этого пункта после достижения breakpoint на строке 48 исходного кода показан на рисунке 3-2.

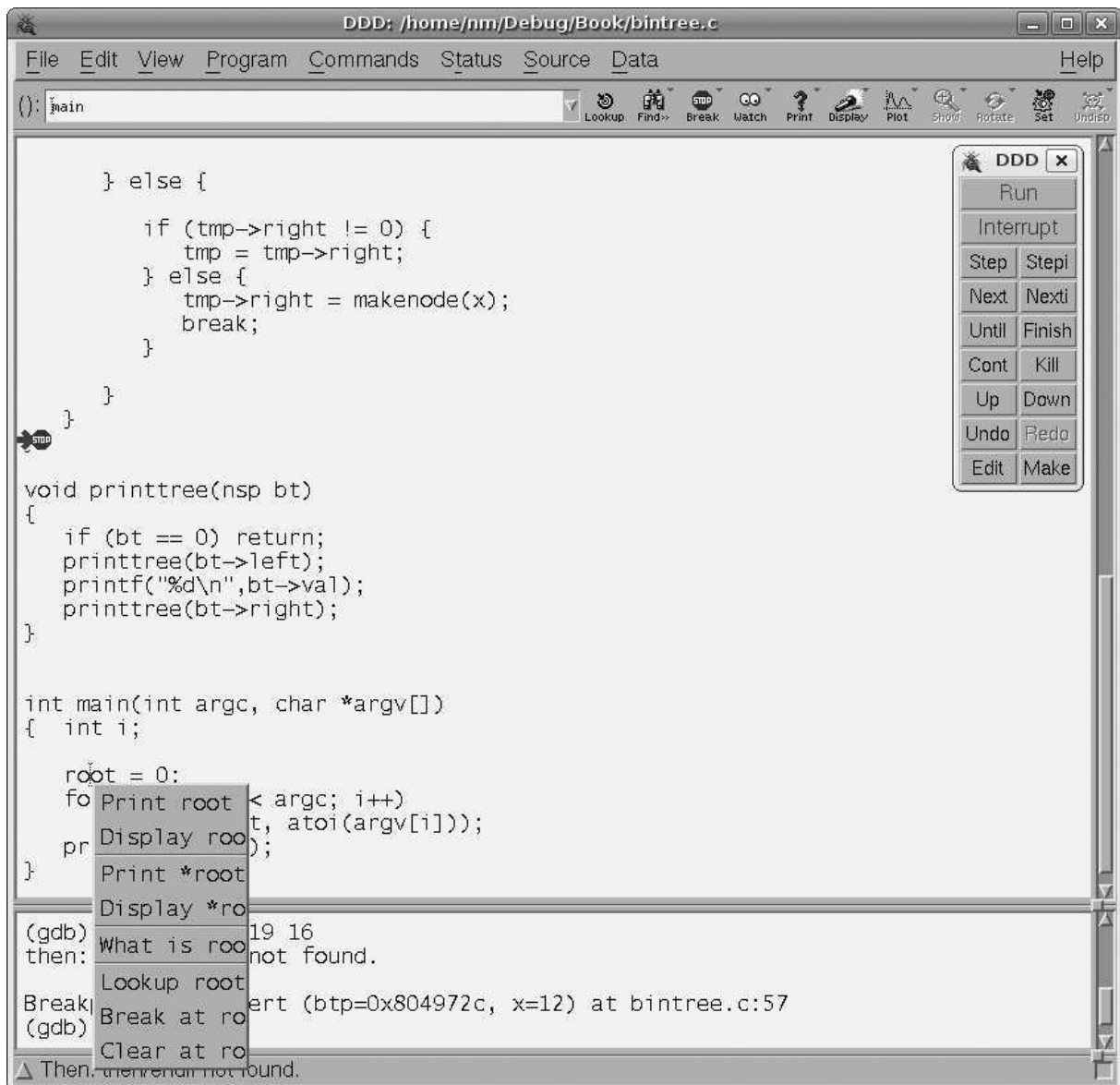


Рисунок 3-1. Всплывающее окно для просмотра переменной

Появилось новое окно DDD - Data Window, с узлом, соответствующим `root`. Пока что это не более чем графический аналог команды GDB `display`. Но действительно приятно здесь то, что можно следовать по ссылкам дерева! Чтобы пройти по левой ветви дерева, например, щелкните правой кнопкой мыши поле `left` отображаемого корневого узла. (С правым узлом в этот момент вы бы так не сделали, поскольку ссылка равна 0.) Затем выберите пункт `Display *()` во всплывающем меню, и теперь DDD выглядит так, как на рисунке 3-3. Итак, DDD представляет вам рисунок дерева (или этой его части) именно так, как вы сами нарисовали бы его на доске - очень здорово!

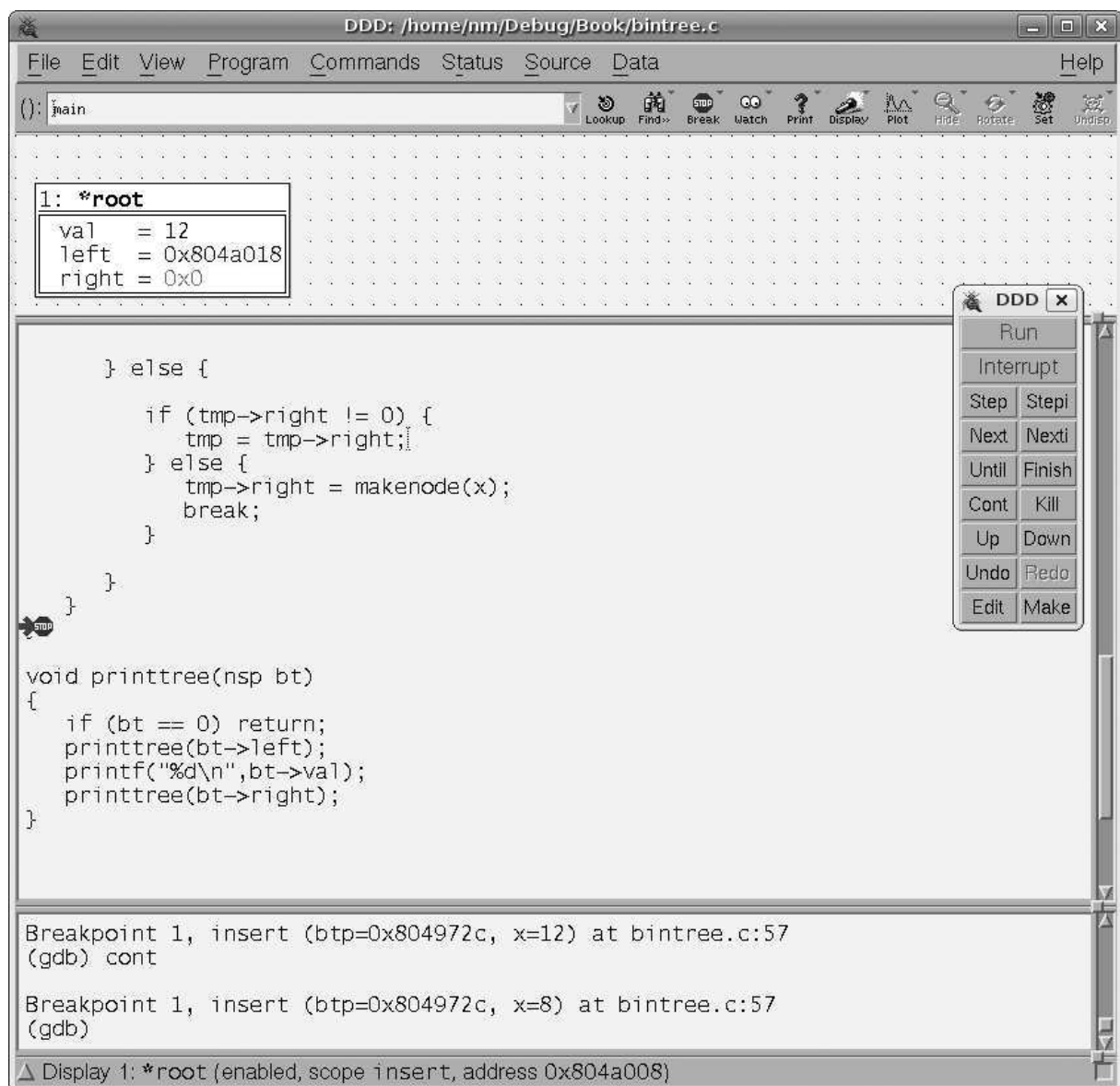


Рисунок 3-2. Отображение узла

Отображение содержимого существующего узла будет автоматически обновляться всякий раз, когда содержимое узла изменяется. Каждый раз, когда ссылка изменяется с нуля на ненулевое значение, можно щелкнуть ее правой кнопкой мыши, чтобы отобразить новый узел.

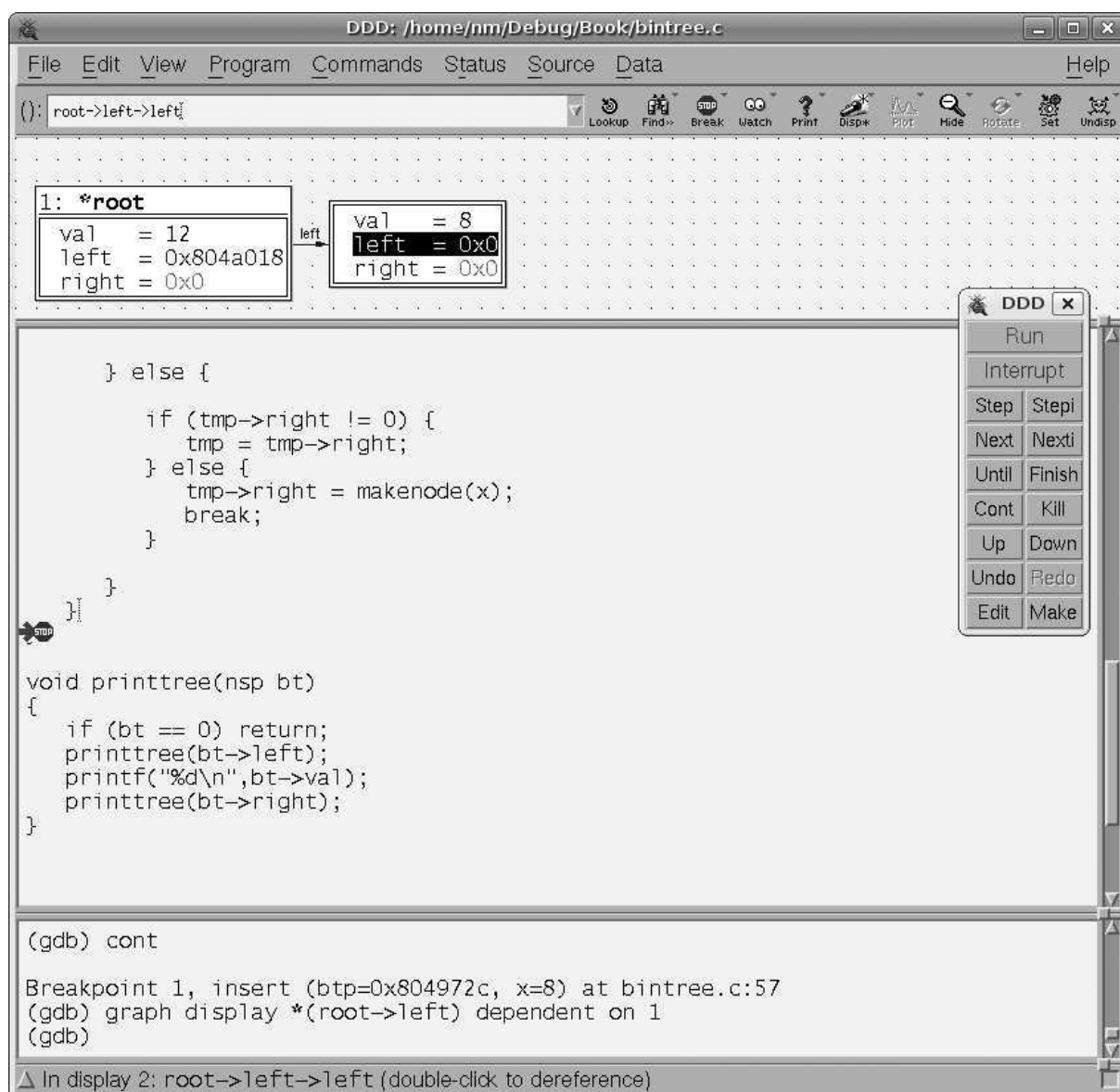


Рисунок 3-3. Переход по ссылкам

Очевидно, Data Window может очень быстро загромождаться. Можно расширить окно, щелкнув и перетаскив маленький квадрат в его правом нижнем углу, но лучшим подходом было бы предвидеть эту ситуацию еще до запуска DDD. Если вызвать DDD с отдельным параметром командной строки:

```
$ ddd --separate bintree
```

то появятся отдельные окна - Source Code, Console и Data, которые можно изменять по размеру как угодно.

Если во время сессии DDD вы хотите удалить часть или все содержимое Data Window, есть много способов сделать это. Например, можно щелкнуть элемент правой кнопкой мыши, а затем выбрать пункт Undisplay.

3.2.3 Проверка в Eclipse

Как и в DDD, чтобы проверить скалярную переменную в Eclipse, просто переместите указатель мыши к любому экземпляру переменной в окне исходного кода. Обратите внимание, что это должна быть самостоятельная скалярная переменная, а не, например, переменная внутри struct. Это показано на рисунке 3-4. Здесь мы успешно запросили значение `x`, но если бы переместили указатель мыши на часть `val` в `tmp->val` в той же строке, это не сказало бы нам, что там находится.

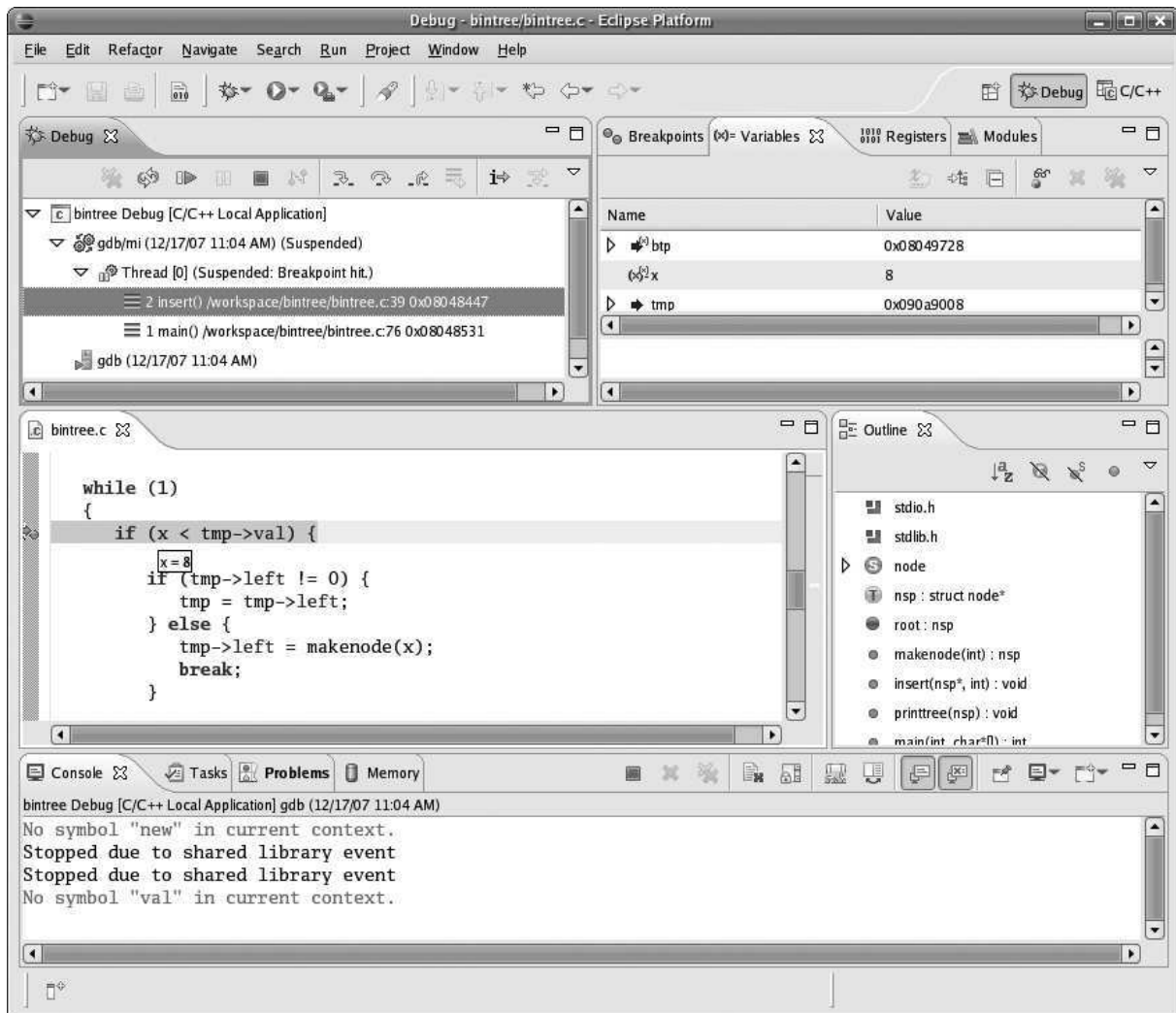


Рисунок 3-4. Проверка скалярной переменной в Eclipse

В этот момент вы воспользовались бы представлением Variables в Eclipse, которое можно увидеть в правой верхней части рисунка 3-5. Щелкните треугольник рядом с `tmp`, чтобы он указывал вниз, затем прокрутите на строку ниже и увидите, что отображается `tmp->val`. (Оказывается, он содержит 12.)

И можно продолжать этот процесс. После щелчка по треугольнику рядом с `left` вы увидите экран, показанный на рисунке 3-6, где будет видно, что `tmp->left->val` равно 8.

По умолчанию представление Variables не показывает глобальные переменные. В здешней программе есть одна такая переменная, `root`. Ее можно добавить в представление Variables, щелкнув правой кнопкой мыши внутри этого представления, выбрав Add Global Variables, отметив флажок для `root` в появившемся всплывающем окне и затем щелкнув OK.

3.2.4 Проверка динамических массивов

Как обсуждалось в главе 1, в GDB можно напечатать весь массив, скажем объявленный как:

```
int x[25];
```

набрав:

```
(gdb) p x
```

Но что, если массив был создан динамически, скажем так:

```
int *x;  
...  
x = (int *) malloc(25*sizeof(int));
```

Если бы вы захотели напечатать массив в GDB, вы не смогли бы набрать:

```
(gdb) p x
```

Это просто напечатало бы адрес массива. Нельзя было бы набрать и:

```
(gdb) p *x
```

Это напечатало бы только один элемент массива, `x[0]`. Вы все еще могли бы печатать отдельные элементы, как в команде `p x[5]`, но не могли бы напечатать весь массив, просто применив команду `print x`.

3.2.4.1 Решения в GDB

В GDB эту проблему можно решить, создав artificial array. Рассмотрим код:

```
1 int *x;  
2  
3 main()  
4 {  
5 x = (int *) malloc(25*sizeof(int));  
6 x[3] = 12;  
7 }
```

Тогда можно было бы сделать что-то вроде этого:

```
Breakpoint 1, main () at artif.c:6  
6 x = (int *) malloc(25*sizeof(int));  
(gdb) n  
7 x[3] = 12;  
(gdb) n  
8}  
(gdb) p *x@25  
$1 = {0, 0, 0, 12, 0 <repeats 21 times>}
```

Как видите, общая форма такова:

```
*pointer@number_of_elements
```

GDB также позволяет использовать приведения типов там, где это уместно, например:

```
(gdb) p (int [25]) *x
$2 = {0, 0, 0, 12, 0 <repeats 21 times>}
```

3.2.4.2 Решения в DDD

Как всегда, через DDD Console можно использовать метод GDB, в данном случае artificial arrays.

Другой вариант - напечатать или отобразить диапазон памяти (см. раздел 3.2.7 ниже).

3.2.4.3 Решения в Eclipse

Здесь можно использовать команду Eclipse Display as Array.

Например, немного расширим наш предыдущий пример:

```
1 int *x;
2
3 main()
4 {
5 int y;
6 x = (int *) malloc(25*sizeof(int));
7 scanf("%d%d",&x[3],&x[8]);
8 y = x[3] + x[8];
9 printf("%d\n",y);
10 }
```

Предположим, сейчас вы находитесь на присваивании у. Сначала вы поместили бы x в представление Variables, щелкнув правой кнопкой мыши в этом представлении и выбрав x. Затем вы снова щелкнули бы правой кнопкой мыши x в представлении Variables и выбрали Display As Array. В появившемся всплывающем окне вы заполнили бы поля Start Index и Length, скажем значениями 0 и 25, чтобы отобразить весь массив. Теперь экран выглядел бы как на рисунке 3-7. Там, в представлении Variables, виден массив, показанный как:

```
(0,0,0,1,0,0,0,0,2,0,<repeats 16 times>)
```

Значения 1 и 2 пришли из нашего ввода в программу, видимого в представлении Console.

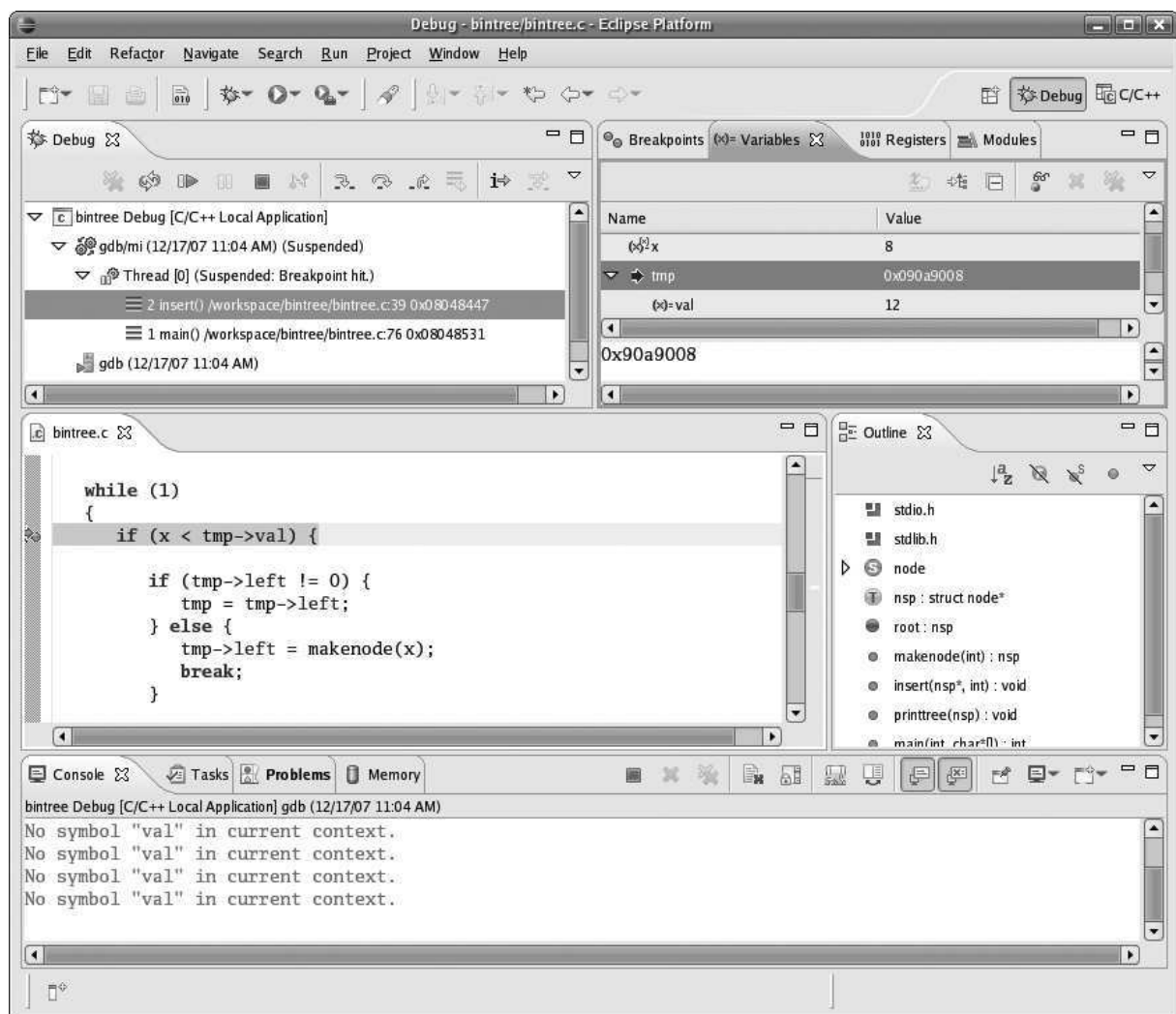


Рисунок 3-5. Проверка поля struct в Eclipse

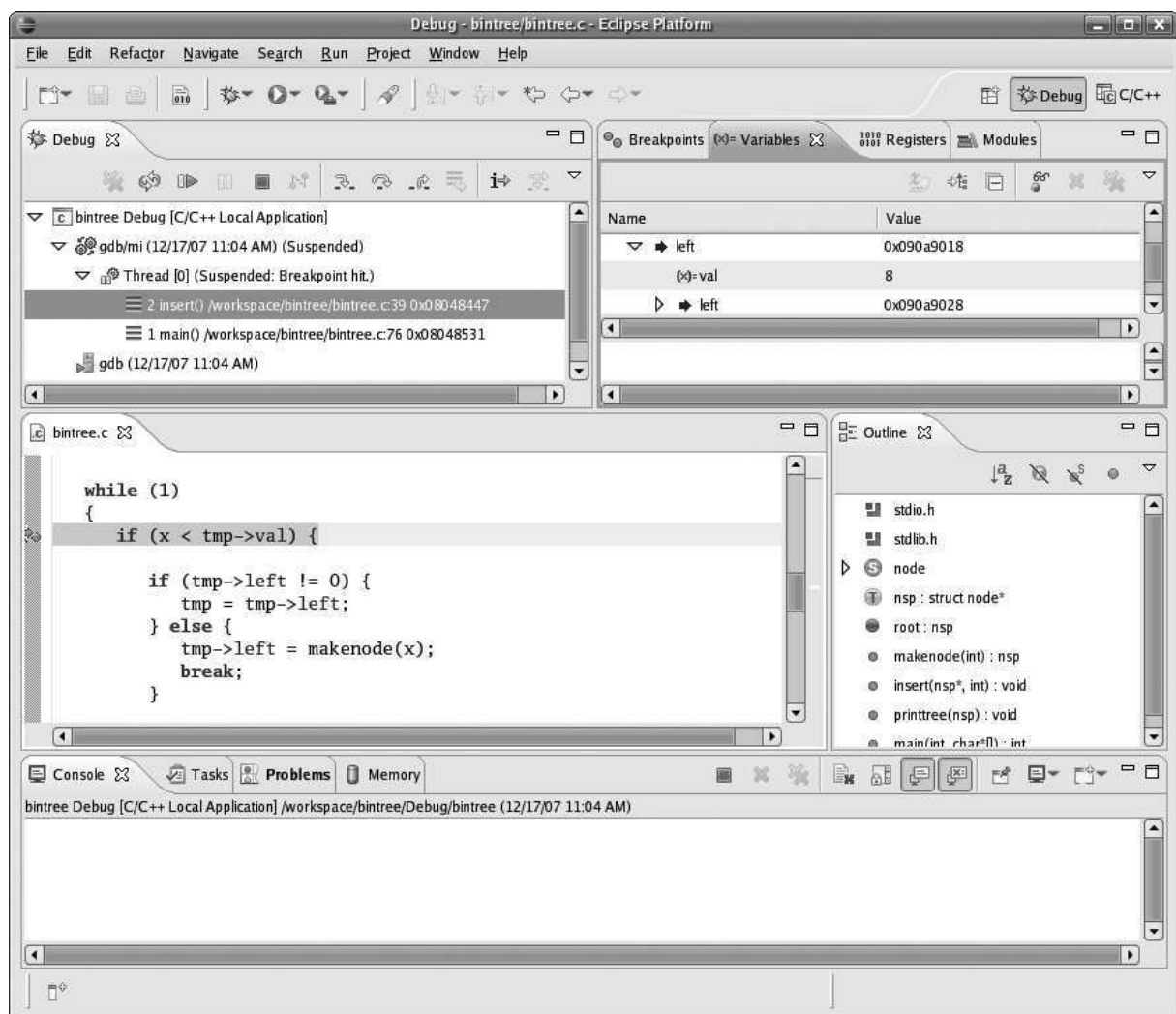


Рисунок 3-6. Переход по ссылкам указателей в Eclipse

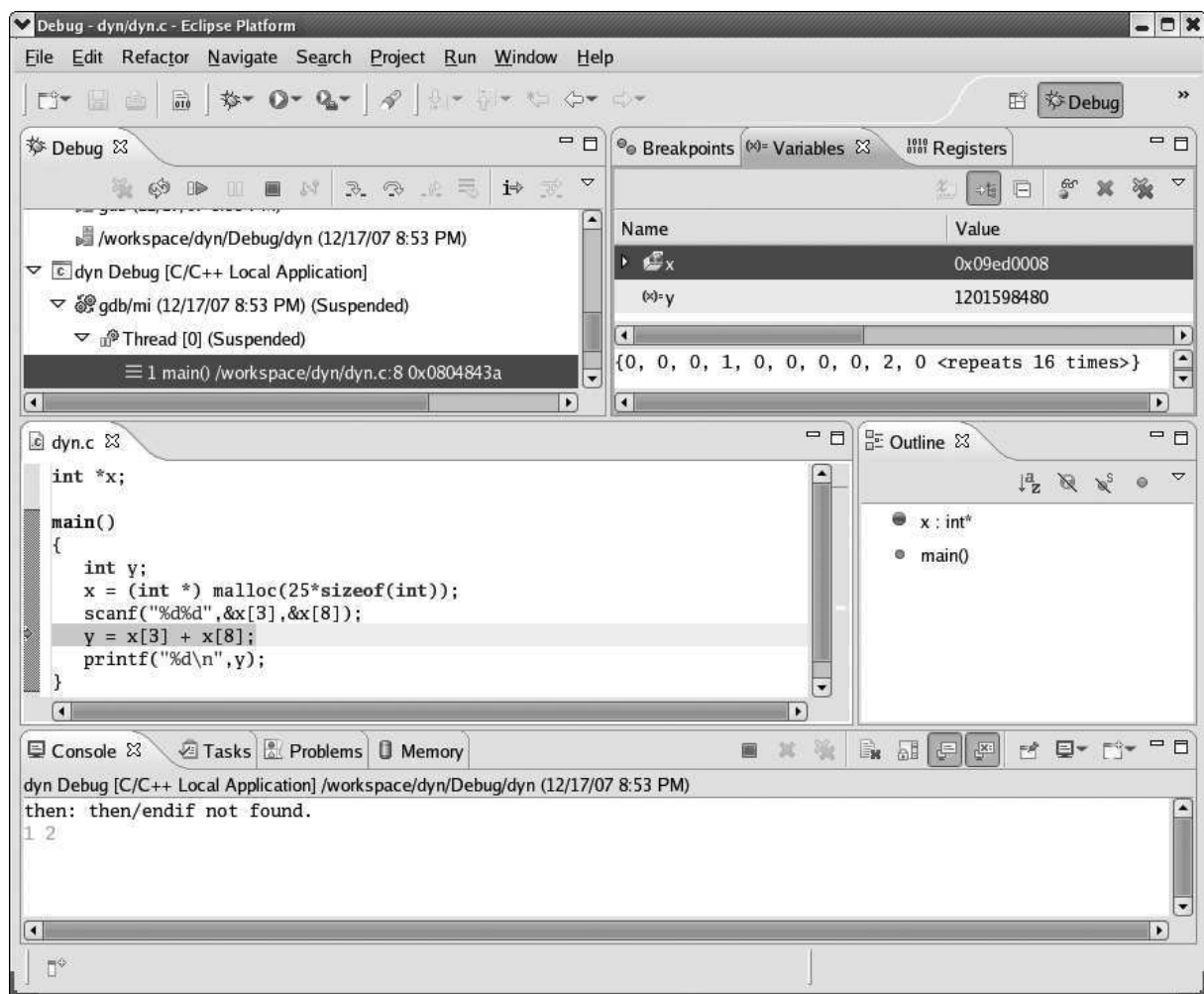


Рисунок 3-7. Отображение динамического массива в Eclipse

3.2.5 А что насчет C++?

Чтобы проиллюстрировать ситуацию для кода C++, вот C++-версия примера с бинарным деревом, использованного ранее:

```
// bintree.cc: routines to do insert and sorted print of a binary tree in C++
#include <iostream.h>
class node {
public:
    static class node *root; // root of the entire tree
    int val; // stored value
    class node *left; // ptr to smaller child
    class node *right; // ptr to larger child
    node(int x); // constructor, setting val = x
    static void insert(int x); // insert x into the tree
    static void printtree(class node *nptr); // print subtree rooted at *nptr
};
class node *node::root = 0;
node::node(int x)
{
    val=x ;
    left = right = 0;
}
void node::insert(int x)
{
    if (node::root == 0) {
        node::root = new node(x);
    }
}
```

```

return;
}
class node *tmp=root;
while (1)
{
if (x < tmp->val)
{
if (tmp->left != 0) {
tmp = tmp->left;
} else {
tmp->left = new node(x);
break;
}
} else {
if (tmp->right != 0) {
tmp = tmp->right;
} else {
tmp->right = new node(x);
break;
}
}
}
}
void node::printtree(class node *np)
{
if (np == 0) return;
node::printtree(np->left);
cout << np->val << endl;
node::printtree(np->right);
}
int main(int argc, char *argv[])
{
for (int i = 1; i < argc; i++)
node::insert(atoi(argv[i]));
node::printtree(node::root);
}

```

Вы по-прежнему компилируете как обычно, убедившись, что сказали компилятору сохранить таблицу символов в исполняемом файле. ^[2]

Те же команды GDB работают, но с несколько другим выводом. Например, снова печатая содержимое объекта, на который указывает tmp внутри insert(), вы получаете следующий вывод:

```

(gdb) p *tmp
$6 = {static root = 0x8049d08, val = 19, left = 0x0, right = 0x0}

```

Это похоже на случай C-программы, за исключением того, что теперь печатается также значение статической переменной node::root (что и должно быть, поскольку она является частью класса).

Конечно, нужно помнить, что GDB требует указывать переменные по тем же правилам областей видимости, которые использует C++. Например:

```

(gdb) p *root
Cannot access memory at address 0x0
(gdb) p *node::root
$8 = {static root = 0x8049d08, val = 12, left = 0x8049d18, right = 0x8049d28}

```

Нам нужно было указать root через ее полное имя, node::root.

В GDB и DDD нет встроенных браузеров классов, но команда GDB `ptype` удобна, чтобы быстро просмотреть структуру класса или `struct`, например:

```
(gdb) ptype node
type = class node {
public:
    static node *root;
    int val;
    node *left;
    node *right;
    node(int);
    static void insert(int);
    static void printtree(node*);
}
```

В DDD можно щелкнуть правой кнопкой мыши имя класса или переменной, а затем выбрать `What Is` во всплывающем меню, чтобы получить ту же информацию.

В Eclipse есть представление `Outline`, так что таким способом можно легко получить информацию о классе.

3.2.6 Мониторинг локальных переменных

В GDB можно получить список значений всех локальных переменных в текущем `stack frame`, вызвав команду `info locals`.

В DDD можно даже отобразить локальные переменные, щелкнув `Data | Display Local Variables`. В результате часть DDD `Data Window` будет отведена под отображение `locals` (обновляемых по мере пошагового прохождения программы). Похоже, что прямого способа сделать это внутри GDB нет, хотя можно делать это для каждого `breakpoint` отдельно, включив команду `info locals` в процедуру `commands` для каждого `breakpoint`, на котором вы хотите автоматически печатать `locals`.

Eclipse отображает локальные переменные в представлении `Variables`, как вы уже видели.

3.2.7 Непосредственное исследование памяти

В некоторых случаях может понадобиться исследовать память по заданному адресу, а не через имя переменной. Для этой цели GDB предоставляет команду `x` ("examine"). В DDD выбирают `Data | Memory`, задают начальную точку и число байтов и выбирают между `Print` и `Display`. В Eclipse есть представление `Memory`, в котором можно создавать `Memory Monitors`.

Обычно это полезно главным образом в контекстах языка ассемблера и подробно обсуждается в главе 8.

3.2.8 Расширенные параметры для Print и Display

Команды `print` и `display` позволяют указывать альтернативные форматы. Например:

```
(gdb) p/x y
```

отобразит переменную `y` в шестнадцатеричном формате вместо десятичного. Другие часто используемые форматы - `c` для символа, `s` для строки и `f` для `floating-point`.

Можно временно отключить элемент display. Например:

```
(gdb) dis disp 1
```

временно отключает элемент 1 в display list. Если вы не знаете номера элементов, их можно проверить командой `info disp`. Чтобы снова включить элемент, используйте `enable`, например:

```
(gdb) enable disp 1
```

Чтобы полностью удалить элемент display, используйте `undisplay`, например:

```
(gdb) undisp 1
```

3.3 Установка переменных изнутри GDB/DDD/Eclipse

В некоторых случаях полезно задавать значение переменной с помощью отладчика прямо в процессе пошагового прохождения программы. Так можно быстро отвечать на вопросы "что если", возникающие, когда вы выдвигаете гипотезы о различных источниках ошибки.

В GDB значения можно задавать очень легко, например:

```
(gdb) set x = 12
```

изменит текущее значение `x` на 12.

Похоже, что в DDD нет способа сделать это с помощью мыши, но, опять же, любую команду GDB можно выдать из DDD в DDD Console Window.

В Eclipse перейдите в представление Variables, щелкните правой кнопкой мыши переменную, значение которой хотите задать, и выберите Change Value. Затем всплывающее окно позволит ввести новое значение.

Аргументы командной строки для вашей программы можно задать с помощью команды GDB `set args`. Однако у этого нет преимущества перед методом, описанным в главе 1, где новые аргументы просто используются при вызове команды GDB `run`. Эти два метода полностью эквивалентны. Например, не происходит так, что:

```
(gdb) set args 1 52 19 11
```

немедленно изменит `argv[1]` на 1, `argv[2]` на 52 и так далее. Эти изменения произойдут только в следующий раз, когда вы выдадите команду `run`.

В GDB есть команда `info args`, которую можно использовать для проверки аргументов текущей функции. DDD предоставляет это, когда вы щелкаете Data | Display Arguments.

3.4 Собственные переменные GDB

Помимо переменных, объявляемых вами в программе, GDB также предоставляет механизмы для других переменных.

3.4.1 Использование value history

Выходные значения команды GDB print помечаются \$1, \$2 и так далее, причем все эти величины вместе называются value history. Их можно использовать для создания сокращений в будущих командах print, которые вы выдаете.

Например, рассмотрим пример bintree из раздела 3.1. Часть сессии GDB могла бы выглядеть так:

```
(gdb) p tmp->left
$1 = (struct node *) 0x80496a8
(gdb) p *(tmp->left)
$2 = {val = 5, left = 0x0, right = 0x0}
(gdb) p *$1
$3 = {val = 5, left = 0x0, right = 0x0}
```

Здесь произошло следующее: после того как мы напечатали значение указателя tmp->left и обнаружили, что оно ненулевое, мы решили напечатать то, на что этот указатель указывает. Мы сделали это дважды: сначала обычным способом, а затем через value history.

В третьей печати мы сослались на \$1 в value history. Если бы мы не делали обычную печать, мы могли бы воспользоваться специальной переменной value history \$:

```
(gdb) p tmp->left
$1 = (struct node *) 0x80496a8
(gdb) p *$
$2 = {val = 5, left = 0x0, right = 0x0}
```

3.4.2 Convenience variables

Предположим, у вас есть переменная-указатель p, которая в разные моменты указывает на разные узлы связанного списка. Во время отладочной сессии вы можете захотеть записать адрес определенного узла, скажем потому, что хотите повторно проверять значение в этом узле в разные моменты процесса отладки. В первый раз, когда p попадет на этот узел, можно сделать что-то вроде:

```
(gdb) set $q = p
```

и с этого момента делать вещи вроде:

```
(gdb) p *$q
```

Переменная \$q здесь называется convenience variable.

Convenience variables могут изменять значения по правилам C. Например, рассмотрим код:

```
int w[4] = {12,5,8,29};
main()
{
    w[2] = 88;
}
```

В GDB можно было бы сделать что-то вроде:

```
Breakpoint 1, main () at cv.c:7
7 w[2] = 88;
(gdb) n
8}
(gdb) set $i = 0
(gdb) p w[$i++]
$ 1=1 2
(gdb)
$ 2=5
(gdb)
$ 3=8 8
(gdb)
$ 4=2 9
```

Чтобы понять, что здесь произошло, вспомните: если в GDB просто нажать клавишу ENTER, не выдавая команды, GDB воспринимает это как просьбу повторить последнюю команду. В приведенной выше сессии GDB вы продолжали нажимать ENTER, что означало просьбу к GDB повторить команду:

```
(gdb) p w[$i++]
```

Это означало не только то, что значение будет напечатано, но и то, что convenience variable `$i` будет увеличиваться.

NB. Можно выбрать почти любое имя для convenience variable, за некоторыми естественными исключениями. Например, нельзя иметь convenience variable с именем `$3`, поскольку оно зарезервировано для элементов value history. Кроме того, не следует использовать имена регистров, если вы работаете на языке ассемблера. Например, для машин на базе Intel x86 одно из имен регистров - EAX, и внутри GDB он обозначается как `$eax`; вы не захотели бы выбрать это имя для convenience variable, если бы работали на уровне языка ассемблера.

Сноски

[1] [\[назад\]](#) Кстати, обратите внимание на typedef в строке 12, `nsp`. Это означает `node struct pointer`, но наш издатель думает, что это No Starch Press.

[2] [\[назад\]](#) В этом отношении, однако, могут возникать различные вопросы. См. руководство GDB относительно форматов исполняемых файлов. Примеры здесь используют компилятор G++ (C++-обертку для GCC), с параметром `-g`, указывающим, что таблица символов должна быть сохранена. Мы также пробовали параметр `-gstabs`, который работал, но с несколько менее желательными результатами.

Глава 4. Когда программа падает

Говорят, что C - язык низкого уровня. Отчасти это означает, что значительная часть управления памятью для прикладной программы оставлена программисту. Хотя такой подход может быть весьма мощным, он также возлагает на программиста большую ответственность.

Также говорят, что C - относительно маленький язык и простой для изучения. Однако C мал только если не учитывать типичную реализацию стандартной библиотеки C, которая огромна, - и многие программисты считают C простым в использовании лишь до тех пор, пока не сталкиваются с указателями.

В общем случае ошибка в программе может привести к одному из двух событий:

- Она может заставить программу сделать что-то, чего программист не намеревался делать. Такие ошибки часто связаны с дефектами логики, как в программе сортировки чисел из главы 3, где мы поместили узел не в ту ветвь дерева. До сих пор мы сосредоточивались именно на этом типе ошибок.
- Она может заставить программу "рухнуть" или "упасть". Такие ошибки часто связаны с неправильной обработкой или неправильным использованием указателей. Именно с этим типом ошибок мы будем иметь дело в этой главе.

4.1 Предварительный материал: управление памятью

Что на самом деле происходит, когда программа падает? Здесь мы объясним это и покажем, как это связано с поиском ошибки, вызывающей падение.

4.1.1 Почему программа падает?

На разговорном языке мира программирования программа падает, когда ошибка заставляет ее прекратить выполнение внезапно и ненормально. Безусловно, самая распространенная причина падения - попытка программы обратиться к области памяти, к которой у нее нет разрешения на доступ. Аппаратное обеспечение обнаружит это и выполнит переход к операционной системе (ОС). На платформах семейства Unix, которые являются нашим основным фокусом здесь и в большей части этой книги, ОС обычно сообщает, что программа вызвала *segmentation fault*, обычно называемый *seg fault*, и прекращает выполнение программы. В системах Microsoft Windows соответствующий термин - *general protection fault*. Как бы это ни называлось, чтобы такая ошибка могла произойти, аппаратное обеспечение должно поддерживать виртуальную память, а ОС должна ее использовать. Хотя это стандартно для современных компьютеров общего назначения, читателю следует помнить, что для небольших специализированных компьютеров, например встроенных компьютеров, управляющих машинами, это часто не так.

Чтобы эффективно использовать GDB/DDO для работы с *seg faults*, важно точно понимать, как возникают ошибки доступа к памяти. На следующих нескольких страницах мы представим краткое введение в роль виртуальной памяти (VM) во время выполнения программ. Наш конкретный фокус будет на том, как вопросы VM связаны с *seg faults*. Поэтому даже если вы изучали VM в курсах по вычислительным системам, здешний фокус может дать вам новые идеи, которые помогут работать с *seg faults* при отладке.

4.1.2 Размещение программы в памяти

Как упоминалось ранее, seg fault возникает, когда у программы есть проблема доступа к памяти. Чтобы обсудить это, важно сначала понять, как программа располагается в памяти.

На платформах Unix набор выделенных программе виртуальных адресов обычно устроен примерно как на схеме на рисунке 4-1.

```
.text
.data
.bss
heap
unused
stack
env
```

Рисунок 4-1. Размещение программы в памяти

Здесь виртуальный адрес 0 находится внизу, а стрелки показывают направление роста двух компонентов, heap и stack, которые по мере роста поглощают свободную область. Роли различных частей таковы:

- Секция text состоит из машинных инструкций, произведенных компилятором из исходного кода вашей программы. Каждая строка C-кода, например, обычно переводится в две или три машинные инструкции, и совокупность всех получившихся инструкций образует text section исполняемого файла. Формальное имя этой секции - .text.

Этот компонент включает статически скомпонованный код, включая /usr/lib/crt0.o, системный код, который выполняет некоторую инициализацию, а затем вызывает ваш main().

- Секция data содержит все переменные программы, выделенные во время компиляции, то есть ваши глобальные переменные.

На самом деле эта секция состоит из различных подсекций. Первая называется .data и состоит из инициализированных переменных, то есть заданных в объявлениях вроде:

```
int x = 5;
```

Также есть секция .bss для неинициализированных данных, заданных в объявлениях вроде:

```
int y;
```

- Когда ваша программа запрашивает у операционной системы дополнительную память во время выполнения - например, когда вы вызываете malloc() в C или используете конструкцию new в C++, - запрошенная память выделяется в области, называемой heap. Если место в heap заканчивается, для расширения heap можно использовать вызов brk() (именно это и делают malloc() и родственные функции).
- Секция stack - это пространство для динамически выделяемых данных. Данные для вызовов функций, включая аргументы, локальные переменные и адреса возврата, хранятся в stack. Stack растет при каждом вызове функции и уменьшается каждый раз, когда функция возвращает управление вызывающему коду.
- Динамически скомпонованный код вашей программы не показан на рисунке выше из-за зависимости его расположения от платформы, но он где-то там находится.

Немного исследуем это. Рассмотрим следующий код:

```
int q[200];
int main( void )
{
    int i, n, *p;
    p = malloc(sizeof(int));
    scanf("%d", &n);
    for (i = 0; i < 200; i++)
        q[i] = i;
    printf("%x %x %x %x %x\n", main, q, p, &i, scanf);
    return 0;
}
```

Сама программа делает немного, но мы написали ее как инструмент для неформального исследования размещения виртуального адресного пространства. Для этого запустим ее:

```
% a.out
5
80483f4 80496a0 9835008 bfb3abec 8048304
```

Видно, что приблизительные расположения text section, data section, heap, stack и динамически скомпонованных функций - соответственно 0x080483f4, 0x080496a0, 0x09835008, 0xbfb3abec и 0x08048304.

Точную картину размещения памяти программы в Linux можно получить, посмотрев файл maps процесса. Номер процесса оказался 21111, поэтому посмотрим соответствующий файл /proc/21111/maps:

```
$ cat /proc/21111/maps
009f1000-009f2000 r-xp 009f1000 00:00 0 [vdso]
009f2000-00a0b000 r-xp 00000000 08:01 4116750 /lib/ld-2.4.so
00a0b000-00a0c000 r-xp 00018000 08:01 4116750 /lib/ld-2.4.so
00a0c000-00a0d000 rwxp 00019000 08:01 4116750 /lib/ld-2.4.so
00a0f000-00b3c000 r-xp 00000000 08:01 4116819 /lib/libc-2.4.so
00b3c000-00b3e000 r-xp 0012d000 08:01 4116819 /lib/libc-2.4.so
00b3e000-00b3f000 rwxp 0012f000 08:01 4116819 /lib/libc-2.4.so
00b3f000-00b42000 rwxp 00b3f000 00:00 0
08048000-08049000 r-xp 00000000 00:16 18815309 /home/matloff/a.out
08049000-0804a000 rw-p 00000000 00:16 18815309 /home/matloff/a.out
09835000-09836000 rw-p 09835000 00:00 0 [heap]
b7ef8000-b7ef9000 rw-p b7ef8000 00:00 0
b7f14000-b7f16000 rw-p b7f14000 00:00 0
bfb27000-bfb3c000 rw-p bfb27000 00:00 0 [stack]
```

Вам не нужно понимать все это. Суть в том, что в этом выводе можно увидеть ваши text и data sections (из файла a.out), а также heap и stack. Также можно увидеть, где была размещена библиотека C (для вызовов scanf(), malloc() и printf()) - из файла /lib/libc-2.4.so. Вы также должны узнать поле разрешений, формат которого похож на знакомые разрешения файлов, показываемые ls, например rw-p. Последнее вскоре будет объяснено.

4.1.3 Понятие страниц

Виртуальное адресное пространство, показанное на рисунке 4-1, концептуально простирается от 0 до $2^w - 1$, где w - размер машинного слова вашей машины в битах. Конечно, ваша программа обычно использует лишь крошечную долю этого пространства, а ОС может резервировать часть пространства для собственной работы. Но ваш код через указатели мог бы породить адрес где угодно в этом диапазоне. Часто такие адреса будут неправильными из-за "энтомологических условий", то есть из-за bugs в вашей программе!

Это виртуальное адресное пространство рассматривается как организованное в фрагменты, называемые pages. На аппаратуре Pentium размер страницы по умолчанию составляет 4096

байтов. Физическая память (и RAM, и ROM) также рассматривается как разделенная на страницы. Когда программа загружается в память для выполнения, ОС организует хранение некоторых страниц программы в страницах физической памяти. Такие страницы называют resident, а остальные хранятся на диске.

В разные моменты выполнения понадобится некоторая страница программы, которая сейчас не resident. Когда это происходит, аппаратное обеспечение обнаруживает ситуацию и передает управление ОС. Последняя приносит нужную страницу в память, возможно заменяя другую страницу программы, которая сейчас resident (если нет свободных страниц памяти), а затем возвращает управление нашей программе. Вытесненная страница программы, если такая есть, становится nonresident и будет храниться на диске.

Для управления всем этим ОС поддерживает page table для каждого процесса. (Page tables Pentium имеют иерархическую структуру, но здесь для простоты мы предполагаем всего один уровень, и большая часть этого обсуждения не будет специфичной для Pentium.) Каждая виртуальная страница процесса имеет запись в таблице, включающую следующую информацию:

- Текущее физическое расположение этой страницы в памяти или на диске. В последнем случае запись укажет, что страница nonresident, и может состоять из указателя на список, который в конечном счете ведет к физическому расположению на диске. Например, она может показывать, что виртуальная страница 12 программы является resident и расположена в физической странице 200 памяти.
- Разрешения - read, write, execute - для этой страницы.

Обратите внимание, что ОС не выделяет программе частичные страницы. Например, если запускаемая программа имеет общий размер около 10000 байтов, она занимала бы три страницы памяти при полной загрузке. Она не занимала бы всего около 2,5 страницы, поскольку страницы - наименьшая единица памяти, которой манипулирует система VM. Это важный момент для понимания при отладке, потому что он подразумевает, что некоторые ошибочные обращения программы к памяти не вызовут seg faults, как вы увидите ниже. Иными словами, во время отладочной сессии нельзя сказать что-то вроде: "Эта строка исходного кода должна быть в порядке, поскольку она не вызвала seg fault".

4.1.4 Подробности о роли page table

Держите в уме виртуальное адресное пространство из таблицы 4-1 и продолжайте предполагать, что размер страницы равен 4096 байтам. Тогда виртуальная страница 0 содержит байты от 0 до 4095 виртуального адресного пространства, страница 1 содержит байты от 4096 до 8191 и так далее.

Как упоминалось, когда мы запускаем программу, ОС создает page table, которую использует для управления виртуальной памятью процесса, выполняющего код программы. (Обзор процессов ОС представлен в материале о потоках в главе 5.) Всякий раз, когда этот процесс выполняется, регистр page table аппаратуры будет указывать на эту таблицу.

Концептуально каждая страница виртуального адресного пространства процесса имеет запись в page table (на практике для сжатия таблицы могут использоваться разные приемы). Эта запись page table хранит различные сведения, относящиеся к странице. Данные, интересующие нас в связи с seg faults, - это разрешения доступа к странице, похожие на разрешения доступа к файлам: read, write и execute. Например, запись page table для страницы 3 укажет, имеет ли ваш процесс право читать данные с этой страницы, право записывать данные на нее и право выполнять инструкции на ней (если страница содержит машинный код).

По мере выполнения программы она будет постоянно обращаться к своим различным секциям, описанным выше, что заставляет аппаратное обеспечение обращаться к page table следующим образом:

- Каждый раз, когда программа использует одну из своих глобальных переменных, требуется доступ read/write к data section.
- Каждый раз, когда программа обращается к локальной переменной, программа обращается к stack, требуя доступа read/write к stack section.
- Каждый раз, когда программа входит в функцию или выходит из нее, она выполняет одно или несколько обращений к stack, требуя доступа read/write к stack section.
- Каждый раз, когда программа обращается к хранилищу, созданному вызовом `malloc()` или `new`, происходит доступ к heap, снова требующий read/write.
- Каждая машинная инструкция, которую выполняет программа, будет извлечена из text section (или из области для динамически скомпонованного кода), тем самым требуя разрешений read и execute.

Во время выполнения программы порождаемые ею адреса будут виртуальными. Когда программа пытается обратиться к памяти по некоторому виртуальному адресу, скажем y , аппаратное обеспечение преобразует его в номер виртуальной страницы v , равный y , деленному на 4096 (где деление использует целочисленную арифметику, отбрасывая остаток). Затем аппаратное обеспечение проверит запись v в page table, чтобы увидеть, соответствуют ли разрешения страницы выполняемой операции. Если соответствуют, аппаратное обеспечение получит фактический номер физической страницы нужного места из этой записи таблицы, а затем выполнит запрошенную операцию с памятью. Но если запись таблицы показывает, что запрошенная операция не имеет должного разрешения, аппаратное обеспечение выполнит внутреннее прерывание. Это вызовет переход к процедуре ОС для обработки ошибок. Обычно ОС затем объявит нарушение доступа к памяти и прекратит выполнение программы (то есть удалит ее из таблицы процессов и из памяти).

Ошибка в вашей программе может привести к несовпадению разрешений и породить seg fault при любом из перечисленных выше типов доступа к памяти. Например, предположим, ваша программа содержит глобальное объявление:

```
int x[100];
```

и предположим, ваш код содержит оператор:

```
x[i] = 3;
```

Вспомните, что в C/C++ выражение $x[i]$ эквивалентно (и на самом деле означает) $*(x+i)$, то есть содержимое места памяти, на которое указывает адрес $x+i$. Если смещение i равно, скажем, 200000, то это, вероятно, породит виртуальный адрес памяти y , находящийся вне набора страниц, которые ОС назначила для data section программы, где компилятор и linker расположили массив $x[]$. Тогда при попытке операции записи произойдет seg fault.

Если бы x вместо этого был локальной переменной, та же проблема возникла бы в вашей stack section.

Нарушения, связанные с разрешением `execute`, могут возникать более тонкими способами. В программе на языке ассемблера, например, у вас может быть элемент данных с именем `sink` и функция с именем `sunk()`. При вызове функции вы можете случайно написать:

```
call sink
```

ВМЕСТО:

```
call sunk
```

Это вызвало бы `seg fault`, потому что программа попыталась бы выполнить инструкцию по адресу `sink`, который лежит в `data section`, а страницы `data section` не имеют включенного разрешения `execute`.

Точный аналог этой ошибки кодирования в C не привел бы к `seg fault`, поскольку компилятор возразил бы против строки вроде:

```
z = sunk(5);
```

когда `sink` объявлен как переменная. Но эта ошибка легко может возникнуть при использовании указателей на функции. Рассмотрим такой код:

```
int f(int x)
{
    return x*x;
}
int (*p)(int);
int main( void )
{
    p=f ;
    u=( *p)(5);
    printf("%d\n", u);
    return 0;
}
```

Если бы вы забыли оператор `p=f` ;, то `p` был бы равен 0, и вы попытались бы выполнить инструкции, лежащие на странице 0, странице, для которой у вас не было бы разрешения `execute` (или любого другого) (вспомните рисунок 4-1).

4.1.5 Небольшая ошибка доступа к памяти может не вызвать `seg fault`

Чтобы углубить понимание того, как возникают `seg faults`, рассмотрим следующий код, поведение которого при выполнении показывает, что `seg faults` не всегда возникают в ситуациях, где вы могли бы их ожидать:

```
int q[200];
main()
{
    int i;
    for (i = 0; i < 2000; i++) {
        q[i] = i;
    }
}
```

Обратите внимание, что программист, по-видимому, допустил опечатку в цикле, задав 2000 итераций вместо 200. Компилятор C не поймает это во время компиляции, и машинный код, сгенерированный компилятором, также не будет проверять во время выполнения, вышел ли индекс массива за границы. (Это поведение GCC по умолчанию, хотя он также предлагает параметр `-fudflap`, который обеспечивает такую проверку индексов во время выполнения.)

Во время выполнения `seg fault` весьма вероятен. Однако момент ошибки может вас удивить. Ошибка, скорее всего, появится не в "естественный" момент, то есть когда $i = 200$; скорее, она, вероятно, произойдет гораздо позже.

Чтобы проиллюстрировать это, мы запустили эту программу на Linux PC под GDB, чтобы удобно запрашивать адреса переменных. Оказалось, что `seg fault` произошел не при $i = 200$, а при $i = 728$. (Ваша система может дать другие результаты, но принципы будут теми же.) Посмотрим почему.

Из запросов к GDB мы выяснили, что массив `q[ ]` заканчивался по адресу `0x80497bf`; то есть последний байт `q[199]` находился в этом месте памяти. С учетом размера страницы Intel 4096 байтов и 32-битного размера слова этой машины виртуальный адрес распадается на 20-битный номер страницы и 12-битное смещение. В нашем случае `q[ ]` заканчивался на виртуальной странице номер $0x8049 = 32841$, смещение $0x7bf = 1983$. Значит, на странице памяти, на которой был выделен `q`, оставалось еще $4096 - 1984 = 2112$ байтов. Это пространство может вместить $2112 / 4 = 528$ целочисленных переменных (поскольку каждая занимает 4 байта на использованной здесь машине), и наш код обращался с ним так, будто оно содержало элементы `q` в "позициях" от 200 до 727.

Эти элементы `q[ ]`, конечно, не существуют, но компилятор не пожаловался. Аппаратное обеспечение тоже не пожаловалось, поскольку записи все еще выполнялись на страницу, для которой у нас определено было разрешение `write` (поскольку на ней лежала часть настоящих элементов `q[ ]`, а `q[ ]` выделен в `data segment`). Только когда i стал равен 728, `q[i]` стал ссылаться на адрес на другой странице. В данном случае это была страница, для которой у нас не было разрешения `write` (или любого другого); аппаратное обеспечение виртуальной памяти обнаружило это и вызвало `seg fault`.

Поскольку каждая целочисленная переменная хранится в 4 байтах, эта страница затем содержит 528 ($2112 / 4$) дополнительных "фантомных" элементов, которые код считает принадлежащими массиву `q[ ]`. Поэтому, хотя мы не намеревались этого делать, доступ к `q[200]`, `q[201]` и так далее, вплоть до элемента $199 + 528 = 727$, то есть `q[727]`, все еще законен - без вызова `seg fault`! Только когда вы пытаетесь обратиться к `q[728]`, вы сталкиваетесь с новой страницей, для которой у вас может быть или не быть требуемых разрешений доступа. Здесь их не было, и поэтому программа получила `seg fault`. Однако следующая страница могла по чистой случайности действительно иметь назначенные ей нужные привилегии, и тогда фантомных элементов массива было бы еще больше.

Мораль: как уже говорилось, из отсутствия `seg fault` нельзя заключать, что операция с памятью корректна.

4.1.6 Seg faults и сигналы Unix

В обсуждении выше мы сказали, что `seg fault` обычно приводит к завершению программы. Это верно, но для серьезной отладки есть еще немного, о чем следует знать в связи с сигналами Unix.

Сигналы указывают на исключительные условия и сообщаются во время выполнения программы, чтобы позволить ОС (или вашему собственному коду) реагировать на различные события. Сигнал может быть поднят для процесса лежащим в основе аппаратным обеспечением системы (как с `SIGSEGV` или `SIGFPE`), операционной системой (как с `SIGTERM` или `SIGABRT`), другим процессом (как с `SIGUSR1` или `SIGUSR2`) или даже может быть отправлен самим процессом самому себе (через библиотечный вызов `raise()`).

Простейший пример сигнала возникает при нажатии CTRL-C на клавиатуре во время работы программы. Нажатие (или отпускание) любой клавиши на клавиатуре порождает аппаратное

прерывание, которое заставляет выполниться процедуру ОС. Когда вы нажимаете CTRL-C, ОС распознает эту комбинацию клавиш как специальный шаблон и поднимает сигнал с именем SIGINT для процесса на управляющем терминале. В обычной речи говорят, что ОС "посылает сигнал процессу". Мы будем использовать эту фразу, но важно понимать, что процессу на самом деле ничего не "посылается". Все, что происходит, - ОС записывает сигнал в свою таблицу процессов, так что в следующий раз, когда процесс, получивший сигнал, получит квант времени на CPU, будет выполнена соответствующая функция signal handler, как объясняется ниже. (Однако, учитывая предполагаемую срочность сигналов, ОС также может решить дать принимающему процессу следующий квант времени раньше, чем сделала бы иначе.)

Существует много различных типов сигналов, которые могут быть подняты для процесса. В Linux весь список сигналов можно увидеть, набрав:

```
man 7 signal
```

в приглашении shell. Сигналы были определены в различных стандартах, таких как POSIX.1, и эти сигналы будут присутствовать во всех совместимых операционных системах. Есть также сигналы, уникальные для отдельных операционных систем.

У каждого сигнала есть свой signal handler - функция, вызываемая, когда этот конкретный сигнал поднят для процесса. Возвращаясь к примеру CTRL-C: когда поднимается SIGINT, ОС устанавливает текущую инструкцию процесса на начало signal handler для этого конкретного сигнала. Таким образом, когда процесс возобновляется, он выполнит handler.

Для каждого типа сигнала есть default signal handler, что удобно избавляет вас от необходимости писать их самостоятельно, если это не нужно. Большинство безвредных сигналов по умолчанию игнорируются. Более серьезные типы сигналов, например возникающие из-за нарушений разрешений доступа к памяти, указывают на условия, делающие дальнейшее выполнение программы нежелательным или даже невозможным. В таких случаях default signal handler просто завершает программу.

Некоторые signal handlers нельзя переопределить, но во многих случаях можно написать собственный handler, чтобы заменить default handler, предоставленный ОС. В Unix это делается с помощью системных вызовов signal() или sigaction().^[1] Ваша пользовательская функция handler может, например, игнорировать сигнал или даже попросить пользователя выбрать дальнейшее действие.

Просто ради забавы мы написали программу, которая показывает, как можно написать собственный signal handler и, используя signal(), вызвать или переопределить default OS handler либо игнорировать сигнал. Мы выбрали SIGINT, но то же самое можно сделать для любого сигнала, который можно перехватить. Программа также демонстрирует, как используется raise().

```
#include <signal.h>
#include <stdio.h>
void my_sigint_handler( int signum )
{
    printf("I received signal %d (that's 'SIGINT' to you).\n", signum);
    puts("Tee Hee! That tickles!\n");
}
int main(void)
{
    char choicestr[20];
    int choice;
    while ( 1 )
    {
        puts("1. Ignore control-C");
        puts("2. Custom handle control-C");
        puts("3. Use the default handler control-C");
        puts("4. Raise a SIGSEGV on myself.");
        printf("Enter your choice: ");
        fgets(choicestr, 20, stdin);
```

```

sscanf(choicestr, "%d", &choice);
if ( choice == 1 )
signal(SIGINT, SIG_IGN); // Ignore control-C
else if ( choice == 2 )
signal(SIGINT, my_sigint_handler);
else if ( choice == 3 )
signal(SIGINT, SIG_DFL);
else if ( choice == 4 )
raise(SIGSEGV);
else
puts("Whatever you say, guv'nor.\n\n");
}
return 0;
}

```

Когда программа совершает нарушение доступа к памяти, для процесса поднимается сигнал SIGSEGV. Default handler для seg fault завершает процесс и записывает на диск "core file", который мы вскоре объясним.

Если вы хотите сохранить программу живой, вместо того чтобы позволить ей завершиться, можно написать пользовательский handler для SIGSEGV. Действительно, вы можете захотеть намеренно вызывать seg faults, чтобы выполнить какую-то работу. Например, некоторые пакеты программного обеспечения для параллельной обработки используют искусственные seg faults, на которые реагирует специальный handler, чтобы поддерживать согласованность между различными узлами системы, как вы увидите в разделе 5. Другое применение специализированных handlers для SIGSEGV, которое будет обсуждаться в главе 7, связано с инструментами для обнаружения seg faults и изящной реакции на них.

Однако пользовательские signal handlers могут создавать осложнения при использовании GDB/DDD/Eclipse. Используется ли GDB сам по себе или через GUI DDD, GDB останавливает процесс всякий раз, когда возникает любой сигнал. В случае приложений, работающих подобно только что упомянутому программному обеспечению для параллельной обработки, это означает, что GDB будет останавливаться очень часто по причинам, не связанным с вашей отладочной работой. Чтобы справиться с этим, нужно сказать GDB не останавливаться при возникновении определенных сигналов, используя команду handle.

4.1.7 Другие типы исключений

Помимо segmentation faults есть и другие источники падений. Floating-point exceptions (FPEs) приводят к поднятию сигнала SIGFPE. Хотя это называется "floating-point" exception, этот сигнал также охватывает целочисленные арифметические исключения, например overflow и divide-by-zero. В системах GNU и BSD FPE handlers получают второй аргумент, который сообщает причину FPE. Default handler при некоторых обстоятельствах игнорирует SIGFPE, например при floating point overflow, а при других обстоятельствах завершает процесс, например при integer divide-by-zero.

Bus error возникает, когда CPU обнаруживает аномальное условие на своей шине во время выполнения машинных инструкций. Разные архитектуры предъявляют разные требования к тому, что должно происходить на шине, и точная причина аномалии зависит от архитектуры. Некоторые примеры ситуаций, которые могут вызвать bus error, включают следующее:

- Обращение к физическому адресу, который не существует. Это отличается от seg fault тем, что seg fault включает обращение к памяти, для которой недостаточно привилегий. Seg faults - это вопрос разрешений; bus errors - это вопрос недопустимого адреса, предъявленного процессору.

- На многих архитектурах машинные инструкции, обращающиеся к 32-битным величинам, должны быть word aligned, то есть адрес памяти этой величины должен быть кратен 4. Ошибка указателя, приводящая к попытке обратиться к 4-байтному числу по адресу с нечетным номером, может вызвать bus error:

```
int main(void)
{
    char *char_ptr;
    int *int_ptr;
    int int_array[2];
    // char_ptr points to first array element
    char_ptr = (char *) int_array;
    // Causes int_ptr to point one byte past the start of an existing int.
    // Since an int can't be only one byte, int_ptr is no longer aligned.
    int_ptr = (int *) (char_ptr+1);
    *int_ptr = 1; // And this might cause a bus error.
    return 0;
}
```

Эта программа не вызовет bus error в Linux, работающем на архитектуре x86, потому что на этих процессорах невыровненные обращения к памяти допустимы; они просто выполняются медленнее, чем выровненные обращения.

В любом случае bus error - это исключение уровня процессора, которое в системе Unix приводит к поднятию сигнала SIGBUS. По умолчанию SIGBUS заставляет процесс dump core и завершиться.

4.2 Core files

Как упоминалось ранее, некоторые сигналы указывают, что процессу нежелательно или даже невозможно продолжать работу. В таких случаях default action - преждевременно завершить процесс и записать файл, называемый core file, что в разговорной речи известно как dumping core. Запись core files может быть подавлена вашим shell (подробности см. в разделе 4.2.2).

Если core file создан во время запуска вашей программы, можно открыть ваш отладчик, скажем GDB, на этом файле, а затем продолжить обычные операции GDB.

4.2.1 Как создаются core files

Core file содержит подробное описание состояния программы в момент ее гибели: содержимое stack (или, если программа многопоточная, stacks для каждого потока), содержимое регистров CPU (снова с одним набором значений регистров на поток, если программа многопоточная), значения статически выделенных переменных программы (глобальных и static variables) и так далее.

Создать core file очень просто. Вот код, который порождает такой файл:

```
int main(void)
{
    abort();
    return 0;
}
```

Листинг 4-1. abort.c

Функция `abort()` заставляет текущий процесс получить сигнал `SIGABRT`, а default signal handler для `SIGABRT` завершает программу и `dumps core`. Вот еще одна короткая программа, которая `dumps core`. В этой программе мы намеренно разыменовываем указатель `NULL`:

```
int main(void)
{
    char *c=0 ;
    printf("%s\n", *c);
    return 0;
}
```

Листинг 4-2. sigsegv.c

Сгенерируем core file. Скомпилируйте и запустите `sigsegv.c`:

```
$ gcc -g -W -Wall sigsegv.c -o sigsegv
$ ./sigsegv
Segmentation fault (core dumped)
```

Если вывести список текущего каталога, вы заметите новый файл с именем `core` (или каким-то его вариантом). Когда вы видите core file где-то в своей файловой системе, может быть неочевидно, какая программа его породила. Команда `Unix file` любезно сообщает нам имя исполняемого файла, который dump этот конкретный core file:

```
$ file core
core: ELF 32-bit LSB core file Intel 80386, version 1 (SYSV), SVR4-style,
SVR4-style, from 'sigsegv'
```

Соглашения об именовании core files

Соглашение об именовании core files раньше было простым: все они назывались `core`.

Затем в GNU/Linux многопоточные программы начали dump core, используя имена файлов вроде `core.3928`, где числовая часть имени файла указывает process ID процесса, который dump core.

Начиная с ядра Linux 2.5, вы можете управлять именами, назначаемыми core files, используя интерфейс `/proc/sys/kernel/`. Механизм довольно прост и хорошо документирован в `Documentation/sysctl/kernel.txt` в дереве исходного кода ядра Linux.

4.2.2 Ваш shell может подавлять создание core file

Во многих, если не в большинстве случаев, процесс отладки не включает core files. Если программа получает `seg fault`, программист просто открывает отладчик, например `GDB`, и запускает программу снова, чтобы воспроизвести fault. По этой причине, а также потому, что core files обычно велики, большинство современных shells имеют механизмы, препятствующие записи core files с самого начала.

В bash созданием core files можно управлять с помощью команды `ulimit`:

```
ulimit -c n
```

где `n` - максимальный размер core file в килобайтах. Любой core file больше `n` KB не будет записан. Если не указать `n`, shell покажет текущее ограничение на core files. Если вы хотите разрешить core file любого размера, можно использовать:

```
ulimit -c unlimited
```

Для пользователей `tcsh` и `csh` размером core files управляет команда `limit`. Например:

```
limit coredumpsize 1000000
```

скажет shell, что вы не хотите создавать core file, если он будет больше миллиона байтов.

Если после запуска `sigsegv` вы не получили core file, проверьте текущие ограничения core file с помощью `ulimit -c` для `bash` или `limit -c` для `tcsh` или `csh`.

Зачем вообще может понадобиться core file? Поскольку можно просто заново запустить программу, получившую `seg fault`, изнутри GDB и воспроизвести `seg fault`, зачем вообще возиться с core files? Ответ в том, что в некоторых ситуациях, например следующих, это предположение неоправданно:

- `Seg fault` возникает только после того, как программа работала долгое время, так что воспроизвести `fault` в отладчике невозможно практически.
- Поведение программы зависит от случайных, средовых событий, так что повторный запуск программы может не воспроизвести `seg fault`.
- `Seg fault` возникает, когда программа запускается наивным пользователем. Здесь пользователь, который обычно не является программистом (или не имеет доступа к исходному коду), не занимался бы отладкой. Однако такой пользователь все еще мог бы отправить core file (если он доступен) программисту для проверки и отладки.

Заметьте, однако, что если исходный код программы недоступен или если исполняемый файл не был скомпилирован с расширенной таблицей символов, или даже когда мы не планируем отлаживать исполняемый файл, core files просто не очень полезны.

4.3 Расширенный пример

В этом разделе мы представляем подробный пример отладки `seg faults`.

Ниже приведен C-код, который может быть частью реализации управляемого строкового типа, похожего на строки C++. Код, содержащийся в исходном файле `cstring.c`, реализует тип с именем `CString`; однако он полон ошибок, очевидных и тонких. Наша цель - найти все эти ошибки и исправить их.

`CString` - это alias, созданный с помощью `typedef`, для структуры, содержащей указатель на хранилище строки `char` вместе с переменной, которая хранит длину строки. Реализовано несколько служебных функций, полезных для обработки строк:

`Init_CString()` Принимает old-style C string как аргумент и использует ее для инициализации нового `CString`.

`Delete_CString()` `CStrings` выделяются в heap, и их память должна освобождаться, когда она больше не нужна. Эта функция заботится о garbage collection.

`Chomp()` Удаляет и возвращает последний символ `CString`.

`Append_Chars_To_CString()` Добавляет C-style string к `CString`.

Наконец, `main()` - наша driver function для проверки реализации `CString`.

Наш код использует чрезвычайно полезную библиотечную функцию `snprintf()`. Если вы еще не встречали эту функцию, она почти как `printf()`, за исключением того, что записывает свой вывод в массив символов, а не в `stdout`. Чтобы помочь предотвратить `buffer overflows` (которые могут происходить с любой функцией, копирующей `null-terminated strings`, если `null character` отсутствует в исходной строке), `snprintf()` также позволяет указать максимальное число байтов для записи, включая завершающий `null character`:

```
#include <stdio.h>
#define STRSIZE 22
int main(void)
{
    char s1[] = "brake";
    char *s2 = "breakpoints";
    char logo[STRSIZE];
    snprintf(logo, STRSIZE, "%c %s %d %s.", 'I', s1, 2+2, s2);
    puts(logo);
    return 0;
}
```

Эта программа запишет строку "I brake 4 breakpoints." в массив символов `logo`, готовую к печати на наклейке для бампера.

А теперь вот реализация нашего `CString`:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
typedef struct {
    char *str;
    int len;
} CString;
CString *Init_CString(char *str)
{
    CString *p = malloc(sizeof(CString));
    p->len = strlen(str);
    strncpy(p->str, str, strlen(str) + 1);
    return p;
}
void Delete_CString(CString *p)
{
    free(p);
    free(p->str);
}
// Removes the last character of a CString and returns it.
//
char Chomp(CString *cstring)
{
    char lastchar = *( cstring->str + cstring->len);
    // Shorten the string by one
    *( cstring->str + cstring->len) = '\0';
    cstring->len = strlen( cstring->str );
    return lastchar;
}
// Appends a char * to a CString
//
CString *Append_Chars_To_CString(CString *p, char *str)
{
    char *newstr = malloc(p->len + 1);
    p->len = p->len + strlen(str);
    // Create the new string to replace p->str
    snprintf(newstr, p->len, "%s%s", p->str, str);
    // Free old string and make CString point to the new string
    free(p->str);
    p->str = newstr;
    return p;
}
int main(void)
```

```

{
CString *mystr;
char c;
mystr = Init_CString("Hello!");
printf("Init:\n str: `%s' len: %d\n", mystr->str, mystr->len);
c = Chomp(mystr);
printf("Chomp '%c':\n str: `%s' len: %d\n", c, mystr->str, mystr->len);
mystr = Append_Chars_To_CString(mystr, " world!");
printf("Append:\n str: `%s' len: %d\n", mystr->str, mystr->len);
Delete_CString(mystr);
return 0;
}

```

Изучите код и попробуйте угадать, каким должен быть вывод. Затем скомпилируйте и запустите его.

```

$ gcc -g -W -Wall cstring.c -o cstring
$ ./cstring
Segmentation fault (core dumped)

```

Упс! Первое, что нам нужно сделать, - выяснить, где произошел seg fault. Затем можно попытаться понять, почему он произошел.

Прежде чем продолжить, хотим упомянуть, что наш офисный коллега из соседней кабинки, Милтон, тоже пытается исправить ошибки в этой программе. В отличие от нас, Милтон не умеет пользоваться GDB, поэтому он собирается открыть Wordpad, вставить вызовы printf() по всему коду и перекомпилировать программу, пытаясь выяснить, где произошел seg fault. Посмотрим, сможем ли мы отладить программу быстрее Милтона.

Пока Милтон открывает Wordpad, мы используем GDB, чтобы исследовать core file:

```

$ gdb cstring core
Core was generated by `cstring'.
Program terminated with signal 11, Segmentation fault.
#0 0x400a9295 in strncpy () from /lib/tls/libc.so.6
(gdb) backtrace
#0 0x400a9295 in strncpy () from /lib/tls/libc.so.6
#1 0x080484df in Init_CString (str=0x80487c5 "Hello!") at cstring.c:15
#2 0x080485e4 in main () at cstring.c:62

```

Согласно выводу backtrace, seg fault произошел на строке 15, в Init_CString(), во время вызова strncpy(). Даже не глядя на код, мы уже знаем, что довольно велика вероятность, что на строке 15 мы передали в strncpy() указатель NULL.

В этот момент Милтон все еще пытается решить, куда вставить первый из множества вызовов printf().

4.3.1 Первая ошибка

GDB сказал нам, что seg fault произошел на строке 15 в Init_CString(), поэтому мы сменим текущий frame на frame для вызова Init_CString().

```

(gdb) frame 1
#1 0x080484df in Init_CString (str=0x80487c5 "Hello!") at cstring.c:15
15 strncpy(p->str, str, strlen(str) + 1);

```

Мы применим Principle of Confirmation, посмотрев на каждый из аргументов-указателей, переданных в `strncpy()`, а именно `str`, `p` и `p->str`, и проверив, что их значения таковы, какими мы их считаем. Сначала печатаем значение `str`:

```
(gdb) print str
$1 = 0x80487c5 "Hello!"
```

Поскольку `str` - указатель, GDB дает нам его значение как шестнадцатеричный адрес `0x80487c5`. А поскольку `str` - указатель на `char`, и тем самым адрес символьной строки, GDB любезно сообщает нам также значение строки: "Hello!" Это было ясно и из вывода `backtrace` выше, но все равно следует проверить. Итак, `str` не `NULL` и указывает на корректную строку, и пока все в порядке.

Теперь обратим внимание на другие аргументы-указатели, `p` и `p->str`:

```
(gdb) print *p
$ 2={
  str = 0x0,
  len = 6
}
```

Теперь проблема ясна: `p->str`, который тоже является указателем на строку, равен `NULL`. Это объясняет `seg fault`: мы попытались записать в место 0 в памяти, которое для нас запрещено.

Но что могло заставить `p->str` (указатель на строку в создаваемом `CString`) быть `NULL`? Что ж, взглянув на код:

```
(gdb) list Init_CString
5 typedef struct {
6   char *str;
7   int len;
8 } CString;
9
10
11 CString *Init_CString(char *str)
12 {
13   CString *p = malloc(sizeof(CString));
14   p->len = strlen(str);
15   strncpy(p->str, str, strlen(str) + 1);
16   return p;
17 }
18
```

мы видим, что до строки, в которой произошел `seg fault`, есть всего две строки кода, и из них строка 13 гораздо более вероятный виновник.

Мы заново запустим программу изнутри GDB, установим `temporary breakpoint` на входе в `Init_CString()` и пройдем эту функцию строка за строкой, наблюдая значение `p->str`.

```
(gdb) tbreak Init_CString
Breakpoint 1 at 0x804849b: file cstring.c, line 13.
(gdb) run
Breakpoint 1, Init_CString (str=0x80487c5 "Hello!") at cstring.c:13
13 CString *p = malloc(sizeof(CString));
(gdb) step
14 p->len = strlen(str);
(gdb) print p->str
$4 = 0x0
(gdb) step
15 strncpy(p->str, str, strlen(str) + 1);
```

Вот проблема: мы собираемся совершить `seg fault`, потому что следующая строка кода разыменовывает `p->str`, а `p->str` все еще `NULL`. Теперь используем маленькие серые клеточки, чтобы понять, что произошло.

Когда мы выделили память для `p`, мы получили достаточно памяти для нашей структуры: указатель, чтобы хранить адрес строки, и `int`, чтобы хранить длину строки, но мы не выделили память для хранения самой строки. Мы совершили обычную ошибку: объявили указатель и не заставили его ни на что указывать! Что нам нужно сделать: сначала выделить достаточно памяти для хранения `str`, а затем заставить `p->str` указывать на эту вновь выделенную память. Вот как это можно сделать (нужно прибавить единицу к длине строки, потому что `strlen()` не считает завершающий `'\0'`):

```
CString *Init_CString(char *str)
{
    // Allocate for the struct
    CString *p = malloc(sizeof(CString));
    p->len = strlen(str);
    // Allocate for the string
    p->str = malloc(p->len + 1);
    strncpy(p->str, str, strlen(str) + 1);
    return p;
}
```

Кстати, Милтон только что закончил вставлять вызовы `printf()` в свой код и собирается перекомпилировать. Если ему повезет, он узнает, где произошел `seg fault`. Если нет, ему придется добавить еще больше вызовов `printf()`.

4.3.2 Не выходите из GDB во время отладочной сессии

Во время отладочной сессии мы никогда не выходим из GDB, пока вносим изменения в код. Как обсуждалось ранее, так мы избегаем затратных по времени запусков, сохраняем наши breakpoints и так далее.

Аналогично, мы держим текстовый редактор открытым. Оставаясь в той же сессии редактора между компиляциями во время отладки, мы можем хорошо использовать возможность "undo" нашего редактора. Например, обычная стратегия в процессе отладки - временно удалить части кода, чтобы сосредоточиться на оставшихся участках, где, как вы думаете, находится ошибка. Закончив проверку, можно просто использовать `undo` редактора, чтобы восстановить удаленные строки.

Итак, на экране у нас обычно будет одно окно для GDB (или DDD) и одно окно для редактора. Также у нас либо было бы открыто третье окно для выдачи команд компилятора, либо, что лучше, мы выполняли бы их через редактор. Например, если вы используете редактор Vim, можно выдать следующую команду, которая сохранит ваши изменения и перекомпилирует программу за один шаг:

```
: make
```

(Мы предполагаем, что вы установили переменную Vim `autowrite`, используя `set autowrite`, в startup file Vim. Эта возможность Vim также переместит курсор к первому сообщенному warning или error компиляции, если такие есть, и вы сможете переходить туда и обратно по списку ошибок компиляции с помощью команд Vim `:cnext` и `:cprev`. Конечно, все это становится проще, если поместить короткие aliases для этих команд в startup file Vim.)

4.3.3 Вторая и третья ошибки

После исправления первой ошибки снова запускаем программу изнутри GDB (помните, что когда GDB замечает, что вы перекомпилировали программу, он автоматически загрузит новый исполняемый файл, так что снова нет нужды выходить из GDB и перезапускать его):

```
(gdb) run
The program being debugged has been started already.
Start it from the beginning? (y or n) y
`cstring' has changed; re-reading symbols.
Starting program: cstring
Init:
str: `Hello!' len: 6
Chomp '':
str: `Hello!0' len: 7
Append:
str: `Hello!0 world' len: 14
Program exited normally.
(gdb)
```

Похоже, с `Chomp()` есть две проблемы. Во-первых, он должен был `chomp` символ '!', но похоже, что он `chomp` непечатаемый символ. Во-вторых, в конце нашей строки появился символ нуля. Поскольку `Chomp()` - очевидное место для поиска этих ошибок, мы запустим программу и поставим temporary breakpoint на входе в `Chomp()`.

```
(gdb) tbreak Chomp
Breakpoint 2 at 0x8048523: file cstring.c, line 32.
(gdb) run
Starting program: cstring
Init:
str: `Hello!' len: 6
Breakpoint 1, Chomp (cstring=0x804a008) at cstring.c:32
32 char lastchar = *( cstring->str + cstring->len);
(gdb)
```

Последним символом строки должен быть '!'. Подтвердим это.

```
(gdb) print lastchar
$1 = 0 '\0'
```

Мы ожидали, что `lastchar` будет '!', но вместо этого это null character. Похоже, это, вероятно, ошибка "off by one". Разберемся. Строку можно представить так:

```
pointer offset: 0 1 2 3 4 5 6
cstring->str: H e l l o ! \0
string length: 1 2 3 4 5 6
```

Последний символ строки хранится по адресу `cstring->str + 5`, но поскольку длина строки - это счетчик символов, а не индекс, адрес `cstring->str + cstring->len` указывает на одну ячейку массива после последнего символа, где находится завершающий NULL, а не туда, куда мы хотели. Можно исправить эту проблему, изменив строку 31 с:

```
char lastchar = *( cstring->str + cstring->len);
```

на:

```
char lastchar = *( cstring->str + cstring->len - 1);
```

В этой части кода скрывается третья ошибка. После вызова `Chomp()` строка "Hello!" стала "Hello!0" (вместо "Hello"). Следующая строка для выполнения в GDB, строка 33, - это место, где мы хотели укоротить строку, заменив ее последний символ завершающим null character:

```
*( cstring->str + cstring->len) = '\0';
```

Мы сразу видим, что эта строка содержит ту же проблему, которую мы только что исправили в строке 31: мы неправильно ссылаемся на последний символ строки. Более того, теперь, когда наши глаза нацелены на эту строку кода, кажется, что мы сохраняем в этом месте символ '0', который не является null character. Мы хотели поместить '\0' в конец строки. После внесения этих двух исправлений строка 33 выглядит так:

```
*( cstring->str + cstring->len - 1) = '\0';
```

К этому моменту Милтон, наш коллега, использующий `printf()`, нашел первый seg fault и только сейчас исправляет проблему выделения памяти в `Init_CString()`. Вместо того чтобы перейти к ошибкам, которые мы только что исправили в `Chomp()`, ему придется удалить все вызовы `printf()` и перекомпилировать программу. Как неудобно!

4.3.4 Четвертая ошибка

Внесем исправления, обсуждавшиеся в предыдущем разделе, перекомпилируем код и снова запустим программу:

```
(gdb) run
`cstring' has changed; re-reading symbols.
Starting program: cstring
Init:
str: `Hello!' len: 6
Chomp '!':
str: `Hello' len: 5
Append:
str: `Hello world' len: 12
Program received signal SIGSEGV, Segmentation fault.
0xb7f08da1 in free () from /lib/tls/libc.so.6
(gdb)
```

Еще один seg fault. Судя по отсутствующему восклицательному знаку после операции `append`, кажется, что следующая ошибка может скрываться в `Append_Chars_To_CString()`. Простой `backtrace` должен подтвердить или опровергнуть эту гипотезу:

```
1 (gdb) backtrace
2 #0 0xb7f08da1 in free () from /lib/tls/libc.so.6
3 #1 0x0804851a in Delete_CString (p=0x804a008) at cstring3.c:24
4 #2 0x08048691 in main () at cstring3.c:70
5 (gdb)
```

Согласно строке 3 вывода `backtrace`, наше предположение неверно: программа на самом деле упала в `Delete_CString()`. Это не означает, что у нас также нет ошибки в `Append_Chars_To_CString()`, но наша непосредственная ошибка, та, которая вызвала seg fault, находится в `Delete_CString()`. Именно поэтому мы используем здесь GDB, чтобы проверять ожидания: он полностью убирает догадки при поиске места, где произошел seg fault. Когда наш друг, использующий `printf()`, догонит нас в своей отладке, он будет вставлять трассировочный код не в ту функцию!

К счастью, `Delete_CString()` короткая, так что мы должны быстро найти, что не так.


```
(gdb) list Delete_CString
20
21 void Delete_CString(CString *p)
22 {
23     free(p);
24     free(p->str);
25 }
26
```

Сначала мы освобождаем `p`, затем освобождаем `p->str`. Это считается не такой уж тонкой ошибкой. После освобождения `p` нет гарантии, что `p->str` все еще указывает на правильное место в памяти; он может указывать куда угодно. В данном случае это "куда угодно" оказалось памятью, к которой мы не могли обратиться, что вызвало `seg fault`. Исправление - поменять порядок вызовов `free()`:

```
void Delete_CString(CString *p)
{
    free(p->str);
    free(p);
}
```

Кстати, Милтон слишком расстроился, пытаясь отследить ошибку `off-by-one` в `Chomp()`, которую мы так легко исправили. Это он сейчас звонит нам по телефону за помощью.

4.3.5 Пятая и шестая ошибки

Мы еще раз исправляем, перекомпилируем и заново запускаем код.

```
(gdb) run
`cstring' has changed; re-reading symbols.
Starting program: cstring
Init:
str: `Hello!' len: 6
Chomp '!':
str: `Hello' len: 5
Append:
str: `Hello world' len: 12
Program exited normally.
(gdb)
```

После операции `append` в строке отсутствует восклицательный знак; строка должна быть "Hello world!". Любопытно, что сообщенная длина строки 12 корректна, хотя сама строка некорректна. Самое логичное место для поиска этой ошибки - `Append_Chars_To_CString()`, поэтому поставим там `breakpoint`:

```
(gdb) tbreak Append_Chars_To_CString
Breakpoint 3 at 0x8048569: file cstring.c, line 45.
(gdb) run
Starting program: cstring
Init:
str: `Hello!' len: 6
Chomp '!':
str: `Hello' len: 5
Breakpoint 1, Append_Chars_To_CString (p=0x804a008, str=0x8048840 " world!")
at cstring.c:45
45 char *newstr = malloc(p->len + 1);
```

C string `newstr` должна быть достаточно большой, чтобы вместить и `p->str`, и `str`. Мы видим, что вызов `malloc()` на строке 45 не выделяет достаточно памяти; он выделяет место только для `p->str` и завершающего `null`. Строку 45 следует изменить на:

```
char *newstr = malloc(p->len + strlen(str) + 1);
```

После внесения этого исправления и перекомпиляции мы получаем следующий вывод:

```
(gdb) run
`cstring' has changed; re-reading symbols.
Starting program: cstring
Init:
str: `Hello!' len: 6
Chomp '!':
str: `Hello' len: 5
Append:
str: `Hello world' len: 12
```

Наше исправление не устранило ошибку, которую мы имели в виду. То, что мы нашли и исправили, было "silent bug". Не сомневайтесь: это была ошибка, а тот факт, что она не проявилась как `seg fault`, был чистой удачей. Весьма вероятно, что оставшаяся ошибка все еще находится в `Append_Chars_To_CString()`, поэтому установим там еще один `breakpoint`:

```
(gdb) tbreak Append_Chars_To_CString
Breakpoint 4 at 0x8048569: file cstring.c, line 45.
(gdb) run
Starting program: cstring
(gdb) run
Init:
str: `Hello!' len: 6
Chomp '!':
str: `Hello' len: 5
Breakpoint 1, Append_Chars_To_CString (p=0x804a008, str=0x8048840 " world!")
at cstring.c:45
45 char *newstr = malloc(p->len + strlen(str) + 1);
(gdb) step
46 p->len = p->len + strlen(str);
```

Строка 46 показывает, почему длина строки корректна, хотя сама строка некорректна: сложение правильно вычисляет длину `p->str`, соединенной с `str`. Здесь проблемы нет, поэтому шагнем дальше.

```
(gdb) step
49 snprintf(newstr, p->len, "%s%s", p->str, str);
```

Следующая строка кода, строка 49, - место, где мы формируем новую строку. Мы ожидаем, что `newstr` будет содержать "Hello world!" после этого шага. Применим Principle of Confirmation и проверим это.

```
(gdb) step
51 free(p->str);
(gdb) print newstr
$2 = 0x804a028 "Hello world"
```

Восклицательный знак отсутствует в строке, построенной на строке 51 кода, так что ошибка, вероятно, происходит на строке 49, но в чем она может быть? В вызове `snprintf()` мы попросили скопировать в `newstr` максимум `p->len` байтов. Значение `p->len`, как подтверждено, было 12, а текст "Hello world!" содержит 12 символов. Мы не сказали `snprintf()` скопировать завершающий `null character` исходной строки. Но тогда разве мы не должны были получить `malformed string`, с восклицательным знаком в последней позиции и без `null`?

Это одна из замечательных вещей в `snprintf()`. Он всегда копирует завершающий `null character` в `target`. Если вы ошиблись и указали максимальное число символов для копирования, меньшее фактического числа символов в `source` (как мы сделали здесь), `snprintf()` скопирует столько символов, сколько сможет, но последний символ, записанный в `target`, гарантированно будет `null`.

character. Чтобы исправить нашу ошибку, нужно сказать `snprintf()` скопировать достаточно байтов, чтобы вместить текст source string и завершающий null.

Поэтому строку 45 нужно изменить. Вот полная исправленная функция:

```
CString *Append_Chars_To_CString(CString *p, char *str)
{
    char *newstr = malloc(p->len + strlen(str) + 1);
    p->len = p->len + strlen(str);
    // Create the new string to replace p->str
    snprintf(newstr, p->len + 1, "%s%s", p->str, str);
    // Free old string and make CString point to the new string
    free(p->str);
    p->str = newstr;
    return p;
}
```

Перекомпилируем исправленный код и запустим программу:

```
(gdb) run
`cstring' has changed; re-reading symbols.
Starting program: cstring
Init:
str: `Hello!' len: 6
Chomp '!':
str: `Hello' len: 5
Append:
str: `Hello world!' len: 12
Program exited normally.
(gdb)
```

Выглядит хорошо!

Мы прошли довольно большую территорию и столкнулись с несколькими сложными понятиями, но это того стоило. Даже если наша ошибочная реализация CString была немного надуманной, наша отладочная сессия была вполне реалистичной и охватывает многие аспекты отладки:

- Principle of Confirmation
- использование core files для post-mortem analysis процесса, который упал
- исправление, компиляция и повторный запуск программы без выхода из GDB
- недостаточность отладки в стиле `printf()`
- использование старой доброй силы мозга - ей нет замены

Если вы пришли из мира отладки в стиле `printf()`, стоит подумать, насколько труднее было бы отследить некоторые из этих ошибок с помощью `printf()`. Диагностический код с `printf()` имеет свое место в отладке, но как универсальный "инструмент" он ужасающе недостаточен и неэффективен для отслеживания большинства ошибок, возникающих в реальном коде.

Сноски

[1] [\[назад\]](#) Используются две функции для переопределения default signal handlers, потому что Linux, как и другие Unix, соответствует нескольким стандартам. Функция `signal()`, которую проще использовать, чем `sigaction()`, соответствует стандарту ANSI, тогда как функция `sigaction()` сложнее, но и более универсальна, и соответствует стандарту POSIX.

Глава 5. Отладка в контексте нескольких активностей

Отладка - трудная задача, и она становится еще сложнее, когда приложение, ведущее себя неправильно, пытается координировать несколько одновременных активностей; сетевое программирование клиент/сервер, программирование с потоками и параллельная обработка - примеры этой парадигмы. В этой главе представлен обзор наиболее часто используемых методов мультипрограммирования и даны некоторые советы о том, как работать с ошибками в программах такого рода, с фокусом на использовании GDB/DDD/Eclipse в процессе отладки.

5.1 Отладка client/server network programs

Компьютерные сети - чрезвычайно сложные системы, и строгая отладка сетевых программных приложений иногда может требовать использования аппаратных monitors для сбора подробной информации о network traffic. Только об этой теме отладки можно было бы написать целую книгу. Наша цель здесь - просто ввести тему.

Наш пример состоит из следующей пары client/server. Клиентское приложение позволяет пользователю проверить load на машине, на которой работает серверное приложение, даже если у пользователя нет учетной записи на этой машине. Клиент отправляет серверу запрос информации - здесь это запрос о load системы сервера через команду Unix w - по сетевому соединению. Затем сервер обрабатывает запрос и возвращает результаты, захватывая вывод w и отправляя его обратно по соединению. В общем случае сервер может принимать запросы от нескольких удаленных клиентов; чтобы сохранить наш пример простым, предположим, что существует только один экземпляр клиента.

Код сервера показан ниже:

```
1 // svr.c
2
3 // a server to remotely run the w command
4 // user can check load on machine without login privileges
5 // usage: svr
6
7 #include <stdio.h>
8 #include <sys/types.h>
9 #include <sys/socket.h>
10 #include <netinet/in.h>
11 #include <netdb.h>
12 #include <fcntl.h>
13 #include <string.h>
14 #include <unistd.h>
15 #include <stdlib.h>
16
17 #define WPORT 2000
18 #define BUFSIZE 1000 // assumed sufficient here
19
20 int clntdesc, // socket descriptor for individual client
21 svrdesc; // general socket descriptor for server
22
23 char outbuf[BUFSIZE]; // messages to client
24
25 void respond()
26 { int fd,nb;
27
28  memset(outbuf,0,sizeof(outbuf)); // clear buffer
29  system("w > tmp.client"); // run 'w' and save results
30  fd = open("tmp.client",O_RDONLY);
```

```

31 nb = read(fd,outbuf,BUFSIZE); // read the entire file
32 write(clntdesc,outbuf,nb); // write it to the client
33 unlink("tmp.client"); // remove the file
34 close(clntdesc);
35 }
36
37 int main()
38 { struct sockaddr_in bindinfo;
39
40 // create socket to be used to accept connections
41 svrdesc = socket(AF_INET,SOCK_STREAM,0);
42 bindinfo.sin_family = AF_INET;
43 bindinfo.sin_port = WPORT;
44 bindinfo.sin_addr.s_addr = INADDR_ANY;
45 bind(svrdesc,(struct sockaddr *) &bindinfo,sizeof(bindinfo));
46
47 // OK, listen in loop for client calls
48 listen(svrdesc,5);
49
50 while (1) {
51 // wait for a call
52 clntdesc = accept(svrdesc,0,0);
53 // process the command
54 respond();
55 }
56 }

```

Вот код клиента:

```

1 // clnt.c
2
3 // usage: clnt server_machine
4
5 #include <stdio.h>
6 #include <sys/types.h>
7 #include <sys/socket.h>
8 #include <netinet/in.h>
9 #include <netdb.h>
10 #include <string.h>
11 #include <unistd.h>
12
13 #define WPORT 2000 // server port number
14 #define BUFSIZE 1000
15
16 int main(int argc, char **argv)
17 { int sd,msgsize;
18
19 struct sockaddr_in addr;
20 struct hostent *hostptr;
21 char buf[BUFSIZE];
22
23 // create socket
24 sd = socket(AF_INET,SOCK_STREAM,0);
25 addr.sin_family = AF_INET;
26 addr.sin_port = WPORT;
27 hostptr = gethostbyname(argv[1]);
28 memcpy(&addr.sin_addr.s_addr,hostptr->h_addr_list[0],hostptr->h_length);
29
30 // OK, now connect
31 connect(sd,(struct sockaddr *) &addr,sizeof(addr));
32
33 // read and display response
34 msgsize = read(sd,buf,BUFSIZE);
35 if (msgsize > 0)
36 write(1,buf,msgsize);
37 printf("\n");
38 return 0;
39 }

```

Для тех, кто не знаком с client/server programming, вот обзор того, как работают эти программы.

На строке 41 кода сервера вы создаете `socket`, абстракцию, похожую на `file descriptor`; так же как `file descriptor` используют для выполнения операций I/O над объектом файловой системы, через `socket` читают из сетевого соединения и пишут в него. На строке 45 `socket` привязывается к конкретному `port number`, произвольно выбранному как 2000. (User-level applications вроде этой ограничены номерами портов 1024 и выше.) Этот номер идентифицирует "mailbox" в системе сервера, куда клиенты отправляют запросы, которые должны быть обработаны для именно этого приложения.

Сервер "открывает дело", вызывая `listen()` на строке 48. Затем он ждет поступления клиентского запроса, вызывая `accept()` на строке 52. Этот вызов `blocks` до прихода запроса. Затем он возвращает новый `socket` для общения с клиентом. (Когда есть несколько клиентов, исходный `socket` продолжает принимать новые запросы даже во время обслуживания существующего запроса, отсюда необходимость отдельных `sockets`. Это потребовало бы реализации сервера в `threaded fashion`.) Сервер обрабатывает запрос клиента функцией `respond()` и отправляет клиенту информацию о `machine load`, локально вызывая команду `w` и записывая результаты в `socket` на строке 32.

Клиент создает `socket` на строке 24, а затем использует его на строке 31 для подключения к порту 2000 сервера. На строке 34 он читает `load information`, отправленную сервером, а затем печатает ее.

Вот как должен выглядеть вывод клиента:

```
$ clnt laura.cs.ucdavis.edu
13:00:15 up 13 days, 39 min, 7 users, load average: 0.25, 0.13, 0.09
USER TTY FROM LOGIN@ IDLE JCPU PCPU WHAT
matloff :0 - 14Jun07 ?xdm? 25:38 0.15s -/bin/tcsh -c /
matloff pts/1 :0.0 14Jun07 17:34 0.46s 0.46s -csh
matloff pts/2 :0.0 14Jun07 18:12 0.39s 0.39s -csh
matloff pts/3 :0.0 14Jun07 58.00s 2.18s 2.01s /usr/bin/mutt
matloff pts/4 :0.0 14Jun07 0.00s 1.85s 0.00s clnt laura.cs.u
matloff pts/5 :0.0 14Jun07 20.00s 1.88s 0.02s script
matloff pts/7 :0.0 19Jun07 4days 22:17 0.16s -csh
```

Теперь предположим, что программист забыл строку 26 в коде клиента, которая задает порт в системе сервера, к которому нужно подключиться:

```
addr.sin_port = WPORT;
```

Представим, что мы не знаем, в чем ошибка, и посмотрим, как можно ее отследить.

Вывод клиента теперь был бы таким:

```
$ clnt laura.cs.ucdavis.edu
$
```

Похоже, клиент вообще ничего не получил обратно от сервера. Конечно, это могло бы быть вызвано множеством причин либо в сервере, либо в клиенте, либо в обоих.

Осмотримся с помощью GDB. Сначала проверим, действительно ли клиент успешно подключился к серверу. Установите `breakpoint` на вызове `connect()` и запустите программу:

```
(gdb) b 31
Breakpoint 1 at 0x8048502: file clnt.c, line 31.
(gdb) r laura.cs.ucdavis.edu
Starting program: /fandrhome/matloff/public_html/matloff/public_html/Debug
/Book/DDD/clnt laura.cs.ucdavis.edu
Breakpoint 1, main (argc=2, argv=0xbf81a344) at clnt.c:31
31 connect(sd,(struct sockaddr *) &addr,sizeof(addr));
```

Используйте GDB, чтобы выполнить `connect()` и проверить возвращаемое значение на condition ошибки:

```
(gdb) p connect(sd,&addr,sizeof(addr))
$ 1=- 1
```

Это действительно -1, код failure. Это большая подсказка. (Конечно, как matter of defensive programming, когда мы писали код клиента, мы проверили бы возвращаемое значение `connect()` и обработали бы случай failure to connect.)

Кстати, обратите внимание: при ручном выполнении вызова `connect()` нужно убрать `cast`. Если сохранить `cast`, вы получили бы ошибку:

```
(gdb) p connect(sd,(struct sockaddr *) &addr,sizeof(addr))
No struct type named sockaddr.
```

Это связано с особенностью GDB и возникает потому, что мы не использовали эту `struct` в другом месте программы.

Также обратите внимание: если бы попытка `connect()` в сессии GDB увенчалась успехом, вы не смогли бы затем продолжить и выполнить строку 31. Попытка открыть уже открытый `socket` является ошибкой.

Вам пришлось бы перепрыгнуть через строку 31 и перейти прямо к строке 34. Это можно сделать с помощью команды GDB `jump`, выдав `jump 34`, но вообще эту команду следует использовать осторожно, поскольку она может привести к пропуску некоторых машинных инструкций, которые нужны ниже по коду. Поэтому, если бы попытка соединения прошла успешно, вы, вероятно, захотели бы перезапустить программу.

Попытаемся отследить причину failure, проверив аргумент `addr` в вызове `connect()`:

```
(gdb) p addr
...
connect(3, {sa_family=AF_INET, sin_port=htons(1032),
sin_addr=inet_addr("127.0.0.1")}, 16) = -1 ECONNREFUSED (Connection refused)
...
```

Aha! Значение `htons(1032)` указывает порт 2052 (см. ниже), а не ожидаемый нами 2000. Это говорит о том, что вы либо неправильно указали порт, либо вообще забыли его указать. Если проверить, вы быстро обнаружите, что имел место второй вариант.

Снова было бы разумно включить в исходный код немного механики, помогающей процессу отладки, например проверку возвращаемых значений системных вызовов. Другой полезный шаг - включить строку:

```
#include <errno.h>
```

которая в нашей системе создает глобальную переменную `errno`, значение которой можно напечатать изнутри кода или изнутри GDB:

```
(gdb) p errno
$1 = 111
```

Из файла `/usr/include/linux/errno.h` вы обнаружите, что этот номер ошибки кодирует `connection refused error`.

Однако реализация библиотеки `errno` может отличаться от платформы к платформе. Например, `header file` может иметь другое имя, или `errno` может быть реализован как вызов макроса, а не как переменная.

Другой подход - использовать `strace`, который трассирует все системные вызовы, выполняемые программой:

```
$ strace cInt laura.cs
...
connect(3, {sa_family=AF_INET, sin_port=htons(1032),
sin_addr=inet_addr("127.0.0.1")}, 16) = -1 ECONNREFUSED (Connection refused)
...
```

Это дает две важные части информации по цене одной. Во-первых, вы сразу видите, что была ошибка `ECONNREFUSED`. Во-вторых, вы также видите, что порт был `htons(1032)`, что имеет значение 2052. Последнее значение можно проверить, выдав команду вроде:

```
(gdb) p htons(1032)
```

изнутри GDB; она показывает значение 2052, что явно не 2000, как ожидалось.

Вы обнаружите, что `strace` - удобный инструмент во многих контекстах (сетевых и не только) для проверки результатов системных вызовов.

В качестве другого примера предположим, что вы случайно опустили запись клиенту в коде сервера (строка 32):

```
write(cIntdesc, outbuf, nb); // write it to the client
```

В этом случае клиентская программа зависла бы, ожидая ответа, который не приходит. Конечно, в этом простоватом примере вы сразу заподозрили бы проблему с вызовом `write()` в сервере и быстро обнаружили бы, что мы его забыли. Но в более сложных программах причина может быть не так очевидна. В таких случаях вы, вероятно, настроили бы две одновременные сессии GDB, одну для клиента и одну для сервера, проходя обе программы в tandem. Вы обнаружили бы, что в некоторый момент их совместной работы клиент зависает, ожидая услышать ответ от сервера, и таким образом получили бы подсказку о вероятном месте ошибки внутри сервера. Затем вы сосредоточили бы внимание на серверной сессии GDB, пытаясь понять, почему в этот момент он не отправил данные клиенту.

В действительно сложных случаях сетевой отладки open source программа `ethereal` может использоваться для отслеживания отдельных TCP/IP packets.

5.2 Отладка threaded code

Threaded programming стало весьма популярным. Для Unix наиболее распространенный пакет потоков - стандарт POSIX Pthreads, поэтому мы будем использовать его для нашего примера в этом разделе. Принципы похожи и для других thread packages.

5.2.1 Обзор процессов и потоков

Современные операционные системы используют timesharing, чтобы управлять несколькими запущенными программами так, чтобы пользователю казалось, будто они выполняются одновременно. Конечно, если у машины больше одного CPU, более одной программы действительно могут выполняться одновременно, но для простоты мы предположим всего один процессор; в этом случае одновременность только кажущаяся.

Каждый экземпляр запущенной программы представлен ОС как process (в терминологии Unix) или task (в Windows). Поэтому несколько запусков одной программы, выполняющихся в одно и то же время (например, одновременные сессии текстового редактора vi), являются отдельными processes. Processes должны "ходить по очереди" на машине с одним CPU. Для конкретности предположим, что "ходы", называемые timeslices, имеют длину 30 миллисекунд.

После того как process выполнялся 30 миллисекунд, аппаратный timer испускает interrupt, который заставляет ОС выполняться. Мы говорим, что process был preempted. ОС сохраняет текущее состояние прерванного process, чтобы его можно было возобновить позже, затем выбирает следующий process, которому дать timeslice. Это известно как context switch, потому что среда выполнения CPU переключилась с одного process на другой. Этот цикл повторяется бесконечно.

Очередь может закончиться рано. Например, когда process должен выполнить input/output, он в конечном счете вызывает функцию ОС, выполняющую низкоуровневые аппаратные операции; например, вызов функции библиотеки C scanf() приводит к вызову системного вызова Unix OS read(), который взаимодействует с keyboard driver. Таким образом process уступает свой ход ОС, и ход заканчивается рано.

Одно из следствий этого состоит в том, что планирование timeslices для данного process довольно случайно. Время, которое требуется пользователю, чтобы подумать, а затем нажать клавишу, случайно, поэтому момент начала его следующего timeslice непредсказуем. Более того, если вы отлаживаете threaded program, вы не знаете порядок, в котором будут scheduled threads; это может усложнить отладку.

Вот немного больше подробностей: ОС поддерживает process table, в которой перечислена информация обо всех текущих processes. Грубо говоря, каждый process помечен в таблице как находящийся либо в состоянии Run, либо в состоянии Sleep. Рассмотрим пример, в котором выполняющаяся программа достигает точки, где ей нужно прочитать ввод с клавиатуры. Как только что отмечено, это завершит ход process. Поскольку process теперь ждет завершения I/O, ОС помечает его как находящийся в состоянии Sleep, делая его недоступным для timeslices.

Таким образом, состояние Sleep означает, что process blocked, ожидая наступления некоторого события. Когда это событие наконец произойдет позже, ОС затем изменит его состояние в process table обратно на Run.

Не-I/O события также могут вызвать переход в состояние Sleep. Например, если parent process создает child process и вызывает wait(), parent будет block, пока child не завершит свою работу и не terminated. Снова, когда именно это произойдет, обычно непредсказуемо.

Кроме того, нахождение в состоянии Run не означает, что process действительно выполняется на CPU; скорее, это просто означает, что он готов выполняться, то есть имеет право на processor timeslice. При context switch ОС выбирает process, которому следующим дается ход на CPU, из тех, кто сейчас находится в состоянии Run, согласно process table. Процедура scheduling, используемая ОС для выбора нового context, гарантирует, что любой данный process продолжит получать timeslices и в итоге завершится, но не обещает, какие именно timeslices он получит. Поэтому то, когда sleeping process фактически "проснется" после наступления ожидаемого события, случайно, как и точная скорость продвижения process к завершению.

Thread очень похож на process, за исключением того, что он спроектирован так, чтобы занимать меньше памяти и требовать меньше времени на создание и переключение, чем processes. Действительно, threads иногда называют "lightweight" processes, и, в зависимости от thread system и run-time environment, они даже могут быть реализованы как processes операционной системы. Подобно программам, которые порождают processes для выполнения работы, multithreaded application обычно выполняет процедуру main(), которая создает один или несколько child threads. Parent, main(), тоже является thread.

Главное различие между processes и threads состоит в том, что хотя у каждого thread есть собственные local variables, как и у process, global variables родительской программы в threaded environment совместно используются всеми threads и служат основным методом коммуникации между threads. (Разделять globals между Unix processes возможно, но делать это неудобно.)

В системе Linux можно увидеть все processes и threads, находящиеся сейчас в системе, выполнив команду `ps axH`.

Существуют nonpreemptive thread systems, но Pthreads использует preemptive thread management policy, и thread в программе может быть прерван в любой момент другим thread. Поэтому элемент случайности, описанный выше для processes в timeshared system, также возникает в поведении threaded program. В результате некоторые bugs в приложениях, разработанных с использованием Pthreads, не так-то просто воспроизводятся.

5.2.2 Базовый пример

Сохраним простоту и используем в качестве примера следующий код для поиска prime numbers. Программа использует классическое Sieve of Eratosthenes. Чтобы найти все primes от 2 до n , мы сначала перечисляем все числа, затем вычеркиваем все multiples of 2, затем все multiples of 3 и так далее. Все числа, оставшиеся в конце, являются prime numbers.

```
1 // finds the primes between 2 and n; uses the Sieve of Eratosthenes,
2 // deleting all multiples of 2, all multiples of 3, all multiples of 5,
3 // etc.; not efficient, e.g. each thread should do deleting for a whole
4 // block of values of base before going to nextbase for more
5
6 // usage: sieve nthreads n
7 // where nthreads is the number of worker threads
8
9 #include <stdio.h>
10 #include <math.h>
11 #include <pthread.h>
12
13 #define MAX_N 100000000
14 #define MAX_THREADS 100
15
16 // shared variables
17 int nthreads, // number of threads (not counting main())
18 n, // upper bound of range in which to find primes
19 prime[MAX_N+1], // in the end, prime[i] = 1 if i prime, else 0
20 nextbase; // next sieve multiplier to be used
21
22 int work[MAX_THREADS]; // to measure how much work each thread does,
23 // in terms of number of sieve multipliers checked
24
25 // lock index for the shared variable nextbase
26 pthread_mutex_t nextbaselock = PTHREAD_MUTEX_INITIALIZER;
27
28 // ID structs for the threads
29 pthread_t id[MAX_THREADS];
30
31 // "crosses out" all multiples of k, from k*ko n
32 void crossout(int k)
33 { int i;
34
35   for (i = k; i*k <= n; i++) {
36     prime[i*k] = 0;
37   }
38 }
39
40 // worker thread routine
41 void *worker(int tn) // tn is the thread number (0,1,...)
42 { int lim,base;
43
```

```

44 // no need to check multipliers bigger than sqrt(n)
45 lim = sqrt(n);
46
47 do {
48 // get next sieve multiplier, avoiding duplication across threads
49 pthread_mutex_lock(&nextbaselock);
50 base = nextbase += 2;
51 pthread_mutex_unlock(&nextbaselock);
52 if (base <= lim) {
53 work[tn]++; // log work done by this thread
54 // don't bother with crossing out if base is known to be
55 // composite
56 if (prime[base])
57 crossout(base);
58 }
59 else return;
60 } while (1);
61 }
62
63 main(int argc, char **argv)
64 { int nprimes, // number of primes found
65 totwork, // number of base values checked
66 i;
67 void *p;
68
69 n = atoi(argv[1]);
70 nthreads = atoi(argv[2]);
71 for (i = 2; i <= n; i++)
72 prime[i] = 1;
73 crossout(2);
74 nextbase = 1;
75 // get threads started
76 for (i = 0; i < nthreads; i++) {
77 pthread_create(&id[i], NULL, (void *) worker, (void *) i);
78 }
79
80 // wait for all done
81 totwork = 0;
82 for (i = 0; i < nthreads; i++) {
83 pthread_join(id[i], &p);
84 printf("%d values of base done\n", work[i]);
85 totwork += work[i];
86 }
87 printf("%d total values of base done\n", totwork);
88
89 // report results
90 nprimes = 0;
91 for (i = 2; i <= n; i++)
92 if (prime[i]) nprimes++;
93 printf("the number of primes found was %d\n", nprimes);
94
95 }

```

В этой программе есть два аргумента командной строки: upper bound n диапазона, проверяемого на primes, и $nthreads$, число worker threads, которые мы хотим создать.

Здесь `main()` создает worker threads, каждый из которых является invocation функции `worker()`. Workers совместно используют три data items: переменную upper bound, n ; переменную, задающую следующее число, whose multiples are to be eliminated from range $2..n$, `nextbase`; и массив `prime[]`, который записывает для каждого числа в диапазоне $2..n$, было оно eliminated или нет. Каждый invocation repeatedly получает еще не обработанный elimination multiplicand, `base`, а затем eliminates all multiples of `base` from range $2..n$. После spawning workers `main()` использует `pthread_join()`, чтобы дождаться завершения работы всех этих threads, прежде чем возобновить себя; в этот момент он считает оставшиеся primes и выдает отчет. Отчет включает не только prime count, но и информацию о том, сколько работы сделал каждый worker thread. Такая оценка полезна для load balancing и performance optimization на multiprocessor system.

Каждый экземпляр `worker()` получает следующее значение `base`, выполняя следующий код (строки 49-51):

```
pthread_mutex_lock(&nextbaselock);
base = nextbase += 2;
pthread_mutex_unlock(&nextbaselock);
```

Здесь global variable `nextbase` обновляется и используется для инициализации значения local variable `base` экземпляра `worker()`; затем `worker` вычеркивает multiples of `base` в массиве `prime[]`. (Обратите внимание, что мы начали с устранения всех multiples of 2 в начале `main()`, и после этого нужно рассматривать только нечетные значения для `base`.)

После того как `worker` знает значение `base`, которое нужно использовать, он может безопасно вычеркивать multiples of `base` из shared array `prime[]`, потому что никакой другой `worker` не будет использовать это значение `base`. Однако мы должны поместить guard statements вокруг операции обновления shared variable `nextbase`, от которой зависит `base` (строка 26). Вспомните, что любой `worker thread` может быть preempted в непредсказуемое время другим `worker thread`, который будет находиться в непредсказуемом месте кода `worker()`. В частности, может случиться так, что текущий `worker` будет прерван посреди оператора:

```
base = nextbase += 2;
```

а следующий timeslice будет дан другому thread, который также выполняет тот же оператор. В таком случае два workers пытаются одновременно изменить shared variable `nextbase`, что может привести к коварным и трудно воспроизводимым bugs.

Обрамление кода, который манипулирует shared variable - известного как critical section, - guard statements предотвращает это. Вызовы `pthread_mutex_lock()` и `pthread_mutex_unlock()` гарантируют, что в enclosed program fragment находится не более одного thread. Они говорят ОС разрешить thread войти в critical section только если никакой другой thread сейчас ее не выполняет, и не preempt этот thread, пока он не завершит всю section. (Lock variable `nextbaselock` используется thread system internally для обеспечения этого "mutual exclusion".)

К сожалению, в threaded code слишком легко не распознать critical sections и/или не защитить их правильно. Посмотрим, как GDB можно использовать для отладки такого рода ошибки в программе Pthreads. Предположим, мы забыли включить оператор `unlock`:

```
pthread_mutex_unlock(&nextbaselock);
```

Это, конечно, заставляет программу зависнуть, как только critical section впервые entered worker thread, поскольку другие worker threads будут ждать вечно, пока lock не будет relinquished. Но представим, что вы этого еще не знаете. Как отследить виновника с помощью GDB?

Скомпилируйте программу, убедившись, что включили flags `-lpthread -lm`, чтобы скомпоновать библиотеки Pthreads и math (последняя нужна для вызова `sqrt()`). Затем запустите код в GDB с `n = 100` и `nthreads = 2`:

```
(gdb) r 100 2
Starting program: /debug/primes 100 2
[New Thread 16384 (LWP 28653)]
[New Thread 32769 (LWP 28676)]
[New Thread 16386 (LWP 28677)]
[New Thread 32771 (LWP 28678)]
```

Каждый раз, когда создается новый thread, GDB сообщает об этом, как показано здесь. Через минуту мы посмотрим, какой thread есть какой.

Программа зависает, и вы прерываете ее, нажав CTRL-C. Сессия GDB теперь выглядит так:

```
(gdb) r 100 2
Starting program: /debug/primes 100 2
[New Thread 16384 (LWP 28653)]
[New Thread 32769 (LWP 28676)]
[New Thread 16386 (LWP 28677)]
[New Thread 32771 (LWP 28678)]
Program received signal SIGINT, Interrupt.
[Switching to Thread 32771 (LWP 28678)]
0x4005ba35 in __pthread_sigsuspend () from /lib/i686/libpthread.so.0
```

В такой момент критически важно знать, что делает каждый thread; это можно определить с помощью команды GDB `info threads`:

```
(gdb) info threads
* 4 Thread 32771 (LWP 28678) 0x4005ba35 in __pthread_sigsuspend ()
  from /lib/i686/libpthread.so.0
  3 Thread 16386 (LWP 28677) 0x4005ba35 in __pthread_sigsuspend ()
  from /lib/i686/libpthread.so.0
  2 Thread 32769 (LWP 28676) 0x420db1a7 in poll () from
  /lib/i686/libc.so.6
  1 Thread 16384 (LWP 28653) 0x4005ba35 in __pthread_sigsuspend ()
  from /lib/i686/libpthread.so.0
```

Звездочка означает, что сейчас вы находитесь в thread 4. Посмотрим, что происходит с этим thread:

```
(gdb) bt
#0 0x4005ba35 in __pthread_sigsuspend () from /lib/i686/libpthread.so.0
#1 0x4005adb8 in __pthread_wait_for_restart_signal ()
  from /lib/i686/libpthread.so.0
#2 0x4005d190 in __pthread_alt_lock () from /lib/i686/libpthread.so.0
#3 0x40059d77 in pthread_mutex_lock () from /lib/i686/libpthread.so.0
#4 0x0804855f in worker (tn=1) at Primes.c:49
#5 0x40059881 in pthread_start_thread () from /lib/i686/libpthread.so.0
#6 0x40059985 in pthread_start_thread_event () from
  /lib/i686/libpthread.so.0
```

(Это работает в реализации Pthreads LinuxThreads, но может быть невозможно на некоторых других платформах.)

Ага - в frames 3 и 4 вы видите, что этот thread находится на строке 49 исходного кода и пытается acquire lock и войти в critical section:

```
pthread_mutex_lock(&nextbaselock);
```

Также обратите внимание из frame 0 выше, что thread, по-видимому, *suspended*, ожидая, пока lock будет relinquished другим thread. Он не будет получать timeslices, пока это не произойдет и thread manager не организует для него acquire lock.

Что делают другие threads? Можно inspect stack любого thread, переключившись на этот thread и затем выдав команду `bt`:

```
(gdb) thread 3
[Switching to thread 3 (Thread 16386 (LWP 28677))]#0 0x4005ba35 in
__pthread_sigsuspend () from /lib/i686/libpthread.so.0
(gdb) bt
#0 0x4005ba35 in __pthread_sigsuspend () from /lib/i686/libpthread.so.0
#1 0x4005adb8 in __pthread_wait_for_restart_signal ()
  from /lib/i686/libpthread.so.0
#2 0x4005d190 in __pthread_alt_lock () from /lib/i686/libpthread.so.0
#3 0x40059d77 in pthread_mutex_lock () from /lib/i686/libpthread.so.0
#4 0x0804855f in worker (tn=0) at Primes.c:49
#5 0x40059881 in pthread_start_thread () from /lib/i686/libpthread.so.0
#6 0x40059985 in pthread_start_thread_event () from
  /lib/i686/libpthread.so.0
```

Вспомните, что мы создали два worker threads. Вы видели выше, что thread 4 был одним из них (frame 4 из его вывода bt), а теперь из frame 4 здесь же вывода видите, что thread 3 - другой. Также видно, что thread 3 тоже пытается acquire lock (frame 3).

Других worker threads быть не должно, но один из фундаментальных принципов отладки состоит в том, что ничто не принимается на веру, а все должно проверяться. Теперь мы делаем это, inspecting status remaining threads. Вы обнаружите, что другие два threads - nonworker threads, как показано ниже:

```
(gdb) thread 2
[Switching to thread 2 (Thread 32769 (LWP 28676))]#0 0x420db1a7 in poll
()
from /lib/i686/libc.so.6
(gdb) bt
#0 0x420db1a7 in poll () from /lib/i686/libc.so.6
#1 0x400589de in __pthread_manager () from /lib/i686/libpthread.so.0
#2 0x4005962b in __pthread_manager_event () from
/lib/i686/libpthread.so.0
```

Итак, thread 2 - threads manager. Это внутреннее для пакета Pthreads. Он определенно не worker thread, что частично подтверждает наше ожидание, что worker threads всего два. Проверяя thread 1:

```
(gdb) thread 1
[Switching to thread 1 (Thread 16384 (LWP 28653))]#0 0x4005ba35 in
__pthread_sigsuspend () from /lib/i686/libpthread.so.0
(gdb) bt
#0 0x4005ba35 in __pthread_sigsuspend () from /lib/i686/libpthread.so.0
#1 0x4005adb8 in __pthread_wait_for_restart_signal ()
from /lib/i686/libpthread.so.0
#2 0x40058551 in pthread_join () from /lib/i686/libpthread.so.0
#3 0x080486aa in main (argc=3, argv=0xbfffe7b4) at Primes.c:83
#4 0x420158f7 in __libc_start_main () from /lib/i686/libc.so.6
```

вы обнаружите, что он выполняет main(), и тем самым можете подтвердить, что worker threads всего два.

Однако оба workers stalled, каждый ожидает, пока lock будет relinquished. Неудивительно, что программа зависла! Этого достаточно, чтобы pinpoint location and nature of the bug, и мы быстро понимаем, что забыли вызов unlocking function.

5.2.3 Вариация

Что если вы вообще не осознали необходимость guarding update shared variable nextbase? Что произошло бы в предыдущем примере, если бы вы опустили и unlock, и lock operations?

Наивный взгляд на этот вопрос мог бы привести к предположению, что с точки зрения корректной работы программы вреда не было бы (то есть мы получили бы точный count числа primes), хотя, возможно, с замедлением из-за duplicate work (то есть использования одного и того же значения base более одного раза). Казалось бы, некоторые threads могут дублировать работу других, а именно когда двум workers случается grab одно и то же значение nextbase, чтобы инициализировать свои local copies of base. Тогда некоторые composite numbers могли бы быть crossed out twice, но результаты (то есть count числа primes) все равно были бы корректны.

Но посмотрим внимательнее. Оператор:

```
base = nextbase += 2;
```

компилируется как минимум в две machine language instructions. Например, при использовании компилятора GCC на машине Pentium под Linux C-оператор выше переводится в следующие инструкции assembly language (полученные запуском GCC с параметром -S и затем просмотром получившегося файла .s):

```
addl $2, nextbase
movl nextbase, %eax
movl %eax, -8(%ebp)
```

Этот код увеличивает nextbase на 2, затем копирует значение nextbase в register EAX и, наконец, копирует значение EAX в место в stack worker, где хранится его local variable base.

Предположим, у вас всего два worker threads, значение nextbase равно, скажем, 9, и timeslice текущего running invocation worker() заканчивается сразу после выполнения машинной инструкции:

```
addl $2, nextbase
```

которая устанавливает shared global variable nextbase в 11. Предположим, следующий timeslice получает другой invocation worker(), который как раз выполняет те же инструкции. Вторым worker теперь увеличивает nextbase до 13, использует это для установки своей local variable base и начинает eliminate all multiples of 13. В конце концов первый invocation worker() получит еще один timeslice и продолжит с места остановки, выполняя machine instructions:

```
movl nextbase, %eax
movl %eax, -8(%ebp)
```

Конечно, значение nextbase теперь 13. Поэтому первый worker устанавливает значение своей local variable base в 13 и продолжает eliminate multiples of this value, а не значения 11, которое он установил во время своего прошлого timeslice. Ни один worker ничего не делает с multiples of 11. В итоге вы не только unnecessarily duplicate work, но и skipping necessary work!

Как можно обнаружить такую ошибку с помощью GDB? Предположительно surfaced "symptom" состоял в том, что reported number of primes было слишком большим. Поэтому вы могли бы заподозрить, что значения base somehow sometimes skipped. Чтобы проверить эту гипотезу, можно поставить breakpoint сразу после строки:

```
base = nextbase += 2;
```

Многократно выдавая команду GDB continue (c) и отображая значение base:

```
(gdb) disp base
```

вы могли бы в конце концов verify, что некоторое значение base действительно skipped.

Ключевое слово здесь - "могли бы". Вспомните наше более раннее обсуждение того, что threaded programs выполняются в несколько случайной манере. В здешнем контексте может оказаться, что в одних запусках программы bug surfaces (то есть reported too many primes), а в других вы можете получать correct answers!

К сожалению, хорошего решения этой проблемы нет. Debugging threaded code часто требует дополнительного терпения и изобретательности.

5.2.4 Краткая сводка thread-команд GDB

Вот краткая сводка использования thread-related commands GDB:

- `info threads` (дает информацию обо всех current threads)
- `thread 3` (переключается на thread 3)
- `break 88 thread 3` (останавливает выполнение, когда thread 3 достигает source line 88)
- `break 88 thread 3 if x==y` (останавливает выполнение, когда thread 3 достигает source line 88 и переменные `x` и `y` равны)

5.2.5 Thread-команды в DDD

В DDD выберите `Status | Threads`, и всплывет окно, отображающее все threads в манере GDB `info threads`, как видно на рисунке 5-1. Можно щелкнуть thread, чтобы переключить focus отладчика на него.

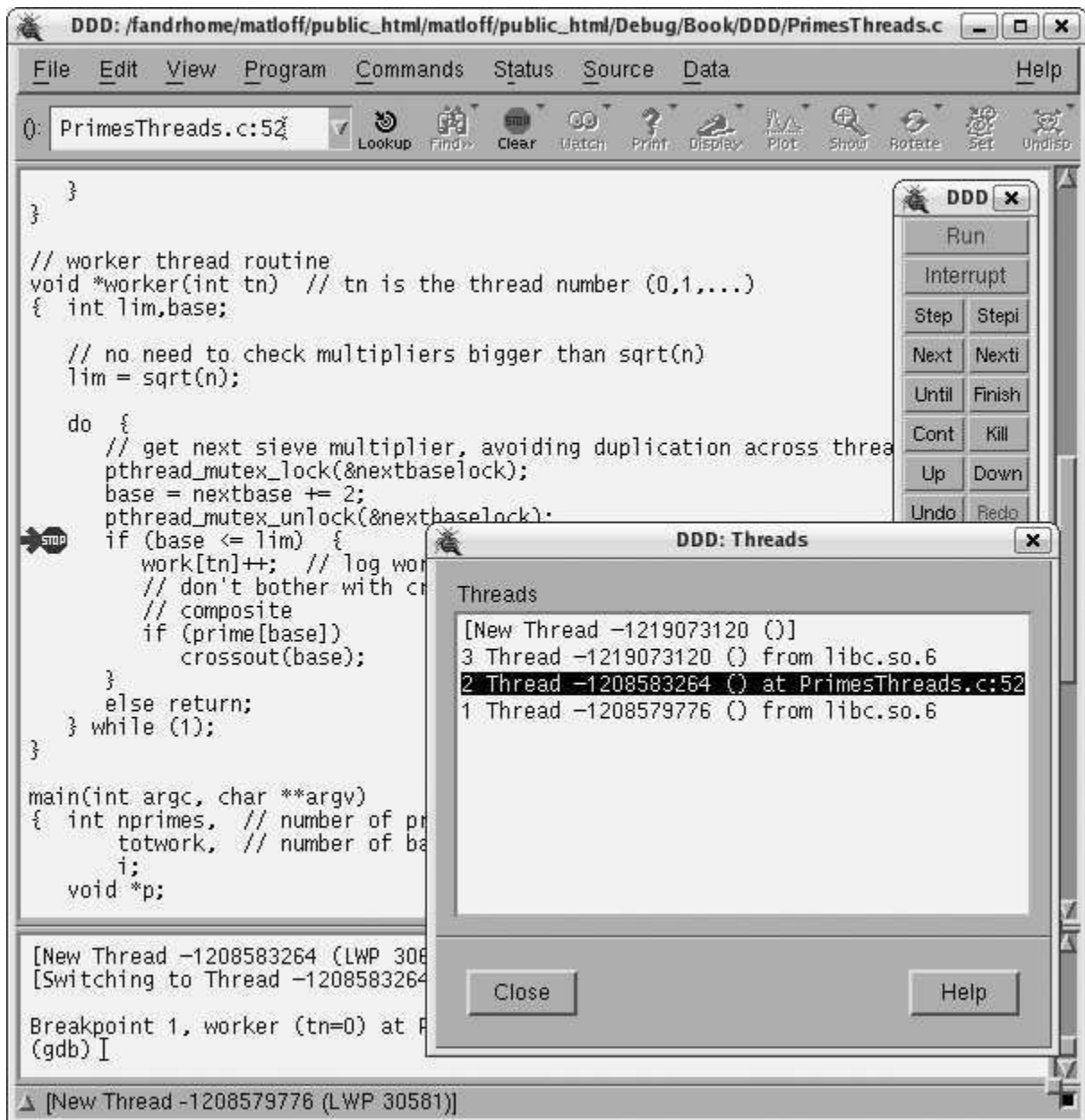


Рисунок 5-1. Окно `Threads`

Вероятно, вам захочется оставить это всплывающее окно открытым, а не использовать его один раз и закрыть. Так не придется снова открывать его каждый раз, когда вы хотите увидеть, какой thread сейчас выполняется, или переключиться на другой thread.

Похоже, в DDD нет способа сделать breakpoint thread-specific, как вы сделали с командой GDB `break 88 thread 3` выше. Вместо этого такую команду выдают GDB через DDD Console.

5.2.6 Thread-команды в Eclipse

Сначала обратите внимание, что default makefile, созданный Eclipse, не будет включать command-line argument `-lpthread` для GCC (и не будет включать arguments для любых других специальных библиотек, которые вам нужны). При желании можно изменить makefile напрямую, но проще сказать Eclipse сделать это за вас. Находясь в C/C++ perspective, щелкните имя проекта правой кнопкой мыши и выберите Properties; направьте треугольник рядом с C/C++ Build вниз; выберите Settings | Tool Settings; направьте треугольник рядом с GCC C Linker вниз и выберите Libraries | Add (последнее - зеленый значок плюс); и заполните library flags без `-l` (например, вписав `m` для `-lm`). Затем build ваш проект.

Вспомните из главы 1, что Eclipse постоянно отображает ваш thread list, в отличие от DDD, где его нужно запрашивать. Более того, вам не нужно запрашивать операцию вроде backtrace, как в DDD; call stack показан в thread list. Это изображено на рисунке 5-2. Как выше, мы запустили программу на некоторое время, затем прервали ее, щелкнув icon Suspend справа от Resume. Thread list находится в Debug view, который обычно расположен в верхней левой части экрана, но здесь показан в расширенном виде, поскольку мы щелкнули Maximize во вкладке Debug. (Можно щелкнуть Restore, чтобы вернуться к стандартной layout.)

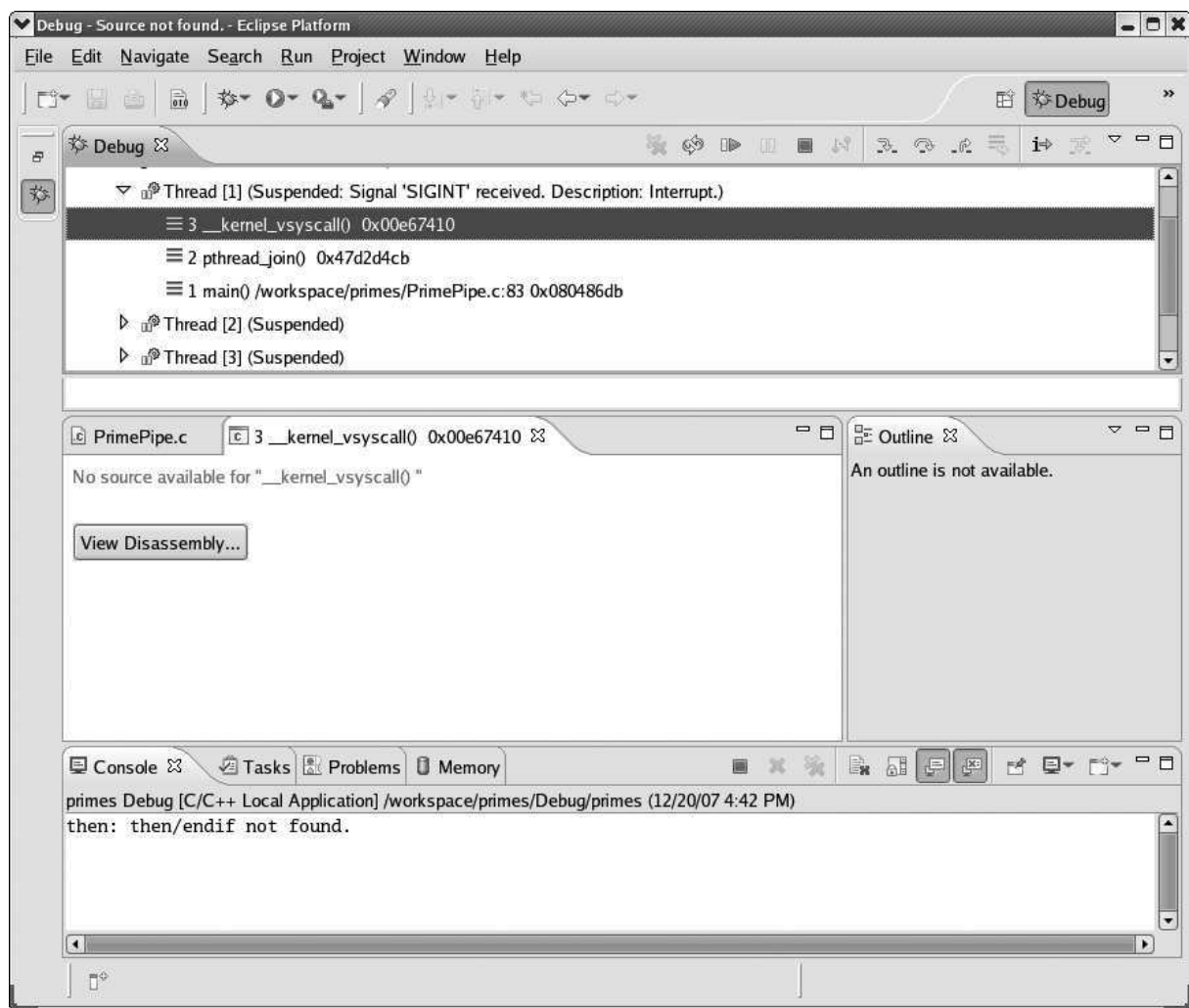


Рисунок 5-2. Отображение threads в Eclipse

Мы видим, что thread 3 выполнялся в момент interruption; он получил сигнал SIGINT, то есть interruption (CTRL-C) signal. Мы также видим, что связанный system call был вызван pthread_join(), который, в свою очередь, был вызван main(). Из того, что мы видели об этой программе ранее, мы знаем, что это действительно main thread.

Чтобы просмотреть информацию для другого thread, мы просто щелкаем треугольник рядом с thread, чтобы направить его вниз. Чтобы переключиться на другой thread, щелкаем его entry в списке.

Мы можем захотеть установить breakpoint, который applies only to a specific thread. Для этого сначала нужно дождаться, пока thread будет создан. Затем, когда execution pauses из-за предыдущей установки breakpoint или из-за interruption, как выше, мы щелкаем правой кнопкой мыши символ breakpoint тем же способом, как если бы делали breakpoint conditional, но на этот раз выбираем Filtering. Появится всплывающее окно, подобное показанному на рисунке 5-3. Мы видим, что этот breakpoint сейчас applies to all three threads. Если мы хотим, чтобы он applies only to thread 2, например, мы сняли бы флажки рядом с entries для двух других threads.

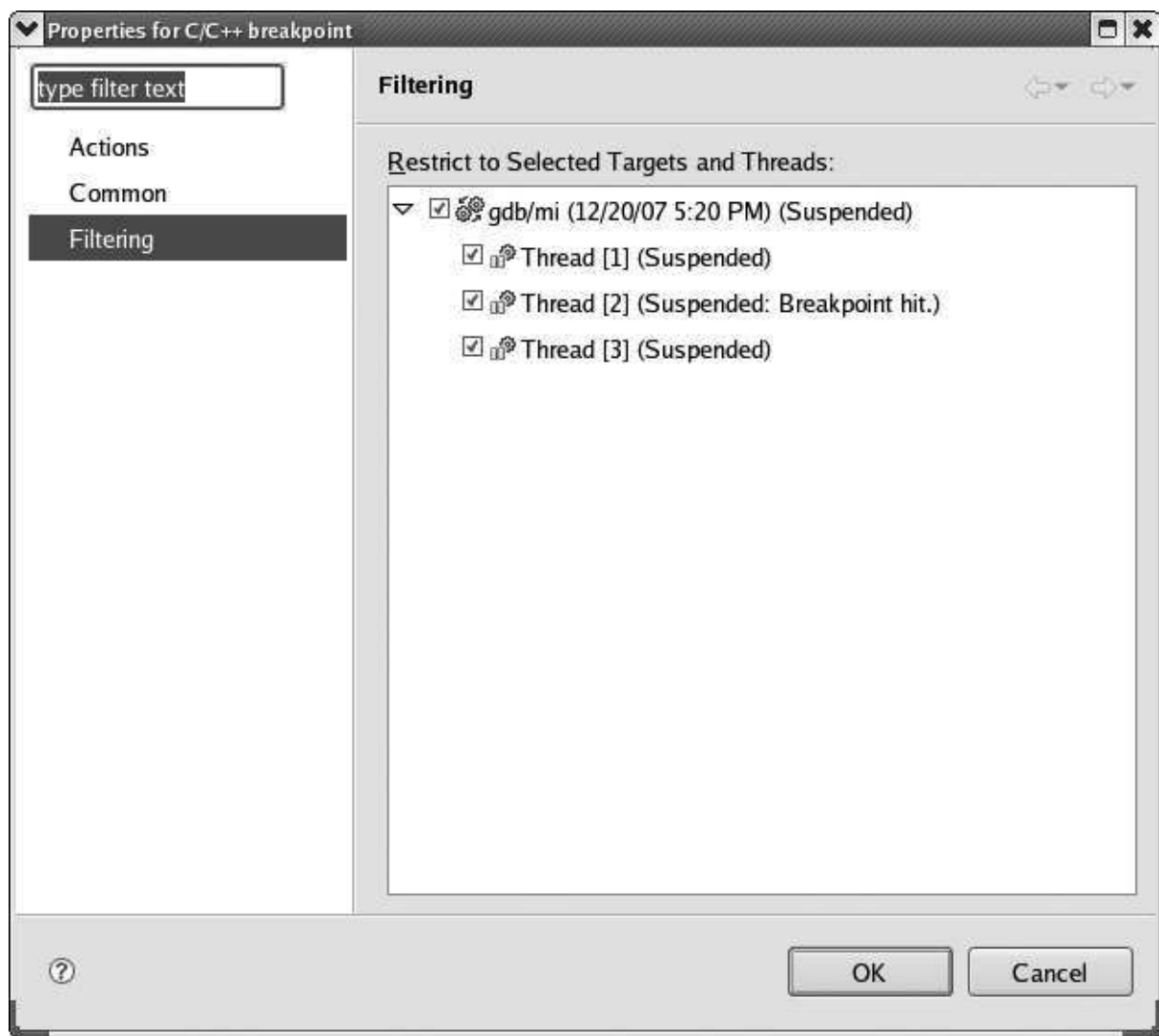


Рисунок 5-3. Установка thread-specific breakpoint в Eclipse

5.3 Отладка parallel applications

Есть два основных типа parallel programming architectures - shared memory и message passing.

Термин shared memory означает именно это: несколько CPUs имеют доступ к некоторой общей physical memory. Код, выполняющийся на одном CPU, общается с кодом, выполняющимся на других, читая из этой shared memory и записывая в нее, во многом так же, как threads в multithreaded application общаются друг с другом через shared address space. (Действительно, threaded programming стал стандартным способом писать application code для shared memory systems.)

В отличие от этого, в message passing environment код, выполняющийся на каждом CPU, может обращаться только к local memory этого CPU, и общается с другими, отправляя strings of bytes, называемые messages, через communication medium. Обычно это какая-то сеть, работающая либо на general-purpose protocol вроде TCP/IP, либо на специализированной software infrastructure, tailored to message-passing applications.

5.3.1 Message-passing systems

Сначала обсудим message passing, используя в качестве примера популярный пакет Message Passing Interface (MPI). Здесь мы используем реализацию MPICH, но те же принципы применимы к LAM и другим реализациям MPI.

Снова рассмотрим программу поиска prime numbers:

```
1 #include <mpi.h>
2
3 // MPI sample program; not intended to be efficient; finds and reports
4 // the number of primes less than or equal to n
5
6 // Uses a pipeline approach: node 0 looks at all the odd numbers (i.e.,
7 // we assume multiples of 2 are already filtered out) and filters out
8 // those that are multiples of 3, passing the rest to node 1; node 1
9 // filters out the multiples of 5, passing the rest to node 2; node 2
10 // filters out the rest of the composites and then reports the number
11 // of primes
12
13 // the command-line arguments are n and debugwait
14
15 #define PIPE_MSG 0 // type of message containing a number to
16 // be checked
17 #define END_MSG 1 // type of message indicating no more data will
18 // be coming
19
20 int nnodes, // number of nodes in computation
21 n, // find all primes from 2 to n
22 me; // my node number
23
24 init(int argc, char **argv)
25 { int debugwait; // if 1, then loop around until the
26 // debugger has been attached
27
28 MPI_Init(&argc, &argv);
29 n = atoi(argv[1]);
30 debugwait = atoi(argv[2]);
31
32 MPI_Comm_size(MPI_COMM_WORLD, &nnodes);
33 MPI_Comm_rank(MPI_COMM_WORLD, &me);
34
35 while (debugwait) ;
36 }
37
38 void node0()
39 { int i, dummy,
40 tocheck; // current number to check for passing on to next node
41 for (i = 1; i <= n/2; i++) {
42 tocheck = 2 * i + 1 ;
43 if (tocheck > n) break;
44 if (tocheck % 3 > 0)
45 MPI_Send(&tocheck, 1, MPI_INT, 1, PIPE_MSG, MPI_COMM_WORLD);
46 }
47 MPI_Send(&dummy, 1, MPI_INT, 1, END_MSG, MPI_COMM_WORLD);
48 }
49
50 void node1()
51 { int tocheck, // current number to check from node 0
52 dummy;
53 MPI_Status status; // see below
54
55 while (1) {
56 MPI_Recv(&tocheck, 1, MPI_INT, 0, MPI_ANY_TAG,
57 MPI_COMM_WORLD, &status);
58 if (status.MPI_TAG == END_MSG) break;
59 if (tocheck % 5 > 0)
60 MPI_Send(&tocheck, 1, MPI_INT, 2, PIPE_MSG, MPI_COMM_WORLD);
61 }
62 // now send our end-of-data signal, which is conveyed in the
```

```

63 // message type, not the message itself
64 MPI_Send(&dummy,1,MPI_INT,2,END_MSG,MPI_COMM_WORLD);
65 }
66
67 void node2()
68 { int tocheck, // current number to check from node 1
69   primecount,i,iscomposite;
70   MPI_Status status;
71
72   primecount = 3; // must account for the primes 2, 3 and 5, which
73   // won't be detected below
74   while (1) {
75     MPI_Recv(&tocheck,1,MPI_INT,1,MPI_ANY_TAG,
76     MPI_COMM_WORLD,&status);
77     if (status.MPI_TAG == END_MSG) break;
78     iscomposite = 0;
79     for (i = 7; i*i <= tocheck; i += 2)
80       if (tocheck % i == 0) {
81         iscomposite = 1;
82         break;
83       }
84     if (!iscomposite) primecount++;
85   }
86   printf("number of primes = %d\n",primecount);
87 }
88
89 main(int argc,char **argv)
90 { init(argc,argv);
91   switch (me) {
92     case 0: node0();
93     break;
94     case 1: node1();
95     break;
96     case 2: node2();
97   };
98   MPI_Finalize();
99 }

```

Как объяснено в комментариях в начале программы, здесь наше Sieve of Eratosthenes выполняется на трех nodes параллельной системы и работает pipeline-образно. Первый node начинает с нечетных чисел и удаляет все multiples of 3, передавая оставшиеся значения дальше; второй node берет output первого и удаляет все multiples of 5; а третий node берет output второго, удаляет остальные nonprimes и reports number of primes that are left.

Здесь pipelining достигается тем, что каждый node передает следующему по одному числу за раз. (Гораздо большей эффективности можно было бы достичь, передавая groups of numbers в каждом MPI message, тем самым reducing communications overhead.) При отправке числа следующему node node отправляет message типа PIPE_MSG. Когда у node больше нет чисел для отправки, он указывает это, отправляя message типа END_MSG.

В качестве примера отладки здесь предположим, что мы забыли включить последнее уведомление в первом node, то есть забыли строку 46 в коде node0():

```
MPI_Send(&dummy,1,MPI_INT,1,END_MSG,MPI_COMM_WORLD);
```

Программа зависнет на downstream nodes. Посмотрим, как можно отследить эту ошибку. (Имейте в виду, что некоторые line numbers в сессии GDB ниже будут отличаться на 1 от номеров в листинге выше.)

MPICH application program запускают, вызывая script с именем mpirun на одном node системы. Затем script запускает application program на каждом node через SSH. Здесь мы сделали это в сети из трех машин, которые назовем Node 0, Node 1 и Node 2, с n, равным 100. Ошибка заставляет программу зависнуть на последних двух nodes. Программа также зависает на первом node, потому что ни один экземпляр MPI program не выйдет, пока все не выполнят функцию MPI_FINALIZE().

Мы хотели бы использовать GDB, но поскольку применили `mpirun` для вызова приложения на каждом из трех `nodes`, а не запускали их напрямую на `nodes`, мы не можем запустить GDB напрямую. Однако GDB позволяет `dynamically attach debugger` к уже `running process`, используя `process number`. Поэтому запустим `ps` на Node 1, чтобы определить номер `process`, который выполняет там наше приложение:

```
$ ps ax
...
2755 ? S 0:00 tcsh -c /home/matloff/primepipe node 1 3
2776 ? S 0:00 /home/matloff/primepipe node1 32812 4
2777 ? S 0:00 /home/matloff/primepipe node1 32812 4
```

MPI program выполняется как `process 2776`, поэтому присоединяем GDB к программе на Node 1:

```
$ gdb primepipe 2776
...
0xfffffe002 in ?? ()
```

Это не слишком информативно! Посмотрим, где мы находимся:

```
(gdb) bt
#0 0xfffffe002 in ?? ()
#1 0x08074a76 in recv_message ()
#2 0x080748ad in p4_recv ()
#3 0x0807ab46 in MPID_CH_Check_incoming ()
#4 0x08076ae9 in MPID_RecvComplete ()
#5 0x0806765f in MPID_RecvDatatype ()
#6 0x0804a29f in PMPI_Recv ()
#7 0x08049ce8 in node1 () at PrimePipe.c:56
#8 0x08049e19 in main (argc=8, argv=0xbffffb24) at PrimePipe.c:94
#9 0x420156a4 in __libc_start_main () from /lib/tls/libc.so.6
```

Из `frame 7` мы видим, что программа зависла на строке 56, ожидая `receive from Node 0`.

Далее было бы полезно знать, сколько работы сделано функцией, выполняющейся на Node 1, `node1()`. Она только начала или почти закончила? Мы можем оценить `progress`, определив последнее обработанное значение переменной `tocheck`:

```
(gdb) frame 7
#7 0x08049ce8 in node1 () at PrimePipe.c:56
56 MPI_Recv(&tocheck,1,MPI_INT,0,MPI_ANY_TAG,
(gdb) p tocheck
$ 1=9 7
```

NB. Сначала нам нужно было перейти к `stack frame` для `node1()`, используя команду GDB `frame`.

Это указывает, что Node 1 находится в конце выполнения, поскольку 97 должно быть последним числом, которое Node 0 передает ему для `prime checking`. Поэтому сейчас мы ожидали бы `message from Node 0` типа `END_MSG`. Тот факт, что программа зависла, *suggests*, что Node 0 мог не отправить такое `message`, что, в свою очередь, привело бы нас к проверке, отправил ли он его. Таким образом мы, будем надеяться, быстро `zero in on bug`, которым было случайное `omission` строки 46.

Кстати, помните, что когда GDB вызывается командой:

```
$ gdb primepipe 2776
```

как мы сделали выше, `command-line processing` GDB сначала проверяет, нет ли `core file` с именем 2776. В маловероятном случае, если такой файл существует, GDB загрузит его вместо присоединения к `intended process`. В качестве альтернативы у GDB также есть команда `attach`.

В этом примере `bug` заставил программу зависнуть. Подход к отладке `parallel program` вроде этой несколько иной, когда `symptom` - `incorrect output`. Предположим, например, что в строке 71 мы неправильно инициализировали `primescount` значением 2 вместо 3. Если попытаться следовать той же `debugging procedure`, программы, выполняющиеся на каждом `node`, завершили бы выполнение и вышли слишком быстро, чтобы вы успели `attach` GDB. (Правда, можно было бы использовать очень

большое значение `n`, но обычно лучше сначала отлаживать `simple cases`.) Нам нужно средство, которое можно использовать, чтобы заставить `programs wait` и дать вам шанс `attach GDB`. Именно для этого предназначена строка 34 в функции `init()`.

Как видно в `source code`, значение `debugwait` берется из `command line`, supplied by user, причем 1 означает `wait`, а 0 означает `no wait`. Если указать 1 для значения `debugwait`, то когда каждый invocation программы достигает строки 34, он остается там. Это дает нам время `attach GDB`. Затем можно выйти из `infinite loop` и продолжить `debug`. Вот что мы делаем на Node 0:

```
node1:~$ gdb primepipe 3124
...
0x08049c53 in init (argc=3, argv=0xbfffe2f4) at PrimePipe.c:34
34 while (debugwait) ;
(gdb) set debugwait = 0
(gdb) c
Continuing.
```

Обычно мы боимся `infinite loops`, но здесь мы намеренно создаем один, чтобы облегчить `debugging`. Мы делаем то же самое на Node 1 и Node 2, а на последнем также пользуемся возможностью поставить `breakpoint` на строке 77 перед продолжением:

```
[matloff@node3 ~]$ gdb primepipe 2944
34 while (debugwait) ;
(gdb) b 77
Breakpoint 1 at 0x8049d7d: file PrimePipe.c, line 77.
(gdb) set debugwait = 0
(gdb) c
Continuing.
Breakpoint 1, node2 () at PrimePipe.c:77
77 if (status.MPI_TAG == END_MSG) break;
(gdb) p tocheck
$ 1=7
(gdb) n
78 iscomposite = 0;
(gdb) n
79 for (i = 7; i *i <= tocheck; i += 2)
(gdb) n
84 if (!iscomposite) primecount++;
(gdb) n
75 MPI_Recv(&tocheck,1,MPI_INT,1,MPI_ANY_TAG,
(gdb) p primecount
$ 2=3
```

В этот момент мы замечаем, что `primecount` должен быть 4, а не 3 - primes through 7 are 2, 3, 5, and 7 - и тем самым нашли `location of the bug`.

5.3.2 Shared-memory systems

А что насчет `shared-memory` типа `parallel programming`? Здесь есть отдельные случаи для `true shared-memory machines` и `software-distributed shared-memory settings`.

5.3.2.1 True shared memory

Как упоминалось ранее, в `true shared-memory environment application programs` часто разрабатываются с использованием `threads`. Тогда применим материал раздела 5.2 об отладке с `GDB/DDD`.

OpenMP стал популярной programming environment на таких машинах. OpenMP предоставляет программисту high-level parallel programming constructs, которые, в свою очередь, используют threads. У программиста все еще есть thread-level access, если он нужен, но по большей части threaded implementation директив OpenMP в значительной степени transparent to programmer.

В разделе 5.4 мы представляем extended example отладки OpenMP application.

5.3.2.2 Software distributed shared-memory systems

Цены на машины с dual-core CPUs теперь доступны обычным потребителям, но large-scale shared-memory systems со многими processors все еще стоят сотни тысяч долларов. Популярная недорогая альтернатива - network of workstations (NOW). NOW architectures используют underlying library, которая дает иллюзию shared memory. Библиотека, в значительной степени transparent to application programmer, занимается network transactions, поддерживающими consistency of copies of shared variables across different nodes.

Этот подход называется software distributed shared memory (SDSM). Наиболее широко используемая SDSM library - Treadmarks, разработанная и поддерживаемая Rice University. Еще один отличный пакет - JIAJIA, доступный от Chinese Academy of Sciences (<http://www-users.cs.umn.edu/~tianhe/paper/dist.htm>).

SDSM applications демонстрируют некоторые виды поведения, которые могут озадачить неосторожного программиста. Они сильно зависят от конкретной системы, поэтому общее изложение здесь дать нельзя, но мы кратко обсудим несколько вопросов, общих для многих из них.

Многие SDSMs являются page based, то есть они опираются на underlying virtual memory hardware на nodes. Действия сложны, но можно дать быстрый обзор. Рассмотрим переменную X, которая должна совместно использоваться NOW nodes. Программист указывает это намерение, делая определенный вызов SDSM library, которая, в свою очередь, делает определенный Unix system call, requesting OS to replace its own seg fault handler with function in SDSM library for page faults involving page containing X. SDSM настраивает все так, что только NOW nodes с valid copies of X имеют соответствующие memory pages, помеченные как resident. Когда к X обращаются на некотором другом node, возникает page fault, и underlying SDSM software получает correct value from node that has it.

Снова, не обязательно знать точную работу SDSM system; скорее, важно просто понимать, что используется underlying VM-based mechanism для поддержания consistency of local copies of shared data across NOW nodes. Если вы этого не понимаете, вы будете сбиты с толку при попытке отлаживать SDSM application code. Debugger будет, кажется, загадочно останавливаться на несуществующих seg faults, потому что SDSM infrastructure deliberately generates seg faults, и когда SDSM application program запускается под debugging tool, tool их detects. Как только вы это понимаете, проблемы нет: в GDB вы просто выдаете команду continue, чтобы resume execution, когда происходит одна из таких странных pauses.

У вас может возникнуть искушение приказать GDB не останавливаться и не выдавать warning messages whenever any seg faults occur, используя команду GDB:

```
handle SIGSEGV nostop noprint
```

Однако этот подход следует использовать с осторожностью, поскольку он может привести к тому, что вы пропустите genuine seg faults, caused by bugs in application program.

Еще одна связанная трудность с debugging applications, работающих на page-based SDSMs, возникает следующим образом. Если node в сети изменяет значение shared variable, то любой другой node, которому нужно значение этой переменной, должен получить updated value through network transaction. Опять же, details of how this happens зависят от SDSM system, но это означает, что если вы single-stepping through code, выполняющийся на одном node, вы можете обнаружить, что GDB загадочно зависает, потому что node теперь ждет update to its local copy of variable, recently modified by another node. Если вы одновременно запускаете отдельную сессию GDB для пошагового прохождения кода на том другом node, update на первом node не произойдет, пока debugging session на втором node не продвинется достаточно далеко. Иными словами, если программист не внимателен и осторожен во время debugging SDSM application, он может сам создать deadlock situation through debugging process itself.

Ситуация SDSM похожа на случай message-passing в одном смысле: нужна переменная вроде debugwait в MPI example выше, которая позволяет программе pause на всех nodes, давая вам шанс attach GDB на каждом node и step through program from beginning.

5.4 Расширенный пример

Этот раздел представляет пример отладки shared-memory application, разработанного с использованием OpenMP. Необходимое знание OpenMP будет объяснено ниже. Все, что нужно, - базовое понимание threads.

5.4.1 Обзор OpenMP

OpenMP по сути является higher-level parallel programming interface к thread-management operations. Число threads задается через environment variable OMP_NUM_THREADS. В C shell, например, вы вводите:

```
% setenv OMP_NUM_THREADS 4
```

в shell prompt, чтобы arrange to have four threads.

Application code состоит из C, перемежаемого директивами OpenMP. Каждая директива applies to block that follows it, delimited by left and right braces. Самая базовая directive:

```
#pragma omp parallel
```

Она создает OMP_NUM_THREADS threads, каждый из которых concurrently executes block of code following pragma. Обычно внутри этого block будут embedded other directives.

Еще одна очень распространенная directive OpenMP:

```
#pragma omp barrier
```

Она задает "meeting point" для всех threads. Когда любой thread reaches this point, он будет block, пока все остальные threads не придут туда.

Часто требуется, чтобы только один thread выполнил некоторый block, а остальные threads его пропустили. Это достигается записью:

```
#pragma omp single
```

Сразу после такого block есть implied barrier.

Есть много других директив OpenMP, но единственная другая, используемая в здешнем примере:

```
#pragma omp critical
```

Как следует из названия, она создает critical section, в которой в любой данный момент разрешен только один thread.

5.4.2 Пример программы OpenMP

Мы реализуем знаменитый algorithm Dijkstra для определения minimum distances between pairs of vertices in weighted graph. Скажем, нам даны distances between adjacent vertices (если две vertices не adjacent, distance between them set to infinity). Цель - найти minimum distances between vertex 0 and all other vertices.

Ниже приведен source file `dijkstra.c`. Он генерирует random edge lengths among specified number of vertices, а затем находит minimum distances from vertex 0 to each of other vertices.

```
1 // dijkstra.c
2
3 // OpenMP example program: Dijkstra shortest-path finder in a
4 // bidirectional graph; finds the shortest path from vertex 0 to all
5 // others
6
7 // usage: dijkstra nv print
8
9 // where nv is the size of the graph, and print is 1 if graph and min
10 // distances are to be printed out, 0 otherwise
11
12 #include <omp.h> // required
13 #include <values.h>
14
15 // including stdlib.h and stdio.h seems to cause a conflict with the
16 // Omni compiler, so declare directly
17 extern void *malloc();
18 extern int printf(char *,...);
19
20 // global variables, shared by all threads
21 int nv, // number of vertices
22 *notdone, // vertices not checked yet
23 nth, // number of threads
24 chunk, // number of vertices handled by each thread
25 md, // current min over all threads
26 mv; // vertex which achieves that min
27
28 int *ohd, // 1-hop distances between vertices; "ohd[i][j]" is
29 // ohd[i*nv+j]
30 *mind; // min distances found so far
31
32 void init(int ac, char **av)
33 { int i,j,tmp;
34   nv = atoi(av[1]);
35   ohd = malloc(nv*nv*sizeof(int));
```

```

36 mind = malloc(nv*sizeof(int));
37 notdone = malloc(nv*sizeof(int));
38 // random graph
39 for (i = 0; i < nv; i++)
40 for (j = i; j < nv; j++) {
41 if (j == i) ohd[i*nv+i] = 0;
42 else {
43 ohd[nv*i+j] = rand() % 20;
44 ohd[nv*j+i] = ohd[nv*i+j];
45 }
46 }
47 for (i = 1; i < nv; i++) {
48 notdone[i] = 1;
49 mind[i] = ohd[i];
50 }
51 }
52
53 // finds closest to 0 among notdone, among s through e; returns min
54 // distance in *d, closest vertex in *v
55 void findmymin(int s, int e, int *d, int *v)
56 { int i;
57 *d = MAXINT;
58 for (i = s; i <= e; i++)
59 if (notdone[i] && mind[i] < *d) {
60 *d = mind[i];
61 *v=i ;
62 }
63 }
64
65 // for each i in {s,...,e}, ask whether a shorter path to i exists, through
66 // mv
67 void updatemind(int s, int e)
68 { int i;
69 for (i = s; i <= e; i++)
70 if (notdone[i])
71 if (mind[mv] + ohd[mv*nv+i] < mind[i])
72 mind[i] = mind[mv] + ohd[mv*nv+i];
73 }
74
75 void dowork()
76 {
77 #pragma omp parallel
78 { int startv,endv, // start, end vertices for this thread
79 step, // whole procedure goes nv steps
80 mymv, // vertex which attains that value
81 me = omp_get_thread_num(),
82 mymd; // min value found by this thread
83 #pragma omp single
84 { nth = omp_get_num_threads(); chunk = nv/nth;
85 printf("there are %d threads\n",nth); }
86 startv = me * chunk;
87 endv = startv + chunk - 1;
88 // the algorithm goes through nv iterations
89 for (step = 0; step < nv; step++) {
90 // find closest vertex to 0 among notdone; each thread finds
91 // closest in its group, then we find overall closest
92 #pragma omp single
93 { md = MAXINT;
94 mv = 0;
95 }
96 findmymin(startv,endv,&mymd,&mymv);
97 // update overall min if mine is smaller
98 #pragma omp critical
99 { if (mymd < md)
100 { md = mymd; }
101 }
102 #pragma omp barrier
103 // mark new vertex as done
104 #pragma omp single
105 { notdone[mv] = 0; }
106 // now update my section of ohd

```

```

107 updatemind(startv,endv);
108 }
109 }
110 }
111
112 int main(int argc, char **argv)
113 { int i,j,print;
114   init(argc,argv);
115   // start parallel
116   dowork();
117   // back to single thread
118   print = atoi(argv[2]);
119   if (print) {
120     printf("graph weights:\n");
121     for (i = 0; i < nv; i++) {
122       for (j = 0; j < nv; j++)
123         printf("%u ",ohd[nv *i+j]);
124       printf("\n");
125     }
126     printf("minimum distances:\n");
127     for (i = 1; i < nv; i++)
128       printf("%u\n",mind[i]);
129   }
130 }

```

Рассмотрим, как работает algorithm. Начните со всех vertices, кроме vertex 0, то есть в этом случае vertices 1 through 5, в "not done" set. В каждой iteration algorithm выполните следующее:

1. Найдите "not done" vertex v , closest to vertex 0, along paths known so far. Эта проверка shared by all threads, причем каждый thread проверяет equal number of vertices. Функция, которая делает эту работу, - `findmymin()`.
2. Затем переместите v в "done" set.
3. Для всех remaining vertices i в "not done" set проверьте, не короче ли путь, сначала идущий от 0 к v по best known path so far, а затем от v к i за one hop, чем current shortest distance from 0 to i . Если да, update that distance accordingly. Функция, выполняющая эти действия, - `updatemind()`.

Iteration продолжается, пока "not done" set не станет empty.

Поскольку OpenMP directives require preprocessing, всегда существует potential problem, что мы потеряем original line numbers и variable and function names. Чтобы увидеть, как с этим справляться, обсудим два разных компилятора. Сначала посмотрим на Omni compiler (<http://www.hpcc.jp/Omni/>), а затем на GCC (требуется version 4.2 or later).

Мы компилируем наш код под Omni следующим образом:

```
$ omcc -g -o dij dijkstra.c
```

После компиляции программы и запуска ее с четырьмя threads мы обнаруживаем, что она работает неправильно:

```

$ dij 6 1
there are 4 threads
graph weights:
0361 71 51 3
301 561 29
61 50127
176101 01 9
151 221 003
13971 930
minimum distances:
3
6
17
15
13

```

Анализ graph by hand показывает, что correct minimum distances должны быть 3, 6, 7, 8 и 11.

Затем запускаем программу в GDB. Здесь очень важно понять последствия того факта, что OpenMP работает через directives. Хотя line numbers, function names и так далее mostly retained двумя обсуждаемыми здесь compilers, между ними есть некоторые discrepancies. Посмотрите, что происходит, когда мы пытаемся установить breakpoint в executable `di` в начале сессии GDB:

```
(gdb) tb main
Breakpoint 1 at 0x80492af
(gdb) r 6 1
Starting program: /debug/di 6 1
[Thread debugging using libthread_db enabled]
[New Thread -1208490304 (LWP 11580)]
[Switching to Thread -1208490304 (LWP 11580)]
0x080492af in main ()
(gdb) l
1 /tmp/omni_C_11486.c: No such file or directory.
in /tmp/omni_C_11486.c
```

Мы обнаруживаем, что breakpoint не находится в source file. Вместо этого он находится в infrastructure code OpenMP у Omni. Иными словами, `main()` здесь - `main()` Omni, а не ваш собственный. Omni compiler mangled имя нашего `main()` в `_ompc_main()`.

Чтобы установить breakpoint на `main()`, мы вводим:

```
(gdb) tb _ompc_main
Breakpoint 2 at 0x80491b3: file dijkstra.c, line 114.
```

и проверяем это, продолжив:

```
(gdb) c
Continuing.
[New Thread -1208493152 (LWP 11614)]
[New Thread -1218983008 (LWP 11615)]
[New Thread -1229472864 (LWP 11616)]
_ompc_main (argc=3, argv=0xbfab6314) at dijkstra.c:114
114 init(argc,argv);
```

Хорошо, вот знакомая строка `init()`. Конечно, мы могли бы выдать команду:

```
(gdb) b dijkstra.c:114
```

Обратите внимание на создание трех новых threads, что дает всего четыре.

Каким бы способом мы ни устанавливали breakpoints, здесь приходится делать чуть больше работы, чем обычно, поэтому особенно важно оставаться в одной сессии GDB между запусками программы, даже когда мы меняем source code и перекомпилируем, чтобы сохранить breakpoints, conditions и так далее. Так нам нужно пройти через хлопоты настройки этих вещей только один раз.

Теперь как отследить bug(s)? Естественно подходить к отладке этой программы, проверяя результаты в конце каждой iteration. Основные результаты находятся в "not done" set (в массиве `notdone[]`) и в текущем списке best-known distances from 0 to other vertices, то есть в массиве `mind[]`. Например, после первой iteration "not done" set должен состоять из vertices 2, 3, 4 и 5, поскольку vertex 1 был выбран в этой iteration.

Вооружившись этой информацией, применим Principle of Confirmation и проверим `notdone[]` и `mind[]` после каждой iteration цикла `for` в `dowork()`.

Нужно быть осторожными в том, где именно мы устанавливаем breakpoints. Хотя natural spot для этого, казалось бы, строка 108, в самом конце main loop algorithm, это может быть не очень хорошо, поскольку GDB будет останавливаться там для каждого thread. Вместо этого выберем placing a breakpoint внутри OpenMP single block, чтобы он остановился только для одного thread.

Поэтому проверяем результаты после каждой iteration, останавливаясь в начале loop, начиная со второй iteration:

```
(gdb) b 92 if step >= 1
Breakpoint 3 at 0x80490e3: file dijkstra.c, line 92.
(gdb) c
Continuing.
there are 4 threads
Breakpoint 3, __ompc_func_0 () at dijkstra.c:93
93 { md = MAXINT;
```

Подтвердим, что первая iteration выбрала correct vertex (vertex 1) для перемещения из "not done" set:

```
(gdb) p mv
$ 1=0
```

Гипотеза, в конце концов, не подтверждена. Inspection of code показывает, что на строке 100 мы забыли установить mv. Исправим ее, чтобы читалось:

```
{ md = mymd; mv = mymv; }
```

Итак, мы перекомпилируем и снова запускаем программу. Как отмечалось ранее в этом разделе (и в других местах этой книги), очень полезно не выходить из GDB при повторном запуске программы. Мы могли бы запустить программу в другом terminal window, но просто для разнообразия выберем здесь другой подход. Мы временно disable наши breakpoints, выдав команду dis, затем запускаем перекомпилированную программу изнутри GDB, а потом re-enable breakpoints с помощью ena:

```
(gdb) dis
(gdb) r
The program being debugged has been started already.
Start it from the beginning? (y or n) y
`/debug/dij' has changed; re-reading symbols.
Starting program: /debug/dij 6 1
[Thread debugging using libthread_db enabled]
[New Thread -1209026880 (LWP 11712)]
[New Thread -1209029728 (LWP 11740)]
[New Thread -1219519584 (LWP 11741)]
[New Thread -1230009440 (LWP 11742)]
there are 4 threads
graph weights:
0361 71 51 3
301 561 29
61 50127
176101 01 9
151 221 003
13971 930
minimum distances:
3
6
17
15
13
Program exited with code 06.
(gdb) ena
```

Мы все еще получаем wrong answers. Проверим things at that breakpoint снова:

```
(gdb) r
Starting program: /debug/dij 6 1
[Thread debugging using libthread_db enabled]
[New Thread -1209014592 (LWP 11744)]
[New Thread -1209017440 (LWP 11772)]
[New Thread -1219507296 (LWP 11773)]
[New Thread -1229997152 (LWP 11774)]
there are 4 threads
[Switching to Thread -1209014592 (LWP 11744)]
Breakpoint 3, __ompc_func_0 () at dijkstra.c:93
93 { md = MAXINT;
(gdb) p mv
$ 2=1
```

По крайней мере, mv теперь имеет правильное значение. Проверим mind[]:

```
(gdb) p *mind@6
$3 = {0, 3, 6, 17, 15, 13}
```

Обратите внимание: поскольку мы constructed массив mind[] dynamically через malloc(), мы не могли использовать команду GDB print в ее обычной форме. Вместо этого мы использовали feature GDB artificial array.

В любом случае mind[] все еще incorrect. Например, mind[3] должен быть $3 + 6 = 9$, но он равен 17. Проверим код, который обновляет mind[]:

```
(gdb) b 107 if me == 1
Breakpoint 4 at 0x8049176: file dijkstra.c, line 107.
(gdb) r
The program being debugged has been started already.
Start it from the beginning? (y or n) y
Starting program: /debug/dij 6 1
[Thread debugging using libthread_db enabled]
[New Thread -1209039168 (LWP 11779)]
[New Thread -1209042016 (LWP 11807)]
[New Thread -1219531872 (LWP 11808)]
[New Thread -1230021728 (LWP 11809)]
there are 4 threads
[Switching to Thread -1230021728 (LWP 11809)]
Breakpoint 4, __ompc_func_0 () at dijkstra.c:107
107 updatemind(startv,endv);
```

Сначала подтвердим, что startv и endv имеют sensible values:

```
(gdb) p startv
$ 4=1
(gdb) p endv
$ 5=1
```

Chunk size всего 1? Посмотрим:

```
(gdb) p chunk
$ 6=1
```

Проверив computation for chunk, мы понимаем, что нам нужно, чтобы number of threads evenly divide nv. Последний имеет значение 6, которое не делится на наш thread count, 4. Мы делаем себе note вставить позже error-catching code и пока уменьшаем thread count до 3.

Снова, мы не хотим выходить из GDB, чтобы сделать это. GDB inherits environment variables при первом вызове, но значения этих variables также можно changed or set within GDB, и именно это мы делаем здесь:

```
(gdb) set environment OMP_NUM_THREADS = 3
```

Теперь снова запускаем:

```
(gdb) dis
(gdb) r
The program being debugged has been started already.
Start it from the beginning? (y or n) y
Starting program: /debug/dij 6 1
[Thread debugging using libthread_db enabled]
[New Thread -1208707392 (LWP 11819)]
[New Thread -1208710240 (LWP 11847)]
[New Thread -1219200096 (LWP 11848)]
there are 3 threads
graph weights:
0361 71 51 3
301 561 29
61 50127
176101 01 9
151 221 003
13971 930
minimum distances:
3
6
7
15
12
Program exited with code 06.
(gdb) ena
```

Ай-яй, все еще те же wrong answers! Продолжаем проверять updating process для mind[]:

```
(gdb) r
Starting program: /debug/dij 6 1
[Thread debugging using libthread_db enabled]
[New Thread -1208113472 (LWP 11851)]
[New Thread -1208116320 (LWP 11879)]
[New Thread -1218606176 (LWP 11880)]
there are 3 threads
[Switching to Thread -1218606176 (LWP 11880)]
Breakpoint 4, __ompc_func_0 () at dijkstra.c:107
107 updatemind(startv,endv);
(gdb) p startv
$ 7=2
(gdb) p endv
$ 8=3
```

Хорошо, это правильные значения для startv и endv в случае me = 1. Поэтому входим в функцию:

```
(gdb) s
[Switching to Thread -1208113472 (LWP 11851)]
Breakpoint 3, __ompc_func_0 () at dijkstra.c:93
93 { md = MAXINT;
(gdb) c
Continuing.
[Switching to Thread -1218606176 (LWP 11880)]
updatemind (s=2, e=3) at dijkstra.c:69
69 for (i = s; i <= e; i++)
```


Обратите внимание: из-за context switches among threads мы не вошли в `updatemind()` immediately. Теперь проверяем случай `i = 3`:

```
(gdb) tb 71 if i == 3
Breakpoint 5 at 0x8048fb2: file dijkstra.c, line 71.
(gdb) c
Continuing.
updatemind (s=2, e=3) at dijkstra.c:71
71 if (mind[mv] + ohd[mv *nv+i] < mind[i])
```

Как обычно, применяем Principle of Confirmation:

```
(gdb) p mv
$ 9=0
```

Что ж, это большая проблема. Вспомните, что в первой iteration `mv` turns out to be 1. Почему здесь он 0?

Через некоторое время мы понимаем, что те context switches должны были быть большой подсказкой. Снова взгляните на вывод GDB выше. Thread, чей system ID 11851, уже был на строке 93, иными словами, он уже был в next iteration of algorithm's main loop. Более того, когда мы нажали `c`, чтобы продолжить, он даже выполнил строку 94:

```
mv=0 ;
```

Этот thread перезаписал previous value `mv`, равное 1, так что thread, который обновляет `mind[3]`, теперь полагается на wrong value of `mv`. Решение - добавить еще один barrier:

```
updatemind(startv,endv);
#pragma omp barrier
```

После этого исправления программа работает правильно.

Предыдущее изложение основывалось на Omni compiler. Как упоминалось, начиная с version 4.2, GCC также handles OpenMP code. Все, что нужно сделать, - добавить flag `-fopenmp` в command line GCC.

В отличие от Omni, GCC генерирует код таким образом, что focus GDB с самого начала находится в вашем собственном source file. Поэтому выдача команды:

```
(gdb) b main
```

в самом начале сессии GDB действительно установит breakpoint в вашем собственном `main()`, в отличие от того, что мы видели для Omni compiler.

Однако на момент написания книги major shortcoming GCC состоит в том, что symbols for local variables inside an OpenMP parallel block (в terminology OpenMP они называются private variables) не будут visible within GDB. Например, команда:

```
(gdb) p mv
```

которую вы выдавали выше для Omni-generated code, будет работать для GCC-generated code, но команда:

```
(gdb) p startv
```

на GCC-generated code завершится failure.

Конечно, есть способы обойти это. Например, если вы хотите узнать значение `startv`, можно query значение `s` внутри `updatemind()`. Надеемся, эта проблема будет решена в следующей версии GCC.

Глава 6. Специальные темы

Во время отладки возникают различные вопросы, которые не относятся непосредственно к отладочным инструментам. Некоторые из этих вопросов мы рассмотрим в этой главе.

6.1 Что если программа даже не компилируется или не загружается?

Как ни бесценны GDB, DDD и Eclipse, они не смогут помочь, если ваша программа даже не компилируется. В этом разделе мы дадим несколько советов о том, как действовать в такой ситуации.

6.1.1 Phantom line numbers в сообщениях о синтаксических ошибках

Иногда компилятор сообщает, что у вас syntax error в строке x, хотя на самом деле строка x совершенно корректна, а настоящая ошибка находится в более ранней строке.

Например, вот source file bintree.c из главы 3, куда добавлена syntax error в месте, которое мы пока не раскроем (ну, если захотите поискать, оно довольно очевидно).

```
1 // bintree.c: routines to do insert and sorted print of a binary tree
2
3 #include <stdio.h>
4 #include <stdlib.h>
5
6 struct node {
7     int val; // stored value
8     struct node *left; // ptr to smaller child
9     struct node *right; // ptr to larger child
10 };
11
12 typedef struct node *nsp;
13
14 nsp root;
15
16 nsp makenode(int x)
17 {
18     nsp tmp;
19
20     tmp = (nsp) malloc(sizeof(struct node));
21     tmp->val = x;
22     tmp->left = tmp->right = 0;
23     return tmp;
24 }
25
26 void insert(nsp *btp, int x)
27 {
28     nsp tmp = *btp;
29
30     if (*btp == 0) {
31         *btp = makenode(x);
32         return;
33     }
34
35     while (1)
36     {
37         if (x < tmp->val) {
38
39             if (tmp->left != 0) {
```

```

40 tmp = tmp->left;
41 } else {
42 tmp->left = makenode(x);
43 break;
44 }
45
46 } else {
47
48 if (tmp->right != 0) {
49 tmp = tmp->right;
50 } else {
51 tmp->right = makenode(x);
52 break;
53 }
54
55 }
56 }
57
58 void printtree(nsp bt)
59 {
60 if (bt == 0) return;
61 printtree(bt->left);
62 printf("%d\n",bt->val);
63 printtree(bt->right);
64 }
65
66 int main(int argc, char *argv[])
67 {
68 int i;
69
70 root = 0;
71 for (i = 1; i < argc; i++)
72 insert(&root, atoi(argv[i]));
73 printtree(root);
74 }

```

Прогон этого через GCC дает:

```

$ gcc -g bintree.c
bintree.c: In function `insert':
bintree.c:75: parse error at end of input

```

Поскольку строка 74 - конец source file, второе сообщение об ошибке, мягко говоря, не слишком информативно. Но первое сообщение suggests, что проблема в insert(), так что это clue, хотя оно не говорит, в чем или где проблема.

В такой ситуации typical culprit - отсутствующая closing brace или semicolon. Можно проверить это напрямую, но в большом source file это может быть трудно. Применим другой подход.

Вспомните Principle of Confirmation из главы 1. Здесь сначала подтвердим, что проблема действительно в insert(). Для этого временно закомментируем эту функцию в source code:

```

...
tmp->val = x;
tmp->left = tmp->right = 0;
return tmp;
}
// void insert(nsp *btp, int x)
// {
// nsp tmp = *btp;
//
// if ( *btp == 0) {
// *btp = makenode(x);
// return;
// }
//
// while (1)
// {
// if (x < tmp->val) {
//

```

```

// if (tmp->left != 0) {
// tmp = tmp->left;
// } else {
// tmp->left = makenode(x);
// break;
// }
//
// } else {
//
// if (tmp->right != 0) {
// tmp = tmp->right;
// } else {
// tmp->right = makenode(x);
// break;
// }
// }
// }
void printtree(nsp bt)
{
if (bt == 0) return;
...

```

NB. Предпочтительно использовать быстрый shortcut для комментирования, например block operation. Shortcuts текстовых редакторов, полезные в debugging contexts, будут обсуждаться в главе 7.

Сохраните файл, а затем снова запустите GCC:

```

$ gcc -g bintree.c
/tmp/ccg0LDCS.o: In function `main':
/home/matloff/public_html/matloff/public_html/Debug/Book/DDD/bintree.c:72:
undefined reference to `insert'
collect2: ld returned 1 exit status

```

Не отвлекайтесь на то, что linker, LD, пожаловался, что не может найти `insert()`. В конце концов, вы знали, что это произойдет, поскольку закомментировали эту функцию. Вместо этого интересующий момент состоит в том, что нет жалобы на `syntax error`, как раньше. Итак, вы действительно подтвердили, что `syntax error` где-то в `insert()`. Теперь раскомментируйте строки этой функции (снова, предпочтительно используя shortcut текстового редактора вроде "undo") и сохраните файл. Также, просто чтобы убедиться, что вы восстановили все `correctly`, снова запустите GCC, чтобы подтвердить, что `syntax error` возвращается (здесь не показано).

В этот момент можно применить еще один принцип, сформулированный в главе 1: Principle of Binary Search. Повторно сужайте область поиска в функции `insert()`, каждый раз cutting the area in half, пока не получите достаточно небольшую область, где можно увидеть `syntax error`.

Для этого сначала закомментируйте примерно половину функции. Разумный способ сделать это - просто закомментировать цикл `while`. Затем снова запустите GCC:

```

$ gcc -g bintree.c
$

```

Aha! Error message исчезло, значит syntax problem должна быть где-то внутри loop. Итак, вы сузили проблему до этой половины функции, и теперь снова делите эту область пополам. Для этого закомментируйте код else:

```
void insert(nsp *btp, int x)
{
    nsp tmp = *btp;
    if (*btp == 0) {
        *btp = makenode(x);
        return;
    }
    while (1)
    {
        if (x < tmp->val) {
            if (tmp->left != 0) {
                tmp = tmp->left;
            } else {
                tmp->left = makenode(x);
                break;
            }
        } // else {
        //
        // if (tmp->right != 0) {
        // tmp = tmp->right;
        // } else {
        // tmp->right = makenode(x);
        // break;
        // }
        //
        // }
    }
}
```

Повторно запустив GCC, вы обнаружите, что проблема появляется снова:

```
$ gcc -g bintree.c
bintree.c: In function `insert':
bintree.c:75: parse error at end of input
```

Итак, syntax error находится либо в block if, либо в конце функции. К этому моменту вы сузили проблему всего до семи строк кода, так что, вероятно, сможете найти проблему визуальной inspection; оказывается, мы случайно пропустили closing brace во внешнем if-then-else.

Principle of Binary Search может быть очень полезен при поиске syntax errors неизвестного location. Но при временном комментировании кода не создавайте собственные новые syntax errors! Комментируйте целую функцию, целый loop и так далее, как мы сделали здесь.

6.1.2 Missing libraries

Иногда GCC - фактически LD, linker, который вызывается GCC во время процесса building вашей программы, - сообщает, что не может найти одну или несколько функций, вызываемых вашим кодом. Обычно это происходит из-за того, что GCC не сообщили location of a function library. Многие, если не большинство, читателей этой книги хорошо разбираются в этой теме, но для тех, кто не разбирается, мы дадим короткое введение. Обратите внимание, что наше обсуждение здесь применимо главным образом к Linux и в разной степени к другим operating systems семейства Unix.

6.1.2.1 Example

Используем в качестве примера следующий очень простой код, состоящий из main program в a.c:

```
// a.c
int f(int x);
main()
{
    int v;
    scanf("%d",&v);
    printf("%d\n",f(v));
}
```

и subprogram в z/b.c:

```
// b.c
int f(int x)
{
    return x*x;
}
```

Если попытаться скомпилировать a.c без какой-либо попытки link in code in b.c, то LD, конечно, пожалуется:

```
$ gcc -g a.c
/tmp/ccIP5WHu.o: In function `main':
/debug/a.c:9: undefined reference to `f'
collect2: ld returned 1 exit status
```

Мы могли бы перейти в z, скомпилировать b.c, а затем link in object file:

```
$ cd z
$ gcc -g -c b.c
$ cd ..
$ gcc -g a.c z/b.o
```

Однако если у вас много функций для linking in, возможно из разных source files, и если эти функции, вероятно, будут полезны для future programs, которые вы можете написать, можно создать library, единый archive file. Есть два вида library files. Когда вы компилируете код, вызывающий функции в static library, эти функции становятся частью resulting executable file. С другой стороны, если library dynamic, функции physically attached to calling code только тогда, когда программа фактически executed.

Вот как можно создать static library, скажем lib88.a, для здешнего примера.

NB. В Unix systems принято давать static library filenames suffix .a, где a означает archive. Также принято давать любой library имя, начинающееся с lib.

```
$ gcc -g -c b.c
$ ar rc lib88.a b.o
```

Команда ar здесь создает library lib88.a из всех функций, которые она находит в файле b.o. Затем вы скомпилировали бы main program:

```
$ gcc -g a.c -l88 -Lz
```

Option -l здесь является shortcut, имеющим тот же effect, что и:

```
$ gcc -g a.c lib88.a -Lz
```

что directs GCC to tell LD, что ему нужно будет найти functions in library lib88.a (или dynamic variant, как вы увидите ниже).

Option -L directs GCC to tell LD искать ваши functions в directories other than current one (and default search directories). В этом случае она говорит, что z является таким directory.

Недостаток этого подхода в том, что если много programs используют одну и ту же library, каждая будет содержать wasteful separate copy of it on disk. Эта проблема решается (ценой небольшого extra load time) использованием dynamic libraries.

В здешнем примере для создания dynamic library вы использовали бы GCC directly, а не ar. В z вы выполнили бы:

```
$ gcc -fPIC -c b.c
$ gcc -shared -o lib88.so b.o
```

Это создает dynamic library lib88.so. (Unix custom - использовать suffix .so, shared object, возможно followed by version number, для именования dynamic libraries.) Link to it как в static case:

```
$ gcc -g a.c -l88 -Lz
```

Однако теперь это работает немного иначе. В static case функции, вызываемые из library, стали бы частью нашего executable file a.out; теперь же a.out merely contains notation, что эта программа uses library lib88.so. И существенно, что эта notation даже не укажет, где эта library located. Единственная причина, по которой GCC (снова, фактически LD) хотел peek at lib88.so во время compile time, состояла в получении информации о library, необходимой для link.

Сам link произойдет at run time. Operating system будет искать lib88.so, а затем link it into your program. Это поднимает вопрос: где OS выполняет этот search?

Сначала используем команду ldd, чтобы проверить, какие libraries нужны программе, и где, если где-либо, OS их находит:

```
$ ldd a.out
lib88.so => not found
libc.so.6 => /lib/tls/libc.so.6 (0x006cd000)
/lib/ld-linux.so.2 (0x006b0000)
```

Программе нужна C library, которую она находит в directory /lib/tls, но OS не находит lib88.so. Последняя находится в directory /Debug/z, но этот directory не является частью normal search path OS.

Один способ исправить это - добавить /Debug/z в этот search path:

```
% setenv LD_LIBRARY_PATH ${LD_LIBRARY_PATH}:/Debug/z
```

Если вы хотите добавить несколько directories, соедините их names через colons as delimiters. (Это для C shell или TC shell.) Для bash выдайте команды:

```
$ LD_LIBRARY_PATH=$LD_LIBRARY_PATH:/Debug/z
$ export LD_LIBRARY_PATH
```

Убедимся, что это работает:

```
$ ldd a.out
lib88.so => /Debug/z/lib88.so (0xf6ffe000)
libc.so.6 => /lib/tls/libc.so.6 (0x006cd000)
/lib/ld-linux.so.2 (0x006b0000)
```

Есть различные другие approaches, но они выходят за рамки этой книги.

6.1.2.2 Library usage in open source software

Open source software стало весьма популярным, особенно среди пользователей Linux. Однако иногда возникает проблема: build script, обычно называемый configure, который поставляется с source code, не может найти некоторые necessary libraries. Попытки решить проблему, устанавливая environment variable LD_LIBRARY_PATH, могут fail. Поскольку это относится к обсуждаемой здесь теме "missing libraries" и обычно не документируется в source packages, краткая заметка об этом может быть полезной.

Часто root of the problem будет находиться в программе, вызываемой configure, с именем pkgconfig. Последняя retrieves information about libraries from certain metadata files, имеющих suffix .pc, причем prefix является name of the library. Например, файл libgcj.pc содержал бы location of library files libgcj.so*.

Default directory, в котором pkgconfig ищет .pc files, зависит от location of pkgconfig itself. Например, если программа находится в /usr/bin, она будет искать /usr/lib. Этого будет недостаточно, если нужная library находится в /usr/local/lib. Чтобы исправить эту проблему, установите environment variable PKG_CONFIG_PATH. В C или TC shell вы выдали бы shell command:

```
% setenv PKG_CONFIG_PATH /usr/lib/pkgconfig:/usr/local/lib/pkgconfig
```

6.2 Отладка GUI programs

В наши дни пользователи привыкли, что application programs поставляются с graphical user interfaces (GUIs). Это, конечно, просто программы, так что общие debugging principles применимы, но в игру вступают special considerations.

GUI programming в значительной степени состоит из вызовов library для выполнения различных операций на screen. Существует очень много таких libraries in widespread use. Мы, очевидно, не можем cover them all, и в любом случае principles similar.

Соответственно, мы выбрали самый простой пример - library curses. Она настолько проста, что многие могут вообще не считать ее GUI - один студент назвал ее "text-based GUI", - но она донесет суть.

6.2.1 Отладка curses programs

Library curses позволяет programmer писать код, который перемещает cursor по screen, меняет colors of characters или меняет их на reverse video, insert and delete text и так далее.

Например, текстовые редакторы вроде Vim и Emacs programmed in curses. В Vim нажатие клавиши j заставит cursor move down one line. Ввод dd приведет к тому, что current line будет erased, строки ниже нее moved up one line each, а строки выше останутся unchanged. Эти actions достигаются вызовами функций in curses library.

Чтобы использовать curses, нужно включить этот statement в source code:

```
#include <curses.h>
```

и нужно link in curses library:

```
gcc -g sourcefile.c -lcurses
```

Возьмем приведенный ниже код как пример. Он выполняет Unix command `ps ax`, чтобы list all processes. В любой данный момент строка, на которой currently lies cursor, будет highlighted. Вы можете move cursor up and down, нажимая keys `u` и `d`, и так далее. Полный список commands смотрите в комментариях к коду.

Не беспокойтесь, если раньше не использовали curses, поскольку комментарии объясняют, что делает эта library.

```
// psax.c; illustration of curses library
// read this code in a "top-down" manner: first these comments and the global
// variables, then main(), then the functions called by main()
// runs the shell command 'ps ax' and saves the last lines of its output,
// as many as the window will fit; allows the user to move up and down
// within the window, with the option to kill whichever process is
// currently highlighted
// usage: psax
// user commands:
// 'u': move highlight up a line
// 'd': move highlight down a line
// 'k': kill process in currently highlighted line
// 'r': re-run 'ps ax' for update
// 'q': quit
// possible extensions: allowing scrolling, so that the user could go
// through all the 'ps ax' output, not just the last lines; allow
// wraparound for long lines; ask user to confirm before killing a
// process
#define MAXROW 1000
#define MAXCOL 500
#include <curses.h> // required
WINDOW *scrn; // will point to curses window object
char cmdoutlines[MAXROW][MAXCOL]; // output of 'ps ax' (better to use
// malloc())
int ncmdlines, // number of rows in cmdoutlines
nwinlines, // number of rows our "ps ax" output occupies in the
// xterm (or equiv.) window
winrow, // current row position in screen
cmdstartrow, // index of first row in cmdoutlines to be displayed
cmdlastrow; // index of last row in cmdoutlines to be displayed
// rewrites the line at winrow in bold font
highlight()
{
    int clinenum;
    attron(A_BOLD); // this curses library call says that whatever we
// write from now on (until we say otherwise)
// will be in bold font
// we'll need to rewrite the cmdoutlines line currently displayed
// at line winrow in the screen, so as to get the bold font
clinenum = cmdstartrow + winrow;
mvaddstr(winrow,0,cmdoutlines[clinenum]);
attroff(A_BOLD); // OK, leave bold mode
refresh(); // make the change appear on the screen
}
// runs "ps ax" and stores the output in cmdoutlines
runpsax()
{
    FILE *p; char ln[MAXCOL]; int row,tmp;
    p = popen("ps ax","r"); // open UNIX pipe (enables one program to read
// output of another as if it were a file)
    for (row = 0; row < MAXROW; row++) {
        tmp = fgets(ln,MAXCOL,p); // read one line from the pipe
        if (tmp == NULL) break; // if end of pipe, break
```

```

// don't want stored line to exceed width of screen, which the
// curses library provides to us in the variable COLS, so truncate
// to at most COLS characters
strncpy(cmdoutlines[row],ln,COLS);
cmdoutlines[row][MAXCOL-1] = 0;
}
ncmdlines = row;
close(p); // close pipe
}
// displays last part of command output (as much as fits in screen)
showlastpart()
{
int row;
clear(); // curses clear-screen call
// prepare to paint the (last part of the) 'ps ax' output on the screen;
// two cases, depending on whether there is more output than screen rows;
// first, the case in which the entire output fits in one screen:
if (ncmdlines <= LINES) { // LINES is an int maintained by the curses
// library, equal to the number of lines in
// the screen
cmdstartrow = 0;
nwinlines = ncmdlines;
}
else { // now the case in which the output is bigger than one screen
cmdstartrow = ncmdlines - LINES;
nwinlines = LINES;
}
cmdlastrow = cmdstartrow + nwinlines - 1;
// now paint the rows to the screen
for (row = cmdstartrow, winrow = 0; row <= cmdlastrow; row++,winrow++)
mvaddstr(winrow,0,cmdoutlines[row]); // curses call to move to the
// specified position and
// paint a string there
refresh(); // now make the changes actually appear on the screen,
// using this call to the curses library
// highlight the last line
winrow--;
highlight();
}
// moves cursor up/down one line
updown(int inc)
{
int tmp = winrow + inc;
// ignore attempts to go off the edge of the screen
if (tmp >= 0 && tmp < LINES) {
// rewrite the current line before moving; since our current font
// is non-BOLD (actually A_NORMAL), the effect is to unhighlight
// this line
mvaddstr(winrow,0,cmdoutlines[winrow]);
// highlight the line we're moving to
winrow = tmp;
highlight();
}
}
// run/re-run "ps ax"
rerun()
{
runpsax();
showlastpart();
}
// kills the highlighted process
prockill()
{
char *pid;
// strtok() is from C library; see man page
pid = strtok(cmdoutlines[cmdstartrow+winrow]," ");
kill(atoi(pid),9); // this is a UNIX system call to send signal 9,
// the kill signal, to the given process
rerun();
}
main()

```

```

{
char c;
// window setup; next 3 lines are curses library calls, a standard
// initializing sequence for curses programs
scrn = initscr();
noecho(); // don't echo keystrokes
cbreak(); // keyboard input valid immediately, not after hit Enter
// run 'ps ax' and process the output
runpsax();
// display in the window
showlastpart();
// user command loop
while (1) {
// get user command
c = getch();
if (c == 'u') updown(-1);
else if (c == 'd') updown(1);
else if (c == 'r') rerun();
else if (c == 'k') prockill();
else break; // quit
}
// restore original settings
endwin();
}

```

Запустив программу, вы обнаружите, что picture looks all right, но когда нажимаете key u, чтобы cursor go up one line, она работает некорректно, как видно на рисунке 6-1.

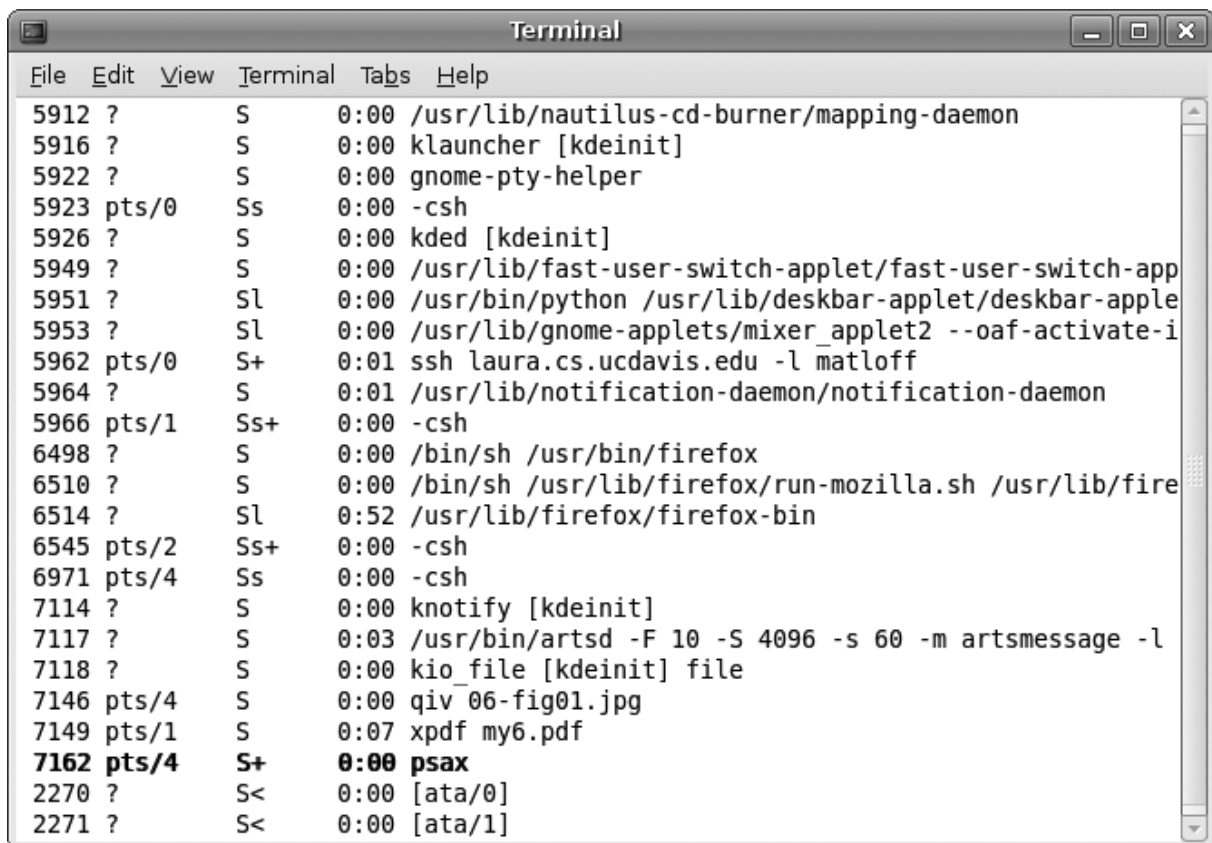


Рисунок 6-1. Программа, работающая в окне терминала

Вывод ps ax находится в ascending order by process number, но внезапно вы видите, что process 2270 displayed after 7162. Отследим bug.

Curses program - мечта автора книги по отладке, потому что она заставляет programmer использовать debugging tool. Programmer не может использовать вызовы printf() или операторы cout, чтобы печатать debugging information, потому что этот output будет mixed in with program output itself, вызывая безнадёжный chaos.

6.2.1.1 Использование GDB

Итак, запускаем GDB, но есть еще одна вещь, которую нужно сделать, связанная с последним моментом. Нужно сказать GDB, чтобы программа выполнялась в другом terminal window, отличном от того, где работает GDB. Это можно сделать командой GDB `tty`. Сначала переходим в другое окно, где будет выполняться program I/O, и запускаем там Unix command `tty`, чтобы определить ID этого window. В данном случае output этой команды сообщает, что window - terminal number `/dev/pts/8`, поэтому в окне GDB вводим:

```
(gdb) tty /dev/pts/8
```

С этого момента весь keyboard input и screen output для программы будет в execution window.

Последнее перед началом: нужно набрать что-то вроде:

```
sleep 10000
```

в execution window, чтобы наш keyboard input в это window шел программе, а не shell.

NB. Есть и другие способы handling problem of separating GDB output from program output. Например, можно сначала запустить execution программы, затем fire up GDB в другом окне, attaching it to the running program.

Затем установите breakpoint в начале функции `updown()`, поскольку error occurs, когда мы пытаемся move cursor up. Затем вводим `run`, и программа начнет выполняться в execution window. Нажмите key `u` в этом окне, и GDB остановится на breakpoint.

```
(gdb) r
Starting program: /Debug/psax
Detaching after fork from child process 3840.
Breakpoint 1, updown (inc=-1) at psax.c:103
103 { int tmp = winrow + inc;
```

Сначала подтвердим, что переменная `tmp` имеет правильное value.

```
(gdb) n
105 if (tmp >= 0 && tmp < LINES) {
(gdb) p tmp
$ 2=2 2
(gdb) p LINES
$ 3=2 4
```

Переменная `winrow` показывает current location of cursor within window. Это location должно быть самым концом window. `LINES` имеет value 24, поэтому `winrow` должно быть 23, поскольку numbering starts at 0. С `inc`, равным -1 (поскольку мы moving cursor up, not down), показанное здесь значение `tmp`, 22, confirmed.

Теперь перейдем к следующей строке.

```
(gdb) n
109 mvaddstr(winrow,0,cmdoutlines[winrow]);
(gdb) p cmdoutlines[winrow]
$4 = " 2270 ? Ss 0:00 nifd -n\n", '\0' <repeats 464 times>
```

Конечно, вот она, строка для process 2270. Мы быстро понимаем, что строка:

```
mvaddstr(winrow,0,cmdoutlines[winrow]);
```

в source code должна быть:

```
mvaddstr(winrow,0,cmdoutlines[cmdstartrow+winrow]);
```

После исправления программа работает нормально.

Когда закончите, нажмите CTRL-C в execution window, чтобы kill command sleep и снова сделать shell usable.

Обратите внимание: если что-то пойдет не так и программа prematurely finishes, execution window может retain some of nonstandard terminal settings, например cbreak mode. Чтобы исправить это, перейдите в это окно и нажмите CTRL-J, затем введите слово reset, затем снова нажмите CTRL-J.

6.2.1.2 Использование DDD

А что насчет DDD? Снова понадобится separate window for execution of the program. Организуйте это, выбрав View | Execution Window, и DDD откроет execution window. Обратите внимание, что команду sleep в это окно вводить не нужно, поскольку DDD делает это за вас. Теперь экран будет выглядеть как на рисунке 6-2.

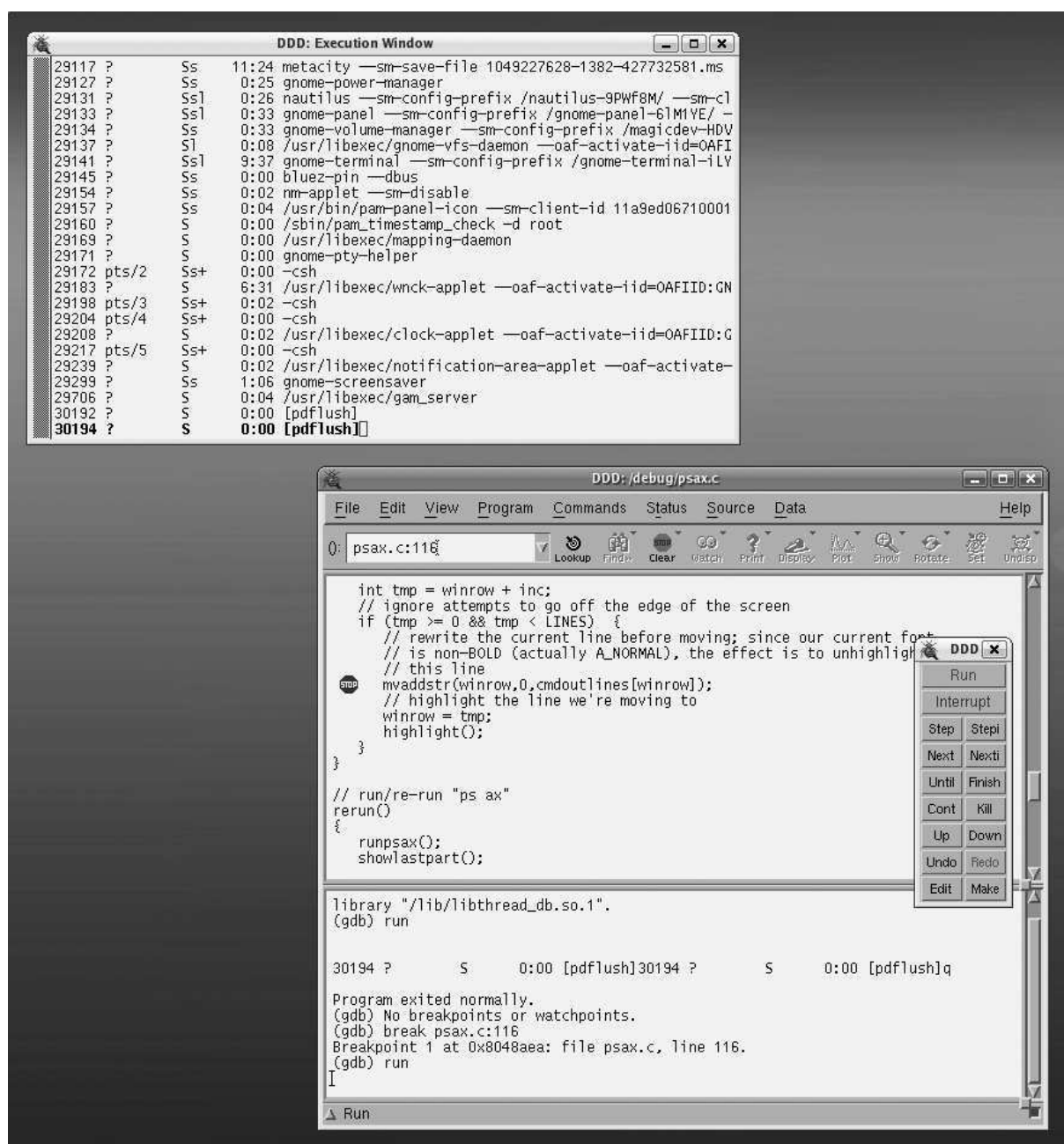


Рисунок 6-2. Присоединение к работающей программе в DDD

Устанавливайте breakpoints как обычно, но помните, что program input нужно вводить в execution window.

6.2.1.3 Использование Eclipse

Сначала обратите внимание, что при building вашего project нужно сказать Eclipse использовать flag `-lcurses` в makefile; procedure for this была показана в главе 5.

Здесь тоже понадобится separate execution window. Это можно сделать при setup вашего debug dialog. После setting up run dialog as usual и выбора Run | Open Debug Dialog мы пойдем немного другим путем, чем до сих пор. На рисунке 6-3 обратите внимание, что помимо обычного выбора C/C++ Local Application есть также option C/C++ Attach to Local Application. Последнее означает, что вы хотите, чтобы Eclipse использовал ability GDB attach itself to already running process (обсуждалось

в главе 5). Щелкните C/C++ Attach to Local Application правой кнопкой, выберите New и продолжайте как раньше.

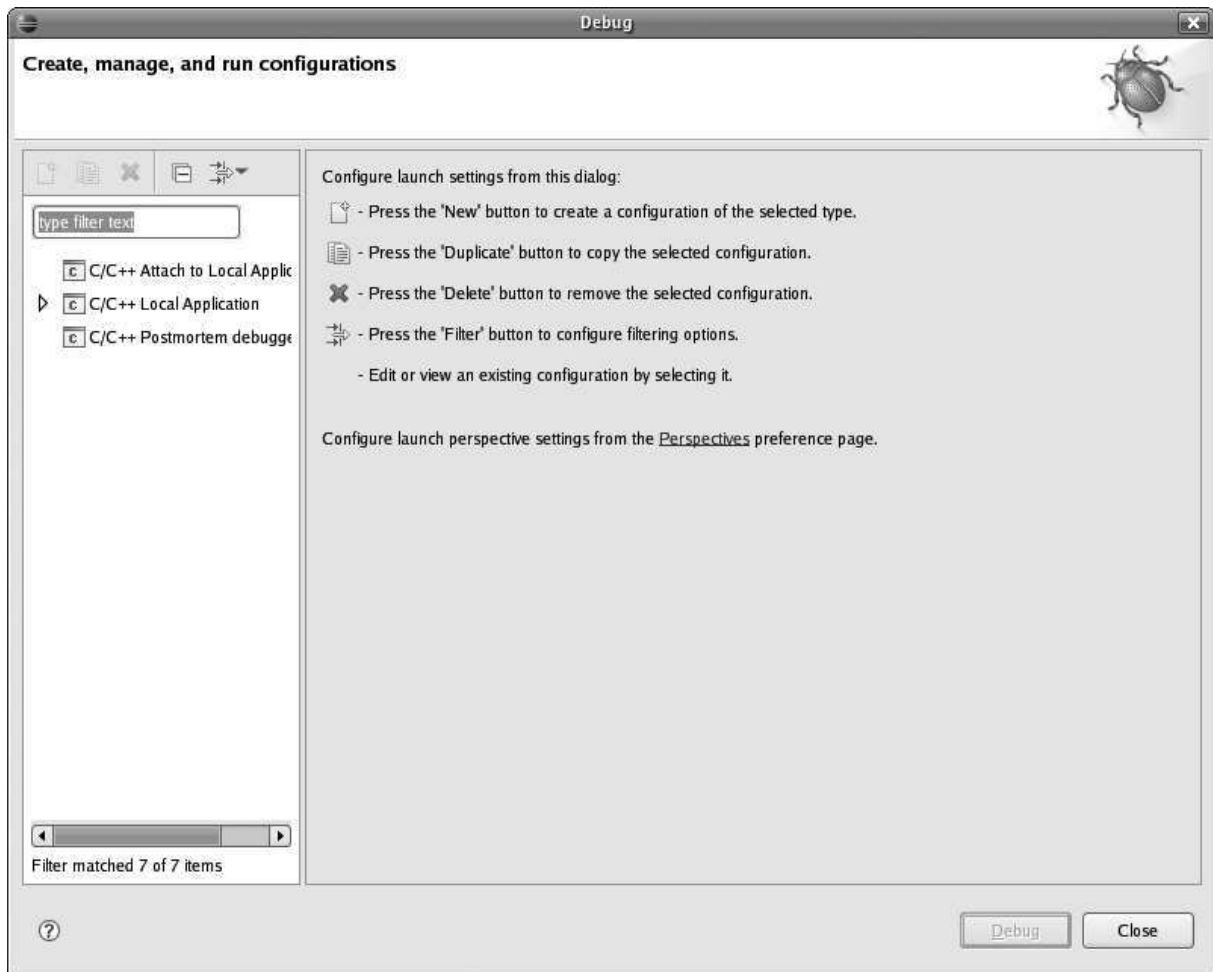


Рисунок 6-3. Присоединение к работающей программе в Eclipse

Когда вы начинаете actual debug run, сначала запустите программу в separate shell window. (Не забудьте, что программа, вероятно, находится в вашем Eclipse workspace directory.) Затем выберите Run | Open Debug Dialog, как обычно делаете при первом прохождении debug run; в этом случае Eclipse откроет окно со списком processes и попросит выбрать тот, к которому вы хотите attach GDB. Это показано на рисунке 6-4, где видно, что ваш process psax имеет ID 12319 (обратите внимание на программу, работающую в другом окне, частично скрытом здесь). Щелкните этот process, затем OK, что приведет к ситуации, изображенной на рисунке 6-5.

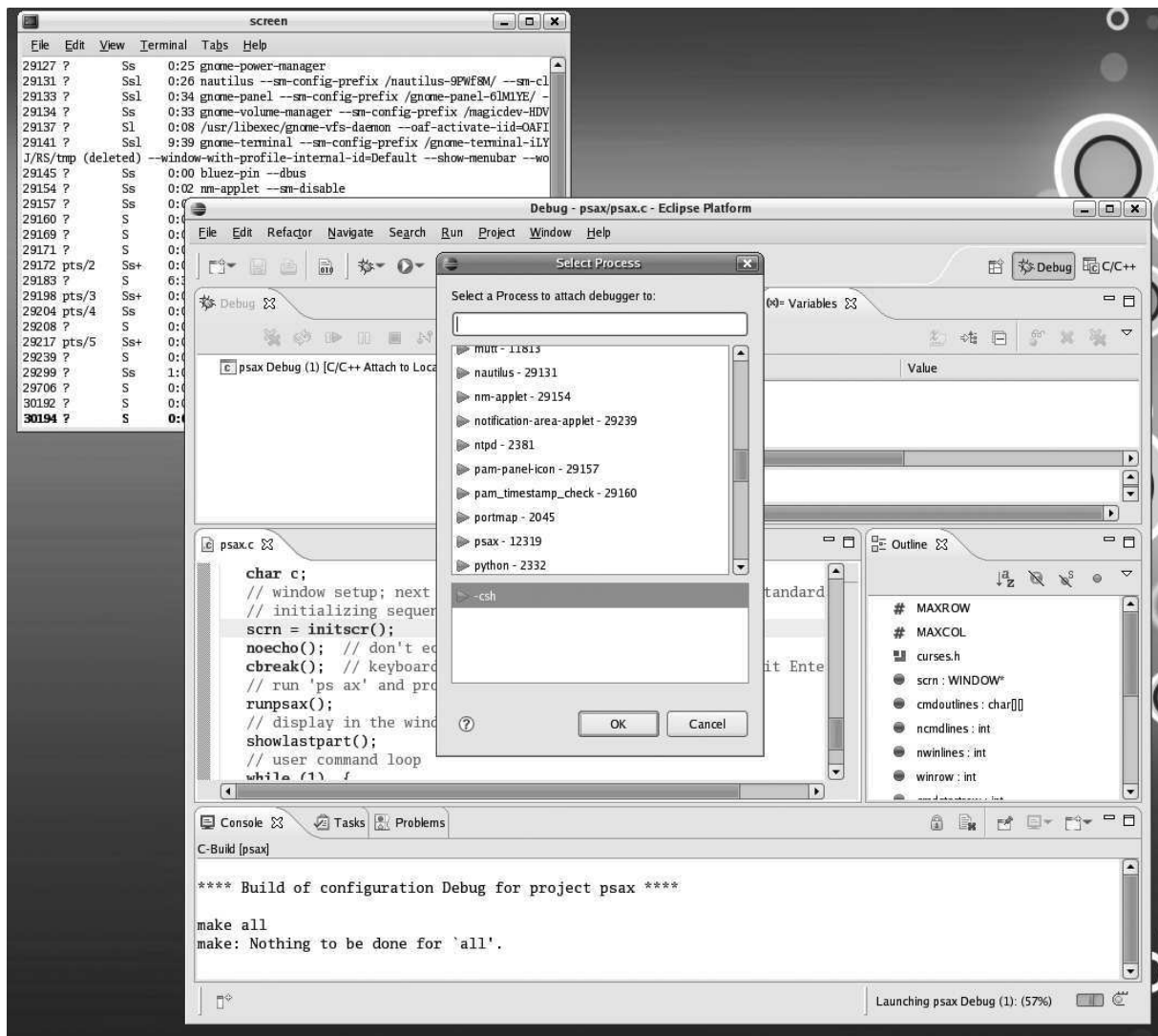


Рисунок 6-4. Выбор process, к которому нужно attach GDB

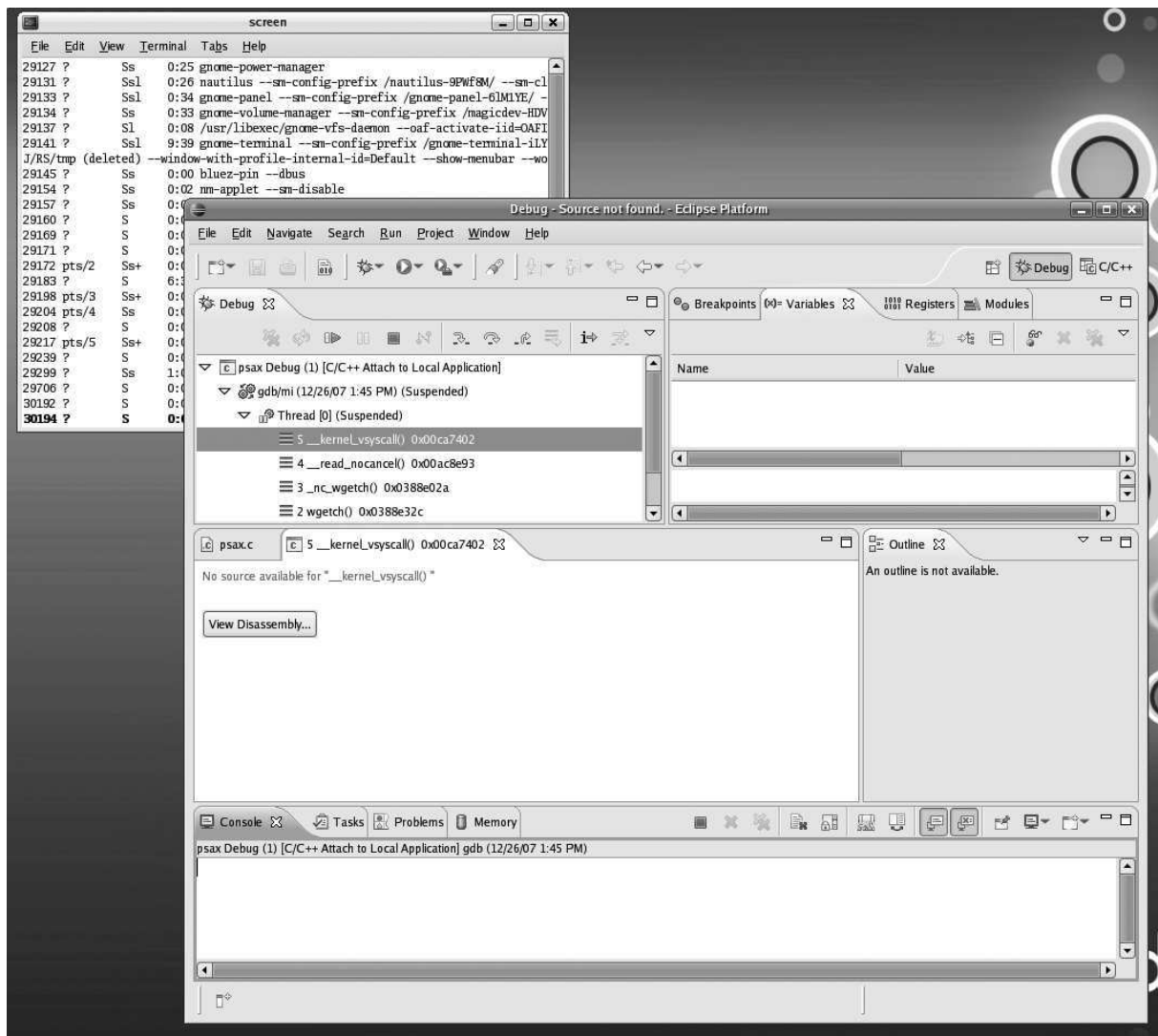


Рисунок 6-5. Остановка в kernel

На этом рисунке видно, что мы остановились во время system call. Eclipse сообщает, что у него нет source code для current instruction, но это ожидаемо и не является проблемой. На самом деле это хорошее время, чтобы установить breakpoints в source file `psax.c`. Сделайте это, затем щелкните icon Resume. Eclipse будет выполняться, пока не достигнет первого breakpoint, и затем можно отлаживать как обычно.

Глава 7. Другие инструменты

Умение отлаживать код не заканчивается освоением отладчика вроде GDB - оно с этого начинается. Существует множество других инструментов, как свободных, так и коммерческих, которые также помогают предотвращать, обнаруживать и устранять bugs в коде. Опытный программист держит несколько таких инструментов в своем наборе приемов, понимает, для чего каждый хорош, и узнает ситуации, когда применение одного из них сэкономит время и усилия при появлении bugs.

До сих пор мы сосредоточивали внимание на использовании symbolic debuggers, но теперь хотим расширить охват до других аспектов отладки, включая defensive programming. Эта глава посвящена некоторым инструментам и приемам, отличным от GDB, которые могут оказаться полезными как для предотвращения bugs с самого начала, так и для их поиска и исправления, когда они появляются.

7.1 Хорошее использование текстового редактора

Лучший метод отладки - вообще не делать programming errors с самого начала! Простое хорошее использование редактора, поддерживающего programming, является одним из самых недооцененных аспектов "pre-debugging".

Если вы проводите много времени за coding, мы настоятельно рекомендуем внимательно подумать о выборе редактора и узнать как можно больше о редакторе, которым будете пользоваться. Есть две основные причины. Во-первых, уверенное владение powerful editor уменьшает время, необходимое для написания кода. Редакторы со specialized features вроде automatic indentation, word completion и global symbol lookup - благо для programmer, и есть несколько вариантов на выбор. Во-вторых, хороший редактор действительно может помочь coder ловить определенные виды bugs прямо во время написания кода. Именно об этом этот раздел.

Оба автора этой книги используют Vim для programming, поэтому на нем мы и сосредоточимся. Однако все популярные редакторы имеют похожие, если не идентичные, feature sets. Если Vim расширяется полезной возможностью, которой пока нет в Emacs, community разработчиков Emacs быстро исправит ситуацию, и наоборот. Поэтому, хотя мы даем specifics для Vim, большая часть сказанного относится и к другим отличным редакторам вроде Emacs.

7.1.1 Syntax highlighting

Vim использует syntax highlighting, чтобы отображать части program file разными colors или fonts, так что code elements вроде keywords, type identifiers, local variables и preprocessor directives имеют собственную схему colors and fonts. Редактор выбирает схему colors and fonts, глядя на filename extension, чтобы определить используемый вами язык. Например, если filename заканчивается на .pl (что указывает на Perl script), occurrences слова die (имя Perl function) highlighted, тогда как если filename заканчивается на .c, они не highlighted.

Лучшим названием для syntax highlighting было бы lexical highlighting, потому что редактор обычно не анализирует syntax слишком глубоко. Он не сможет сказать, что вы передали неправильное число arguments или arguments wrong type при вызове function. Вместо этого он лишь понимает, например, что слова вроде bless и foreach являются Perl keywords, а fmt и dimension - Fortran keywords, и отображает их соответствующим образом.

Тем не менее syntax highlighting очень полезен для ловли простых, но легких для совершения ошибок. Например, на нашем компьютере default color для type identifiers, таких как keywords FILE или float в C, - зеленый. Когда глаз привыкает к font and color scheme, вы заметите color inconsistency, возникающую при misspelled type name, и автоматически исправите ошибку без лишнего compile cycle.

Пример использования syntax highlighting для проверки keywords, который нравится нам (авторам), встречается с makefiles. Keyword patsubst - очень полезная text search-and-replace command для makefiles. Одно из ее самых обычных применений - generate list of .o files from .c files исходного кода проекта:

```
TARGET = CoolApplication
OBSJ = $(patsubst %.c, %.o, $(wildcard *.c))
$(TARGET): $(OBSJ)
```

Один из авторов никак не может запомнить, пишется ли это patsubst, pathsubst или patsub. Зная, что makefile keywords displayed in a light color (yellow), сможете ли вы понять, какая версия строки ниже incorrect? Даже если вы не умеете писать makefiles, один только syntax highlighting должен сделать это ясным ^[1]

```
OBSJ = $(patsub %.c, %.o $(wildcard *.c))
OBSJ = $(patsubst %.c, %.o $(wildcard *.c))
```

Рисунок 7-1. Syntax highlighting находит мою ошибку в makefile.

Более того, вот пример, где syntax highlighting немного умнее. В figure ниже есть настоящая syntax error. Попробуйте найти, не слишком задумываясь, что (или хотя бы где) является ошибкой, основываясь только на colors:

```
if (fp == NULL) {
    puts("This was a \"bad\" file pointer.");
    exit(1);
}
```

Рисунок 7-2. Syntax highlighting выявляет распространенную ошибку.

И вот еще одна иллюстрация похожей ошибки. Попробуйте позволить color направить взгляд к ошибке.

```
fprintf(fp, "argument %d is \"%s\".\n", i, argv[i]);
printf("I just wrote \"%s\" which is argument %d\n", argv[i], i);
```

Рисунок 7-3. Syntax highlighting выявляет еще одну распространенную ошибку.

Вы можете обнаружить, что некоторые colors в syntax highlighting scheme трудно читать. Если так, можно выключить highlighting, набрав:

```
: syntax off
```

Команда для обратного включения, конечно:

```
: syntax on
```

Лучшим вариантом было бы изменить syntax file, чтобы использовать другой, более подходящий color для этого типа keyword, но это выходит за рамки нашего обсуждения.

7.1.2 Matching brackets

Ошибки unbalanced bracket чрезвычайно распространены, и их бывает очень трудно поймать. Рассмотрим следующий код:

```
mytype *myvar;
if ((myvar = (mytype *)malloc(sizeof(mytype))) == NULL) {
    exit(-1);
}
```

NB. В этом разделе слово bracket относится к parentheses, square brackets и braces (или curly brackets): `()`, `[]` и `{ }` соответственно.

Быстро: parentheses balanced? ^[2] Приходилось ли вам когда-нибудь отслеживать unbalanced brackets в длинных blocks of code с множеством conditionals? Или пытаться использовать TEX? (Мы содрогаяемся, вспоминая некоторые наши прошлые LATEX files с missing braces!) Тогда вы должны согласиться с нами: это именно то, для чего нужны компьютеры - избавлять нас от такой tedious work! У Vim есть отличные features, которые могут помочь.

- Каждый раз, когда вы вводите bracket с клавиатуры, option Vim showmatch заставляет Vim на мгновение поместить cursor на matching bracket (если matching bracket exists and is visible on screen). Можно даже управлять тем, как долго cursor задерживается на matching bracket, задав variable matchtime, которая specifies duration in tenths of seconds.
- Ввод percent symbol, когда cursor находится на bracket, переместит cursor к парному bracket. Эта command - отличный способ отслеживать unbalanced brackets.
- Когда вы помещаете cursor на bracket, Vim highlighted его mate, как показано на рисунке ???. Это также отличный способ отслеживать unbalanced bracket problems.

Option showmatch полезна, когда вы programming, но в иных случаях может раздражать. Можно использовать autocommands, чтобы устанавливать эту option только when you program. Например, чтобы установить showmatch только для editing sessions with C/C++ source code files, можно поместить строки вроде этих в файл .vimrc (подробнее см. Vim helpfiles):

```
au BufNewFile,BufRead *.c set showmatch
au BufNewFile,BufRead *.cc set showmatch
```

Что если unbalanced bracket попал в ваш код или, хуже того, нужно поймать unbalanced brackets в чужом spaghetti code? Упомянутая ранее editor command `%` searches for balanced grouping characters. Например, если поместить cursor над left square bracket `[` и набрать `%` из command mode, Vim reposition cursor to next `]` character. Если поставить cursor на right curly brace `}` и invoke `%`, то Vim reposition cursor на previous matching `{` character. Таким образом можно проверить не только то, что у каждого given round, curly или square bracket есть corresponding matching partner, но и то, что partner matches semantically. Можно даже определить другие matching pairs of "brackets", например HTML comment delimiters `<!--` и `-->`, используя команду Vim `matchpair`. Дополнительную информацию смотрите в Vim help pages.

7.1.3 Vim и makefiles

Utility make управляет compilation и building executables в Linux/Unix systems, и небольшое усилие по изучению ее использования может принести programmer большие dividends. Однако это также introduces new opportunities for errors. У Vim есть несколько features, которые могут действительно помочь в debugging process, если вы используете make. Рассмотрим следующий makefile snippet:

```
all: yerror.o main.o
gcc -o myprogram yerror.o main.o
yerror.o: yerror.c yerror.h
gcc -c yerror.c
main.o: main.c main.h
gcc -c main.c
```

В этом makefile есть ошибка, но ее трудно увидеть. make очень picky about formatting. Command line target должна начинаться с tab character - не spaces. Если вы выдадите команду `set list` из Vim, сразу увидите, что не так:

```
all: yerror.o main.o$
^Igcc -o myprogram yerror.o main.o$
$
yerror.o: yerror.c yerror.h$
^Igcc -c yerror.c$
$
main.o: main.c main.h$
gcc -c main.c$
```

В list mode Vim displays nonprintable characters. По умолчанию end-of-line character shows up as \$, а control characters displayed with caret symbol (^); следовательно tab character, то есть CTRL-I, displayed as ^I. Поэтому можно отличить spaces от tabs, и ошибка plainly visible: command line для make target main.o begins with spaces.

Можно управлять отображением через option Vim listchars. Например, если вы хотите изменить end-of-line character на = вместо \$, можно использовать `:set listchars=eol:=`.

7.1.4 Makefiles и compiler warnings

Вызов make из Vim может быть очень удобен. Например, вместо manual saving file и ввода make clean в другом окне, все, что нужно, - набрать `:make clean` из command mode. (Убедитесь, что autowrite установлен, чтобы Vim automatically saves file before running make command.) В общем случае, когда вы вводите:

```
:make arguments
```

из command mode, Vim запустит make и передаст ему arguments.

Дальше еще лучше. Когда вы build your program из Vim, editor captures all messages, которые compiler issues. Он understands syntax of GCC output и знает, когда возникает compiler warning или error. Посмотрим это в действии. Рассмотрим следующий код:

```
#include <stdio.h>
int main(void)
{
    printf("There were %d arguments.\n", argc);
    if (argc .gt. 5) then
        print *, 'You seem argumentative today';
    end if
    return 0;
}
```

Листинг 7-1. main.c

Похоже, кто-то немного занимался concurrent Fortran and C coding! Предположим, вы сейчас editing main.c и хотите build program. Выдайте command `:make` из Vim и увидите все error messages (рисунок 7-4).

```

:!make 2>&1| tee /tmp/v243244/1
gcc -std=c99 -W -Wall    main.c    -o main
main.c: In function 'main':
main.c:12: error: 'argc' undeclared (first use in this function)
main.c:12: error: (Each undeclared identifier is reported only once
main.c:12: error: for each function it appears in.)
main.c:14: error: expected identifier before numeric constant
main.c:14: error: 'then' undeclared (first use in this function)
main.c:15: error: expected ';' before 'print'
main.c:15:18: warning: character constant too long for its type
main.c:16: error: 'end' undeclared (first use in this function)
main.c:16: error: expected ';' before 'if'
make: *** [main] Error 1
(3 of 12): error: 'argc' undeclared (first use in this function)
Press ENTER or type command to continue

```

Рисунок 7-4. Сообщения об ошибках

Теперь, если нажать ENTER или spacebar, вы вернетесь к editing program, но cursor будет positioned on line that generated first warning or error (в этом случае сообщение, что argc undeclared), как показано на рисунке 7-5.

```

#include <stdio.h>

int main(void)
{
    printf("There were %d arguments.\n", argc);

    if (argc .gt. 5) then
        print *, 'You seem argumentative today';
    end if

    return 0;
}

```

Рисунок 7-5. Курсор расположен на первой ошибке.

После исправления ошибки есть два способа перейти к следующей:

- Можно remake program, и Vim снова display remaining warnings and errors и reposition cursor on first one. Это имеет смысл, если build time negligible, особенно если вы mapped single keystroke to build program, например так:

```
au BufNewFile,BufRead *.c map <F1> :make<CR>
```

- Можно также использовать :cnext, что displays next error or warning. Аналогично :cprevious displays last error or warning, а :cc displays current error or warning. Все три commands conveniently reposition cursor to location of "active" error or warning.

7.1.5 Заключительные мысли о текстовом редакторе как IDE

Уверенное владение chosen editor настолько самоочевидно, что им часто пренебрегают, но это действительно первый шаг в learning to program in a particular environment. Рискую преувеличить, editor для programmers - как musical instruments для musicians. Даже самым creative composers нужно знать basics of how to play an instrument, чтобы realize their ideas так, чтобы другие могли извлечь из них пользу. Освоение редактора в полной мере позволяет программировать быстрее, effective figure out other people's code и reduce number of compile cycles, которые нужно выполнять при debugging code.

Если вы используете Vim, мы рекомендуем книгу Steve Oualline *Vi IMproved - Vim* (New Riders, 2001). Книга thorough and well written. (К сожалению, она написана для Vim 6.0, и features Vim 7.0 and later вроде folding не covered.) Наша цель здесь была лишь дать вкус того, что Vim может делать для programmer, но книга Steve - great resource для learning details.

У Vim есть много features, которые авторы находят obscenely useful. Например, мы хотели бы cover:

- использование K для lookup functions in man pages
- поиск declarations переменных с gd и gD
- jumping to macro definitions with [^D and]^D
- displaying macro definitions with]d,]d, [D and]D
- window splitting для одновременного просмотра .c and .h files, чтобы check prototypes
- ... и многое другое

Но это книга по debugging, а не по Vim, и нам нужно вернуться к обсуждению additional software tools.

7.2 Хорошее использование компилятора

Если editor - ваше первое оружие в battle against bugs, то compiler - второе. Все compilers способны scan code and find common errors, но обычно нужно enable this error checking, invoking appropriate option.

Многие compiler warning options, вроде switch GCC -Wtraditional, вероятно, overkill except in special situations. Однако даже не думайте использовать GCC без -Wall каждый раз. Например, одна из самых common mistakes, которые делают новые C programmers, illustrated by following statement:

```
if (a = b)
    printf("Equality for all!\n");
```

Это valid C code, и GCC бодро его скомпилирует. Variable a assigned value of b, и это value используется в conditional. Однако это почти наверняка не то, что programmer meant to do. Используя switches GCC -Wall, вы хотя бы получите warning, alerting you that this code might be a mistake:

```
$ gcc try.c
$ gcc -Wall try.c
try.c: In function `main':
try.c:8: warning: suggest parentheses around assignment used as truth value
```

GCC suggests, чтобы вы parenthesize assignment a=b before using it as truth value, так же как вы делаете, когда assign value and perform comparison: if ((fp = fopen("myfile", "w")) == NULL). GCC по сути спрашивает: "Вы уверены, что хотите assignment a=b здесь, а не test for equality a==b?"

Всегда используйте error-checking options вашего compiler, и если вы teach programming, требуйте, чтобы students тоже их использовали, чтобы instill good habits. Пользователи GCC должны всегда использовать -Wall, даже для smallest "Hello, world!" program. Мы обнаружили, что prudent также использовать -Wmissing-prototypes and -Wmissing-declarations. Более того, если у вас есть spare 10 minutes, scanning GCC man page and reading compiler warning section - отличный способ провести время, особенно если вы собираетесь programming under Unix to any significant extent.

7.3 Error reporting в C

Error reporting в C выполняется через старый mechanism с именем errno. Хотя errno shows its age и имеет некоторые deficiencies, в целом он справляется. Вы можете wonder, зачем вообще нужен error-reporting mechanism, поскольку у большинства C functions есть удобное return value, сообщающее, succeeded or failed вызов. Ответ в том, что return value может warn you, что function не сделала то, что вы хотели, но may or may not tell you why. Чтобы сделать это more concrete, рассмотрим code snippet:

```
FILE *fp
fp = fopen("myfile.dat", "r");
retval = fwrite(&data, sizeof(DataStruct), 1, fp);
```

Предположим, вы inspect retval и find it to be zero. Из man page вы видите, что fwrite() должна return number of items (not bytes or characters) written, so retval should be 1. Сколькими способами может fail fwrite()? Многими! Для начала, filesystem may be full, или у вас может не быть write permissions on file. В этом случае, однако, в коде есть bug, который causes fwrite() to fail. Можете ли вы его найти? ^[3] Error-reporting system вроде errno может provide diagnostic information, чтобы help you figure out what happened в таких cases. (Operating system may announce certain errors as well.)

7.3.1 Использование errno

System and library calls, которые fail, обычно устанавливают globally defined integer variable named errno. В большинстве GNU/Linux systems errno declared in /usr/include/errno.h, так что include this header file избавляет от необходимости объявлять extern int errno в собственном коде.

Когда system or library call fails, он sets errno to value, indicating type of failure. Вы должны check value of errno и take appropriate action. Рассмотрим следующий код:

```
#include <stdio.h>
#include <errno.h>
#include <math.h>
int main(void)
{
    double trouble = exp(1000.0);
    if (errno) {
        printf("trouble: %f (errno: %d)\n", trouble, errno);
        exit(-1);
    }
    return 0;
}
```

Листинг 7-2. double-trouble.c

В нашей системе `exp(1000.0)` больше, чем может store `double`, поэтому assignment results in floating-point overflow. Из output вы видите, что value `errno 34` indicates floating-point overflow error:

```
$ ./a.out
trouble: inf (errno: 34)
```

Это pretty much illustrates how `errno` works. По convention, когда library function or system call fails, он sets `errno` to value, describing why call failed. Вы только что видели, что value 34 means result of `exp(1000.0)` was not representable by `double`, и есть множество other codes, indicating underflow, permission problems, file not found and other error conditions. Однако перед тем как начать использовать `errno` в programs, нужно знать о нескольких issues.

Во-первых, code that uses `errno` may not be completely portable. Например, ISO C standard определяет лишь несколько error codes, а POSIX standard определяет much more. В man page `errno` можно увидеть, какие error codes defined by which standards. Более того, standards не specify numeric values, вроде 34, for error codes. Они prescribe symbolic error codes, macro constants whose names are prefixed by `E` and defined in `errno` header file (or in files included by `errno` header). Единственное consistent across platforms свойство их values - они nonzero. Поэтому нельзя assume, что particular value always indicates same error condition.^[4] Всегда используйте symbolic names для refer to `errno` values.

В дополнение к ISO and POSIX `errno` values конкретные implementations of C library, вроде GNU `glibc`, могут define even more `errno` values. На GNU/Linux section `errno` in `libc` info pages^[5] является canonical source для all available `errno` values on that platform: ISO, POSIX and `glibc`. Вот несколько error code defines, взятых нами из `/usr/include/asm/errno.h` на GNU/Linux machine:

```
#define EPIPE 32 /* Broken pipe */
#define EDOM 33 /* Math arg out of domain of func */
#define ERANGE 34 /* Math result not representable */
#define EDEADLK 35 /* Resource deadlock would occur */
#define ENAMETOOLONG 36 /* File name too long */
#define ENOLCK 37 /* No record locks available */
#define ENOSYS 38 /* Function not implemented */
```

Далее есть несколько important facts, которые нужно помнить о том, как используется `errno`. `errno` может быть set any library function or system call, whether it fails or succeeds! Поскольку even successful function calls могут set `errno`, вы не можете rely on `errno`, чтобы tell you whether an error occurs. Можно rely only on it to tell you why an error happened. Поэтому safest way to use `errno` is as follows:^[6]

1. Perform call to library or system function.
2. Use function's return value to determine whether or not an error occurred.
3. If an error occurred, use `errno` to determine why.

В pseudocode:

```
retval = syscall();
if (retval indicates an error) {
    examine_errno();
    take_action();
}
```

Это подводит нас к man pages. Предположим, вы coding и хотите throw in some error checking после вызова `ptrace()`. Step two says использовать return value `ptrace()`, чтобы determine if an error occurred. Если вы похожи на нас, return values of `ptrace()` у вас не memorized. Что делать? В каждой man page есть section named "Return Value". Можно quickly go to it by typing `man function name` and searching for string return value.

Хотя у `errno` есть drawbacks, есть и good news.

В GNU C library идет extensive work to save errno when a function is entered and then restore it to original value if function call succeeds. Похоже, glibc очень старается not to write over errno for successful function calls. Однако мир еще не GNU, поэтому portable code не должен rely on this fact.

Также, хотя каждый раз лезть в documentation, чтобы увидеть, что означает particular error code, надоедает, есть две functions, упрощающие interpretation error codes: perror() и strerror(). Они делают same thing, но разными ways. Function perror() takes string argument and has no return value:

```
#include <stdio.h>
void perror(const char *s);
```

Argument of perror() - user-supplied string. Когда perror() called, она prints this string, followed by colon and space, then description of type of error based on value of errno. Вот простой example of how to use perror():

```
int main(void)
{
    FILE *fp;
    fp = fopen("/foo/bar", "r");
    if (fp == NULL)
        perror("I found an error");
    return 0;
}
```

Листинг 7-3. perror-example.c

Если на вашей системе нет файла /foo/bar, output выглядит так:

```
$ ./a.out
I found an error: No such file or directory
```

Output of perror() goes to standard error. Помните об этом, если хотите redirect program's error output to a file.

Еще одна function, помогающая translate errno codes into descriptive messages, - strerror():

```
#include <string.h>
char *strerror(int errnum);
```

Эта function takes value of errno as argument and returns string describing error. Вот example of how to use strerror():

```
int main(void)
{
    close(5);
    printf("%s\n", strerror(errno));
    return 0;
}
```

Листинг 7-4. strerror-example.c

Вот output этой программы:

```
$ ./a.out
Bad file descriptor
```

7.4 Более удобная жизнь со strace и ltrace

Важно понимать difference between library functions and system calls. Library functions are higher level, run completely in user space, and provide more convenient interface for programmer to functions that do real work - system calls. System calls do work in kernel mode on user's behalf and are provided by kernel of operating system itself. Library function `printf()` может выглядеть как very general printing function, но все, что она реально делает, - formats data into strings and writes string data using low-level system call `write()`, который then sends data to file associated with terminal's standard output.

Utility `strace` prints out each system call that your program makes, along with its arguments and return value. Хотите увидеть, какие system calls made by `printf()`? Легко! Напишите "Hello, world!" program, но запустите ее так:

```
$ strace ./a.out
```

Не впечатлены тем, как усердно ваш computer работает, чтобы просто напечатать something on screen?

Каждая line of `strace` output corresponds to one system call. Большая часть output `strace` shows calls to `mmap()` and `open()` with filenames like `ld.so` and `libc`. Это связано с system-level things вроде mapping disk files into memory and loading shared libraries. Most likely это вас не интересует. Для наших purposes есть ровно две interesting lines toward the end of output:

```
write(1, "hello world\n", 12hello world) = 12
_exit(0) = ?
```

Эти lines illustrate general format of `strace` output:

- name of system function being called
- arguments to system call between parentheses
- return value of system call ^[7] following = symbol

Вот и все, но какое богатство information! Вы также можете видеть errors. Например, на нашей системе мы получаем следующие строки:

```
open("/etc/ld.so.preload", O_RDONLY) = -1 ENOENT (No such file or directory)
open("/etc/ld.so.cache", O_RDONLY) = 3
```

Первый call to `open()` tries to open file named `/etc/ld.so.preload`. Return value of call to `open()` (which should be non-negative file descriptor) is `-1`, indicating error of some sort. `strace` helpfully tells us that error that caused `open()` to fail is `ENOENT`: file `/etc/ld.so.preload` does not exist.

`strace` tells us that second call to `open()` returned value `3`. This is valid file descriptor, so apparently call to open file `/etc/ld.so.cache` succeeded. Accordingly, no error codes on second line of `strace` output.

Кстати, не беспокойтесь об errors like this. То, что вы видите, связано с dynamic library loading and is not really an error per se. File `ld.so.preload` can be used to override system's default shared libraries. Since I have no desire to fiddle with such things, file simply doesn't exist on my system. По мере gaining experience with `strace`, вы будете все лучше filtering out this kind of "noise" and concentrating on parts of output really interested in.

У `strace` есть несколько options, которые at some point понадобятся, поэтому кратко опишем их здесь. Если вы посмотрели complete `strace` output of "Hello, world!" program, могли заметить, что `strace` can be a bit ... verbose. Гораздо удобнее save all that output in a file, чем смотреть на screen. Один способ, конечно, redirect `stderr`, но можно также использовать switch `-o logfile`, чтобы `strace` wrote all its output to logfile. Также `strace` normally truncates strings to 32 characters. Иногда это может hide important information. To force `strace` truncate strings at N characters, use option `-s N`. Наконец, если running `strace` on program that forks child processes, можно capture `strace` output for individual children to file named `LOG.xxx` with `-o LOG -ff` switch, where xxx is child's process ID.

Также есть utility named `ltrace`, which is like `strace`, but shows library calls rather than system calls. `ltrace` and `strace` have many options in common, so knowing how to use one takes you far toward learning the other.

Utilities `strace` and `ltrace` очень полезны, когда вы хотите send bug reports and diagnostic information to maintainers of programs, for which you don't have source code, and even if source files are available, using these tools can sometimes be faster than digging through code.

Один из авторов впервые stumbled across usefulness of these tools, когда пытался install and run poorly documented proprietary application on his system. При запуске application immediately returned to shell, apparently without doing anything. Он хотел отправить company something more informative than observation "Your program immediately exits." Running `strace` on application yielded clue:

```
open(umovestr: Input/output error 0, O_RDONLY) = -1 EFAULT (Bad address)
```

a running `ltrace` yielded even more clues:

```
fopen(NULL, "r") = 0
```

Согласно output, это был very first call to `fopen()`. Application presumably wanted to open some kind of configuration file, but `fopen()` was passed `NULL`. Application had some kind of internal fault handler that exited but produced no error message. Автор смог написать detailed bug report company, и, как оказалось, problem была в том, что application shipped with faulty global configuration file, pointing to non-existent local configuration file. Patch was issued next day.

С тех пор авторы found `strace` and `ltrace` immensely useful for tracking down bugs and figuring out stubborn, mysterious program behavior that can cause lots of head scratching.

7.5 Static code checkers: `lint` и родственники

Существует ряд free and commercial tools, которые scan your code without compiling it and warn you about errors, possible errors, and deviations from strict C coding standards. Они appropriately named static code checkers. Canonical static code checker for C, written by S.C. Johnson in late 1970s, назывался `lint`. Он был written mainly to check function calls, because early versions of C didn't support prototyping. `lint` spawned many derivative static checkers. One such checker, written by Dave Evans of University of Virginia computer science department, named `lclint`, был popular on modern systems like Linux. In January 2002, Dave renamed `lclint` to `splint`, чтобы emphasize increased focus on secure programming (and because `splint` easier to pronounce than `lclint`).

Aim of `splint` - помочь писать most defensive, secure, and error-free program possible. `splint`, like its predecessors, can be very picky about what constitutes good code.^[8] В качестве упражнения попробуйте найти что-нибудь в следующем коде, что может cause warning:

```
int main(void)
{
    int i;
    scanf("%d", &i);
    return 0;
}
```

Листинг 7-5. scan.c

Когда этот код run through `splint`, он warns, что вы discarding return value of `scanf()`.^[9]

```
$ splint test.c
Splint 3.0.1.6 --- 11 Feb 2002
test.c: (in function main)
test.c:8:2: Return value (type int) ignored: scanf("%d", &i)
Result returned by function call is not used. If this is intended, can cast
result to (void) to eliminate message. (Use -retvalint to inhibit warning)
Finished checking --- 1 code warning
```

Бве warnings splint have fixed format. First line of splint warning tells filename and function where warning occurs. Next line gives line number and position of warning. After that is description of warning, along with instructions on how to suppress this kind of warning. Как видно здесь, invoking `splint -retvalint test.c` turns off all warnings about discarding integer function return values. Alternatively, if you don't want to turn off all reports about discarded int return values, можно suppress warning for just this call to `scanf()` by typecasting `scanf()` to void. То есть replace `scanf("%d", &i);` with `(void) scanf("%d", &i);`. (Есть еще один способ suppress this warning, using annotations; interested reader can learn from splint documentation.)

7.5.1 Как использовать splint

splint поставляется with staggering number of switches, но по мере use you'll get feel for which ones typically suit needs. Многие switches Boolean in nature - they turn feature on or off. These types of switches are preceded by + to turn them on or - to turn them off. Например, `-retvalint` turns off reporting of discarded int return values, and `+retvalint` turns this reporting on (default behavior).

Есть два способа использовать splint. Первый informal и used mainly when running splint on someone else's code or code that's mostly final:

```
$ splint +weak *.c
```

Без switch `+weak`, splint typically too picky to be useful. Есть три levels of checking in addition to `+weak`. Подробности есть в manual splint, но briefly:

`+weak` Weak checking, typically for unannotated C code

`+standard` Default mode

`+checks` Moderately strict checking

`+strict` Absurdly strict checking

The more formal use of splint involves documenting your code specifically for use with splint. This splint-specific documentation is called annotation. Annotation - large topic, and it would take us too far afield to include it in this book, but it is well covered in splint documentation.

7.5.2 Последние слова

splint supports many, but not all, of C99 library extensions. It also doesn't support some C99 language changes. For instance, splint doesn't know about complex data types or about defining an int variable in initializer of for loop.

splint released under GNU GPL. Its homepage is at <http://www.splint.org/>.

7.6 Отладка dynamically allocated memory

Как вы, возможно, знаете, dynamically allocated memory (DAM) - это memory, которую program requests from heap with functions like `malloc()` and `calloc()`.^[10] Dynamically allocated memory typically used for data structures like binary trees and linked lists, and it is also at work behind the scenes when you create object in object-oriented programming. Even standard C library uses DAM for its own internal purposes. Вы также можете помнить, что dynamic memory must be freed when you're done with it.^[11]

DAM problems notoriously difficult to find and fall into several general categories:

- Dynamically allocated memory is not freed.
- Call to `malloc()` fails (this is easy to detect by checking return value of `malloc()`).
- Read or write is performed to address outside DAM segment.
- Read or write is performed to memory within DAM region after segment has been freed.
- `free()` is called twice on same segment of dynamic memory.

These errors may not cause your program to crash in obvious way. Обсудим это немного подробнее.

Чтобы discussion more concrete, вот example illustrating these problems:

```
int main( void )
{
    int *a = (int *) malloc( 3*sizeof(int) ); // malloc return not checked
    int *b = (int *) malloc( 3*sizeof(int) ); // malloc return not checked
    for (int i = -1; i <= 3; ++i)
        a[i] = i; // a bad write for i = -1 and 3
    free(a);
    printf("%d\n", a[1]); // a read from freed memory
    free(a); // a double free on pointer a
    return 0; // program ends without freeing *b.
}
```

Листинг 7-6. memprobs.c

Первая проблема называется memory leak. Например, рассмотрим следующий код:

```
int main( void )
{
    ... lots of previous code ...
    myFunction();
    ... lots of future code ...
}
void myFunction( void )
{
    char *name = (char *) malloc( 10*sizeof(char) );
}
```

Листинг 7-7. Example of a memory leak

Когда `myFunction()` executes, вы allocate memory for 10 char values. Единственный способ refer to this memory - use address returned by `malloc()`, который stored in pointer variable `name`. If, for some reason, you lose track of address - for example, `name` goes out of scope when `myFunction()` exits, and you haven't saved copy of its value elsewhere - then you have no way to access allocated memory and, in particular, no way to free it.

Но именно это происходит в коде. Dynamically allocated memory doesn't simply disappear or go out of scope the way storage for stack-allocated variable like `name` does, and so each time `myFunction()` is called, it gobbles up memory for 10 chars, which then is never released. Net result is that available heap space gets smaller and smaller. That's why this bug is called memory leak.

Memory leaks decrease amount of memory available to program. On most modern systems, like GNU/Linux, this memory is reclaimed by operating system when application with memory leak terminates. On ancient systems, like Microsoft DOS and Microsoft Windows 3.1, leaked memory is lost until operating system rebooted. In either case, memory leaks cause degradation in system performance due to increased paging. Over time, they can cause program with leak, or even entire system, to crash.

Second problem encountered with dynamically allocated memory is that call to `malloc()` may fail. There are lots of ways this can happen. For example, bug in computation might cause request for amount of DAM too large or negative. Or perhaps system really is out of memory. If you don't realize this happened and continue to try to read and write to what you mistakenly believe to be valid DAM, complications develop in already unpleasant situation. This is a type of access violation, which we'll discuss shortly. However, to avoid it, always check whether `malloc()` returns non-NULL pointer and try to exit program gracefully if it does not.

Third and fourth problems are called access errors. Both are basically versions of same thing: program tries to read from or write to memory address not available to it. Third problem involves accessing memory address above or below DAM segment. Fourth problem involves accessing memory address that used to be available, but was freed prior to access attempt.

Calling `free()` on same segment of DAM twice is colloquially known as double free. C library has internal memory management structures that describe boundaries of each allocated DAM segment. When you call `free()` twice on same pointer to dynamic memory, program's memory management structure becomes corrupted, which can lead to program crashing or, in some cases, allow malicious programmer to exploit bug to produce buffer overflow. In a sense, this is also access violation, but for C library itself rather than program.

Access violations cause one of two things to happen: program can crash, possibly writing core file ^[12] (usually after receiving segmentation fault signal), or, much worse, it can continue to execute, leading to data corruption.

Of these two consequences, former infinitely more desirable. In fact, there are tools available that will cause your program to seg fault and dump core whenever any problem with DAM is detected, rather than risk alternative!

You may wonder: "Why in the world would I want my program to seg fault?" You should get comfortable with idea that if code's handling of DAM is buggy, it is a Very Good Thing (TM) when it crashes, because the other option is intermittent, puzzling, and irreproducible bad behavior. Memory corruption can go unnoticed for long time before effects are felt. Often problem manifests itself in parts of program quite far from bug, and tracking it down can be nightmare. If that weren't bad enough, memory corruption can give rise to breaches of security. Applications known to cause buffer overruns and double frees can, in some instances, be exploited by malicious crackers to run arbitrary code and are responsible for many operating system security vulnerabilities.

On other hand, when your program seg faults and dumps core, you can perform postmortem on core file and learn precise source file and line number of code that caused seg fault. And that, dear reader, is preferable to bug hunting.

In short, catching DAM problems as soon as possible is of extreme importance.

7.6.1 Strategies for detecting DAM problems

In this section we'll discuss Electric Fence, a library that enforces a "fence" around allocated memory addresses. Access to memory outside these fences typically results in seg fault and core dump. We'll also discuss two GNU tools, `mtrace()` and `MALLOC_CHECK_`, that add hooks into standard libc allocation functions to keep records about currently allocated memory. This allows libc to perform checks on memory you're about to read, write, or free. Keep in mind that care is needed when using several software tools, each of which uses hooks to heap-related function calls, because one facility may install one of its hooks over a previously installed hook. ^[13]

7.6.2 Electric Fence

Electric Fence, or EFence, is a library written by Bruce Perens in 1988 and released under GNU GPL license while he worked at Pixar. When linked into your code, it causes program to immediately seg fault and dump core ^[14] when any of following occur:

- A read or write is performed outside boundary of DAM.
- A read or write is performed to DAM that has already been freed.
- A `free()` is performed on pointer that doesn't point to DAM allocated by `malloc()` (this includes double frees as special case).

Let's see how to use Electric Fence to track down `malloc()` problems. Consider program `outOfBound.c`:

```
int main(void)
{
    int *a = (int *) malloc( 2*sizeof(int) );
    for (int i=0; i<=2; ++i) {
        a[i] = i;
        printf("%d\n ", a[i]);
    }
    free(a);
    return 0;
}
```

Листинг 7-8. outOfBound.c

Although program contains archetypal `malloc()` bug, it will probably compile without warnings. Chances are, it will even run without problems: ^[15]

```
$ gcc -g3 -Wall -std=c99 outOfBound.c -o outOfBound_without_efence -lefence
$ ./outOfBound_without_efence
0
1
2
```

We were able to write beyond last element of array `a[]`. Everything looks fine now, but that just means this bug manifests itself unpredictably and will be difficult to nail down later.

Now we'll link `outOfBound` with `EFence` and run it. By default, `EFence` only catches reads or writes beyond last element of dynamically allocated region. That means `outOfBound` should seg fault when you try to write to `a[2]`:

```
$ gcc -g3 -Wall -std=c99 outOfBound.c -o outOfBound_with_efence -lefence
$ ./outOfBound_with_efence
Electric Fence 2.1 Copyright (C) 1987-1998 Bruce Perens.
0
1
Segmentation fault (core dumped)
```

Sure enough, `EFence` found write operation past last element of array.

Accidentally accessing memory before first element of array (for example, specifying "element" `a[-1]`) is less common, but it can certainly occur as result of buggy index calculations. `EFence` provides global `int` named `EF_PROTECT_BELOW`. When you set this variable to 1, `EFence` catches only array underruns and does not check for array overruns:

```
extern int EF_PROTECT_BELOW;
double myFunction( void )
{
    EF_PROTECT_BELOW = 1; // Check from below
    int *a = (int *) malloc( 2*sizeof(int) );
    for (int i=-2; i<2; ++i) {
        a[i] = i;
        printf("%d\n", a[i]);
    }
    ...
}
```

Because of the way `EFence` works, you can catch either attempts to access memory beyond dynamically allocated blocks or attempts to access memory before allocated blocks, but not both types of access errors at same time.

To be thorough, run program twice using `EFence`: once in default mode to check for dynamic memory overruns and a second time with `EF_PROTECT_BELOW` set to 1 to check for underruns.^[16]

In addition to `EF_PROTECT_BELOW`, `EFence` has a few other global integer variables that you can set to control behavior:

`EF_DISABLE_BANNER` Setting this variable to 1 hides banner displayed when you run program linked with `EFence`. Doing this is not recommended, because banner warns you that `EFence` is linked into application and executable should not be used for production release, because executables linked to `EFence` are larger, run more slowly, and produce very large core files.

`EF_PROTECT_BELOW` As discussed, `EFence` checks for DAM overruns by default. Setting this variable to 1 will cause `EFence` to check for memory underruns.

`EF_PROTECT_FREE` By default, `EFence` will not check for access to DAM that has already been freed. Setting this variable to 1 enables protection of freed memory.

`EF_FREE_WIPES` By default, `EFence` will not change values stored in memory that is freed. Setting this variable to nonzero value causes `EFence` to fill segments of dynamically allocated memory with `0xbd` before they are released. This makes improper accesses to freed memory easier to detect by `EFence`.

`EF_ALLOW_MALLOC_0` By default, `EFence` will trap any call to `malloc()` you make with argument 0 (that is, any request for zero bytes of memory). Rationale is that writing something like `char *p = (char *) malloc(0);` is probably a bug. However, if for some reason you really do mean to pass zero to `malloc()`, then setting this variable to nonzero value will cause `EFence` to ignore such calls.

As exercise, try writing program that accesses DAM already freed, and use `EFence` to catch error.

Whenever you change one of these global variables, you need to recompile program, which can be inconvenient. Thankfully, there is easier way. You can also set shell environment variables with same names as EFence's global variables. EFence will detect shell variables and take appropriate action.

As demonstration, we'll set environment variable `EF_DISABLE_BANNER` to suppress printing of EFence banner page. (As mentioned before, you shouldn't do this; do as I say, not as I do!) If you use Bash, execute:

```
$ export EF_DISABLE_BANNER=1
```

C shell users should execute:

```
% setenv EF_DISABLE_BANNER 1
```

Затем снова запустите листинг 7-8 и убедитесь, что баннер отключен.

Еще один прием - задавать переменные EFence из GDB во время отладочной сессии. Это работает потому, что переменные EFence глобальные; однако это также означает, что программа должна уже выполняться, но быть приостановленной.

7.6.3 Debugging DAM problems with GNU C Library tools

If you're working on GNU platform, such as GNU/Linux, there are GNU C library-specific features, similar to EFence, that you can use to catch and recover from dynamic memory problems. We'll discuss them briefly here.

7.6.3.1 The `MALLOC_CHECK_` environment variable

GNU C library provides shell environment variable named `MALLOC_CHECK_` that can be used, like EFence, to catch DAM access violations, but you don't need to recompile your program to use it. Settings and effects:

- 0 All DAM checking is turned off (also the case if variable undefined).
- 1 Diagnostic message is printed on `stderr` when heap corruption detected.
- 2 Program aborts immediately and dumps core when heap corruption detected.
- 3 Combined effects of 1 and 2. ^[17]

Since `MALLOC_CHECK_` is environment variable, using it to find heap-related problems is as simple as typing:

```
$ export MALLOC_CHECK_=3
```

Although `MALLOC_CHECK_` is more convenient than `EFence`, it has serious drawbacks. First, `MALLOC_CHECK_` only reports dynamic memory problem upon next execution of heap-related function (such as `malloc()`, `realloc()` or `free()`, for example) following illegal memory access. This means not only that you don't know source file and line number of problematic code; often you don't even know which pointer is problem variable. To illustrate, consider this code:

```
1 int main(void)
2 {
3     int *p = (int *) malloc(sizeof(int));
4     int *q = (int *) malloc(sizeof(int));
5
6     for (int i=0; i<400; ++i)
7         p[i] = i;
8
9     q[0] = 0;
10
11     free(q);
12     free(p);
13     return 0;
14 }
```

Листинг 7-9. malloc-check-0.c

Program aborts at line 11 when problem really occurs on line 7. Examining core file might lead you to believe problem lies with `q`, not `p`:

```
$ MALLOC_CHECK_=3 ./malloc-check-0
malloc: using debugging hooks
free(): invalid pointer 0x8049680!
Aborted (core dumped)
$ gdb malloc-check-0 core
Core was generated by `./malloc-check-0'.
Program terminated with signal 6, Aborted.
Reading symbols from /lib/libc.so.6...done.
Loaded symbols for /lib/libc.so.6
Reading symbols from /lib/ld-linux.so.2...done.
Loaded symbols for /lib/ld-linux.so.2
#0 0x40046a51 in kill () from /lib/libc.so.6
(gdb) bt
#0 0x40046a51 in kill () from /lib/libc.so.6
#1 0x40046872 in raise () from /lib/libc.so.6
#2 0x40047986 in abort () from /lib/libc.so.6
#3 0x400881d2 in _IO_file_xsputn () from /lib/libc.so.6
#4 0x40089278 in free () from /lib/libc.so.6
#5 0x080484bc in main () at malloc-check-0.c:13
```

You might live with this drawback when debugging 14-line programs, but it can be serious issue with real application. Nevertheless, knowing that DAM problem exists at all is useful information.

Second, this implies that if no heap-related function is called after access error occurs, `MALLOC_CHECK_` will not report error at all.

Third, `MALLOC_CHECK_` error messages don't seem very meaningful. Although previous program listing had array overrun error, error message simply "invalid pointer." Technically true, but not useful.

Finally, `MALLOC_CHECK_` disabled for `setuid` and `setgid` programs, because this combination of features could be used in security exploit. It can be re-enabled by creating file `/etc/suid-debug`. Contents of this file aren't important; only file's existence matters.

In conclusion, `MALLOC_CHECK_` is convenient tool during code development to catch heap-related programming blunders. However, if you suspect DAM problem or want to carefully scan your code for possible DAM problems, use another utility.

7.6.3.2 Using the mcheck() facility

Alternative to MALLOC_CHECK_ facility for catching DAM problems is mcheck() facility. We've found this method more satisfactory than MALLOC_CHECK_.

Prototype for mcheck() is:

```
#include <mcheck.h>
int mcheck (void (*ABORTHANDLER) (enum mcheck_status STATUS))
```

You must call mcheck() before calling any heap-related functions, otherwise call to mcheck() will fail. Therefore this function should be invoked very early in program. Call to mcheck() returns 0 upon success and -1 if called too late.

Argument *ABORTHANDLER is pointer to user-supplied function called when inconsistency in DAM detected. If you pass NULL to mcheck(), default handler used. Like MALLOC_CHECK_, this default handler prints error message to stdout and calls abort() to produce core file. Unlike MALLOC_CHECK_, error message is useful. For instance, trampling past end of dynamically allocated segment in following example:

```
int main(void)
{
    mcheck(NULL);
    int *p = (int *) malloc(sizeof(int));
    p[1] = 0;
    free(p);
    return 0;
}
```

Листинг 7-10. mcheckTest.c

produces error message shown here:

```
$ gcc -g3 -Wall -std=c99 mcheckTest.c -o mcheckTest -lmcheck
$ ./mcheckTest
memory clobbered past end of allocated block
Aborted (core dumped)
```

Other types of problems have similarly descriptive error messages.

7.6.3.3 Using mtrace() to catch memory leaks and double frees

The mtrace() facility is part of GNU C library and is used to catch memory leaks and double frees in C and C++ programs. Using it involves five steps:

1. Set environment variable MALLOC_TRACE to valid filename. This is name of file in which mtrace() places messages. If variable isn't set to valid filename or write permissions are not set for file, mtrace() will do nothing.
2. Include mcheck.h header file.
3. Call mtrace() at top of program. Its prototype is:

```
#include <mcheck.h>
void mtrace(void);
```
4. Run program. If any problems detected, they'll be documented in non human-readable form in file pointed to by MALLOC_TRACE. Also, for security reasons, mtrace() won't do anything for setuid or setgid executables.
5. The mtrace() facility comes with Perl script called mtrace, used to parse log file and print contents to standard output in human-readable form.

Note that there is also `muntrace()` call, used to stop memory tracing, but glibc info page recommends not using it. C library, which may also use DAM for your program, is notified that your program has terminated only after `main()` has returned or call to `exit()` made. Memory that C library uses for your program is not released until this happens. Call to `muntrace()` before this memory is released may lead to false positives.

Let's look at simple example. Here's code illustrating both problems that `mtrace()` catches. In following code, we never free memory allocated on line 6 and pointed to by `p`, and on line 10 we call `free()` on pointer `q`, even though it doesn't point to dynamically allocated memory.

```
1 int main(void)
2 {
3     int *p, *q;
4
5     mtrace();
6     p = (int *) malloc(sizeof(int));
7     printf("p points to %p\n", p);
8     printf("q points to %p\n", q);
9
10    free(q);
11    return 0;
12 }
```

Листинг 7-11. `mtrace1.c`

We compile this program and run it after setting `MALLOC_TRACE` variable.

```
$ gcc -g3 -Wall -Wextra -std=c99 -o mtrace1 mtrace1.c
$ MALLOC_TRACE="./mtrace.log" ./mtrace1
p points to 0x8049a58
q points to 0x804968c
```

If you look at contents of `mtrace.log`, it makes no sense at all. However, running Perl script `mtrace()` produces understandable output:

```
$ cat mtrace.log
= Start
@ ./mtrace1:(mtrace+0x120)[0x80484d4] + 0x8049a58 0x4
@ ./mtrace1:(mtrace+0x157)[0x804850b] - 0x804968c
p@satan$ mtrace mtrace.log
- 0x0804968c Free 3 was never alloc'd 0x804850b
Memory not freed:
-----
Address Size Caller
0x08049a58 0x4 at 0x80484d4
```

However, this is only slightly helpful, because although `mtrace()` found problems, it reported them as pointer addresses. Fortunately, `mtrace()` can do better. The `mtrace()` script also takes executable's filename as optional argument. Using this option, you get line numbers along with associated problems.

```
- 0x0804968c Free 3 was never alloc'd
/home/p/codeTests/mtrace1.c:15
Memory not freed:
-----
Address Size Caller
0x08049a58 0x4 at /home/p/codeTests/mtrace1.c:11
```

Вот теперь это именно то, что мы хотели увидеть!

Like `MALLOC_CHECK_` and `mcheck()` utilities, `mtrace()` won't prevent program from crashing. It simply checks for problems. If program crashes, some output of `mtrace()` may become lost or garbled, which could produce puzzling error reports. Best way to cope with this is to catch and handle seg faults in order to give `mtrace()` a shot at shutting down gracefully. Following example illustrates how to do so.

```

void sigsegv_handler(int signum);
int main(void)
{
    int *p;
    signal(SIGSEGV, sigsegv_handler);
    mtrace();
    p = (int *) malloc(sizeof(int));
    raise(SIGSEGV);
    return 0;
}
void sigsegv_handler(int signum)
{
    printf("Caught sigsegv: signal %d. Shutting down gracefully.\n", signum);
    muntrace();
    abort();
}

```

Листинг 7-12. mtrace2.c

Сноски

- [1] [\[назад\]](#) Этот рисунок пришлось преобразовать в grayscale для книги. На практике identifying error would be even easier.
- [2] [\[назад\]](#) Признайтесь. Вы ожидали, что они не balanced. Наша мысль не в том, что вы ошиблись, а в том, что ответ, вероятно, занял у вас больше времени, чем должен был.
- [3] [\[назад\]](#) Мы открыли файл в read mode, а затем попытались write to it.
- [4] [\[назад\]](#) Например, некоторые systems differentiate between EWOULDBLOCK and EAGAIN, but GNU/Linux does not.
- [5] [\[назад\]](#) Помимо libc info pages, можно посмотреть system header files, чтобы examine errno values. Это не только safe and natural thing to do, но actually encouraged practice!
- [6] [\[назад\]](#) Наше использование errno в листинге 7-2 не было good practice.
- [7] [\[назад\]](#) Вас может удивить question mark return value of exit(). Все, что strace здесь говорит, - _exit returns void.
- [8] [\[назад\]](#) Многие programmers считают badge of honor, когда splint reports no warnings. Когда это происходит, code declared lint free.
- [9] [\[назад\]](#) Return value of scanf() is number of input items that were assigned.
- [10] [\[назад\]](#) В остальной части этого раздела мы будем говорить только о malloc(), но на самом деле имеем в виду malloc() and friends, like calloc() and realloc().
- [11] [\[назад\]](#) Одно notable exception - function alloca(), which requests dynamic memory from current stack frame rather than heap. Memory in frame automatically freed when function returns. Thus, you don't have to free memory allocated by alloca().
- [12] [\[назад\]](#) This is colloquially known as dumping core.
- [13] [\[назад\]](#) Actually, you can use mtrace() and MALLOC_CHECK_ together safely, because mtrace() is careful to preserve any existing hooks it finds.
- [14] [\[назад\]](#) If you run program linked with EFence from within GDB, rather than invoking it on command line, program will seg fault without dumping core. This is desirable because core files of executables linked to EFence can be quite large, and you don't need core file anyway, because you'll already be inside GDB and staring at source code file and line number where seg fault occurred.
- [15] [\[назад\]](#) That doesn't mean malloc() overruns won't wreak havoc with your code! This example is contrived to show how you use EFence. In real program, writing beyond array bounds can cause serious problems!
- [16] [\[назад\]](#) If you want to be really careful, read "Word-Alignment and Overrun Detection" and "Instructions for Debugging Your Program" sections of EFence man page.
- [17] [\[назад\]](#) This is undocumented on authors' system. Thanks to Gianluca Insolvibile for reading glibc sources and finding this option!

Глава 8. Использование GDB/DDD/Eclipse для других языков

GDB и DDD обычно известны как отладчики для программ C/C++, но их можно использовать и для разработки на других языках. Eclipse изначально был спроектирован для разработки на Java, но имеет подключаемые модули для многих других языков. Эта глава покажет, как использовать эту многоязыковую возможность.

GDB/DDD/Eclipse не обязательно являются "лучшими" отладчиками для какого-либо конкретного языка. Для конкретных языков доступно большое число превосходных инструментов отладки. Однако мы говорим о том, что было бы удобно использовать один и тот же интерфейс отладки независимо от того, на каком языке вы пишете: C, C++, Java, Python, Perl или другие языки/отладчики, с которыми эти инструменты могут использоваться. DDD был "перенесен" ко всем ним.

Например, рассмотрим Python. Интерпретатор Python включает собственный простой текстовый отладчик. И снова, существует ряд отличных специфичных для Python графических отладчиков и IDE, но другой вариант - использовать DDD как интерфейс к встроенному отладчику Python. Это позволяет получить удобство графического интерфейса, при этом используя интерфейс, знакомый вам по программированию на C/C++ (DDD).

Откуда берется многоязыковая универсальность этих инструментов?

- Хотя GDB изначально был создан как отладчик для C/C++, разработчики GNU позже также предложили компилятор Java, GCJ.
- Вспомните, что DDD не является отладчиком сам по себе, а скорее графическим интерфейсом, через который можно выдавать команды нижележащему отладчику. Для C и C++ нижележащим отладчиком обычно является GDB. Однако DDD может быть, и часто бывает, оболочкой для отладчиков, специфичных для других языков.
- Eclipse тоже всего лишь оболочка. Подключаемые модули для различных языков дают ему возможность управлять разработкой и отладкой кода на этих языках.

В этой главе мы дадим обзор отладки Java, Perl, Python и языка ассемблера с помощью этих инструментов. Следует отметить, что в каждом случае есть дополнительные возможности, не рассмотренные здесь, и мы советуем вам изучить подробности для того языка, который вы используете.

8.1 Java

В качестве примера рассмотрим прикладную программу, которая работает со связным списком. Здесь объекты класса Node представляют узлы связного списка чисел, поддерживаемого в порядке возрастания значения ключа. Сам список является объектом класса LinkedList. Тестовая программа TestLL.java считывает числа из командной строки, строит связный список, состоящий из этих чисел в отсортированном порядке, затем печатает отсортированный список. Вот исходные файлы:

TestLL.java

```

1 // usage: [java] TestLL list_of_test_integers
2
3 // simple example program; reads integers from the command line,
4 // storing them in a linear linked list, maintaining ascending order,
5 // and then prints out the final list to the screen
6
7 public class TestLL
8 {
9     public static void main(String[] Args) {
10         int NumElements = Args.length;
11         LinkedList LL = new LinkedList();
12         for (int I = 1; I <= NumElements; I++) {
13             int Num;
14             // do C's "atoi()", using parseInt()
15             Num = Integer.parseInt(Args[I-1]);
16             Node NN = new Node(Num);
17             LL.Insert(NN);
18         }
19         System.out.println("final sorted list:");
20         LL.PrintList();
21     }
22 }

```

LinkedList.java

```

1 // LinkedList.java, implementing an ordered linked list of integers
2
3 public class LinkedList
4 {
5     public static Node Head = null;
6
7     public LinkedList() {
8         Head = null;
9     }
10
11     // inserts a node N into this list
12     public void Insert(Node N) {
13         if (Head == null) {
14             Head = N;
15             return;
16         }
17         if (N.Value < Head.Value) {
18             N.Next = Head;
19             Head = N;
20             return;
21         }
22         else if (Head.Next == null) {
23             Head.Next = N;
24             return;
25         }
26         for (Node D = Head; D.Next != null; D = D.Next) {
27             if (N.Value < D.Next.Value) {
28                 N.Next = D.Next;
29                 D.Next = N;
30                 return;
31             }
32         }
33     }
34
35     public static void PrintList() {
36         if (Head == null) return;
37         for (Node D = Head; D != null; D = D.Next)
38             System.out.println(D.Value);
39     }
40 }

```

Node.java


```

1 // Node.java, class for a node in an ordered linked list of integers
2
3 public class Node
4 {
5     int Value;
6     Node Next; // "pointer" to next item in list
7
8     // constructor
9     public Node(int V) {
10         Value = V;
11         Next = null;
12     }
13 }

```

В коде есть ошибка. Попробуем ее найти.

8.1.1 Прямое использование GDB для отладки Java

Java обычно воспринимается как интерпретируемый язык, но с компилятором GNU GCJ можно компилировать исходный код Java в родной машинный код. Это позволяет приложениям Java выполняться гораздо быстрее, а также означает, что можно использовать GDB для отладки. (Убедитесь, что у вас GDB версии 5.1 или более поздней.) GDB, напрямую или через DDD, мощнее, чем JDB, отладчик, который поставляется с Java Development Kit. Например, JDB не позволяет задавать условные точки останова, а это базовый прием отладки GDB, как вы уже видели. Поэтому вы не только выигрываете, изучая на один отладчик меньше, но и получаете лучшую функциональность.

Сначала скомпилируйте приложение в родной машинный код:

```

$ gcj -c -g Node.java
$ gcj -c -g LinkedList.java
$ gcj -g --main=TestLL TestLL.java Node.o LinkedList.o -o TestLL

```

Эти строки аналогичны обычным командам GCC, за исключением параметра `-main=TestLL`, который задает класс, чья функция `main()` является точкой входа для выполнения программы. (Мы компилировали два исходных файла по одному. Мы сочли это необходимым, чтобы GDB правильно отслеживал исходные файлы.) Запуск программы на тестовых входных данных дает:

```

$ TestLL 8 5 12
final sorted list:
5
8

```

Каким-то образом входное значение 12 исчезло. Посмотрим, как использовать GDB, чтобы найти ошибку. Запустите GDB как обычно и сначала скажите ему не останавливаться и не печатать объявления на экран, когда операции сборки мусора Java генерируют сигналы Unix. Такие действия являются помехой и могут мешать возможности выполнять пошаговое выполнение с помощью GDB.

```

(gdb) handle SIGPWR nostop noprint
Signal Stop Print Pass to program Description
SIGPWR No No Yes Power fail/restart
(gdb) handle SIGXCPU nostop noprint
Signal Stop Print Pass to program Description
SIGXCPU No No Yes CPU time limit exceeded

```

Теперь, поскольку первой очевидной жертвой ошибки было число 12 во входных данных, установим точку останова в начале метода `Insert()`, с условием, что значение ключа узла равно 12:

```
(gdb) b LinkedList.java:13 if N.Value == 12
Breakpoint 1 at 0x8048bb4: file LinkedList.java, line 13.
```

В качестве альтернативы вы могли бы подумать о том, чтобы попробовать команду:

```
(gdb) b LinkedList.java:Insert if N.Value == 12
```

Однако, хотя это сработало бы позже, в этот момент класс `LinkedList` еще не загружен.

Теперь запустите программу в GDB:

```
(gdb) r 8 5 12
Starting program: /debug/TestLL 8 5 12
[Thread debugging using libthread_db enabled]
[New Thread -1208596160 (LWP 12846)]
[New Thread -1210696800 (LWP 12876)]
[Switching to Thread -1208596160 (LWP 12846)]
Breakpoint 1, LinkedList.Insert(Node) (this=@47da8, N=@11a610)
at LinkedList.java:13
13 if (Head == null) {
Current language: auto; currently java
```

Вспоминая Принцип подтверждения, подтвердим, что значение, которое вот-вот будет вставлено, равно 12:

```
(gdb) p N.Value
$ 1=1 2
```

Теперь пройдите по коду пошагово:

```
(gdb) n
17 if (N.Value < Head.Value) {
(gdb) n
22 else if (Head.Next == null) {
(gdb) n
26 for (Node D = Head; D.Next != null; D = D.Next) {
(gdb) n
27 if (N.Value < D.Next.Value) {
(gdb) p D.Next.Value
$ 2=8
(gdb) n
26 for (Node D = Head; D.Next != null; D = D.Next) {
(gdb) n
12 public void Insert(Node N) {
(gdb) n
33 }
(gdb) n
TestLL.main(java.lang.String[]) (Args=@ab480) at TestLL.java:12
12 for (int I = 1; I <= NumElements; I++) {
```

Хм, это нехорошо. Мы прошли весь `Insert()` без вставки 12.

При более внимательном взгляде видно, что в цикле, начинающемся со строки 26 `LinkedList.java`, мы сравнили вставляемое значение, 12, с двумя значениями, которые сейчас находятся в списке, 5 и 8, и обнаружили, что в обоих случаях новое значение было больше. Именно здесь и находится ошибка. Мы не обработали случай, когда вставляемое значение больше всех значений, уже находящихся в списке. Нужно добавить код после строки 31, чтобы обработать эту ситуацию:

```
else if (D.Next.Next == null) {
    D.Next.Next = N;
    return;
}
```

После этого исправления программа работает как нужно.

8.1.2 Использование DDD с GDB для отладки Java

Эти шаги гораздо проще и приятнее, если использовать DDD как интерфейс к GDB (снова предполагая, что исходный код скомпилирован с помощью GCJ). Запустите DDD как обычно:

```
$ ddd TestLL
```

(Игнорируйте сообщение об ошибке, касающееся временного файла.)

Исходный код не сразу появляется в окне Source Text DDD, поэтому можно начать с консоли DDD:

```
(gdb) handle SIGPWR nostop noprint
(gdb) handle SIGXCPU nostop noprint
(gdb) b LinkedList.java:13
Breakpoint 1 at 0x8048b80: file LinkedList.java, line 13.
(gdb) cond 1 N.Value == 12
(gdb) r 8 5 12
```

В этот момент исходный код появляется, и вы находитесь в точке останова. Затем можно выполнять операции DDD как обычно.

8.1.3 Использование DDD как графического интерфейса для JDB

DDD можно использовать напрямую с отладчиком JDB из Java Development Kit. Команда:

```
$ ddd -jdb TestLL.java
```

запустит DDD, который затем вызовет JDB.

Однако мы обнаружили, что после этого работа становится довольно громоздкой, поэтому не рекомендуем такое использование DDD.

8.1.4 Отладка Java в Eclipse

Если вы изначально загрузили и установили версию Eclipse, приспособленную для разработки на C/C++, вам нужно получить подключаемый модуль JDT (Java Development Tools).

Базовые операции такие же, как описанные ранее для C/C++, но обратите внимание на следующее:

- При создании проекта обязательно выберите Java Project. Также мы рекомендуем отметить Use Project Folder As Root for Sources and Classes.
- Используйте представление Package Explorer как навигатор.
- Исходные файлы будут компилироваться в .class, как только они сохранены (или импортированы).
- Когда создаете диалог запуска, щелкните правой кнопкой Java Application и выберите New. В поле Main Class впишите класс, в котором определена main().
- Когда создаете диалог отладки, отметьте Stop in Main.
- При отладочных запусках Eclipse остановится перед main(). Просто нажмите кнопку Resume, чтобы продолжить.

Рисунок 8-1 показывает типичную картину отладки Java в Eclipse. Обратите внимание, что, как и с C/C++, в представлении Variables мы отображали значения узла N, направив треугольник рядом с N вниз.

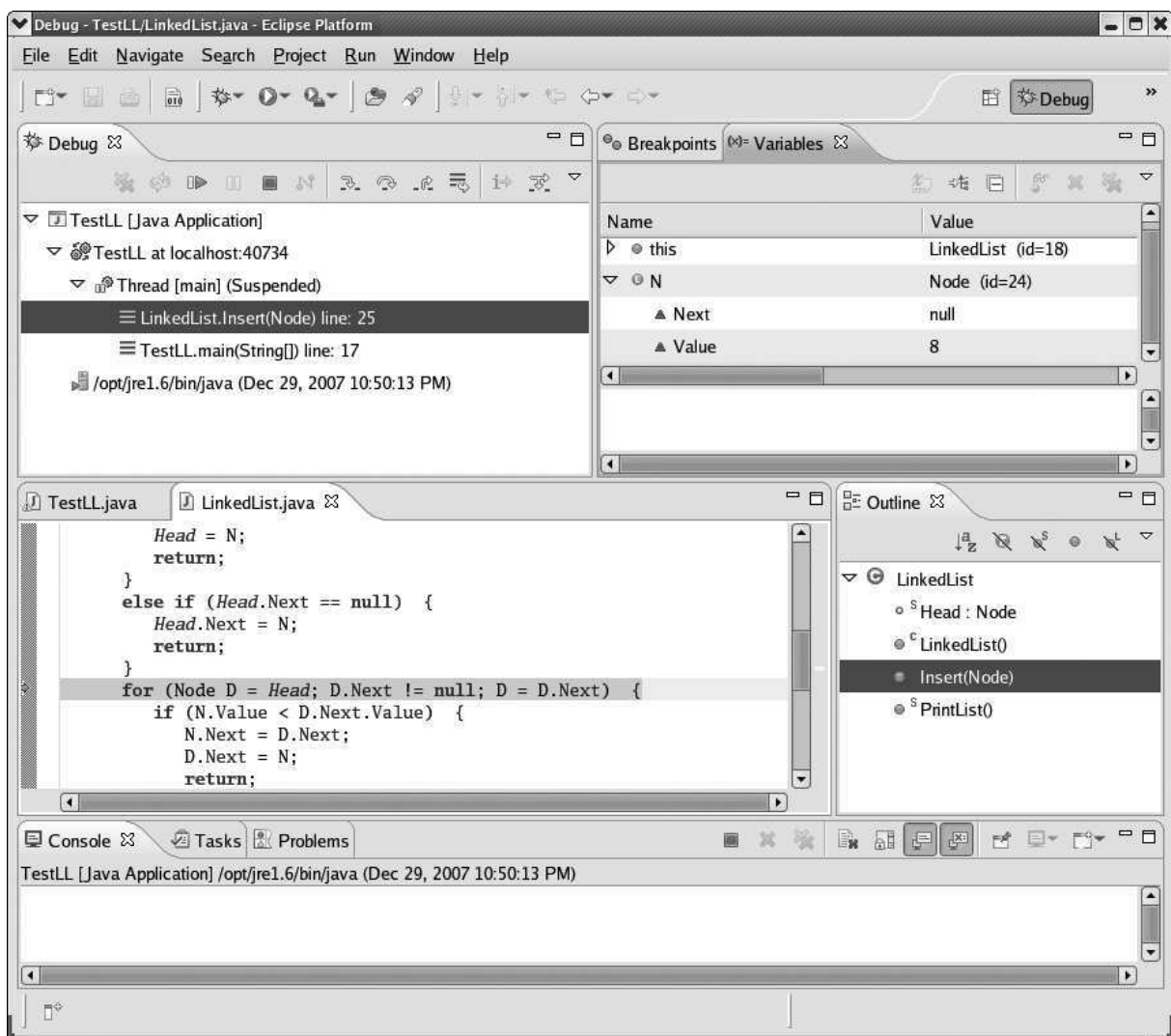


Рисунок 8-1. Отладка Java в Eclipse

8.2 Perl

Используем следующий пример, `textcount.pl`, который подсчитывает статистику по текстовым файлам:

```
1 #! /usr/bin/perl
2
3 # reads the text file given on the command line, and counts words,
4 # lines, and paragraphs
5
6 open(INFILE,@ARGV[0]);
7
8 $line_count = 0;
9 $word_count = 0;
10 $par_count = 0;
11
12 $now_in_par = 0; # not inside a paragraph right now
13
14 while ($line = <INFILE>) {
15     $line_count++;
16     if ($line ne "\n") {
17         if ($now_in_par == 0) {
18             $par_count++;
19             $now_in_par = 1;
20         }
21         @words_on_this_line = split(" ", $line);
22         $word_count += scalar(@words_on_this_line);
23     }
24     else {
25         $now_in_par = 0;
26     }
27 }
28
29 print "$word_count $line_count $par_count\n";
```

Программа подсчитывает число слов, строк и абзацев в текстовом файле, заданном в командной строке. В качестве тестового случая используем файл `test.txt`, показанный ниже (вы можете узнать в нем текст, похожий на текст из главы 1):

```
In this chapter we set out some basic principles of debugging, both
general and also with regard to the GDB and DDD debuggers. At least one
of our ``rules'' will be formal in nature, The Fundamental Principle of
Debugging.
```

```
Beginners should of course read this chapter carefully, since the
material here will be used throughout the remainder of the book.
```

```
Professionals may be tempted to skip the chapter. We suggest, though,
that they at least skim through it. Many professionals will find at
least some new material, and in any case it is important that all
readers begin with a common background.
```

Вверху есть одна пустая строка, две после `Debugging` и одна перед `Professionals`. Вывод нашего кода Perl при запуске на этом файле должен быть таким:

```
$ perl textcount.pl test.txt
102 14 3
```

Теперь предположим, что мы забыли предложение `else`:

```
else {  
  $now_in_par = 0;  
}
```

Тогда вывод будет таким:

```
$ perl textcount.pl test.txt  
102 14 1
```

Количество слов и строк правильно, но количество абзацев неверно.

8.2.1 Отладка Perl через DDD

Perl имеет собственный встроенный отладчик, вызываемый через параметр `-d` в командной строке:

```
$ perl -d myprog.pl
```

Один недостаток встроенного отладчика в том, что у него нет графического интерфейса, но это можно исправить, запуская отладчик через DDD. Посмотрим, как можно использовать DDD для поиска ошибки. Вызовите DDD, набрав:

```
$ ddd textcount.pl
```

DDD автоматически замечает, что это сценарий Perl, вызывает отладчик Perl и ставит зеленую стрелку "вы здесь" на первой исполняемой строке кода.

Теперь задайте аргумент командной строки, `test.txt`, щелкнув Program | Run и заполнив раздел Run with Arguments во всплывающем окне, как показано на рис. 8-2. (В консоли DDD вы можете получить сообщение вроде "... do not know how to create a new TTY ..." Просто игнорируйте его.) В качестве альтернативы мы могли бы задать аргумент "вручную", просто набрав команду отладчика Perl:

```
@ARGV[0] = "test.txt"
```

в консоли DDD.

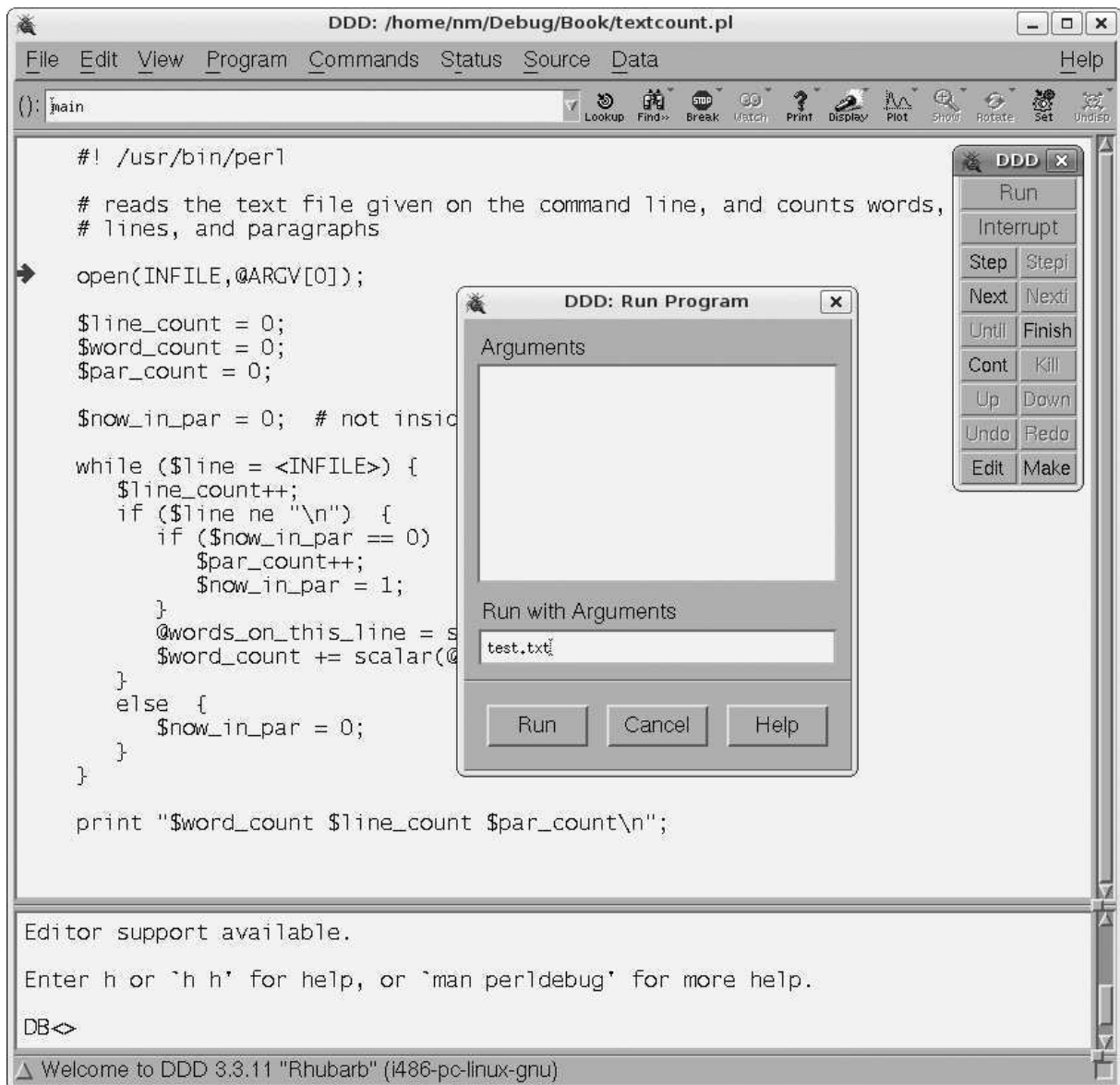


Рисунок 8-2. Задание аргументов командной строки

Поскольку ошибка находится в подсчете абзацев, посмотрим, что происходит, когда программа достигает конца первого абзаца. Это случится сразу после того, как условие:

```
$words_on_this_line[0] eq "Debugging."
```

становится истинным. Поставим точку останова около начала цикла `while`. Мы делаем это точно так же, как для программ C/C++: щелкаем строку правой кнопкой и выбираем Set Breakpoint.

Нужно также наложить приведенное выше условие на эту точку останова. Снова щелкните Source | Breakpoints, убедитесь, что нужная точка останова выделена во всплывающем окне Breakpoints and Watchpoints, а затем щелкните значок Props. Затем заполните нужное условие. См. рис. 8-3.

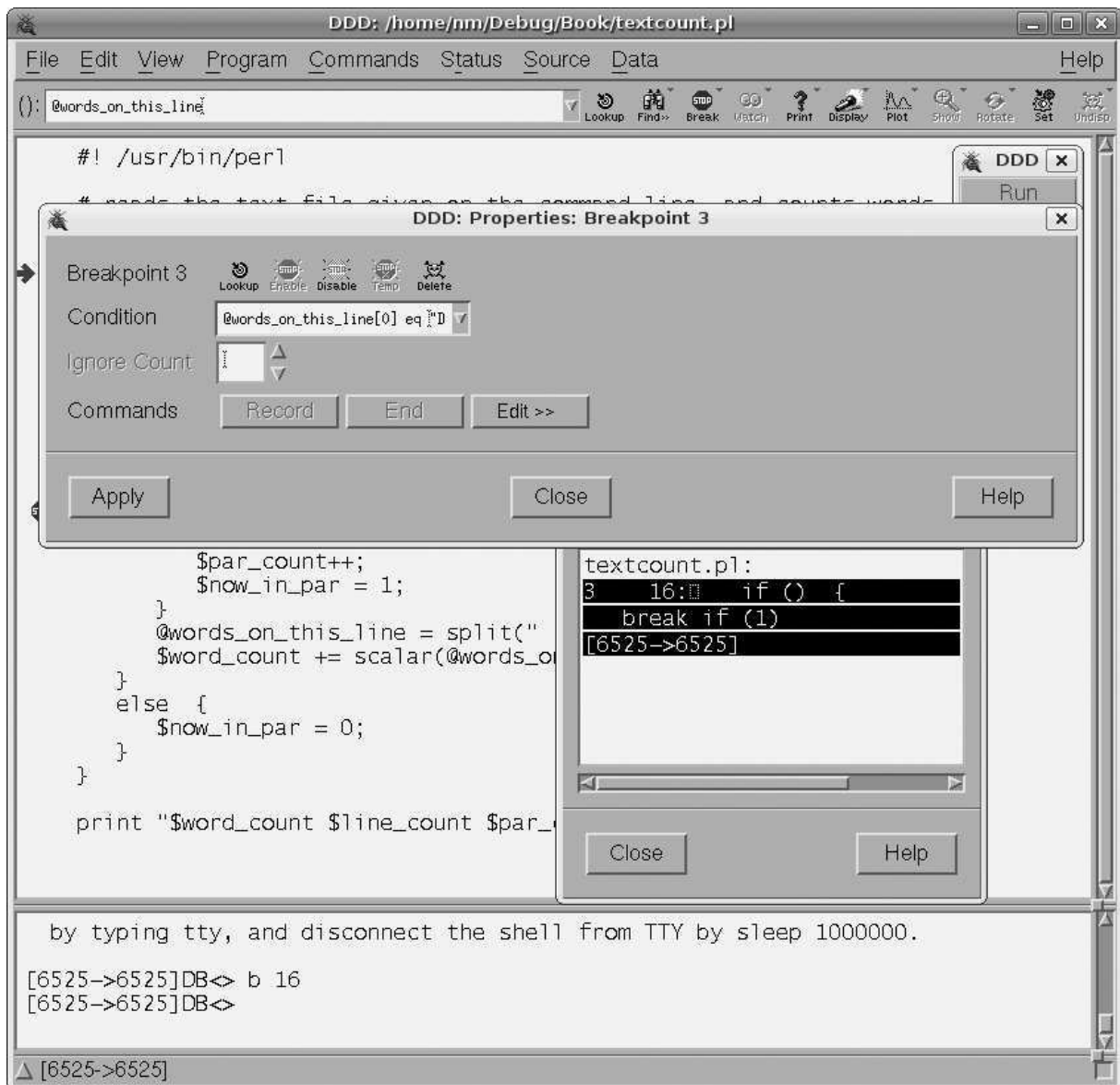


Рисунок 8-3. Задание условия для точки останова

Затем выберите Program | Run. (Мы не выбираем Run Again, поскольку это, похоже, уводит в недра Perl.) Мы перемещаем указатель мыши к экземпляру переменной `$words_on_this_line`, и обычное желтое окно DDD всплывает, показывая значение этой переменной. Так мы подтверждаем, что условие точки останова выполняется, как и должно быть. См. рис. 8-4.

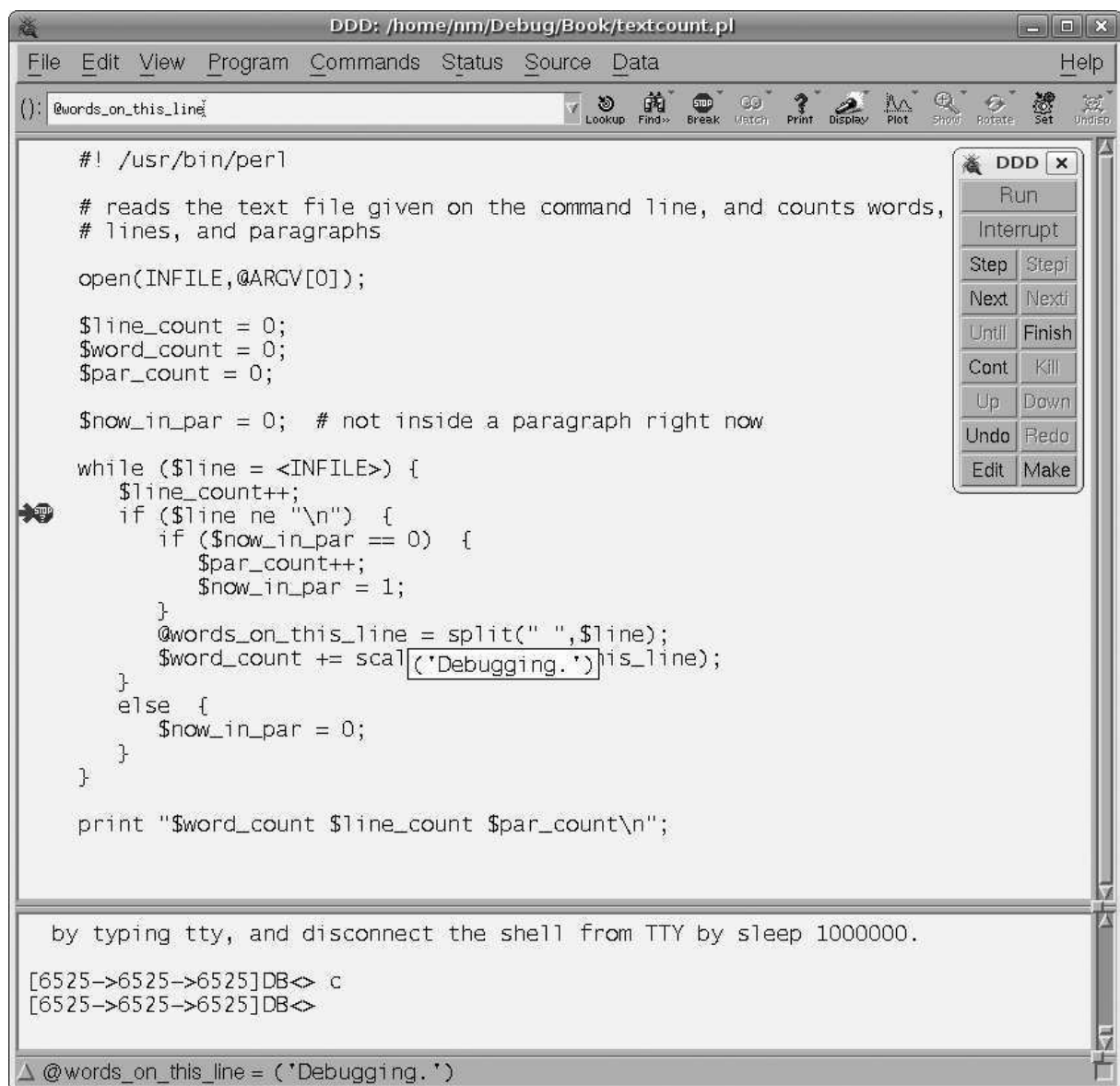


Рисунок 8-4. При прибытии в точку останова

После нескольких нажатий Next, чтобы пропустить пустые строки в текстовом файле, вы заметите, что мы также пропускаем строку:

```
$par_count++;
```

которая должна была увеличивать счетчик абзацев. Двигаясь назад, мы видим, что это было вызвано тем, что переменная `$now_in_par` равна 0, и, развивая это наблюдение, вскоре понимаем, как исправить ошибку.

Точку останова также можно отключить, включить или удалить в DDD, выбрав Source | Breakpoints, выделив точку останова и затем щелкнув нужный вариант.

Если вы изменили исходный файл, уведомите DDD, что нужно обновиться, выбрав File | Restart.

8.2.2 Отладка Perl в Eclipse

Для разработки кода Perl в Eclipse вам понадобится пакет Perl PadWalker, который можно загрузить из CPAN, и подключаемый модуль EPIC Eclipse для Perl.

Опять же, базовые операции такие же, как описанные ранее для C/C++, но обратите внимание на следующее:

- При создании проекта выберите Perl Project.
- Используйте Navigator как представление навигатора.
- Фазы сборки нет, поскольку Perl не производит байт-код.
- В перспективе Debug значения переменных доступны только в представлении Variables, а не действием мыши в представлении исходного кода, и только для локальных переменных.

Вам нужно будет использовать ключевое слово `Perl my` при первом появлении каждой переменной в исходном коде, чтобы видеть глобальные переменные (именно здесь вступает в дело пакет PadWalker).

Рисунок 8-5 показывает типичный экран отладки Perl. Обратите внимание, что мы добавили ключевое слово `my` к глобальным переменным.

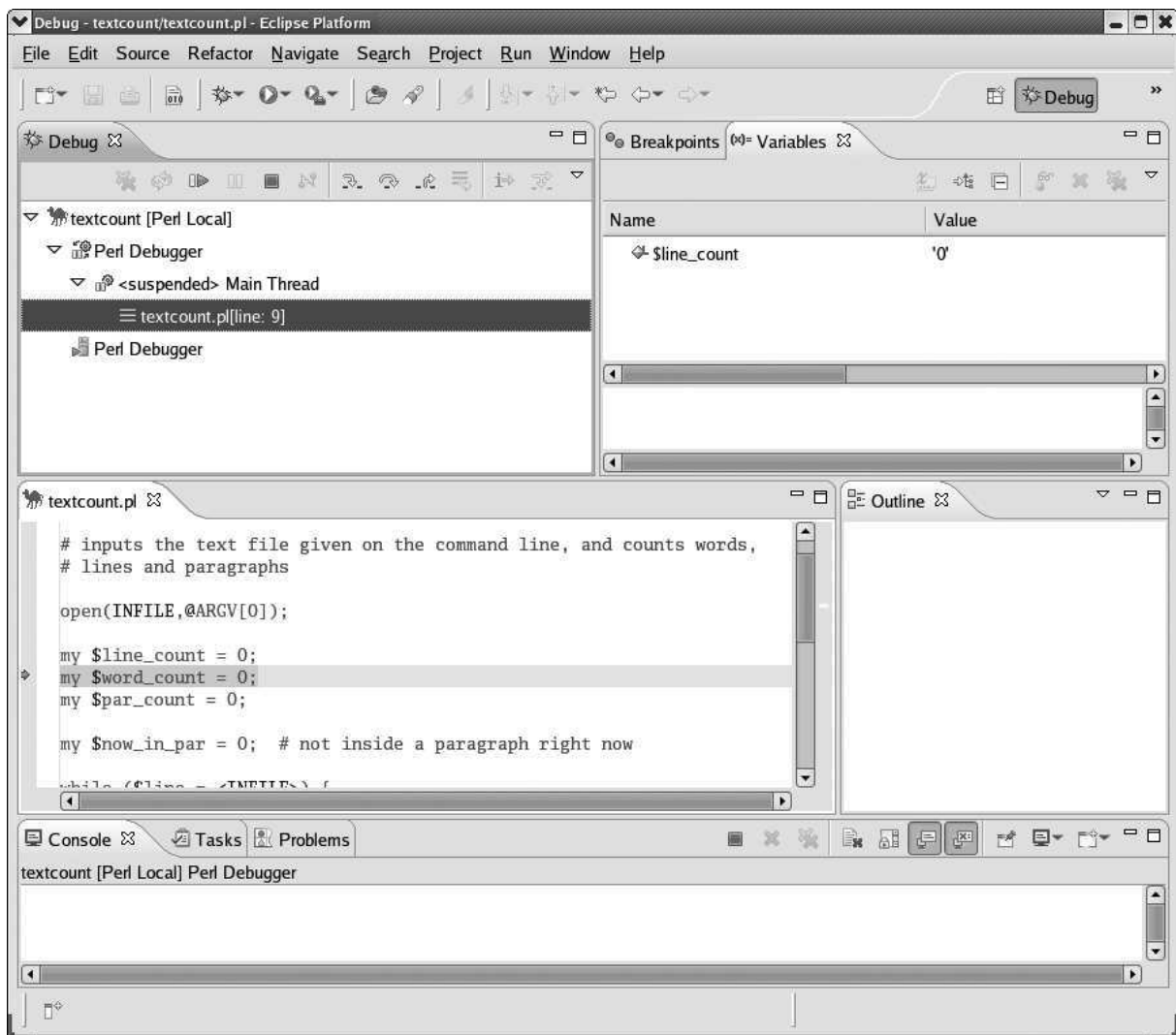


Рисунок 8-5. Экран отладки Perl

8.3 Python

Возьмем исходный пример `tf.py`, который подсчитывает слова, строки и абзацы в текстовом файле, как в приведенном выше примере Perl.

```
1 class textfile:
2     ntfiles = 0 # count of number of textfile objects
3     def __init__(self,fname):
4         textfile.ntfiles += 1
5         self.name = fname # name
6         self.fh = open(fname)
7         self.nlines = 0 # number of lines
8         self.nwords = 0 # number of words
9         self.npars = 0 # number of words
10        self.lines = self.fh.readlines()
11        self.wordlineparcount()
12        def wordlineparcount(self):
13            "finds the number of lines and words in the file"
14            self.nlines = len(self.lines)
15            inparagraph = 0
16            for l in self.lines:
17                w = l.split()
18                self.nwords += len(w)
19                if l == '\n':
20                    if inparagraph:
21                        inparagraph = 0
22                    elif not inparagraph:
23                        self.npars += 1
24                        inparagraph = 1
25
26        def grep(self,target):
27            "prints out all lines in the file containing target"
28            for l in self.lines:
29                if l.find(target) >= 0:
30                    print l
31                    print i
32
33        def main():
34            t = textfile('test.txt')
35            print t.nwords, t.nlines, t.npars
36
37        if __name__ == '__main__':
38            main()
```

Обязательно включите в свой исходный код две строки, показанные в конце программы:

```
if __name__ == '__main__':
    main()
```

NB. Если вы опытный программист Python, вы, вероятно, хорошо знаете этот шаблон. Если Python вам незнаком, краткое объяснение таково: нужно проверить, вызывается ли ваша программа Python сама по себе или импортируется как модуль в какую-то другую программу. Этот вопрос возникает при запуске программы через отладчик.

8.3.1 Отладка Python в DDD

Базовый отладчик Python - это PDB (`pdb.py`), текстовый инструмент. Его полезность сильно повышается при использовании DDD как графической оболочки.

Однако перед началом есть небольшая подготовительная работа. Чтобы позволить DDD правильно обращаться к PDB, Richard Wolff написал PYDB (`pydb.py`), слегка измененную версию PDB. После этого DDD запускался бы с параметром `-pydb`. Но Python развивался, и исходный PYDB перестал работать правильно.

Хорошее решение разработал Rocky Bernstein. На момент написания, летом 2007 года, его измененный (и значительно расширенный) PYDB планировалось включить в следующий выпуск DDD. В качестве альтернативы можно использовать исправление, которое любезно предоставил Richard Wolff. Вам понадобятся следующие файлы:

- <http://heather.cs.ucdavis.edu/~matloff/Python/DDD/pydb.py>
- <http://heather.cs.ucdavis.edu/~matloff/Python/DDD/pydbcmd.py>
- <http://heather.cs.ucdavis.edu/~matloff/Python/DDD/pybsupt.py>

Поместите файлы в какой-нибудь каталог, скажем `/usr/bin`, дайте им разрешение на выполнение и создайте файл с именем `pydb`, состоящий из одной строки:

```
/usr/bin/pydb.py
```

Убедитесь, что вы дали разрешение на выполнение и файлу `pydb`. Когда это сделано, можно использовать DDD для программ Python. Затем запустите DDD:

```
$ ddd --pydb
```

NB. Если всплывает окно ошибки с сообщением "PYDB couldn't start", у вас могут быть проблемы с путями - например, может существовать другой файл с именем `pydb`. Просто убедитесь, что предназначенный для DDD файл будет найден первым в пути поиска.

Затем загрузите исходный файл `tf.py`, либо выбрав `File | Open Source` и дважды щелкнув имя файла, либо набрав:

```
file tf.py
```

в консоли. Приглашение (`Pdb`) может сначала не появиться, но все равно введите команду.

Ваш исходный код появится в окне `Source Text`, и затем можно ставить точки останова как обычно. Предположим, вы поставили точку останова на строке:

```
w = l.split()
```

Чтобы запустить программу, щелкните `Program | Run`, задайте любые аргументы командной строки во всплывающем окне `Run`, а затем щелкните `Run`. После щелчка `Run` вам нужно будет дважды щелкнуть `Continue`. Это связано с работой нижележащего отладчика `PDB/PYDB`. (Если вы забудете, `PDB/PYDB` напомнит вам в консоли DDD.)

Если вы внесете изменение в исходный код, выдайте команду `file` в консоли или выберите `Source | Reload Source`. Ваши точки останова из последнего запуска будут сохранены.

8.3.2 Отладка Python в Eclipse

И снова процедуры для Python похожи на процедуры для других языков. Это, конечно, иллюстрирует ценность единого инструмента, применимого к нескольким языкам: как только вы научитесь использовать инструмент с одним языком, довольно легко использовать его с другим. Основные моменты, специфичные для Python:

- Нужно установить подключаемый модуль `Pydev`. После этого выберите `Window | Preferences | Pydev` и сообщите Eclipse расположение интерпретатора Python, например `/usr/bin/python`.
- Используйте `Pydev Package Explorer` как перспективу навигатора.
- Постоянные файлы байт-кода не создаются, поэтому процедуры сборки нет.
- При настройке диалога запуска/отладки обратите внимание на следующее: заполните `Main Module` именем исходного файла, с которого должно начаться выполнение. На вкладке `Arguments` укажите в `Base Directory` каталог, в котором находятся входные файлы (или корень дерева таких каталогов), и поместите аргументы командной строки в `Program Arguments`.

- В перспективе Debug значения переменных доступны только в представлении Variables и только для локальных переменных.

Одно крупное преимущество Eclipse перед DDD состоит в том, что PDB, нижележащий механизм отладки, используемый DDD, не работает с многопоточными программами, тогда как Eclipse работает.

8.4 Отладка кода SWIG

SWIG (Simplified Wrapper and Interface Generator) - популярный инструмент с открытым исходным кодом для сопряжения Java, Perl, Python и ряда других интерпретируемых языков с C/C++. Он входит в большинство дистрибутивов Linux, а также может быть загружен из Web. Он позволяет писать большую часть кода приложения на интерпретируемом языке и включать отдельные разделы, написанные на C/C++, например для повышения производительности.

Возникает вопрос, как запускать GDB/DDD на таком коде. Здесь мы представим небольшой пример с использованием Python и C. Код C будет управлять очередью "первым пришел - первым вышел" (FIFO):

```
1 // fifo.c, SWIG example; manages a FIFO queue of characters
2
3 char *fifo; // the queue
4
5 int nfifo = 0, // current length of the queue
6 maxfifo; // max length of the queue
7
8 int fifoinit(int spfifo) // allocate space for a max of spfifo elements
9 { fifo = malloc(spfifo);
10 if (fifo == 0) return 0; // failure
11 else {
12 maxfifo = spfifo;
13 return 1; // success
14 }
15 }
16
17 int fifoput(char c) // append c to queue
18 { if (nfifo < maxfifo) {
19 fifo[nfifo] = c;
20 return 1; // success
21 }
22 else return 0; // failure
23 }
24
25 char fifoget() // delete head of queue and return
26 { char c;
27 if (nfifo > 0) {
28 c = fifo[0];
29 memmove(fifo, fifo+1, --nfifo);
30 return c;
31 }
32 else return 0; // assume no null characters ever in queue
33 }
```

Помимо файла .c, SWIG также требует интерфейсный файл, в данном случае fifo.i. Он состоит из глобальных символов, перечисленных один раз в стиле SWIG и один раз в стиле C:

```
%module fifo
%{extern char *fifo;
extern int nfifo,
maxfifo;
extern int fifoinit(int);
extern int fifoput(char);
extern char fifoget(); %}
extern char *fifo;
extern int nfifo,
maxfifo;
extern int fifoinit(int);
extern int fifoput(char);
extern char fifoget();
```

Чтобы скомпилировать код, сначала запустите swig, который сгенерирует дополнительный файл .c и файл Python. Затем используйте GCC и ld, чтобы получить динамическую библиотеку разделяемого объекта .so. Вот Makefile:

```
_fifo.so: fifo.o fifo_wrap.o
gcc -shared fifo.o fifo_wrap.o -o _fifo.so
fifo.o fifo_wrap.o: fifo.c fifo_wrap.c
gcc -fPIC -g -c fifo.c fifo_wrap.c -I/usr/include/python2.4
fifo.py fifo_wrap.c: fifo.i
swig -python fifo.i
```

Библиотека импортируется в Python как модуль, как показано в тестовой программе:

```
# testfifo.py
import fifo
def main():
    fifo.fifoinit(100)
    fifo.fifoput('x')
    fifo.fifoput('c')
    c = fifo.fifoget()
    print c
    c = fifo.fifoget()
    print c
if __name__ == '__main__': main()
```

Вывод этой программы должен быть "x" и "c", но мы получаем пустой результат:

```
$ python testfifo.py
$
```

Чтобы использовать GDB, помните, что фактическая программа, которую вы запускаете, - это интерпретатор Python, python. Поэтому запускайте GDB для интерпретатора:

```
$ gdb python
```

Теперь мы хотели бы поставить точку останова внутри присоединенной библиотеки FIFO, но библиотека еще не загружена. Загрузка не произойдет, пока строка импорта:

```
import fifo
```

не будет выполнена интерпретатором Python. К счастью, мы все равно можем попросить GDB остановиться на функции в библиотеке:

```
(gdb) b fifoput
Function "fifoput" not defined.
Make breakpoint pending on future shared library load? (y or [n]) y
Breakpoint 1 (fifoput) pending.
```

Теперь запустите интерпретатор, аргументом которого является тестовая программа `testfifo.py`:

```
(gdb) r testfifo.py
Starting program: /usr/bin/python testfifo.py
Reading symbols from shared object read from target memory...(no debugging
symbols found)...done.
Loaded system supplied DSO at 0x164000
(no debugging symbols found)
(no debugging symbols found)
(no debugging symbols found)
[Thread debugging using libthread_db enabled]
[New Thread -1208383808 (LWP 15912)]
(no debugging symbols found)
(no debugging symbols found)
(no debugging symbols found)
(no debugging symbols found)
Breakpoint 2 at 0x3b25f8: file fifo.c, line 18.
Pending breakpoint "fifoput" resolved
[Switching to Thread -1208383808 (LWP 15912)]
Breakpoint 2, fifoput (c=120 'x') at fifo.c:18
18 { if (nfifo < maxfifo) {
```

Теперь можно делать то, что вы уже так хорошо знаете:

```
(gdb) n
19 fifo[nfifo] = c;
(gdb) p nfifo
$ 1=0
(gdb) c
Continuing.
Breakpoint 2, fifoput (c=99 'c') at fifo.c:18
18 { if (nfifo < maxfifo) {
(gdb) n
19 fifo[nfifo] = c;
(gdb) p nfifo
$ 2=0
```

Что ж, здесь есть проблема. Каждый раз, когда вы пытаетесь добавить символ в очередь, вы просто перезаписываете ранее добавленный символ. Строка 19 должна быть такой:

```
fifo[nfifo++] = c;
```

После внесения этого изменения код работает нормально.

8.5 Язык ассемблера

GDB и DDD могут быть чрезвычайно полезны при отладке кода на языке ассемблера. Есть ряд особых соображений, которые нужно иметь в виду; они описаны в этом разделе.

Возьмем в качестве примера код в файле testff.s:

```
1 # the subroutine findfirst(v,w,b) finds the first instance of a value v
2 # in a block of w consecutive words of memory beginning at b, returning
3 # either the index of the word where v was found (0, 1, 2, ...) or -1 if
4 # v was not found; beginning with _start, we have a short test of the
5 # subroutine
6
7 .data # data segment
8 x:
9 .long 1
10 .long 5
11 .long 3
12 .long 168
13 .long 8888
14 .text # code segment
15 .globl _start # required
16 _start: # required to use this label unless special action taken
17 # push the arguments on the stack, then make the call
18 push $x+4 # start search at the 5
19 push $168 # search for 168 (deliberately out of order)
20 push $4 # search 4 words
21 call findfirst
22 done:
23 movl %edi, %edi # dummy instruction for breakpoint
24 findfirst:
25 # finds first instance of a specified value in a block of words
26 # EBX will contain the value to be searched for
27 # ECX will contain the number of words to be searched
28 # EAX will point to the current word to search
29 # return value (EAX) will be index of the word found (-1 if not found)
30 # fetch the arguments from the stack
31 movl 4(%esp), %ebx
32 movl 8(%esp), %ecx
33 movl 12(%esp), %eax
34 movl %eax, %edx # save block start location
35 # top of loop; compare the current word to the search value
36 top: cmpl (%eax), %ebx
37 jz found
38 decl %ecx # decrement counter of number of words left to search
39 jz notthere # if counter has reached 0, the search value isn't there
40 addl $4, %eax # otherwise, go on to the next word
41 jmp top
42 found:
43 subl %edx, %eax # get offset from start of block
44 shr $2, %eax # divide by 4, to convert from byte offset to index
45 ret
46 notthere:
47 movl $-1, %eax
48 ret
```

Это язык ассемблера Linux Intel, использующий синтаксис AT&T, но пользователи, знакомые с другими синтаксисами Intel, должны без труда проследить код. (Команда GDB `set disassembly-flavor intel` заставит GDB отображать весь вывод команды `disassemble` в синтаксисе Intel, таком как синтаксис, используемый, например, компилятором NASM. Кстати, поскольку это платформа Linux, программа выполняется в 32-битном плоском режиме процессора Intel.)

Как указано в комментариях, подпрограмма `findfirst` находит первое вхождение заданного значения внутри заданного блока последовательных слов памяти. Возвращаемое значение подпрограммы - это индекс (0, 1, 2, ...) слова, в котором значение было найдено, или -1, если оно не найдено.

Подпрограмма ожидает, что аргументы будут помещены в стек, поэтому при входе стек выглядит так:

```
адрес начала блока, в котором выполняется поиск
число слов в блоке
искомое значение
адрес возврата
```

NB. Стеки Intel растут вниз, то есть к адресу 0 в памяти. Слова с меньшими адресами появляются на рисунке ниже.

Чтобы внести ошибку, которую мы сможем найти с помощью GDB, мы намеренно перепутали элементы в последовательности вызова в "главной" программе:

```
push $x+4 # start search at the 5
push $168 # search for 168 (deliberately out of order)
push $4 # search 4 words
```

Инструкции, предшествующие вызову, вместо этого должны быть такими:

```
push $x+4 # start search at the 5
push $4 # search 4 words
push $168 # search for 168
```

Так же как вы используете параметр `-g` при компиляции кода C/C++ для использования с GDB/DDO, здесь на уровне ассемблера используйте `-gstabs`:

```
$ as -a --gstabs -o testff.o testff.s
```

Это создает объектный файл `testff.o` и печатает бок о бок сравнение исходного кода на ассемблере и соответствующего машинного кода. Также показываются смещения элементов данных и другая информация, потенциально полезная для процесса отладки.

Затем выполните компоновку:

```
$ ld testff.o
```

Это создает исполняемый файл с именем по умолчанию `a.out`.

Запустим этот код под GDB:

```
(gdb) b done
Breakpoint 1 at 0x8048085: file testff.s, line 18.
(gdb) r
Starting program: /debug/a.out
Breakpoint 1, done () at testff.s:18
18 movl %edi, %edi # dummy for breakpoint
Current language: auto; currently asm
(gdb) p $eax
$ 1=- 1
```

Как видите, к регистрам можно обращаться через префиксы со знаком доллара, в данном случае `$eax` для регистра EAX. К сожалению, значение в этом регистре равно -1, что указывает, что нужное значение, 168, не было найдено в заданном блоке.

При отладке программ на языке ассемблера одна из первых вещей, которые нужно сделать, - проверить правильность стека. Поэтому поставьте точку останова на подпрограмме и изучите стек, когда туда попадете:

```
(gdb) b findfirst
Breakpoint 2 at 0x8048087: file testff.s, line 25.
(gdb) r
The program being debugged has been started already.
Start it from the beginning? (y or n) y
Starting program: /debug/a.out
Breakpoint 2, findfirst () at testff.s:25
25 movl 4(%esp), %ebx
(gdb) x/4w $esp
0xbffffd9a0: 0x08048085 0x00000004 0x000000a8 0x080490b4
```

Стек, конечно, является частью памяти, поэтому для его изучения нужно использовать команду GDB `x`, которая исследует память. Здесь мы попросили GDB вывести четыре слова, начиная с местоположения, указанного указателем стека ESP (обратите внимание, что приведенное выше изображение стека показывает четыре слова). Команда `x` выводит память в порядке возрастания адресов. Именно это вам и нужно, поскольку на архитектуре Intel, как и на многих других, стек растет к 0.

Из приведенного выше изображения стека видно, что первое слово должно быть адресом возврата. Это ожидание можно проверить различными способами. Один подход - использовать команду GDB `disassemble`, которая перечисляет инструкции языка ассемблера (обратно переведенные из машинного кода) и их адреса. Войдя пошагово в подпрограмму, вы затем могли бы проверить, совпадает ли содержимое первого слова в стеке с адресом функции, следующей за вызовом.

Вы увидите, что совпадает. Однако вы обнаружите, что второе число, 4, которое должно быть искомым значением (168), на самом деле является размером блока поиска (4). Из этой информации вы быстро поймете, что мы случайно поменяли местами две инструкции `push` перед вызовом.